

Final Presentation - Overcoming Mislabeled Deep Learning

Jose Mancera

Overview

Problem Statement

- Due to human error, automated labeling flaws, or inherent ambiguities, mislabeled or noisy data in datasets is common in the real world
- Noisy data causes issues during the training process for deep learning models
 - Can overfit to noise
 - Can degrade generalizability

Methodology

- Creating multiple corrupted versions of each dataset for noise levels of 10%-60%
- Measure the variance in the outcomes of each noise level on each architecture
- Attack noise using noise robust loss functions (Cross Entropy, Normalized Focal Loss, etc.)
- Classification metrics (F1 score, accuracy, precision, recall)
- Plots/Graphs (Accuracy plot, Loss plot, Confusion matrices)

Objective

- Determine how different architectures (CNN, ResNet, DenseNet, Vision Transformers (ViT), MLP) respond to different noise levels on standard benchmarks (CIFAR-10, CIFAR-100, MNIST)

Overcoming Mislabeled Deep Learning

How does data get mislabeled?

Why is this relevant?

Automation

Labeling software can misclassify samples for several reasons:

- Associated keywords might not be directly linked to the label
- Faulty sensors, OCR mistakes, pipeline errors

Human Error

Crowdsourcing: using many people to label or annotate data

- Leads to challenges in coordinating large numbers of contributors
- Labelers may be underqualified, rushed, biased, etc

Data and Performance

- Real world data will contain some level of label noise
- Noisy Labels -> Overfitting to incorrect labels
- More robust models -> more reliable models

“Garbage in, garbage out”

- Re-labeling can be expensive and slow
- Datasets are massive, 100's * thousands and/or millions of samples

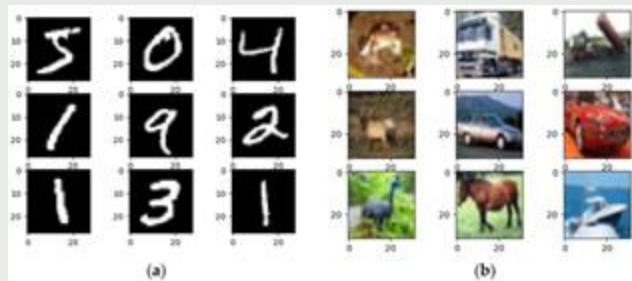
Cleaning data isn't always realistic :(

Mislabeled deep learning is about making models resilient to the imperfections of real world data

Introducing Label Noise

- Label noise is the presence of incorrect or inaccurate labels within a dataset
- Noise can significantly hinder the performance of machine learning models
- An incorrect label is a label that does not accurately reflect the true class of a data instance
- 3 Types
 - Symmetric
 - Labels are randomly flipped to any other class with equal probability
 - Asymmetric
 - Certain classes are more likely to be mislabeled to another class
 - Ex: Dog -> Cat
 - Instance Dependent
 - Likelihood of mislabeling depends on the characteristics of the image
 - Ex: Blurry or ambiguous images

Observations



Different Datasets, Different Classification Metrics

- MNIST's handwritten digits have clear, structured shapes, making classification easier
- CIFAR-10 / CIFAR-100 images have color, texture, and background noise,
- DNNs do not capture object positioning and orientation well

$$d = \log_2(1 + a) + \log_2(1 + \frac{s+p}{2})$$

*Decreasing D - Index as Label Noise % Increases

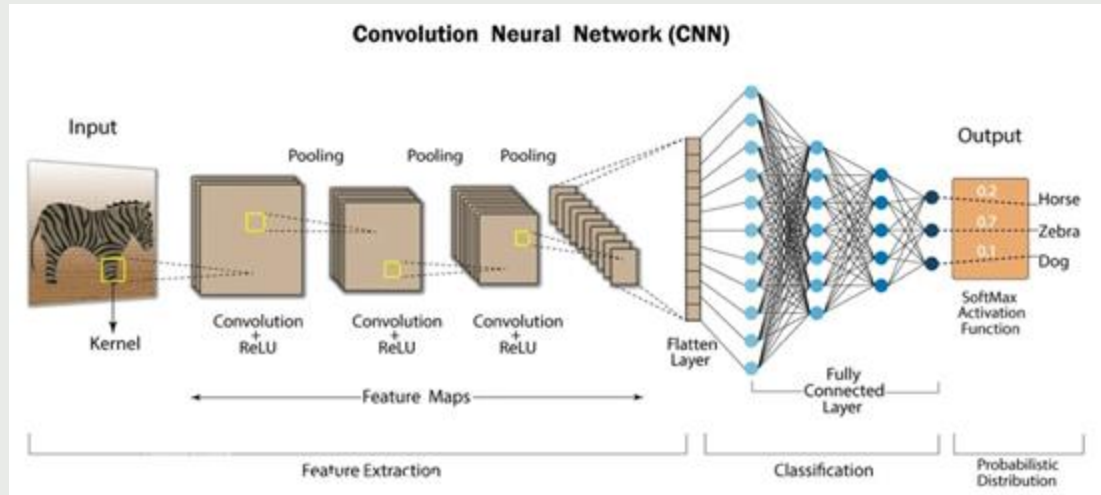
- Increasing label noise lowers the model's ability to learn meaningful patterns declines
- Models struggle to extract consistent patterns when labels are unreliable
- A high d-index indicates the model is learning meaningful features; (0, 2]



Training Loss >>> Validation Loss

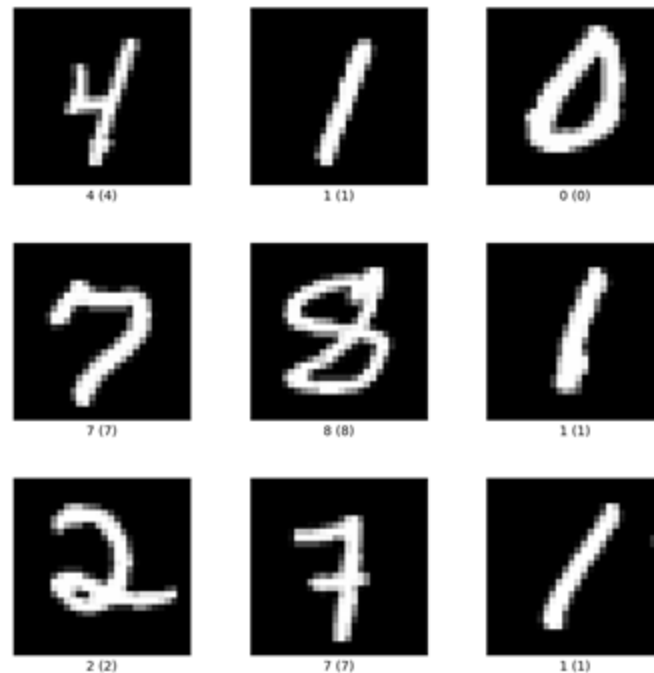
- Model struggle to fit incorrect labels in the training set -> higher loss compared to the clean validation set
- The model can't memorize noisy labels 100%, prevents validation loss from increasing.

Convolutional Neural Networks (CNN)

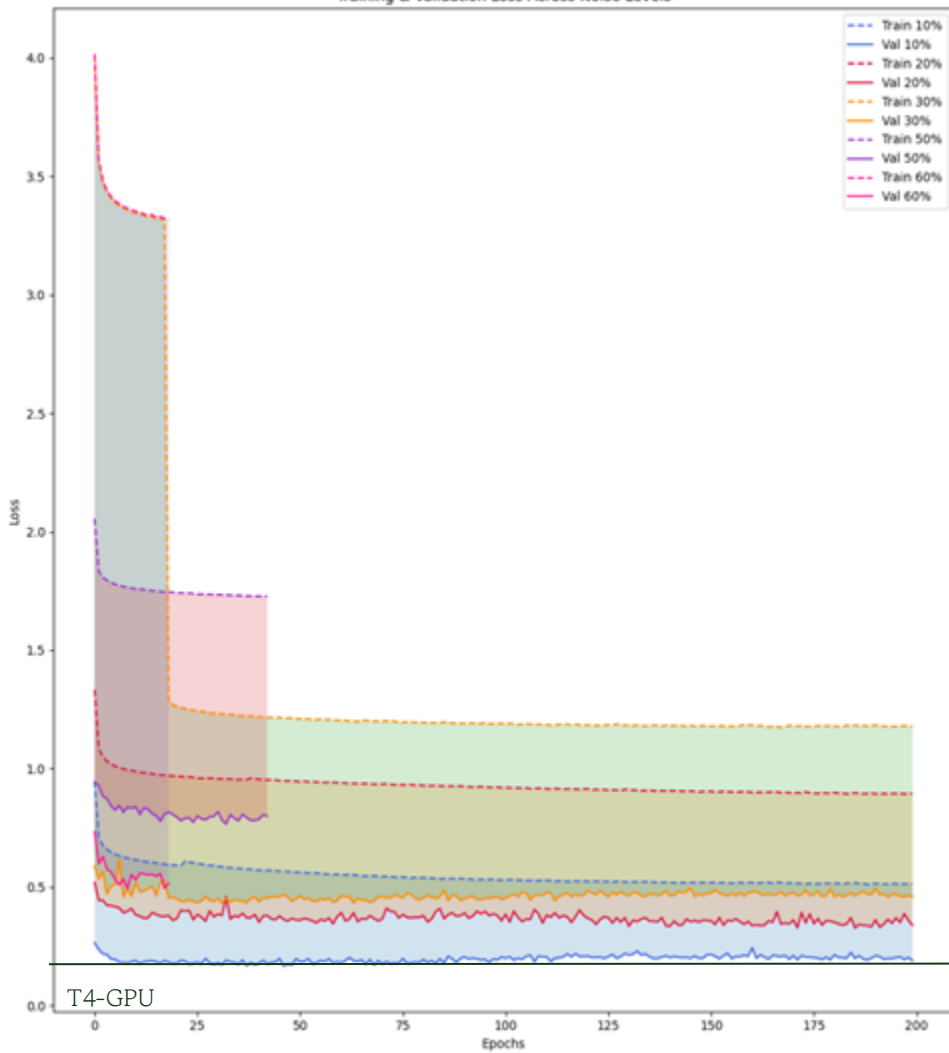


- Specialized DNN designed for image processing
- Partially connected DNN
- Neurons are only connected to neurons in previous layers interested in them
- Sequence of convolutional layers + pooling layers + fully connected layers
- Capable of both feature mapping and feature extraction; DNNs can only do feature extraction

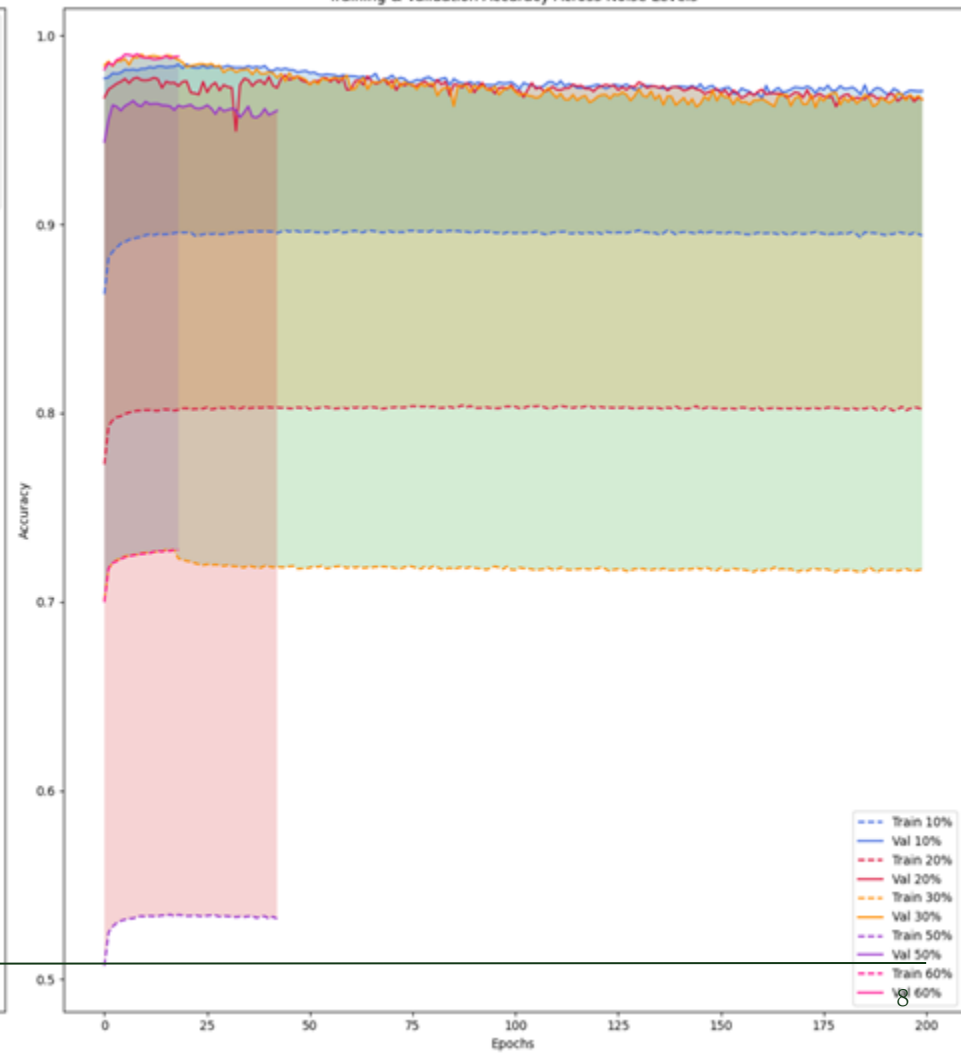
Comparing CNN Performance with MNIST Variants



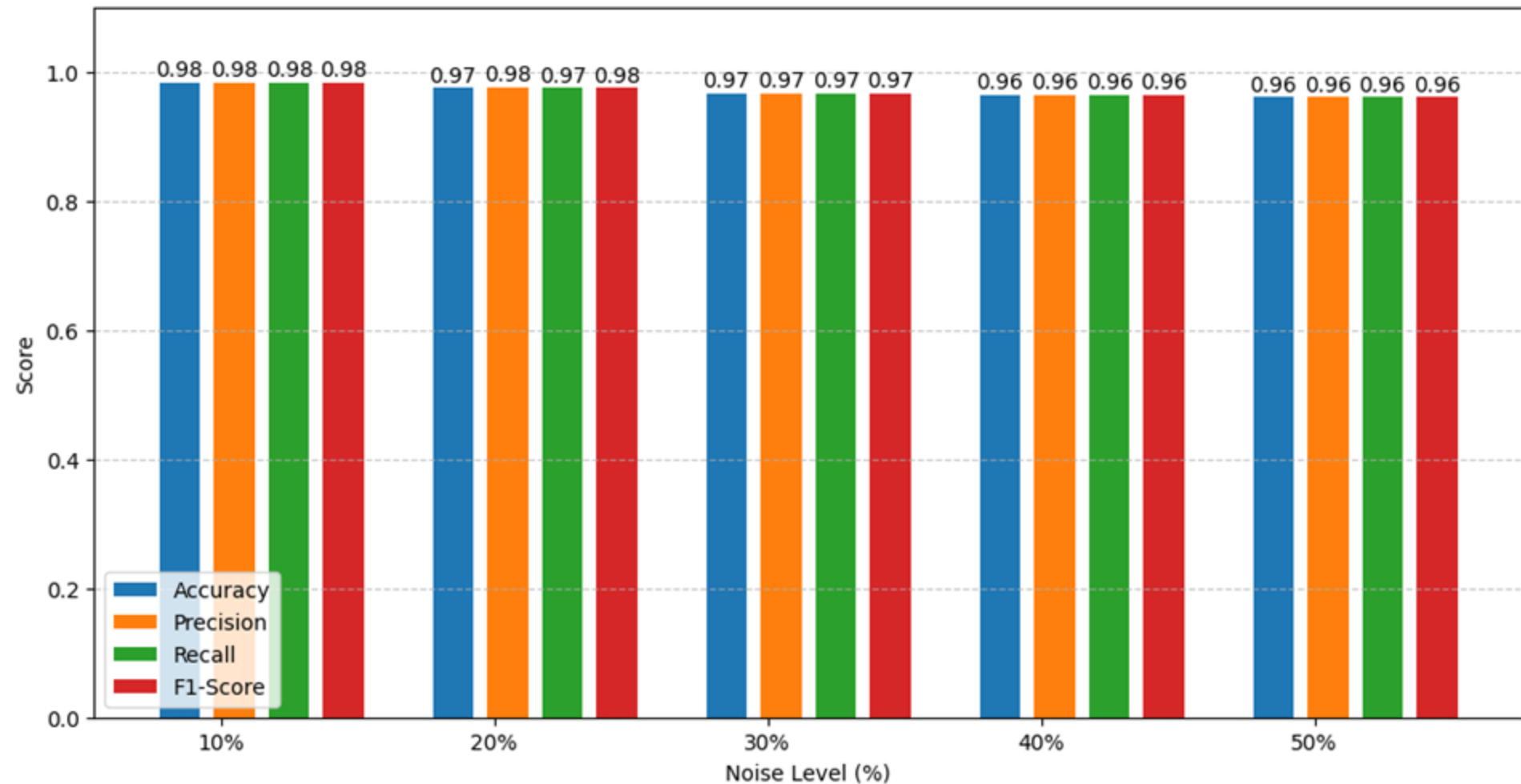
Training & Validation Loss Across Noise Levels



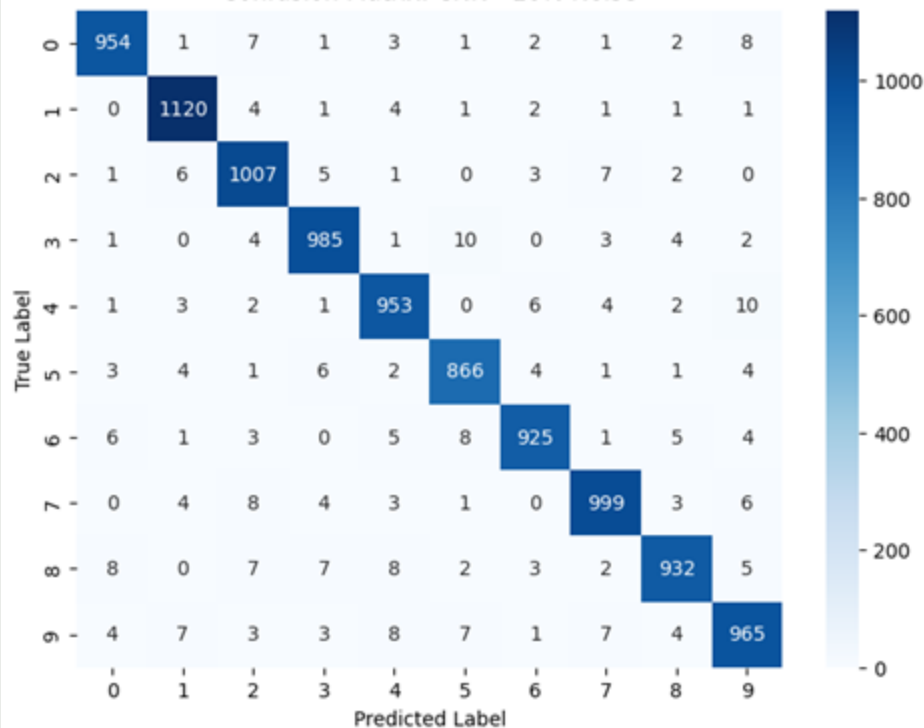
Training & Validation Accuracy Across Noise Levels



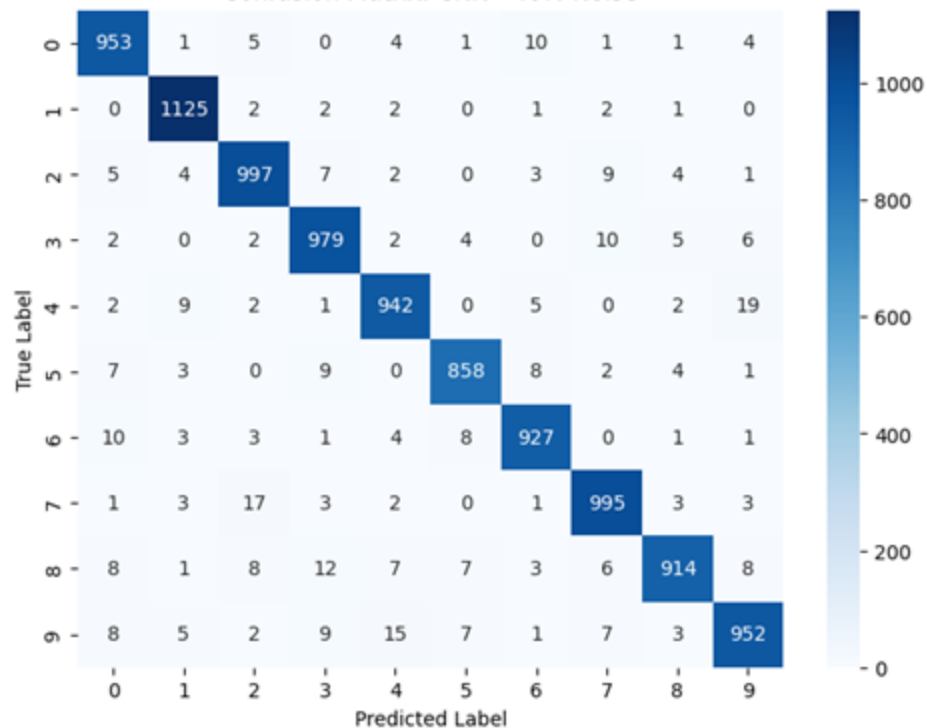
Model Performance Metrics Across Noise Levels



Confusion Matrix: CNN - 10% Noise



Confusion Matrix: CNN - 40% Noise

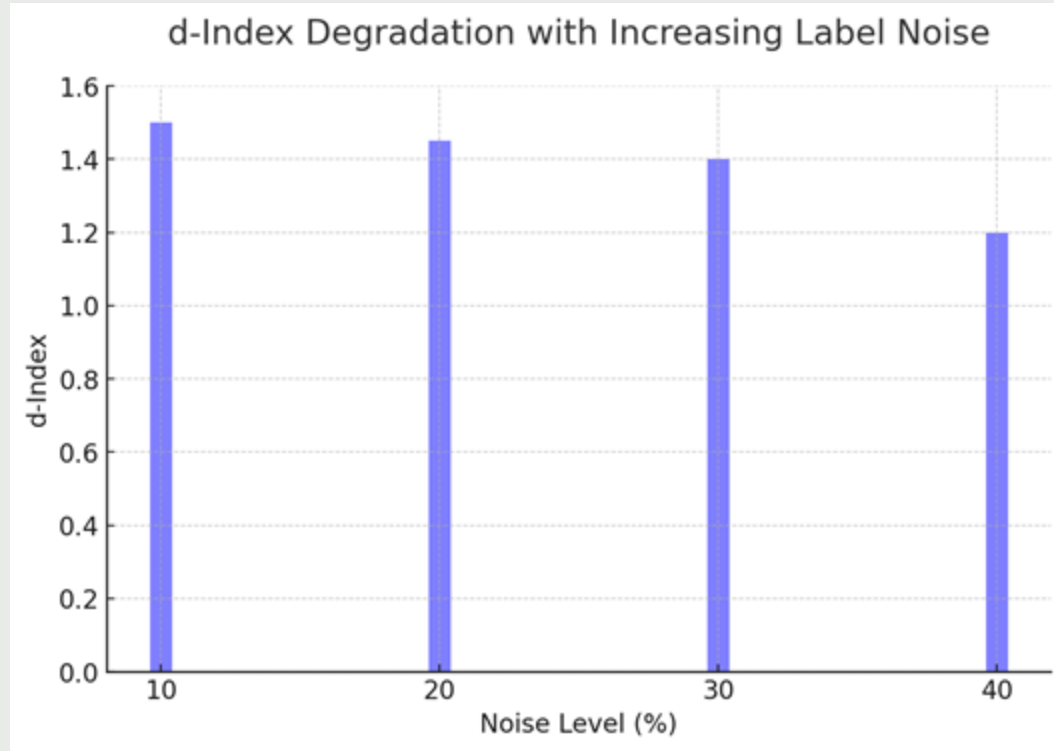


CNN - 10% Noise - Most Misclassified Classes:

- 3 → 5 (10 times)
- 4 → 9 (10 times)
- 0 → 9 (8 times)

CNN - 40% Noise - Most Misclassified Classes:

- 4 → 9 (19 times)
- 7 → 2 (17 times)
- 9 → 4 (15 times)

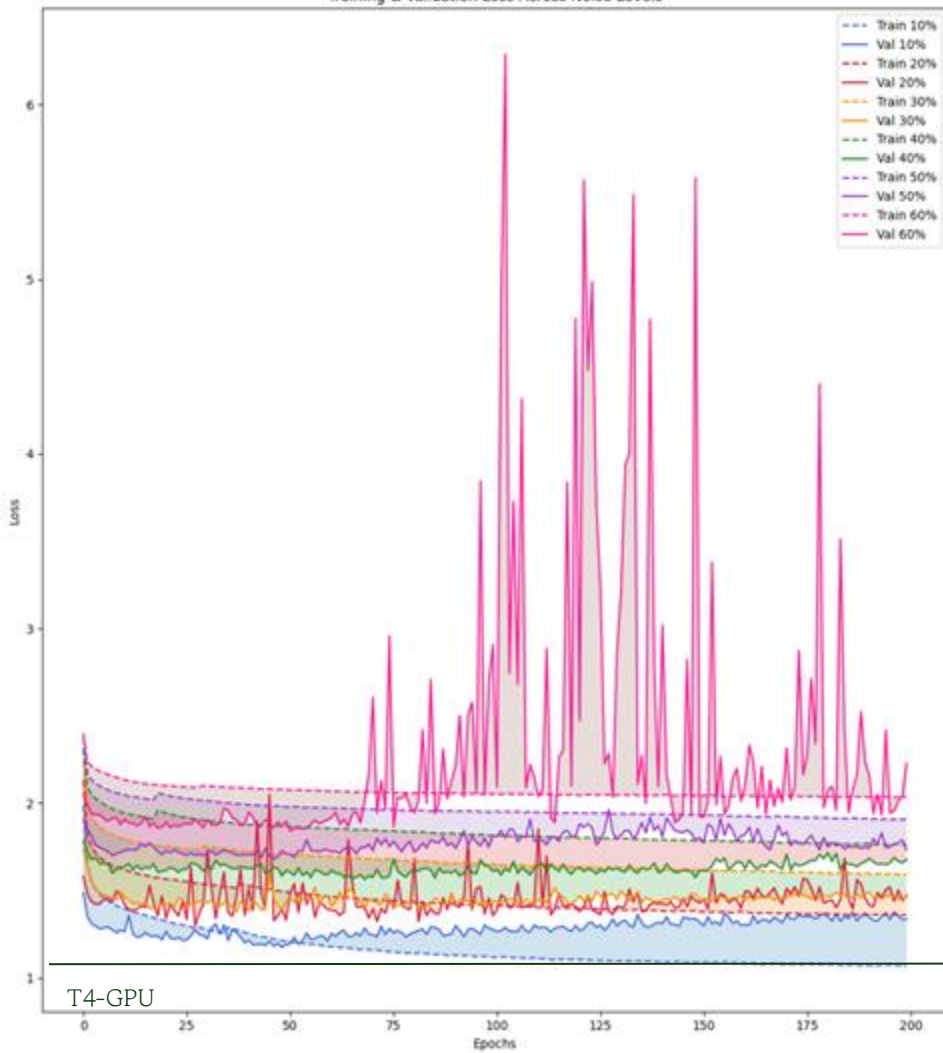


Diagnostic Index

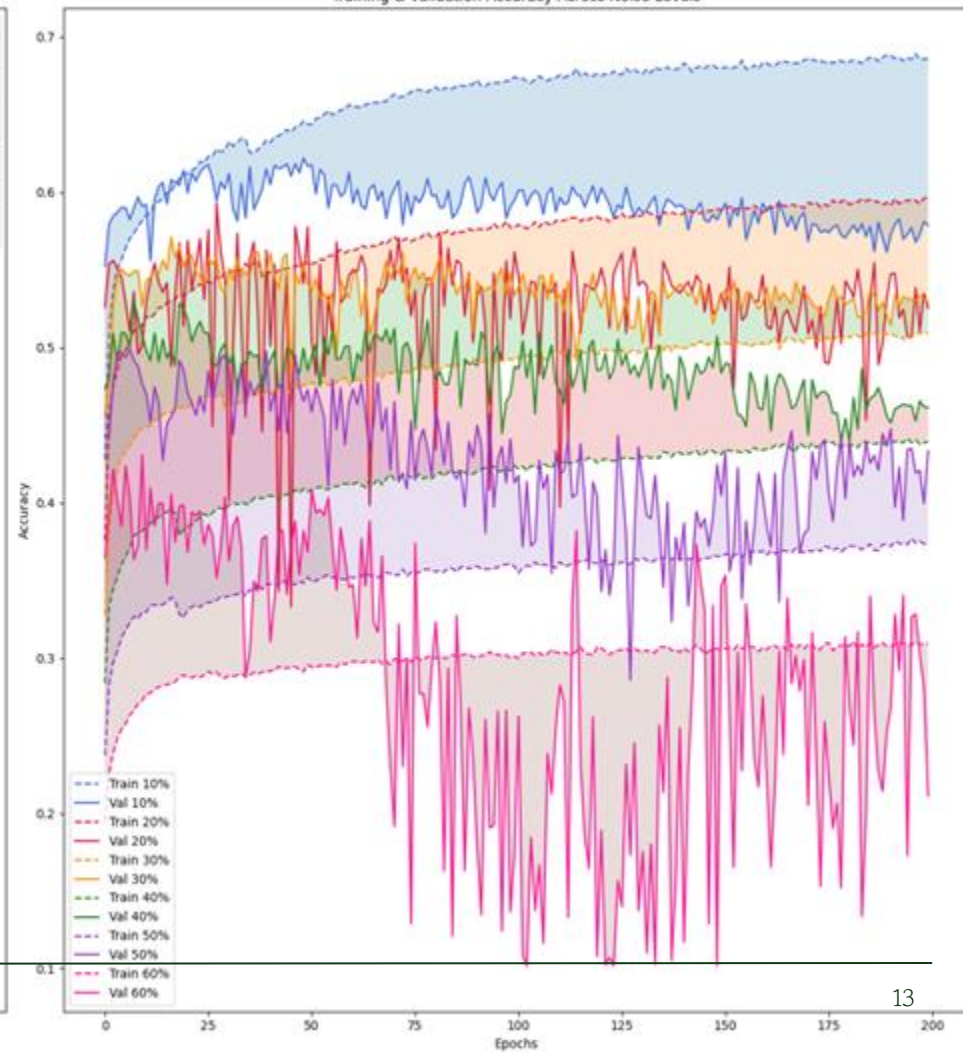
Comparing CNN Performance with CIFAR-10 Variants

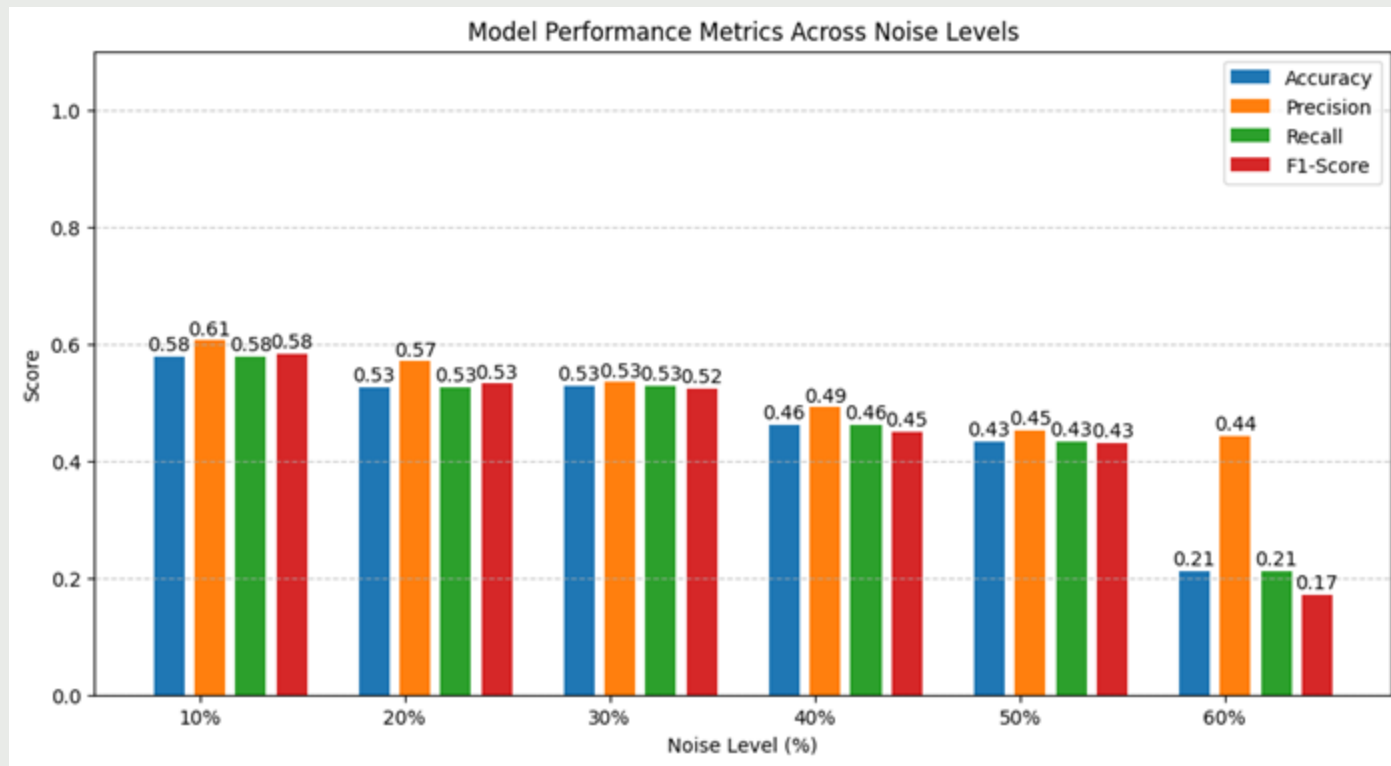


Training & Validation Loss Across Noise Levels



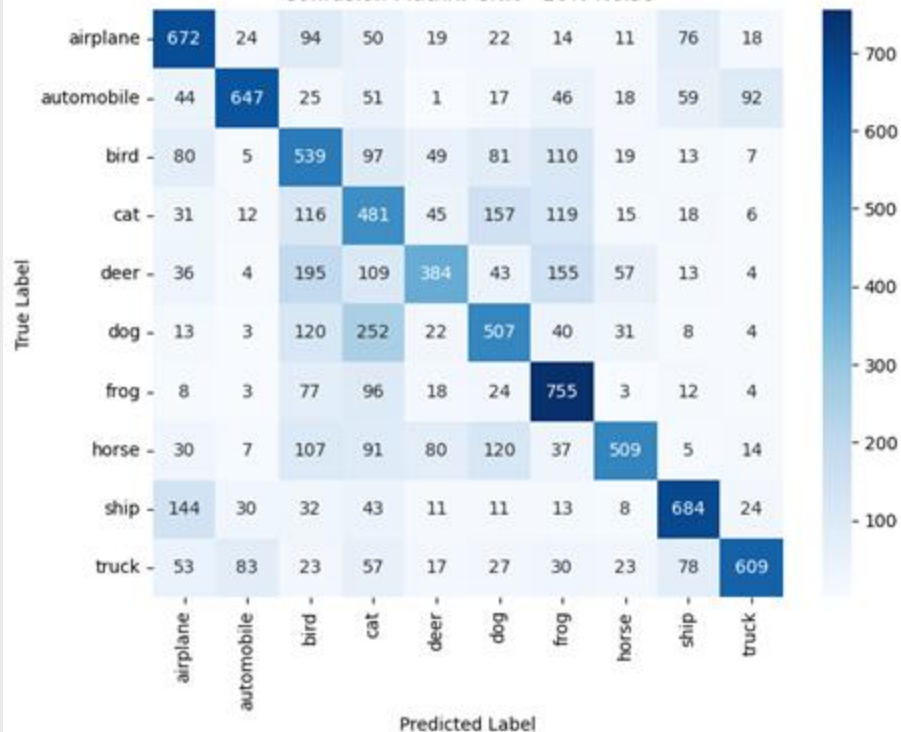
Training & Validation Accuracy Across Noise Levels



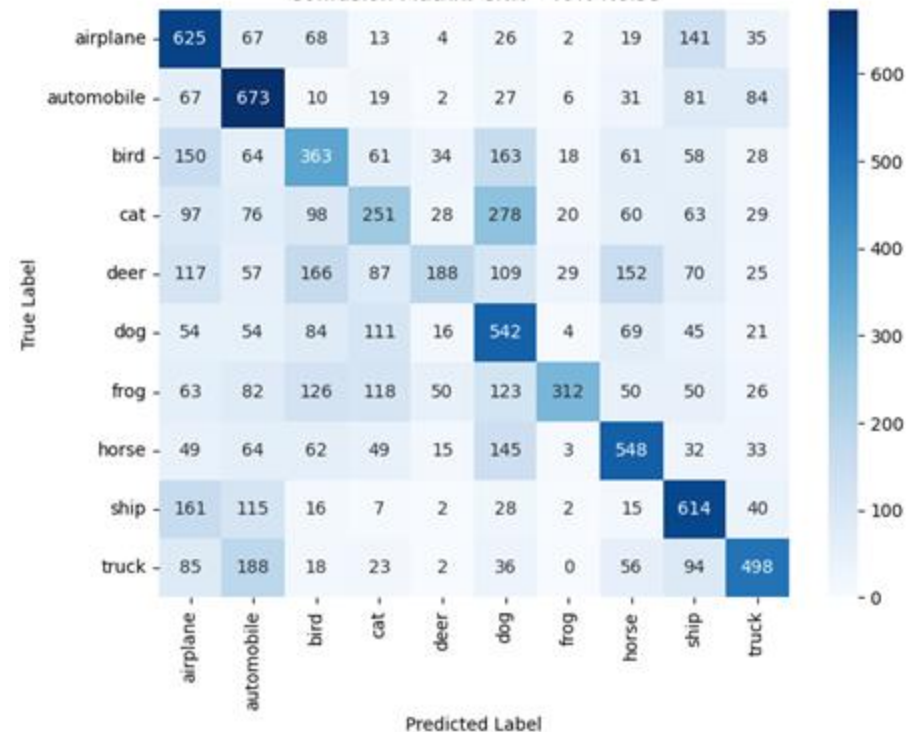


Classification Metrics: CNN + CIFAR-10

Confusion Matrix: CNN - 10% Noise



Confusion Matrix: CNN - 40% Noise

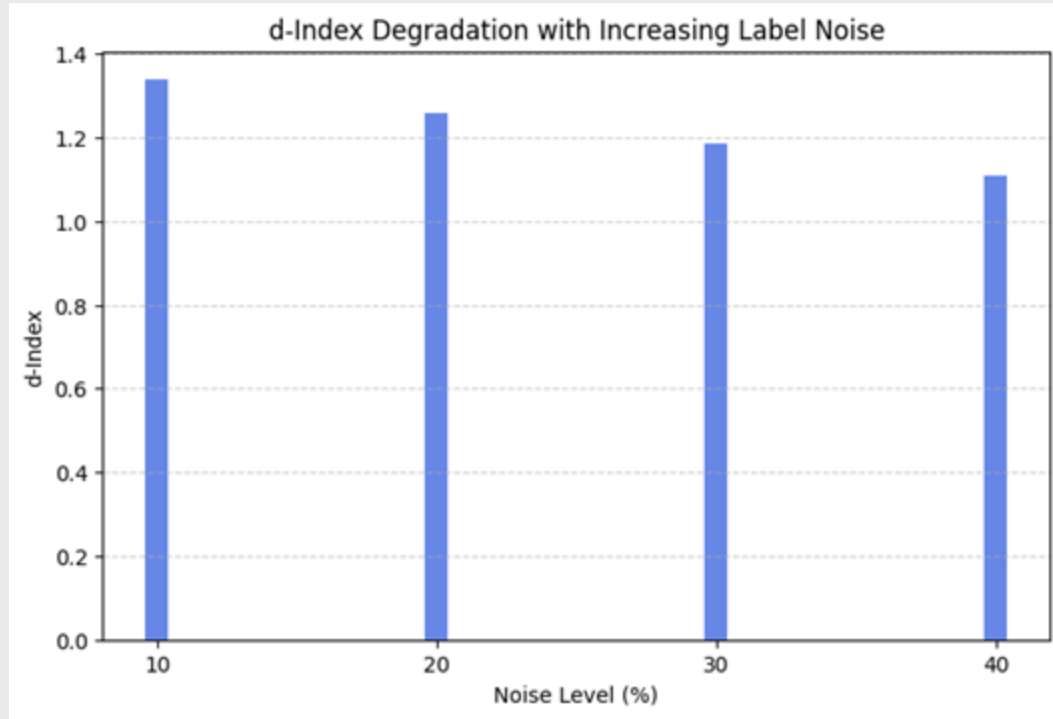


CNN - 10% Noise - Most Misclassified Classes:

- dog → cat (252 times)
- deer → bird (195 times)
- cat → dog (157 times)

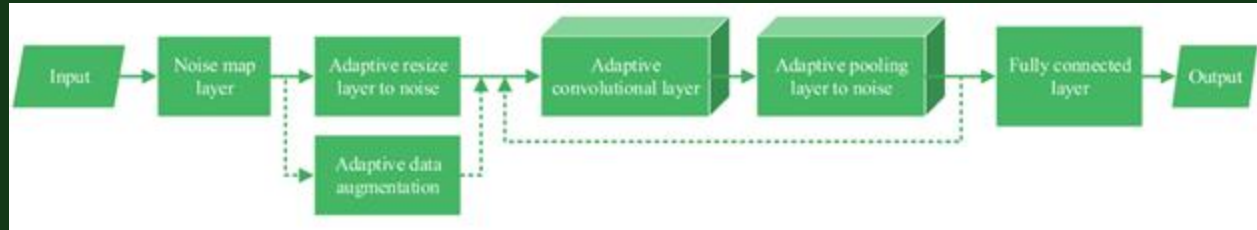
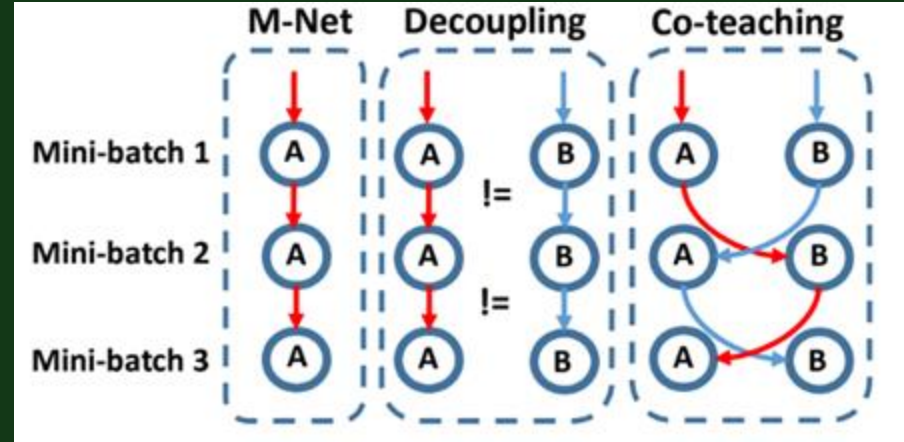
CNN - 40% Noise - Most Misclassified Classes:

- cat → dog (278 times)
- truck → automobile (188 times)
- deer → bird (166 times)

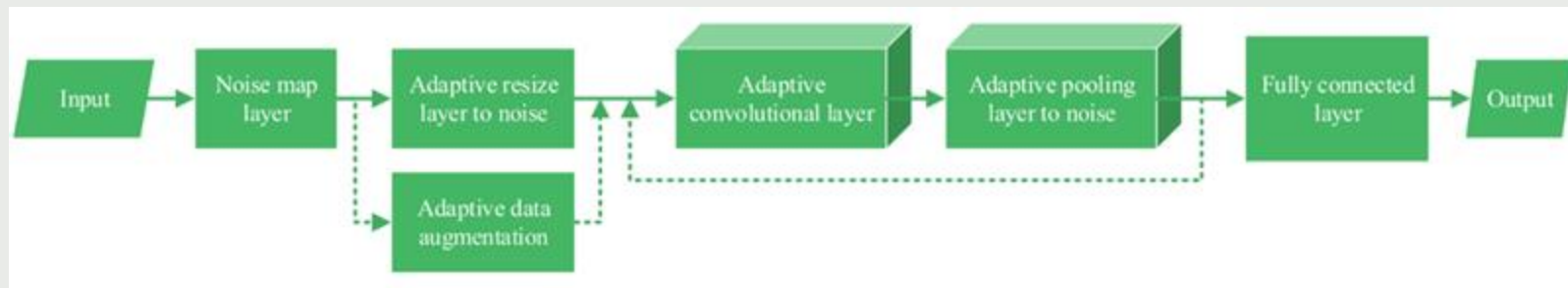


Diagnostic Index

Co-Teaching Noise-Robust CNNs



Noise-Robust CNNs:



Noise Map Layer

- Noise Transition Matrix: Estimates the probability that a noisy label corresponds to a clean label
- CNN adjusts predictions by applying NTM

NML Positioning

- Positioned at the end of the functions, not the beginning
- NML is applied to final predictions, not weights, avoids weight instability
- Likely to correct misclassifications

CNN Architecture

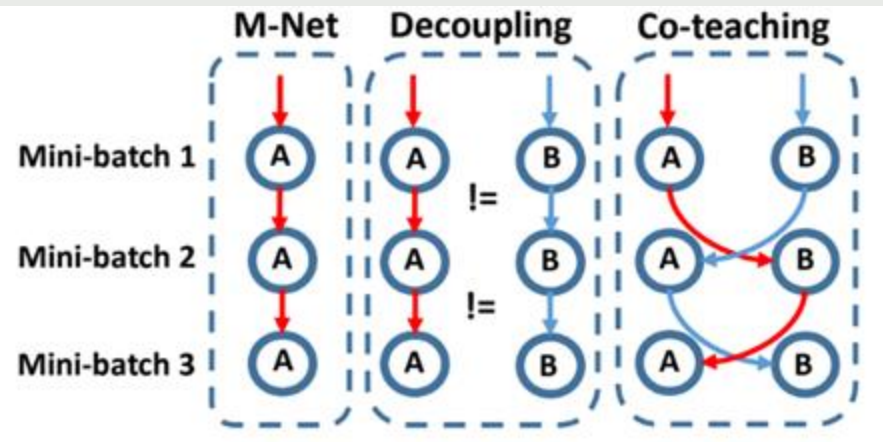
- Five Layers
- Dropout(0.25)
- L2 Regularization
- Batch Normalization
- Adam Optimizer
- Loss Function: APL framework*

Co-teaching

Algorithm 1 Co-teaching Algorithm.

```

1: Input  $w_f$  and  $w_g$ , learning rate  $\eta$ , fixed  $\tau$ , epoch  $T_k$  and  $T_{\max}$ , iteration  $N_{\max}$ ;
for  $T = 1, 2, \dots, T_{\max}$  do
2: Shuffle training set  $\mathcal{D}$ ; //noisy dataset
   for  $N = 1, \dots, N_{\max}$  do
3: Fetch mini-batch  $\mathcal{D}$  from  $\mathcal{D}$ ;
4: Obtain  $\hat{\mathcal{D}}_f = \arg \min_{\mathcal{D}' \subseteq \mathcal{D} | |\mathcal{D}'| \geq R(T)|\mathcal{D}|} \ell(f, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances
5: Obtain  $\hat{\mathcal{D}}_g = \arg \min_{\mathcal{D}' \subseteq \mathcal{D} | |\mathcal{D}'| \geq R(T)|\mathcal{D}|} \ell(g, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances
6: Update  $w_f = w_f - \eta \nabla \ell(f, \hat{\mathcal{D}}_g)$ ; //update  $w_f$  by  $\hat{\mathcal{D}}_g$ ;
7: Update  $w_g = w_g - \eta \nabla \ell(g, \hat{\mathcal{D}}_f)$ ; //update  $w_g$  by  $\hat{\mathcal{D}}_f$ ;
   end
8: Update  $R(T) = 1 - \min \left\{ \frac{T}{T_k}, \tau \right\}$ ;
end
9: Output  $w_f$  and  $w_g$ .
  
```



Two Networks

- For each mini-batch, each network samples instances of small loss
- This teaches useful instances to the other network for future training

Retain - Ratio

- Keeping the top % of confident samples
- RR = 0.8: Keeping 80% of the samples where a network is most confident

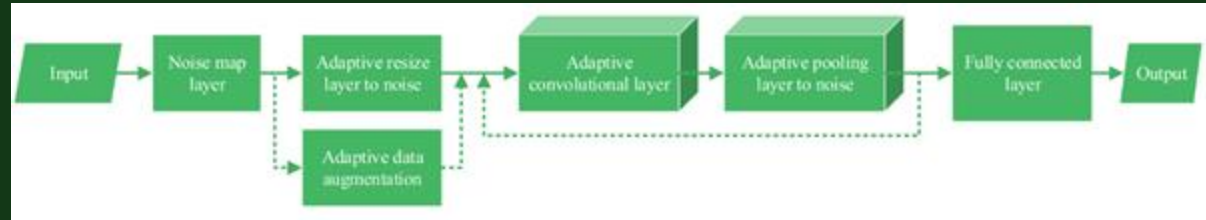
Friendly - Hot Potato

- Network A shares samples having the lowest loss to Network B to train on
- Network B does the same thing
- A decreasing Retain Ratio keeps the networks from using the exact same samples

Noise Robust CNN with Active Passive Loss Framework (NFL + RCE)

$$FL = -(1 - P_t)^\gamma \log(P_t)$$

$$CE = -\log(P_t)$$



Loss Functions: Active Passive Loss Framework

$$\mathcal{L}_{\text{norm}} = \frac{\mathcal{L}(f(\mathbf{x}), y)}{\sum_{j=1}^K \mathcal{L}(f(\mathbf{x}), j)}.$$

$$\mathcal{L}_{\text{APL}} = \alpha \cdot \mathcal{L}_{\text{Active}} + \beta \cdot \mathcal{L}_{\text{Passive}},$$



$$\text{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t).$$



$$\mathcal{L}_{CE}^R(x, y) = -R_y^\top \log F(x).$$

APL Functions

- Active Loss: explicitly maximizes the probability of a sample being in a label class
- Passive Loss: explicitly minimizes probability of a sample being in a label class
- Normalized Functions: Scaled versions that keep their robustness property

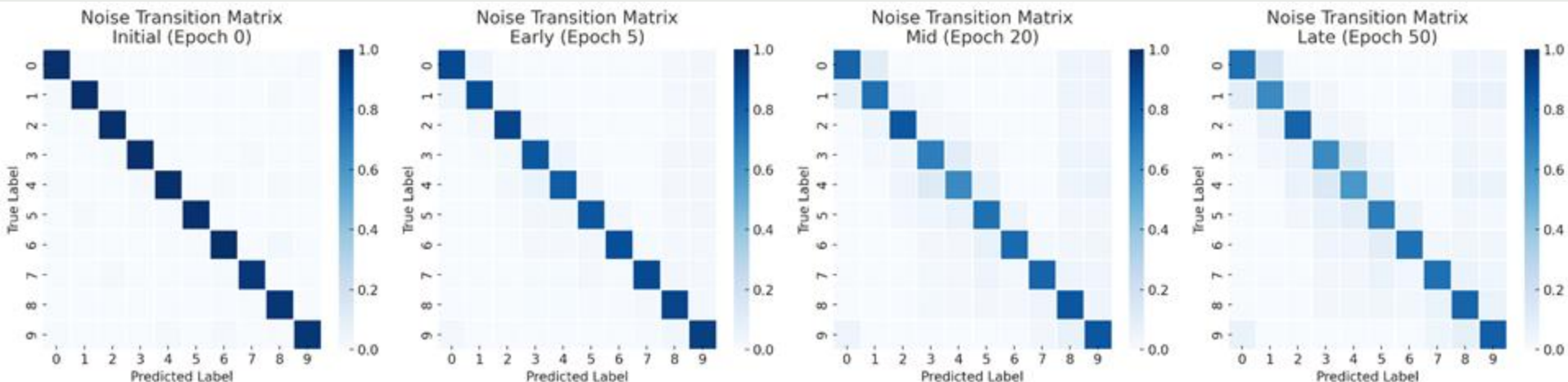
Normalized Focal Loss

- Encourages uncertainty for low confidence predictions
- Reduces weight of high confidence predictions to focus on low confidence predictions
- Less time spend reinforcing what is known

Normalized Reverse Cross Entropy

- CE rewards models for correct predictions
 - RCE punishes model on overconfident predictions
- Prevents CNN from blindly memorizing potentially noisy labels
- Prevents overconfidence across all predictions

Noise-Robust CNN:



Noise Map Layer

- Noise Transition Matrix: Estimates the probability that a noisy label corresponds to a clean label
- CNN adjusts predictions by applying NTM

NML Positioning

- Positioned at the end of the functions, not the beginning
- NML is applied to final predictions, not weights, avoids weight instability
- Likely to correct misclassifications

CNN Architecture

- Eight Layers
- Dropout(0.3)
- L2 Regularization
- Batch Normalization
- Adam Optimizer
- Loss $F(x)$: Focal Loss + Reverse CE

Noise-Robust CNN: Noise Map Layer

```
class NoiseMapLayer(Layer):
    def __init__(self, num_classes, **kwargs):
        super(NoiseMapLayer, self).__init__(**kwargs)
        self.num_classes = num_classes
        self.T = None

    def build(self, input_shape):
        init_T = np.eye(self.num_classes) + np.random.uniform(-0.02, 0.02, (self.num_classes, self.num_classes))
        self.T = self.add_weight(
            shape=(self.num_classes, self.num_classes),
            initializer=tf.keras.initializers.Constant(init_T),
            trainable=True,
            constraint=MinMaxNorm(min_value=0.0, max_value=1.0),
            name="noise_transition_matrix",
            dtype=tf.float32
        )

    def call(self, y_pred):
        y_pred_noise = tf.linalg.matmul(y_pred, self.T)
        return y_pred_noise

    def compute_label_confidence(y_pred, T):
        epsilon = 1e-6
        num_classes = tf.shape(T)[0]

        stabilized_T = T + tf.eye(num_classes) * epsilon
        y_pred_clean = tf.linalg.matmul(y_pred, tf.linalg.pinv(stabilized_T))
        confidence_scores = tf.reduce_max(y_pred_clean, axis=-1)

        confidence_scores = tf.reduce_mean(confidence_scores, axis=-1)

        return confidence_scores

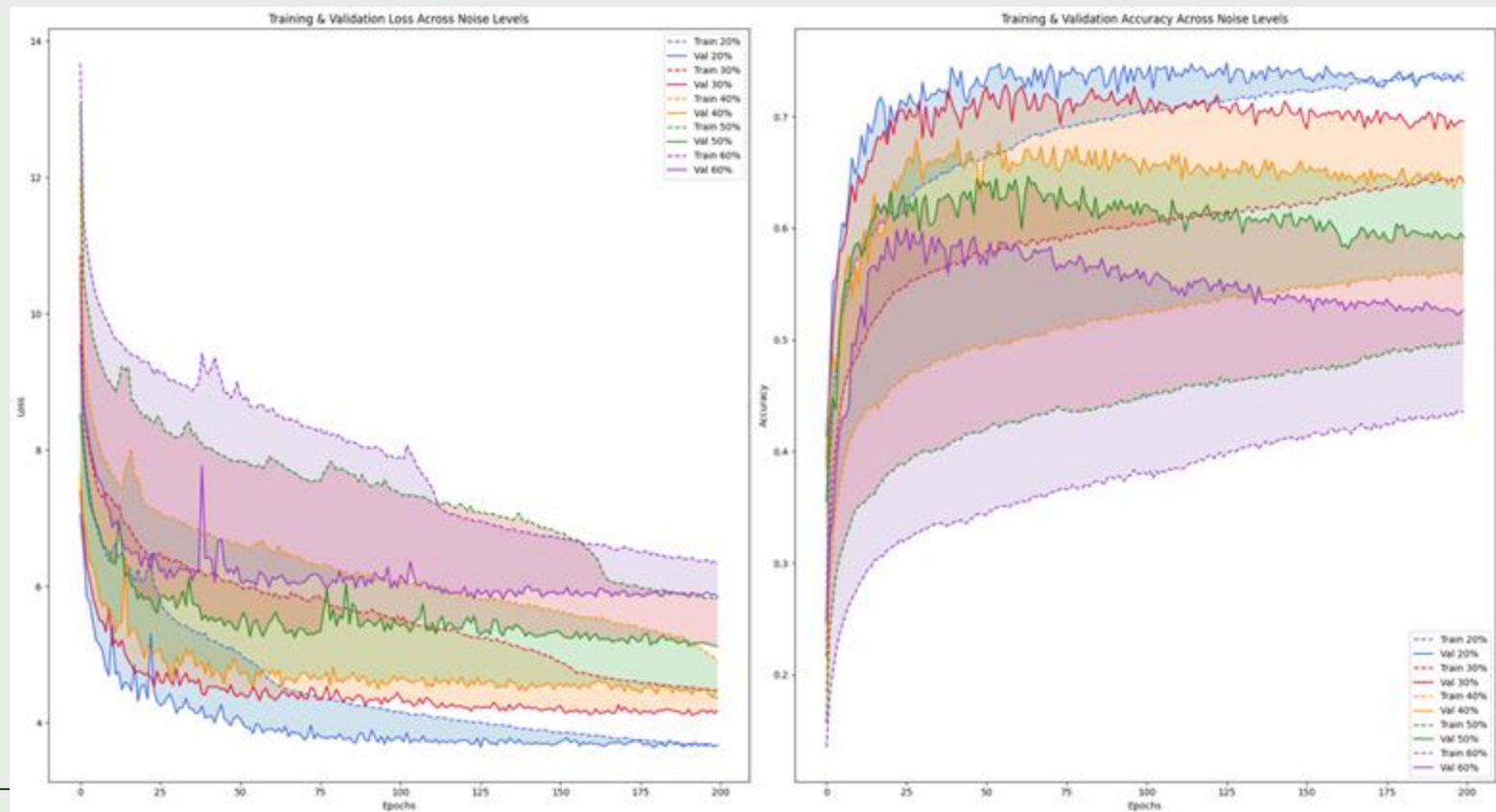
    def noise_aware_loss(y_true, y_pred, noise_layer, base_loss_func):
        y_pred_noise = noise_layer(y_pred)
        y_pred_noise = tf.clip_by_value(y_pred_noise, 1e-7, 1.0)
        confidence_scores = compute_label_confidence(y_pred_noise, noise_layer.T)

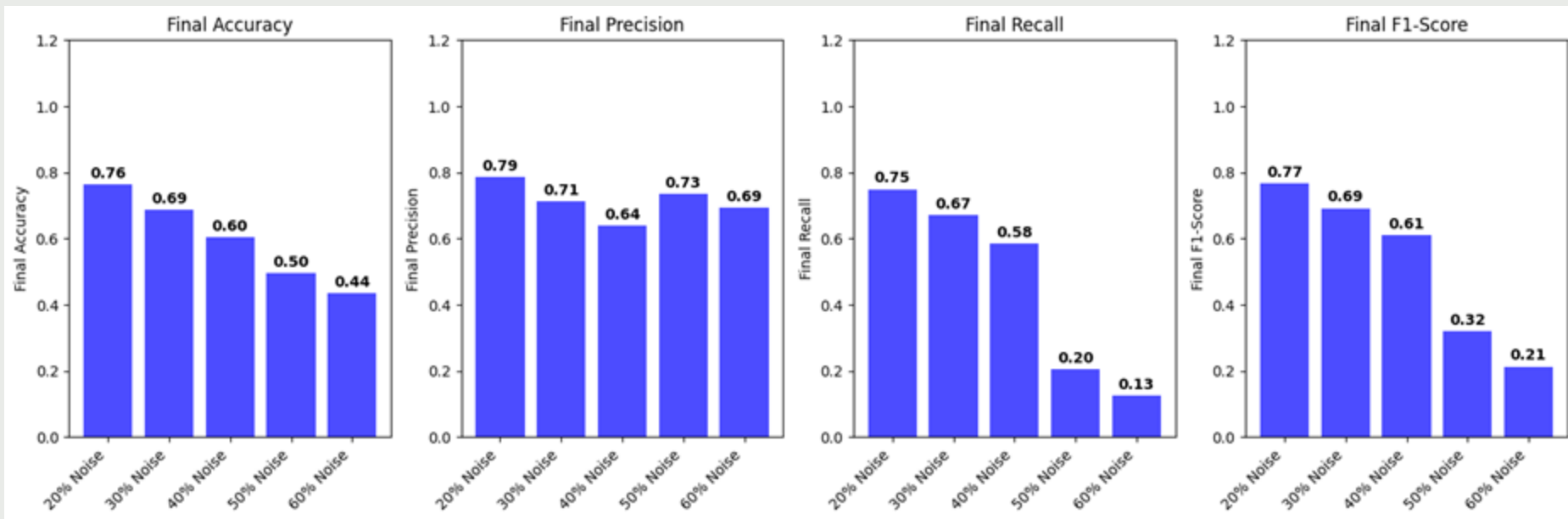
        base_loss = base_loss_func(y_true, y_pred_noise)
        confidence_scores = tf.reshape(confidence_scores, (-1, 1))

        weighted_loss = base_loss * confidence_scores
        return tf.reduce_mean(weighted_loss)
```

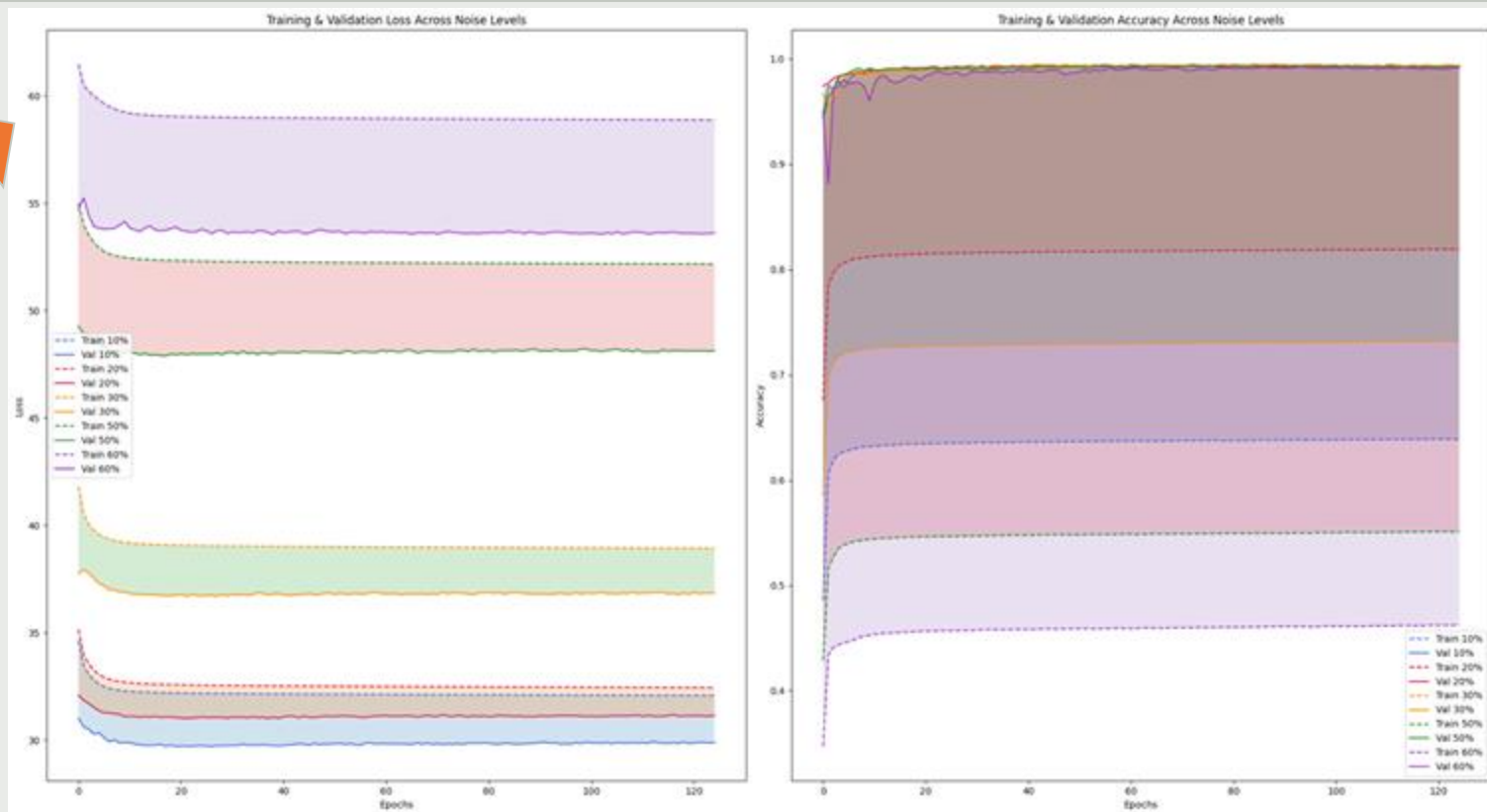
- NoiseMapLayer: uses “num_classes” as the size of the “Noise Transition Matrix” (NTM)
 - Initially an “identity matrix” with small random noise to allow adjustments
- `def call(self, y_pred)`: modifies the CNN’s predictions by multiplying them with the NTM
 - Allows CNN to model label corruption and compensate
- `def compute_label_confidence(y_pred, T)`: estimates how confident CNN is that predictions have clean labels
 - Inverts T to estimate what predictions look like noise-less
- `def noise_aware_loss(y_true, y_pred, noise_layer, loss_func)`: modifies final predictions with NML

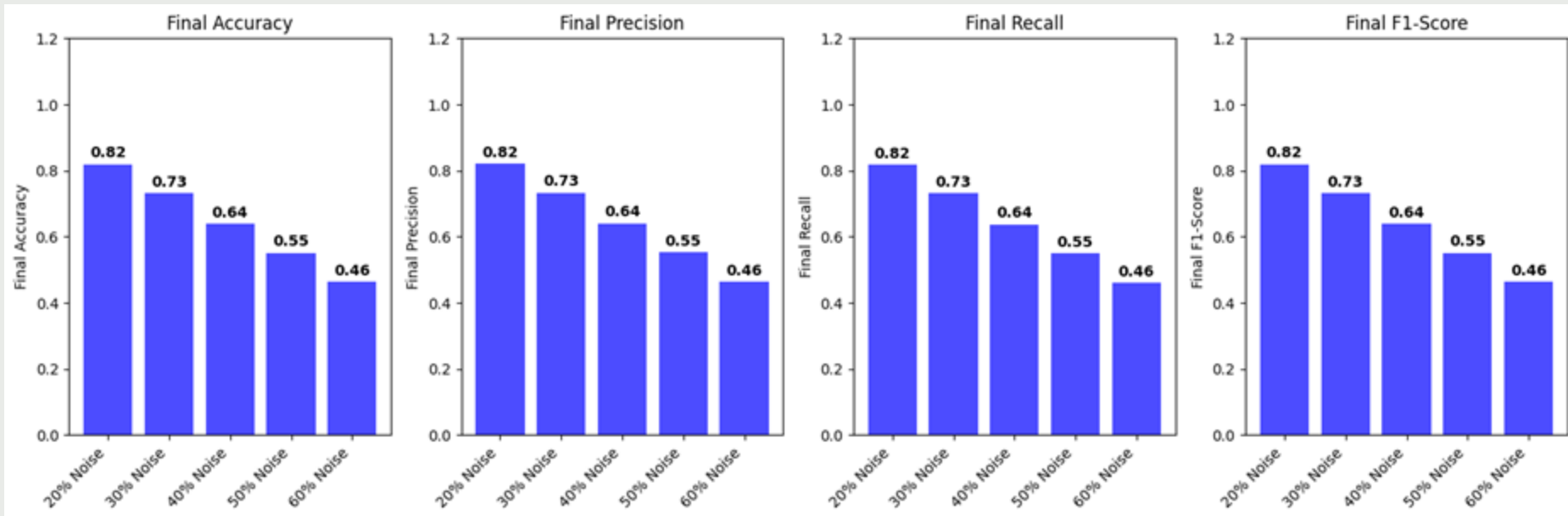
Noise - Robust CNN: CIFAR 10 (20% - 60% Label Corruption)





Noise - Robust CNN: MNIST (10% - 60% Label Corruption)

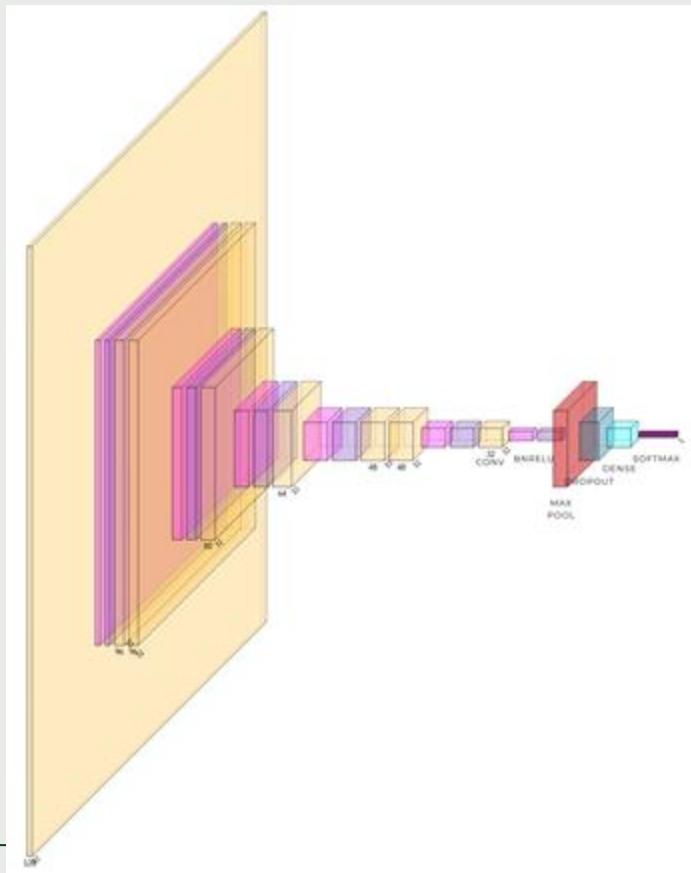




Comparing Co-Taught CNN Performance with CIFAR-10 Variants Using Cross Entropy



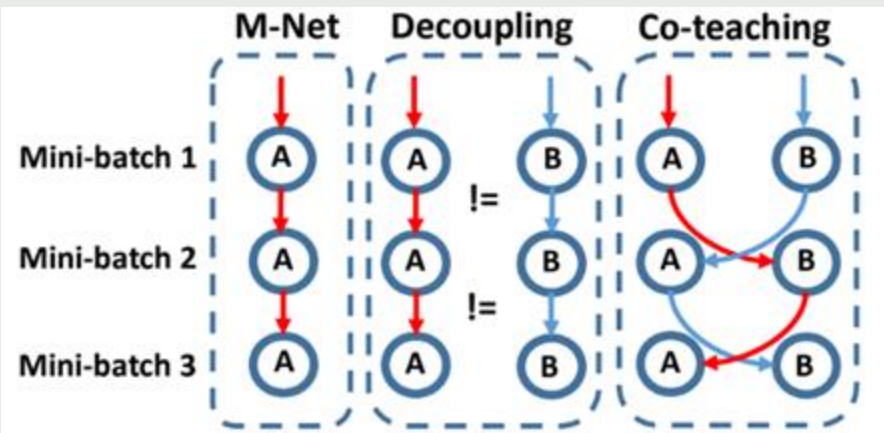
Co Teaching with Convolutional Neural Networks



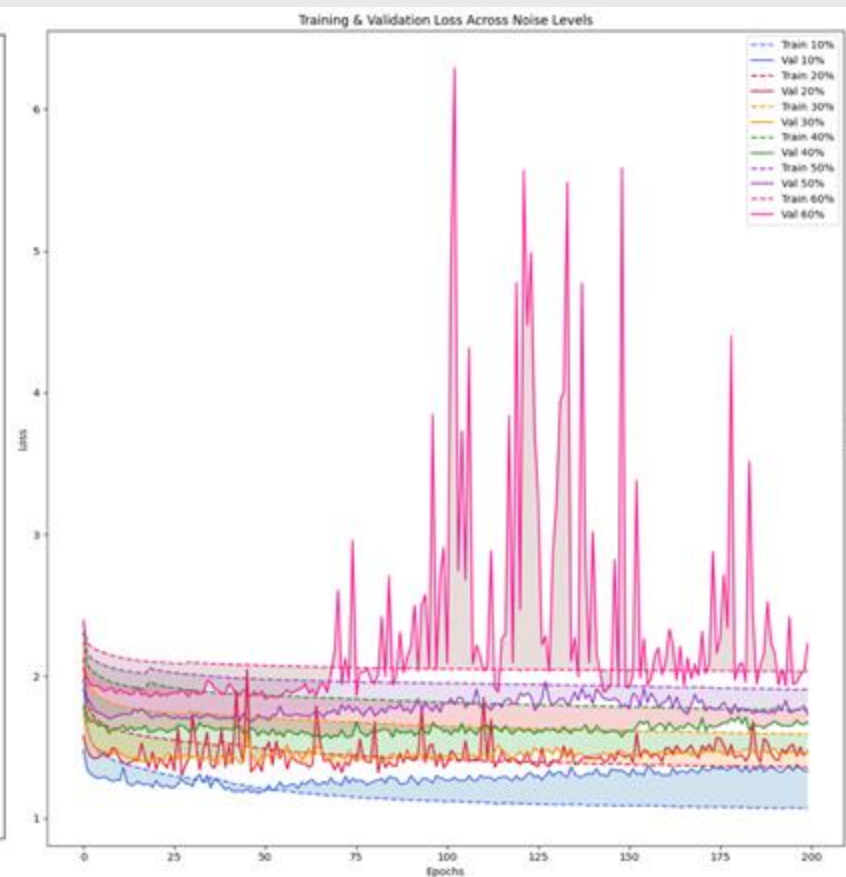
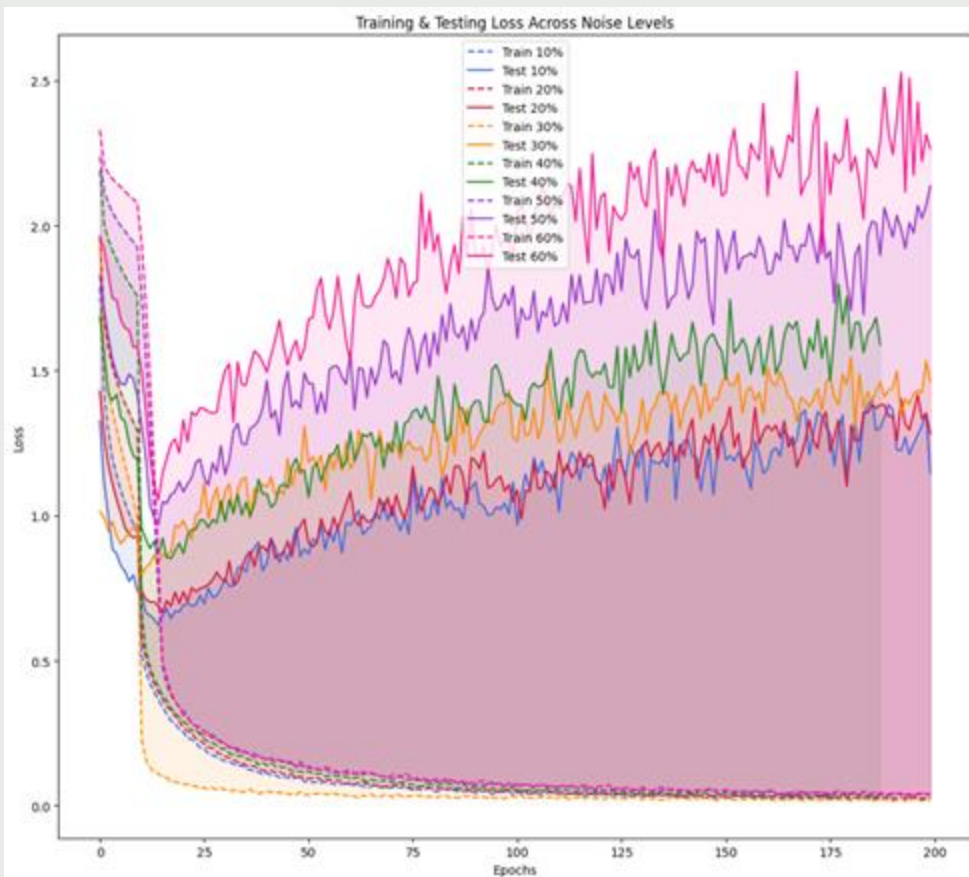
Algorithm 1 Co-teaching Algorithm.

```

1: Input  $w_f$  and  $w_g$ , learning rate  $\eta$ , fixed  $\tau$ , epoch  $T_k$  and  $T_{\max}$ , iteration  $N_{\max}$ ;
for  $T = 1, 2, \dots, T_{\max}$  do
2: Shuffle training set  $\mathcal{D}$ ; //noisy dataset
for  $N = 1, \dots, N_{\max}$  do
3: Fetch mini-batch  $\mathcal{D}$  from  $\mathcal{D}$ ;
4: Obtain  $\mathcal{D}_f = \arg \min_{\mathcal{D}' : |\mathcal{D}'| \geq R(T)|\mathcal{D}|} \ell(f, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances
5: Obtain  $\mathcal{D}_g = \arg \min_{\mathcal{D}' : |\mathcal{D}'| \geq R(T)|\mathcal{D}|} \ell(g, \mathcal{D}')$ ; //sample  $R(T)\%$  small-loss instances
6: Update  $w_f = w_f - \eta \nabla \ell(f, \mathcal{D}_g)$ ; //update  $w_f$  by  $\mathcal{D}_g$ ;
7: Update  $w_g = w_g - \eta \nabla \ell(g, \mathcal{D}_f)$ ; //update  $w_g$  by  $\mathcal{D}_f$ ;
end
8: Update  $R(T) = 1 - \min \left\{ \frac{T}{T_k}, \tau \right\}$ ;
end
9: Output  $w_f$  and  $w_g$ .
    
```

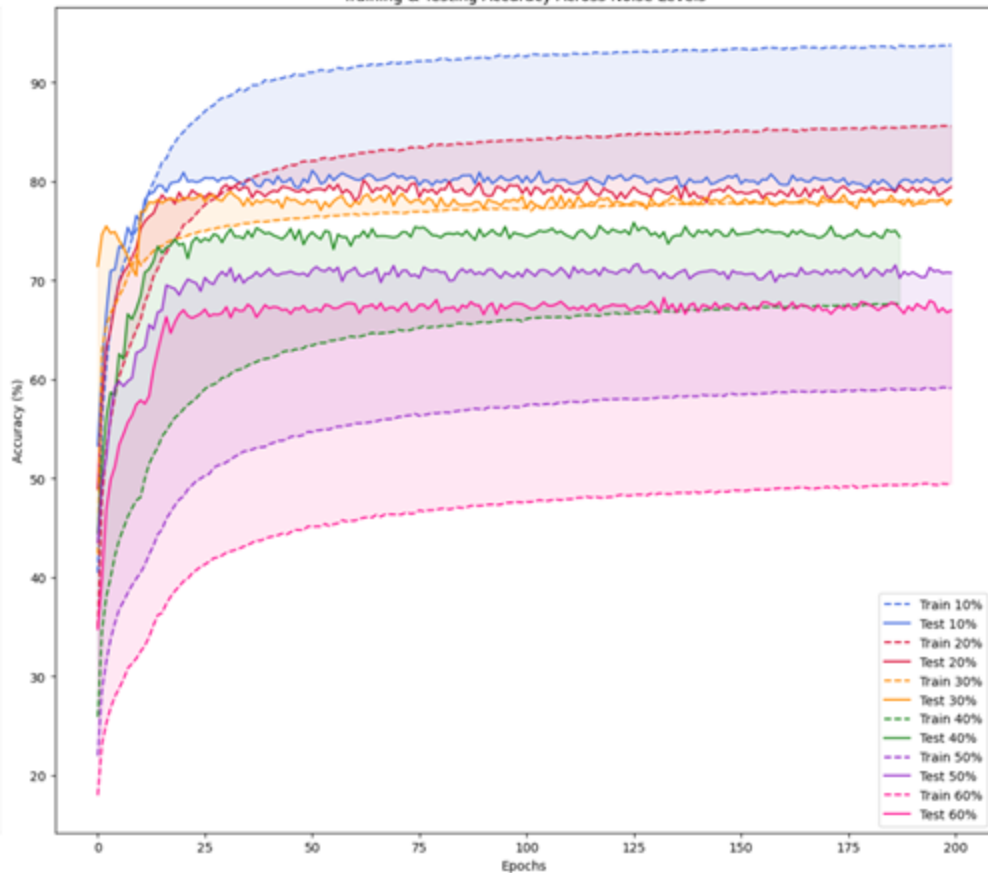


Loss Plots: Co-Teaching (L), CNN (R)

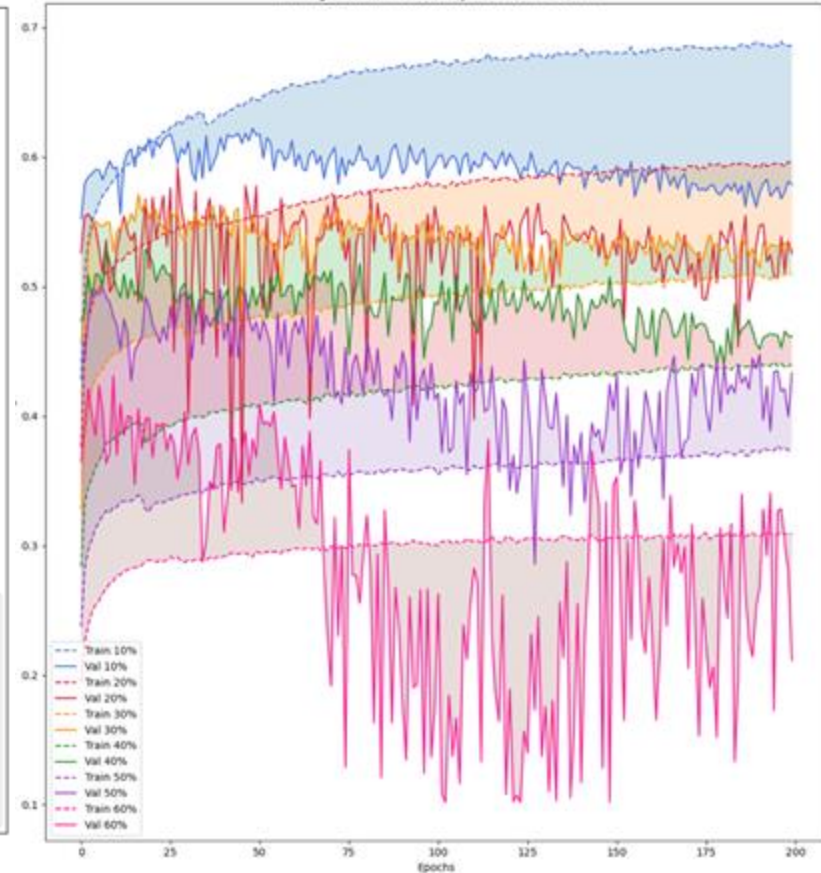


Accuracy Plots: Co-Teaching (L), CNN (R)

Training & Testing Accuracy Across Noise Levels

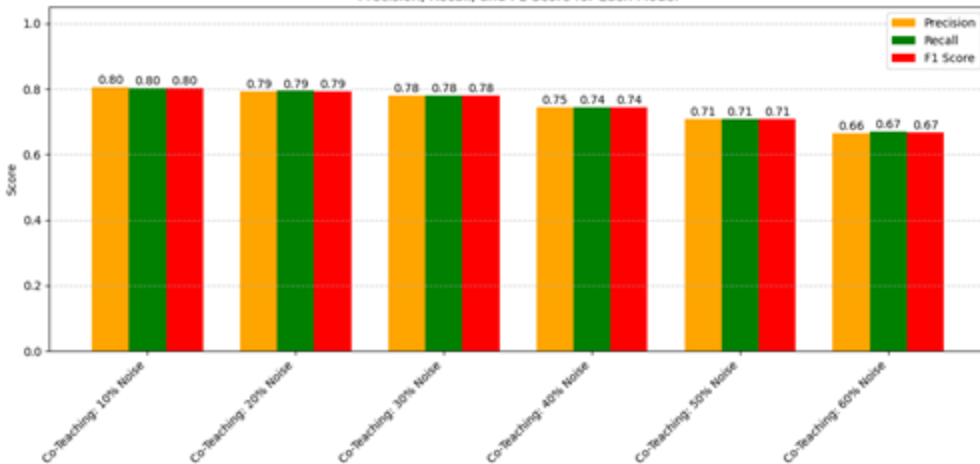


Training & Validation Accuracy Across Noise Levels

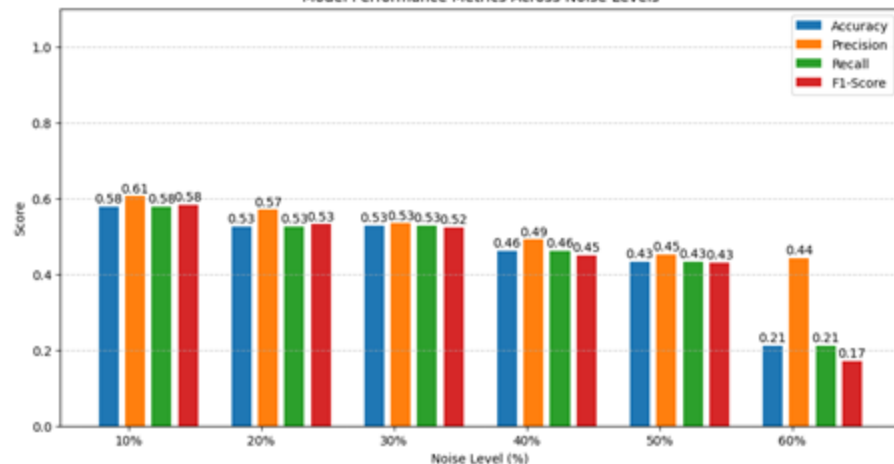


Classification Metrics: Co-Teaching (L), CNN (R)

Precision, Recall, and F1 Score for Each Model



Model Performance Metrics Across Noise Levels



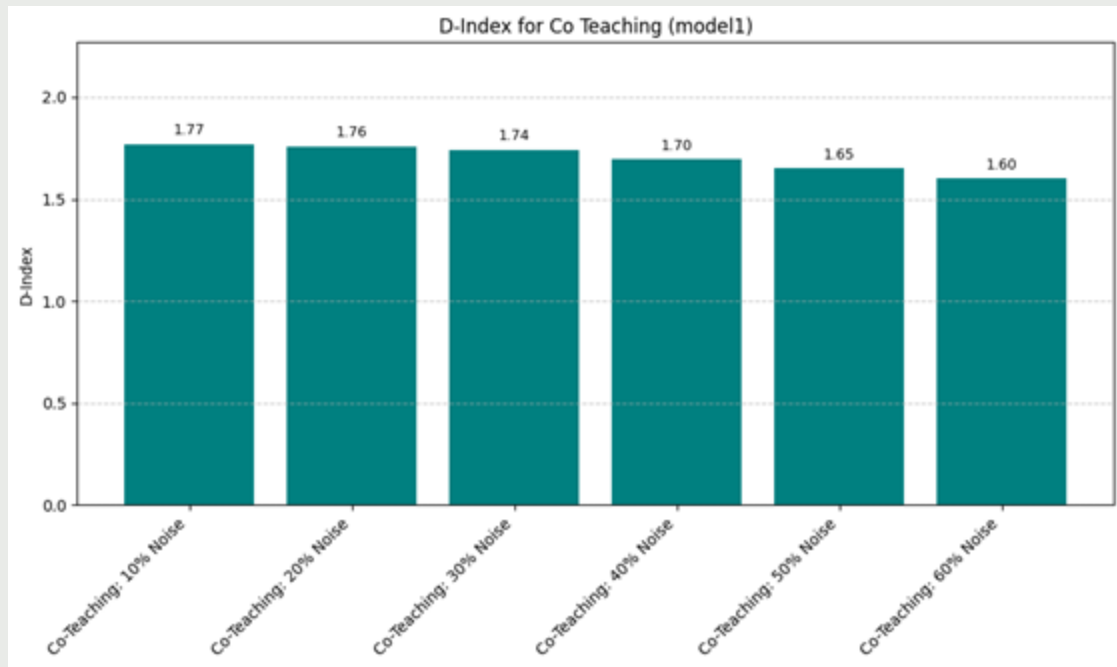
- Precision - accuracy of positive predictions
- Recall - ability to find correct instances
- F1 Score - harmonic mean of precision and recall

		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

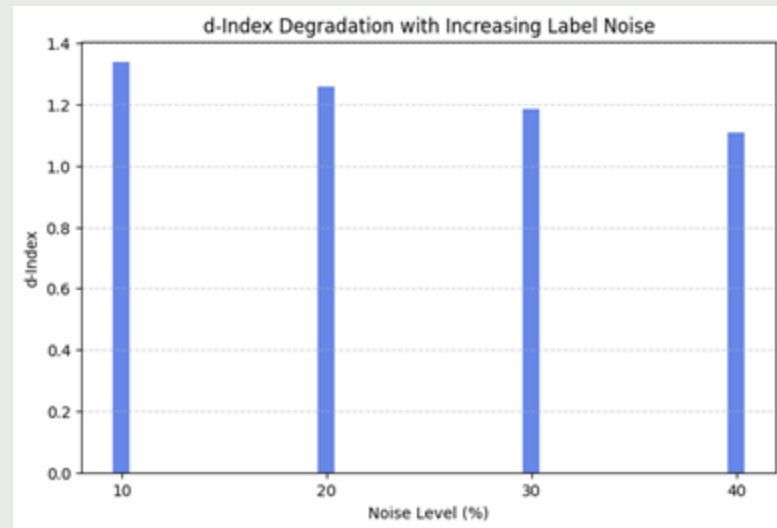
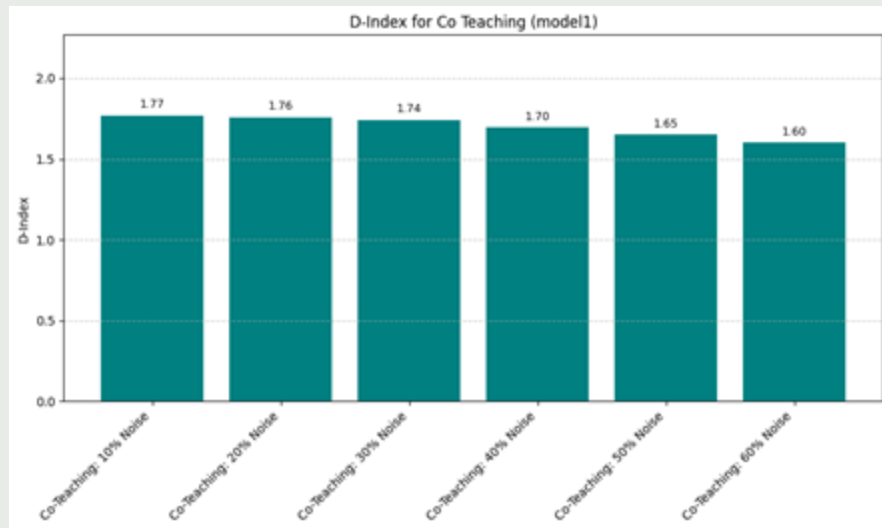
Diagnostic Index

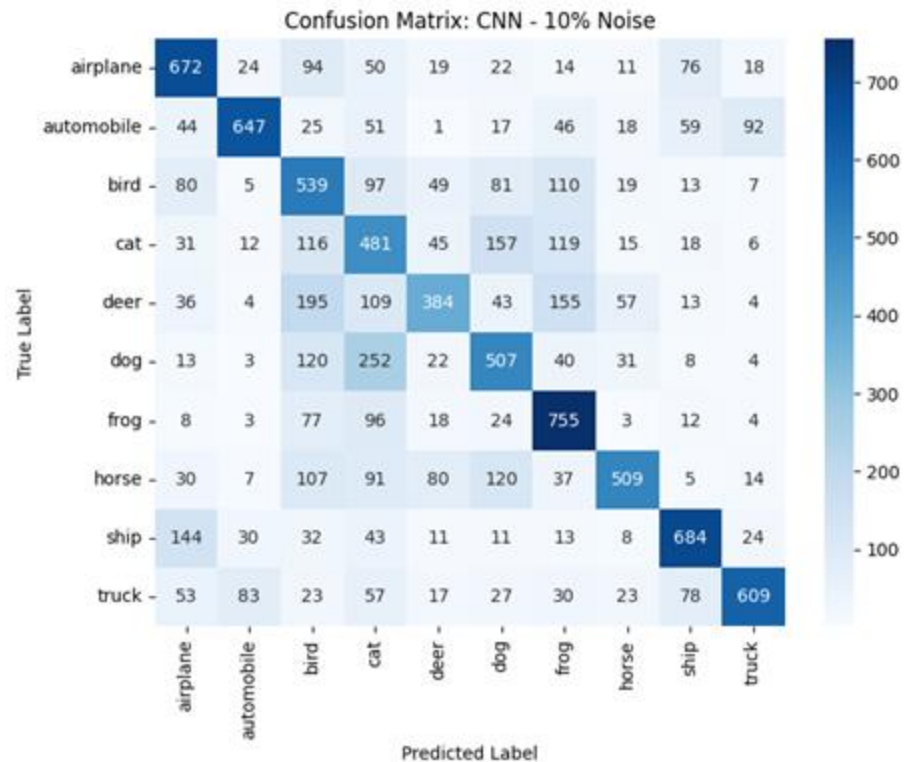
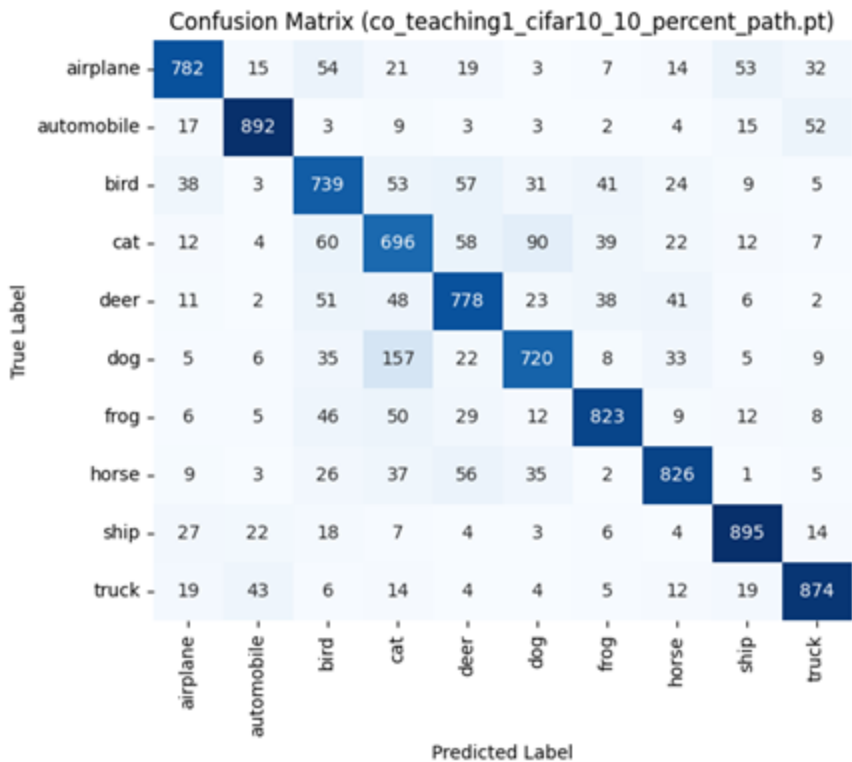


- Increasing label noise lowers the model's ability to learn meaningful patterns declines
- Models struggle to extract consistent patterns when labels are unreliable
- A high d-index indicates the model is learning meaningful features; (0, 2]

$$d = \log_2(1 + a) + \log_2(1 + \frac{s+p}{2})$$

Diagnostic Index: Co Teaching (Left), CNN (Right)





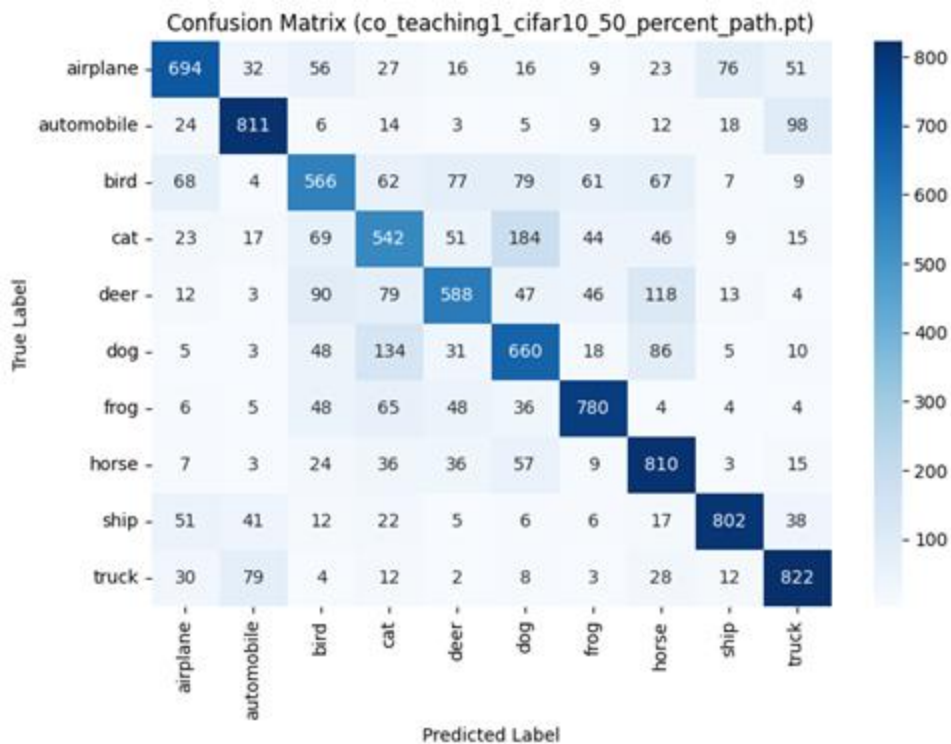
co_teaching1_cifar10_10_percent_path.pt

- dog → cat (157 times)
- cat → dog (90 times)
- cat → bird (60 times)



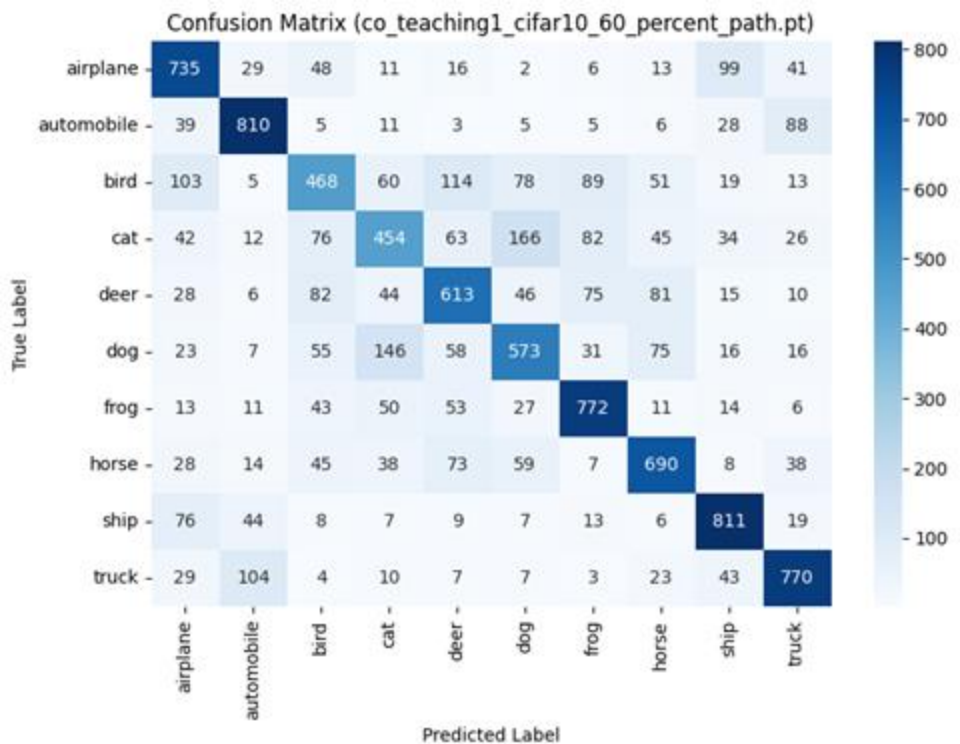
CNN - 10% Noise - Most Misclassified Classes:

- dog → cat (252 times)
- deer → bird (195 times)
- cat → dog (157 times)



co_teaching1_cifar10_50_percent_path.pt

- cat → dog (184 times)
- dog → cat (134 times)
- deer → horse (118 times)

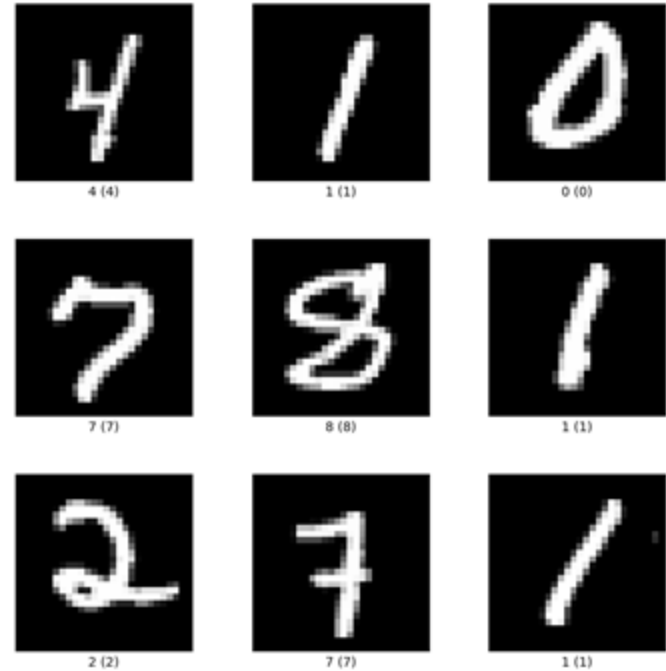


co_teaching1_cifar10_60_percent_path.pt

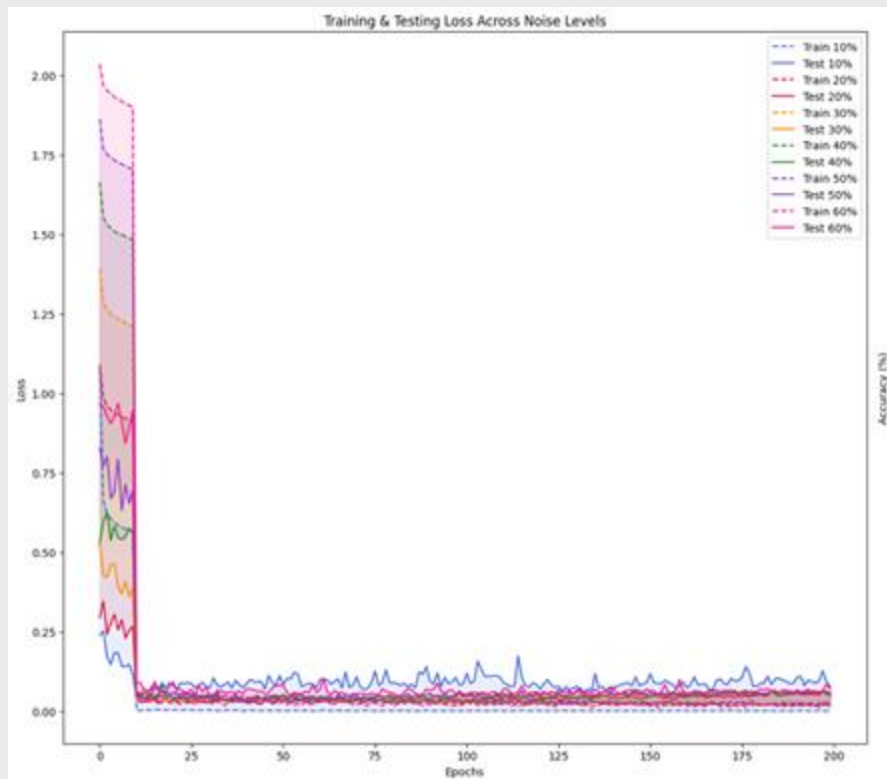
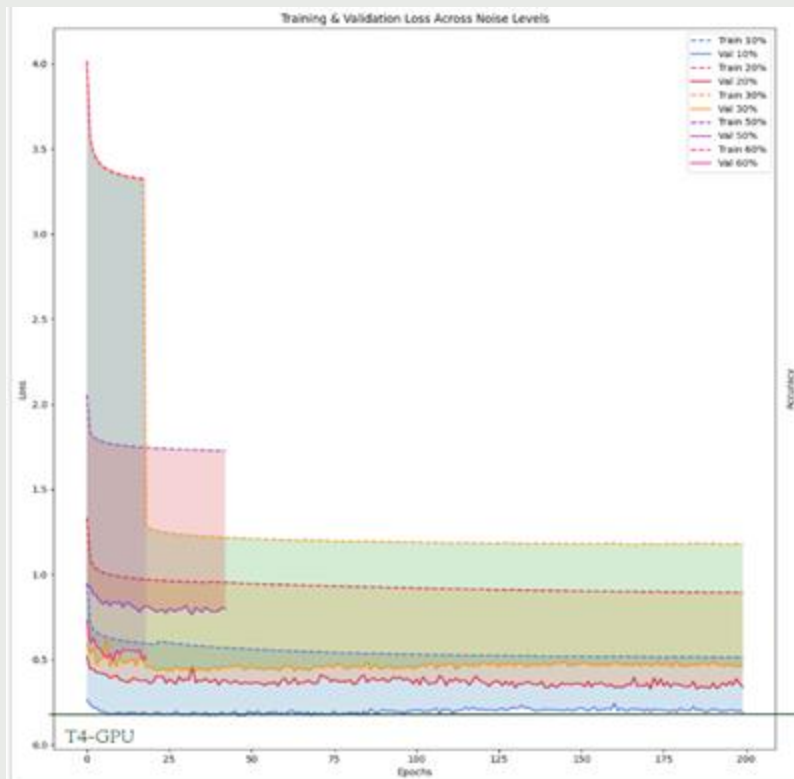
- cat → dog (166 times)
- dog → cat (146 times)
- bird → deer (114 times)



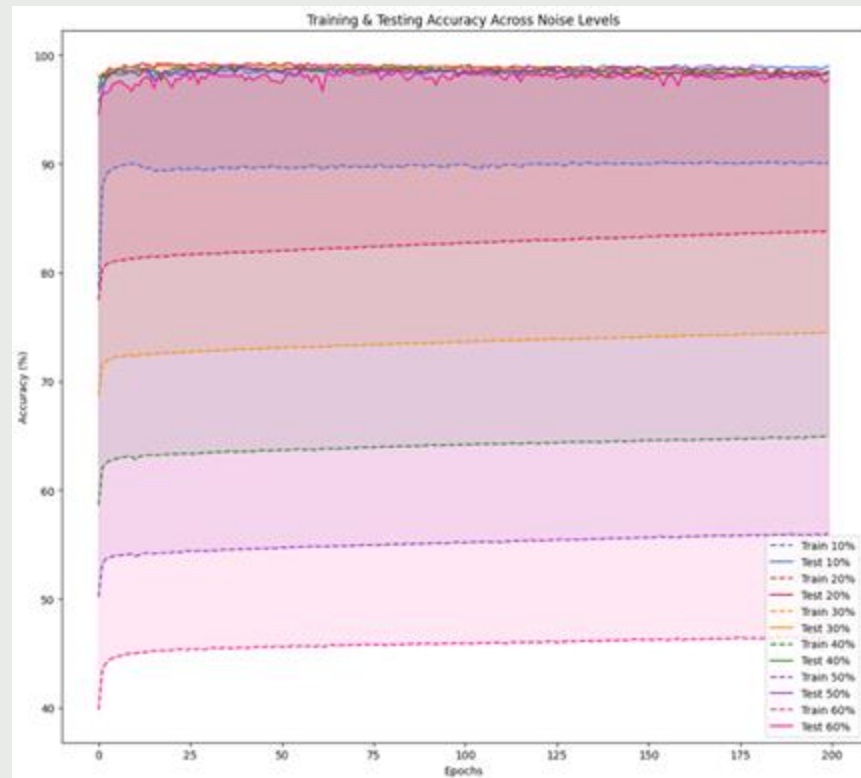
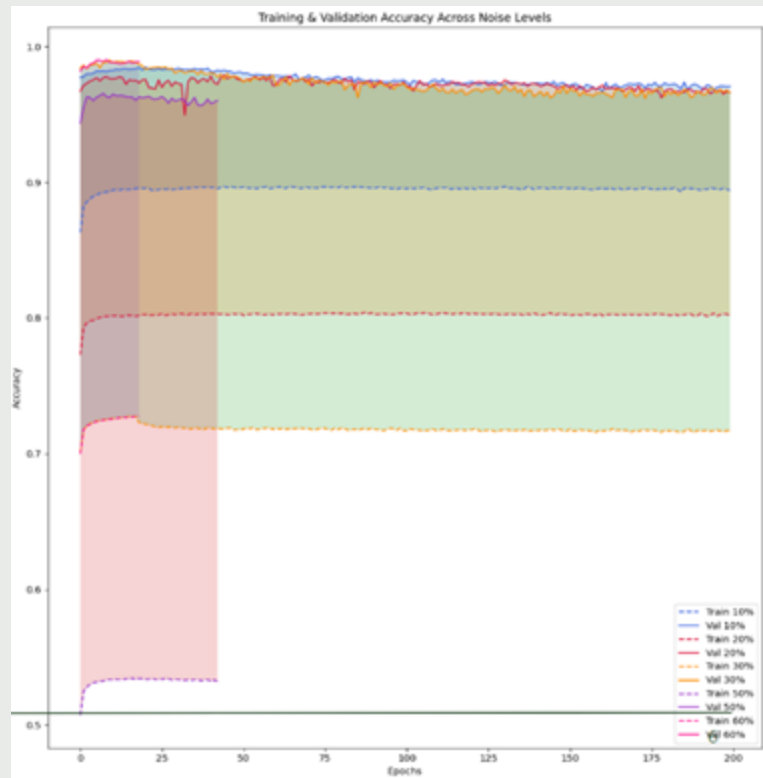
Comparing Co-Taught CNN Performance with MNIST Variants using Cross Entropy



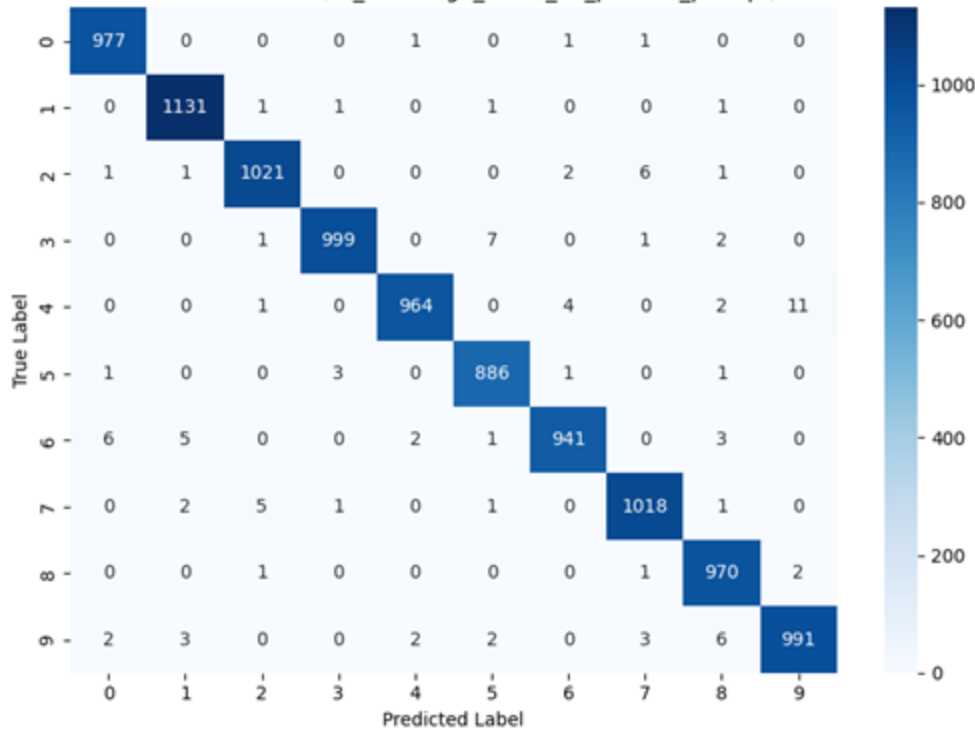
Loss Plots: Co-Teaching (R), CNN (L)



Accuracy Plots: Co-Teaching (R), CNN (L)



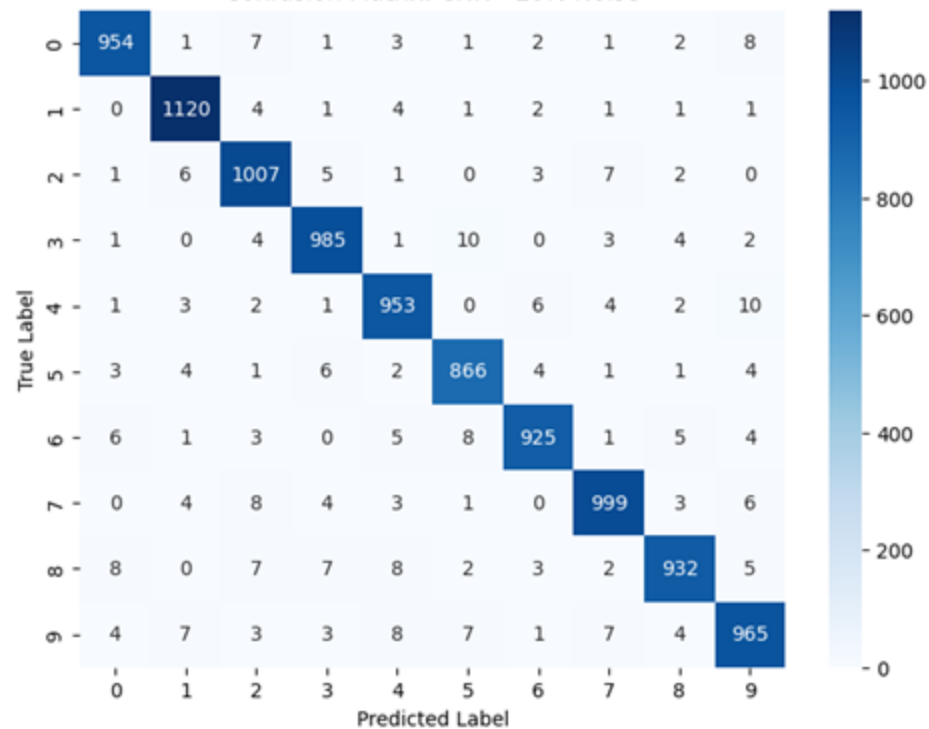
Confusion Matrix (co_teaching1_mnist_10_percent_path.pt)



co_teaching1_mnist_10_percent_path.pt

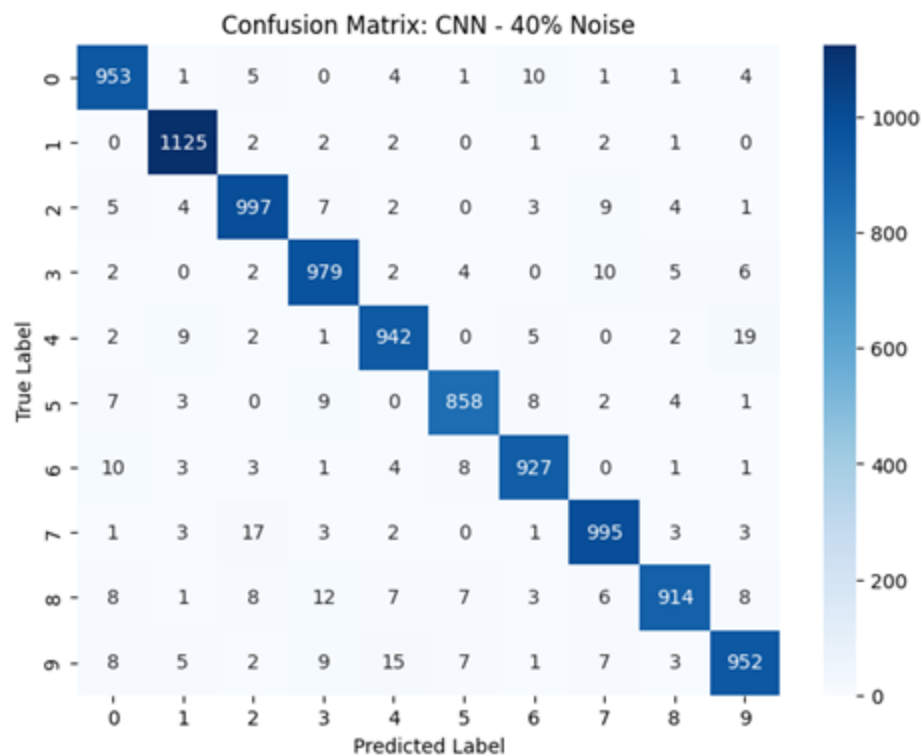
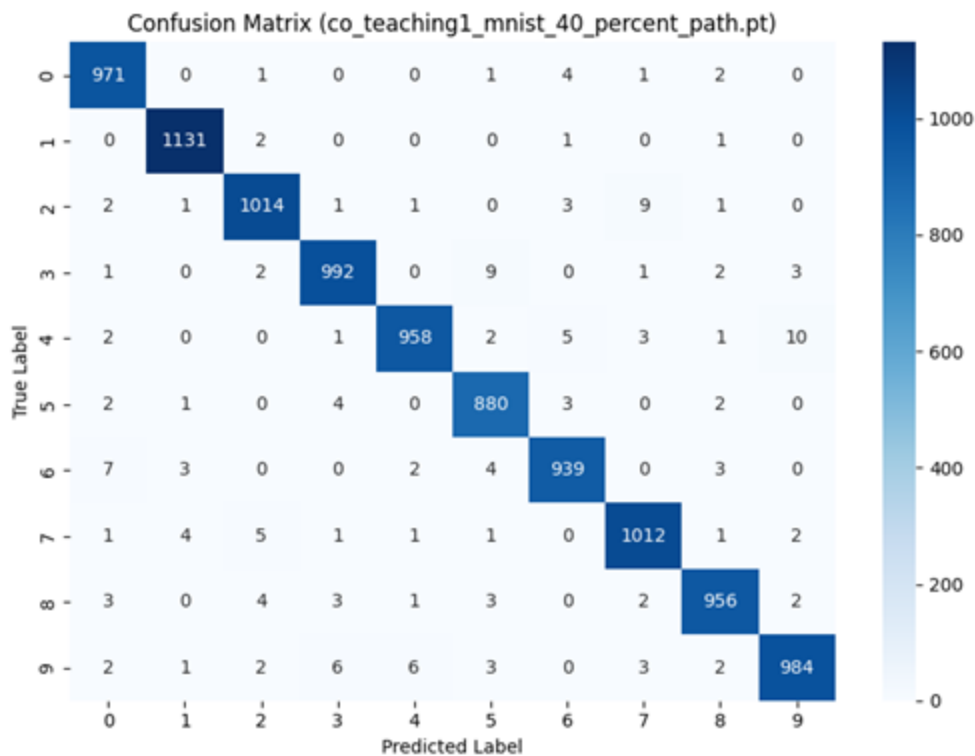
- 4 → 9 (11 times)
- 3 → 5 (7 times)
- 2 → 7 (6 times)

Confusion Matrix: CNN - 10% Noise



CNN - 10% Noise - Most Misclassified Classes:

- 3 → 5 (10 times)
- 4 → 9 (10 times)
- 0 → 9 (8 times)



co_teaching1_mnist_40_percent_path.pt

- 4 → 9 (10 times)
- 2 → 7 (9 times)
- 3 → 5 (9 times)

CNN - 40% Noise - Most Misclassified Classes:

- 4 → 9 (19 times)
- 7 → 2 (17 times)
- 9 → 4 (15 times)

Comparing Co-Taught CNN Performance with CIFAR-10 Variants Using NFL + RCE

$$\text{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t).$$

$$\mathcal{L}_{CE}^R(x, y) = -R_y^\top \log F(x).$$



Loss Functions: Active Passive Loss Framework

$$\mathcal{L}_{\text{APL}} = \alpha \cdot \mathcal{L}_{\text{Active}} + \beta \cdot \mathcal{L}_{\text{Passive}},$$



$$\text{FL}(p_t) = -(1 - p_t)^\gamma \log(p_t).$$

Normalized Focal Loss

- Encourages uncertainty for low confidence predictions
- Reduces weight of high confidence predictions to focus on low confidence predictions
- Less time spent reinforcing what is known

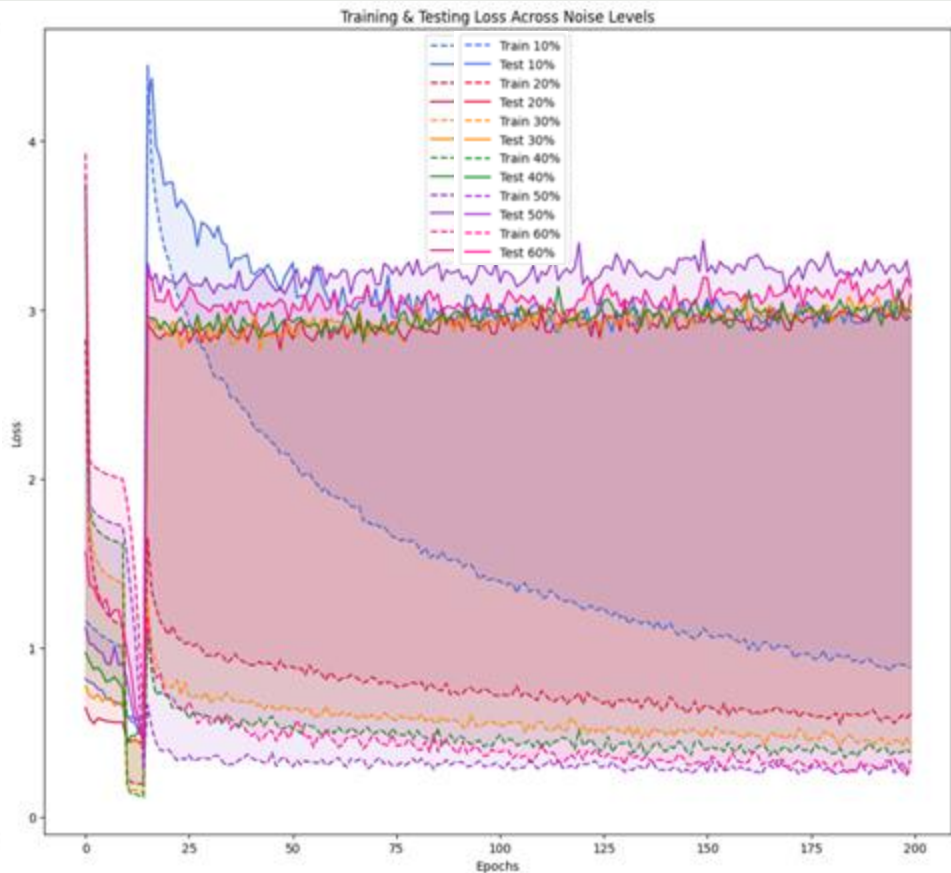


$$\mathcal{L}_{\text{CE}}^R(x, y) = -R_y^\top \log F(x).$$

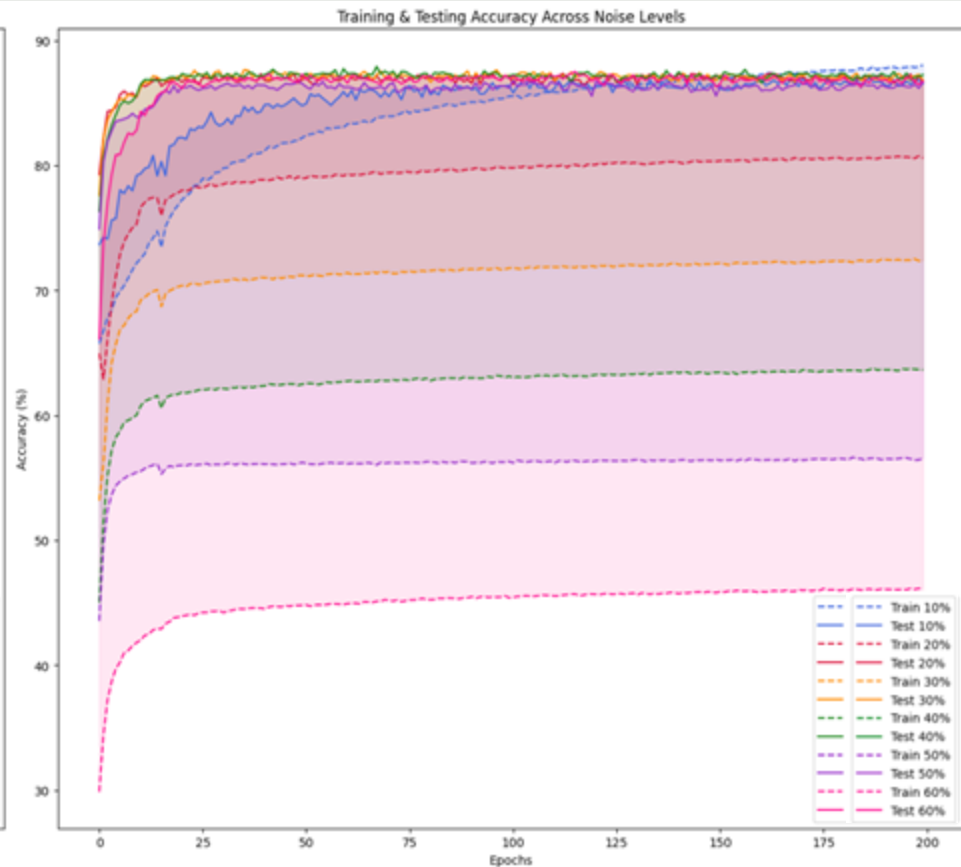
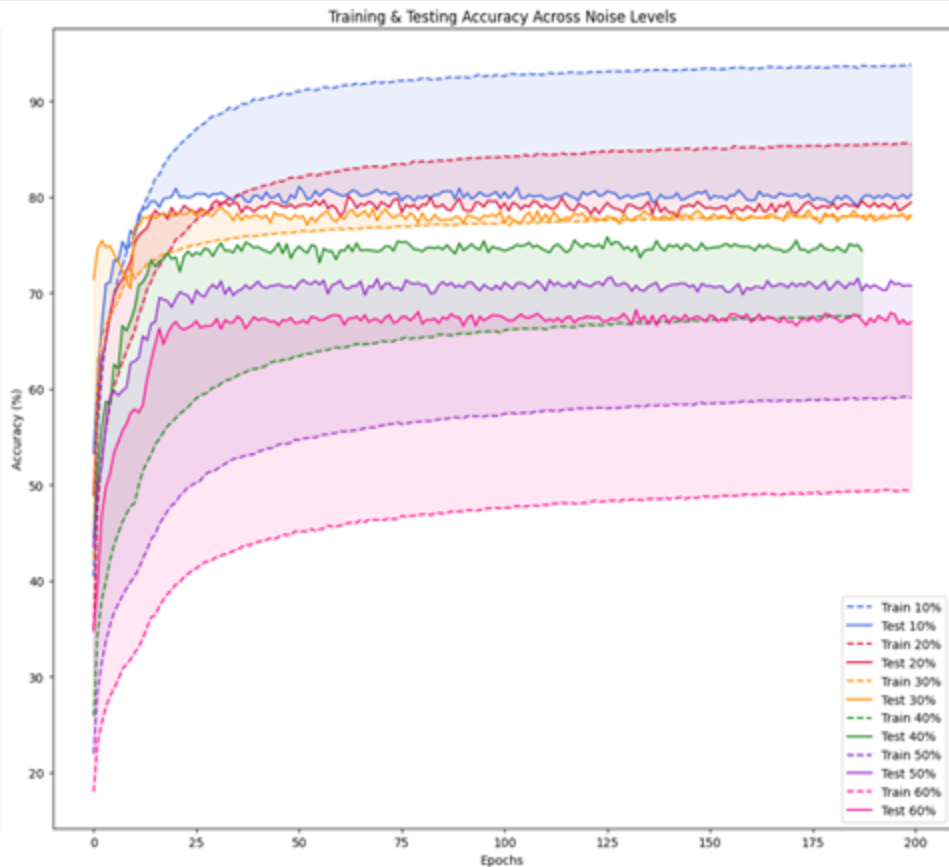
Normalized Reverse Cross Entropy

- CE rewards models for correct predictions
 - RCE punishes model on overconfident predictions
- Prevents CNN from blindly memorizing potentially noisy labels
- Prevents overconfidence across all predictions

Loss Plots: Cross-Entropy (L), NFL + RCE (R)

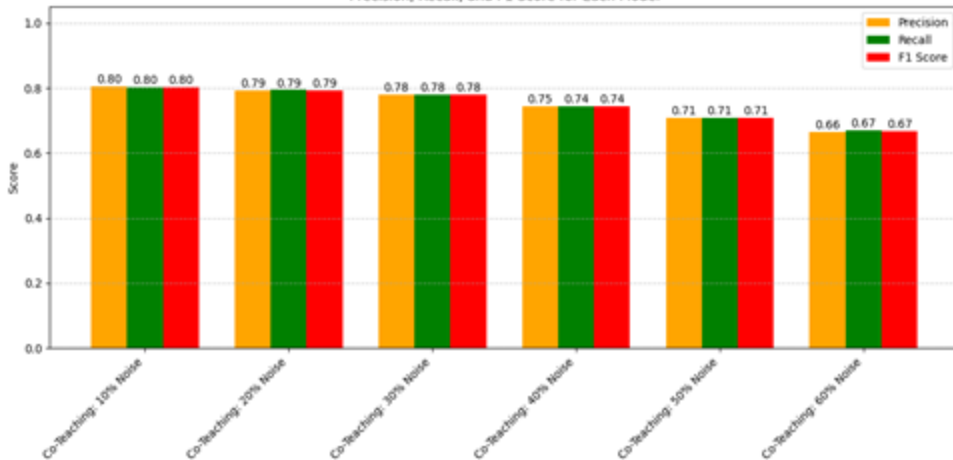


Accuracy Plots: Cross Entropy (L), NFL + RCE (R)

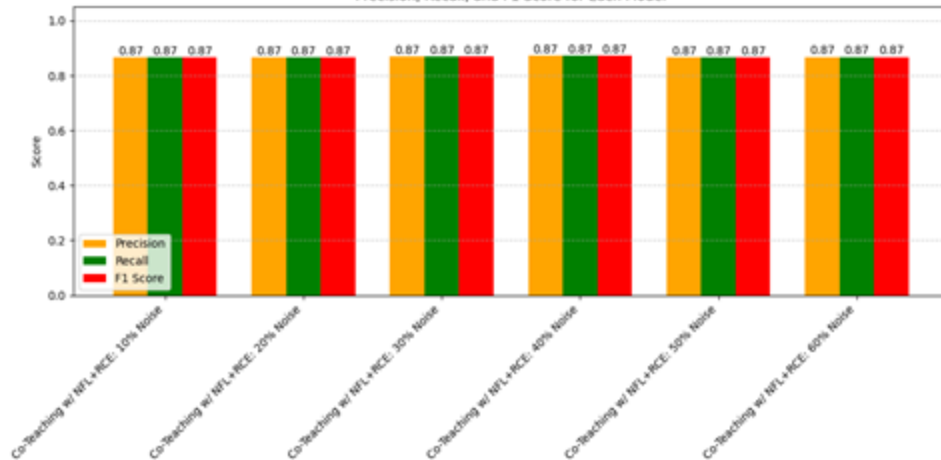


Classification Metrics: Cross Entropy (L), NFL + RCE (R)

Precision, Recall, and F1 Score for Each Model



Precision, Recall, and F1 Score for Each Model



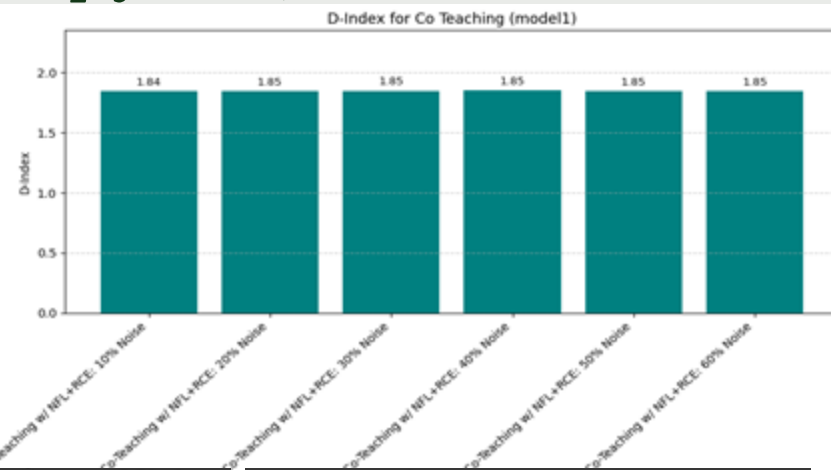
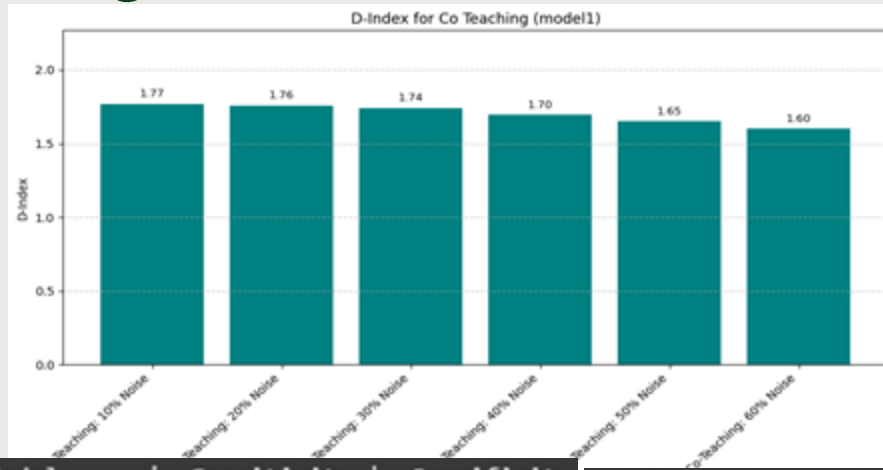
		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

- Uniform noise -> every class has the same likelihood of being misclassified
 - Leads to much fewer instances of FP & FN -> FP ≈ FN
- Macro averaging ---> averaging class wise metrics equally
- NFL + RCE forces the model to ignore incorrect samples while training

Diagnostic Index: Cross Entropy (L), NFL + RCE (R)



Model	Sensitivity	Specificity
Model 1	0.8025	0.9781
Model 2	0.8033	0.9781
Model	Sensitivity	Specificity
Model 1	0.8656	0.9851
Model 2	0.8646	0.9850

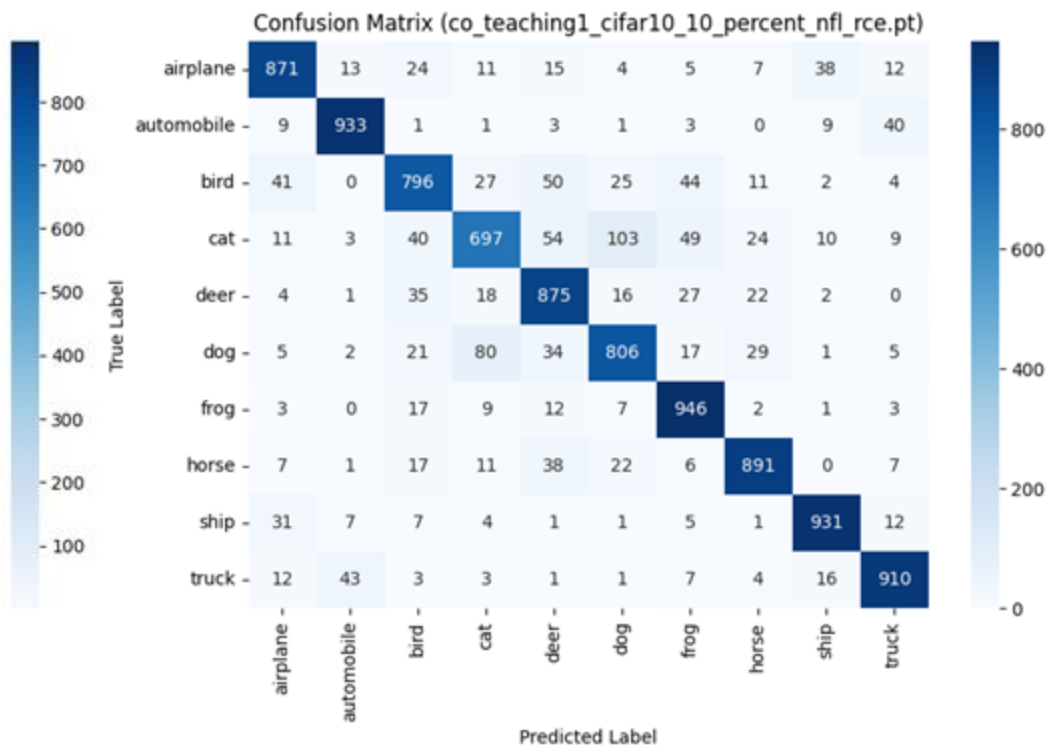
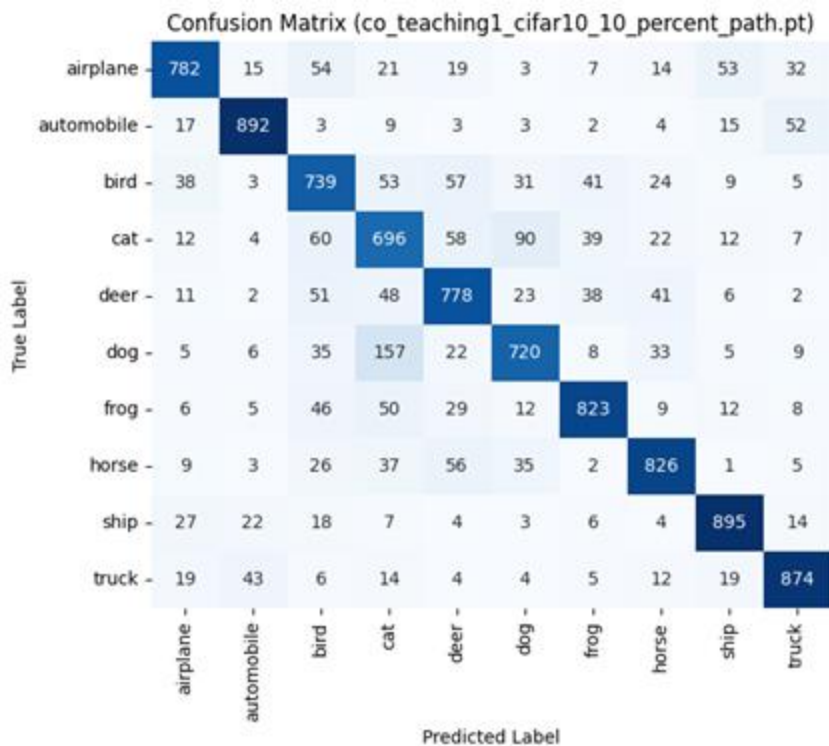
Model	Sensitivity	Specificity
Model 1	0.7941	0.9771
Model 2	0.7928	0.9770
Model	Sensitivity	Specificity
Model 1	0.8664	0.9852
Model 2	0.8664	0.9852

Model	Sensitivity	Specificity
Model 1	0.7805	0.9756
Model 2	0.7771	0.9752
Model	Sensitivity	Specificity
Model 1	0.8683	0.9854
Model 2	0.8713	0.9857

10% Label Noise (CE - TOP, APL - BOT)

20% Label Noise (CE - TOP, APL - BOT)

30% Label Noise (CE - TOP, APL - BOT)



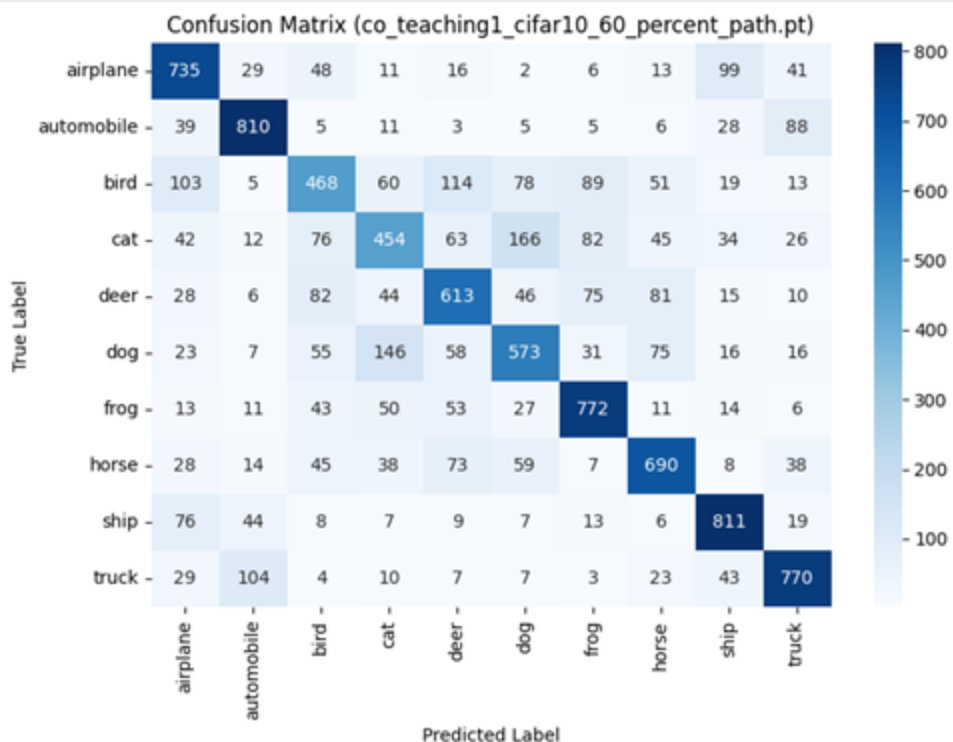
co_teaching1_cifar10_10_percent_path.pt

- dog → cat (157 times)
- cat → dog (90 times)
- cat → bird (60 times)



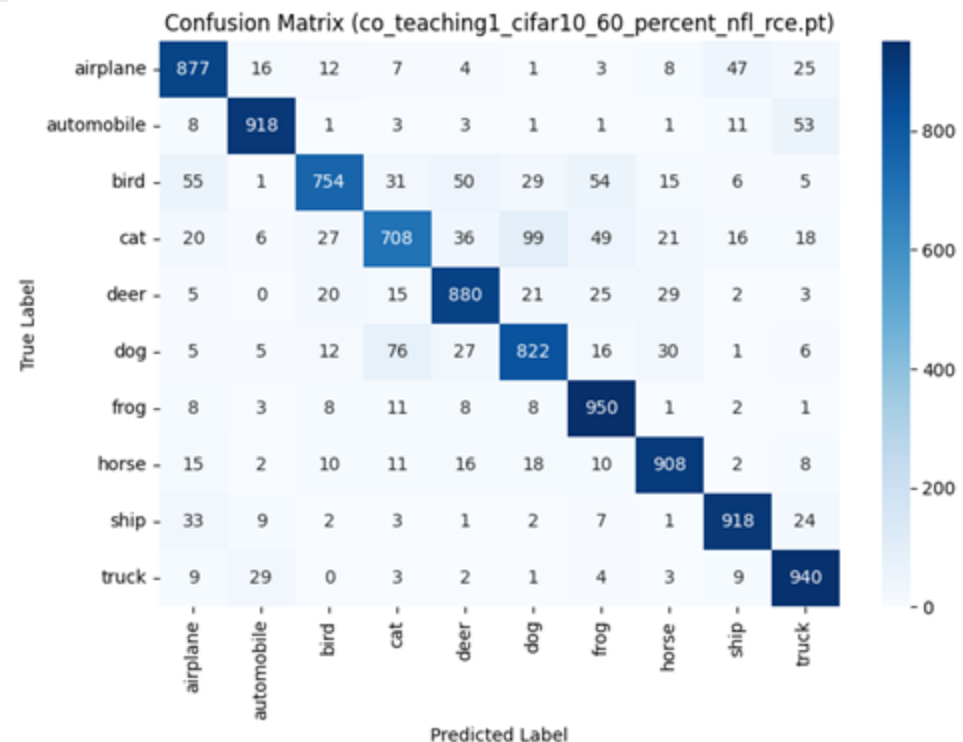
co_teaching1_cifar10_10_percent_nfl_rce.pt

- cat → dog (103 times)
- dog → cat (80 times)
- cat → deer (54 times)



co_teaching1_cifar10_60_percent_path.pt

- cat → dog (166 times)
- dog → cat (146 times)
- bird → deer (114 times)



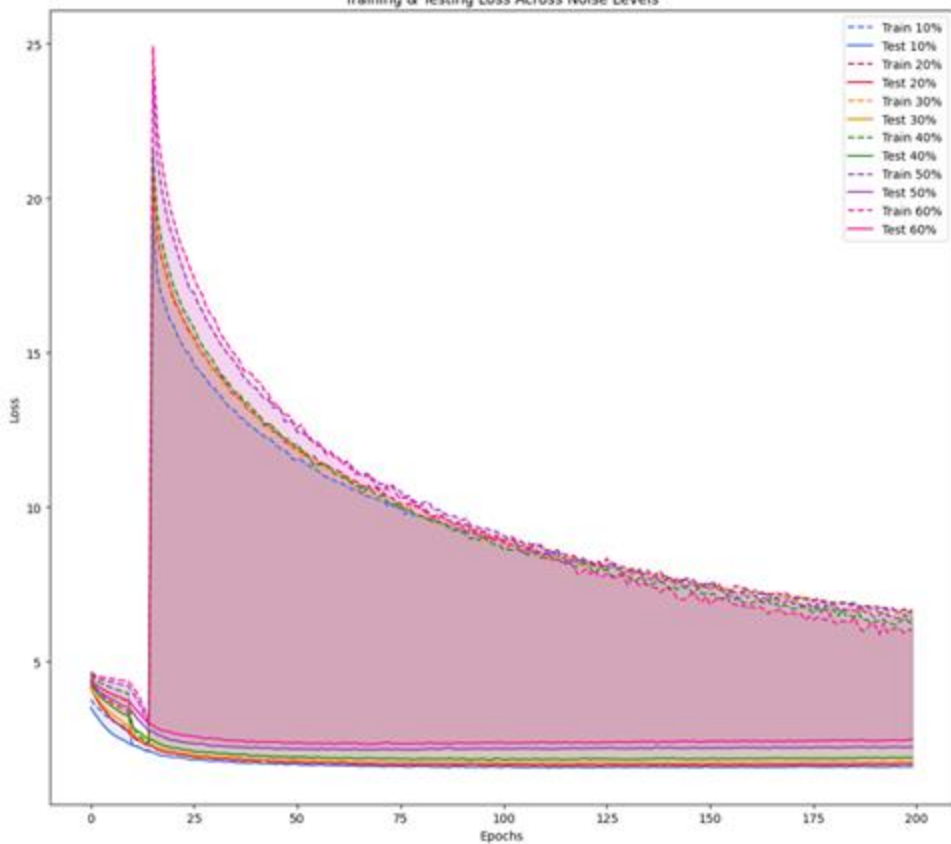
co_teaching1_cifar10_60_percent_nfl_rce.pt

- cat → dog (99 times)
- dog → cat (76 times)
- bird → airplane (55 times)

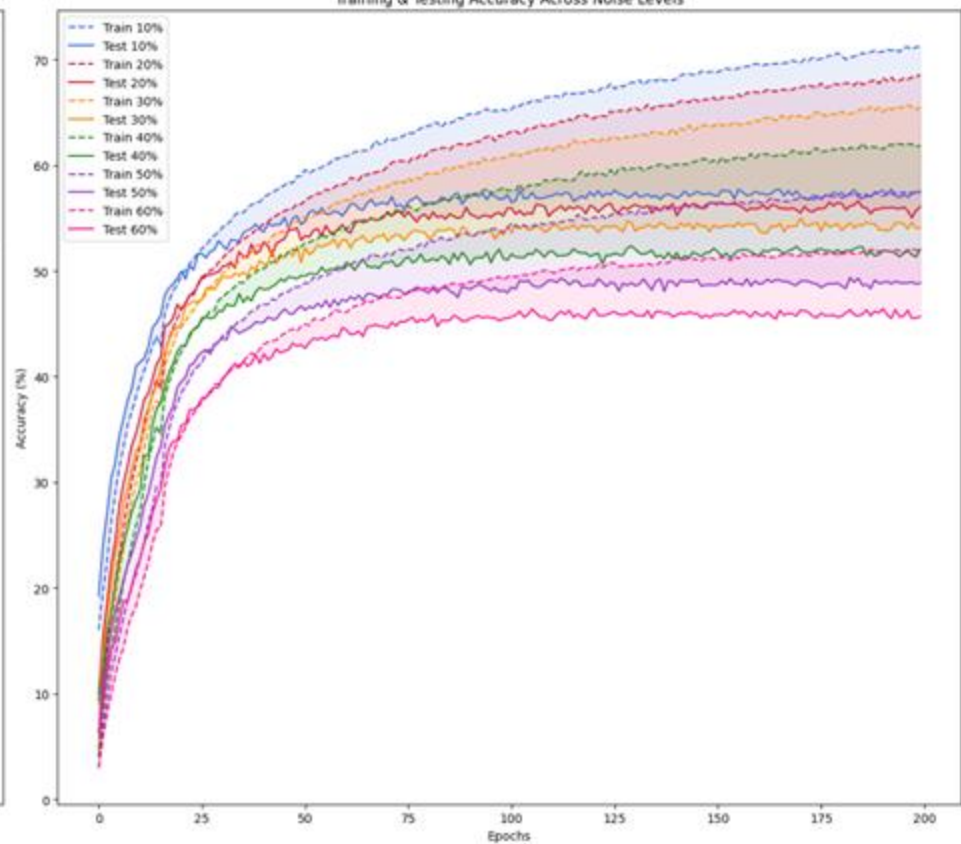
Comparing Co Teaching CNN Performance with CIFAR-100 Variants Using NFL_RCE



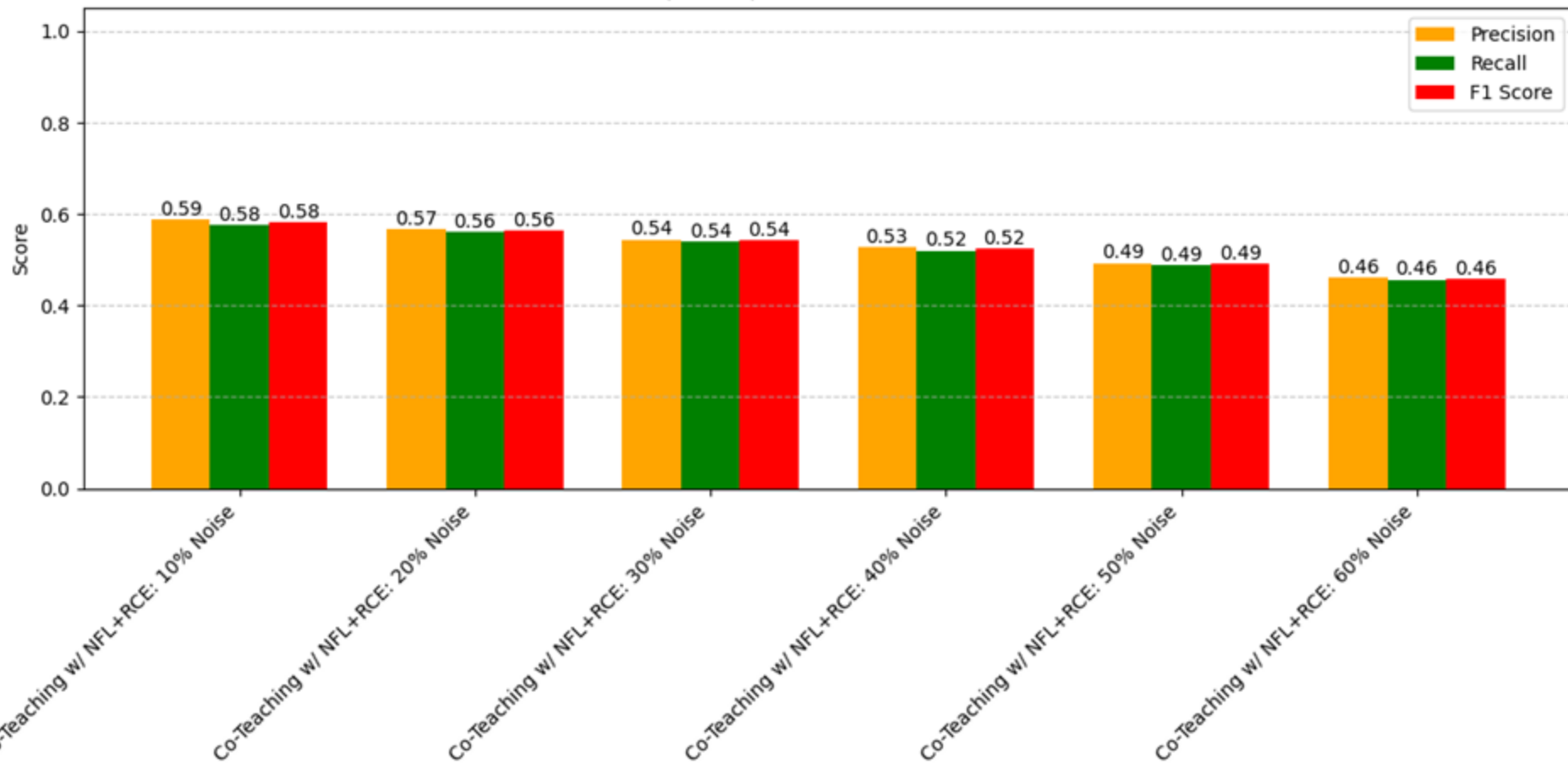
Training & Testing Loss Across Noise Levels



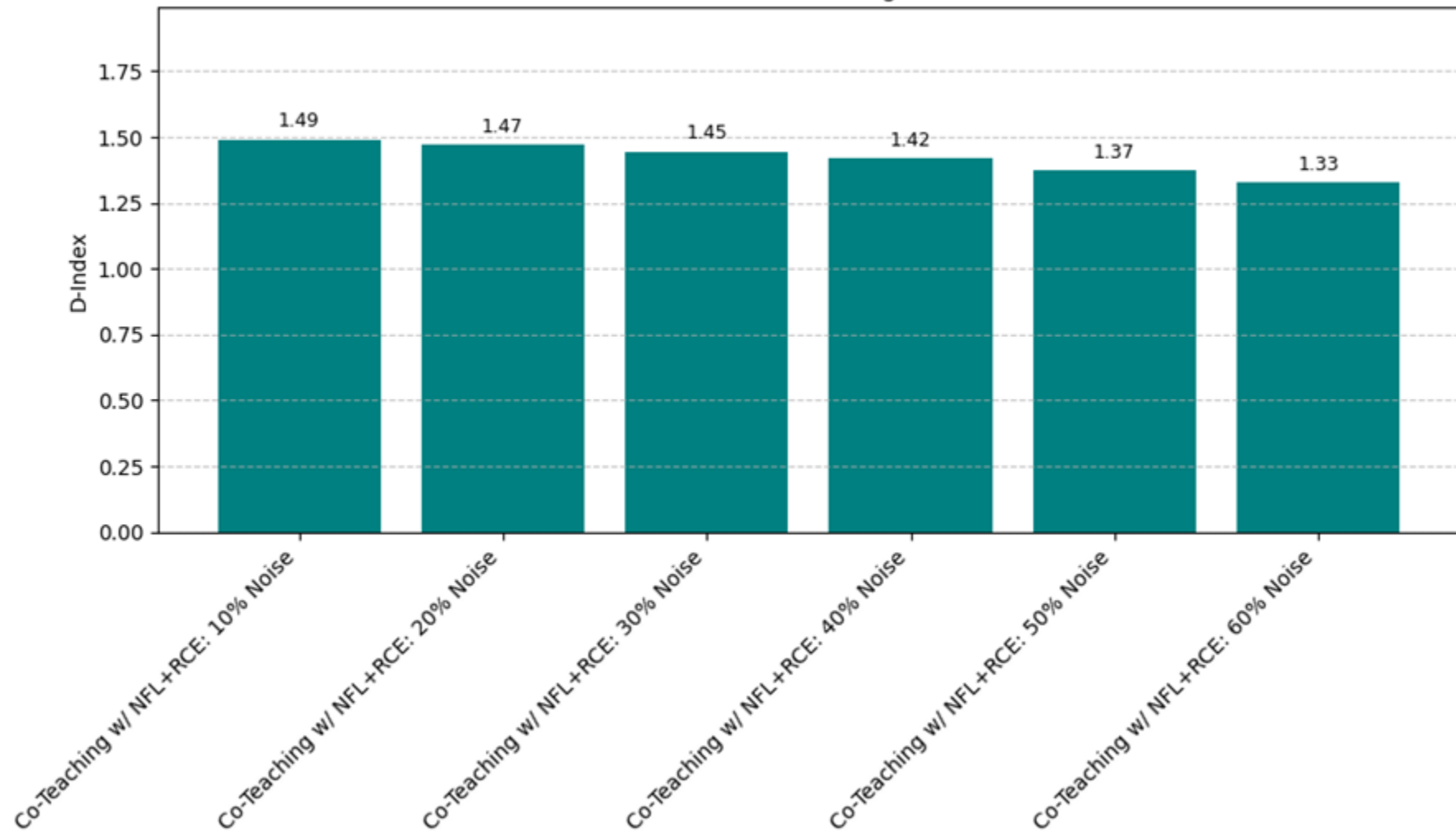
Training & Testing Accuracy Across Noise Levels

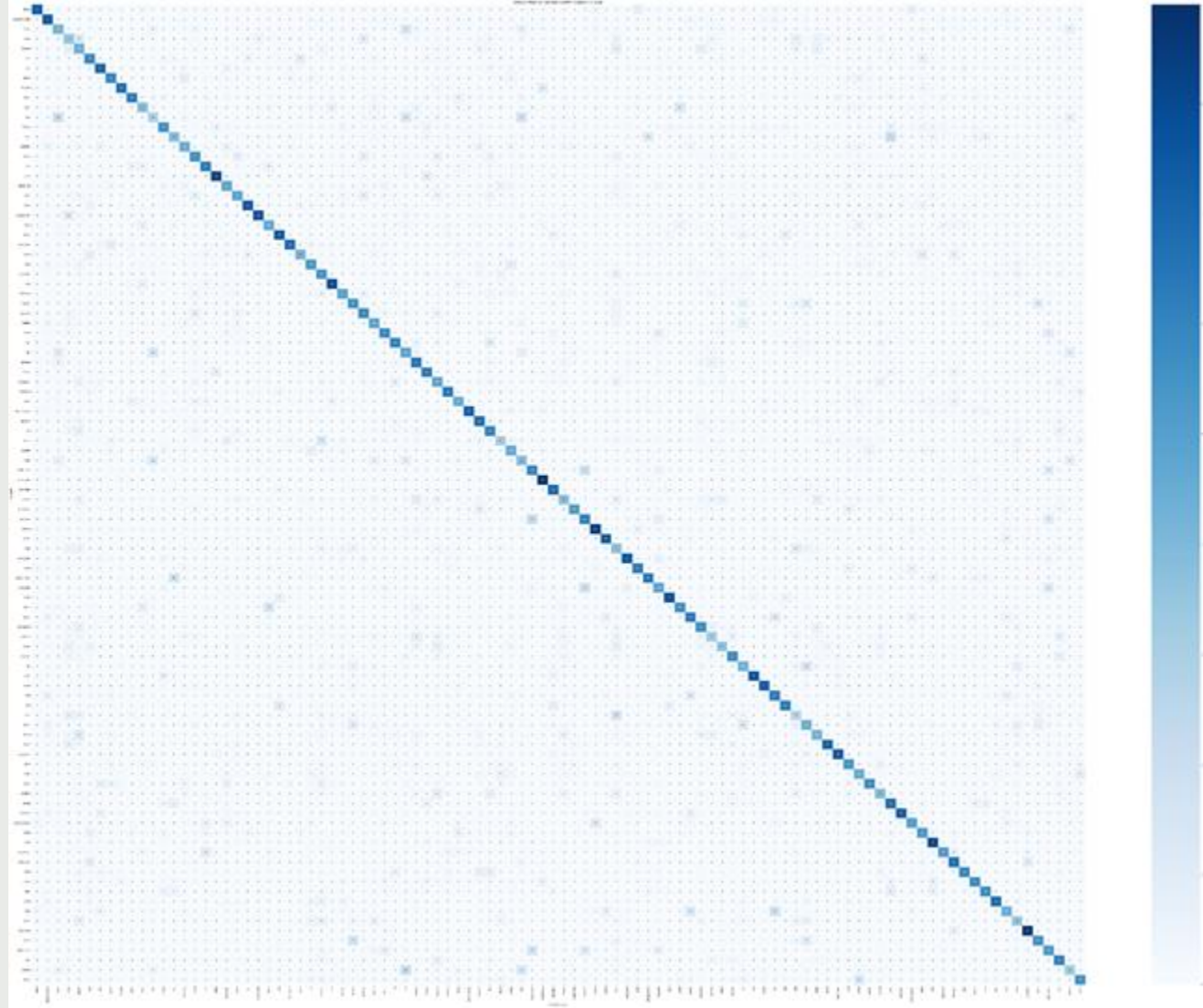


Precision, Recall, and F1 Score for Each Model



D-Index for Co Teaching (model1)

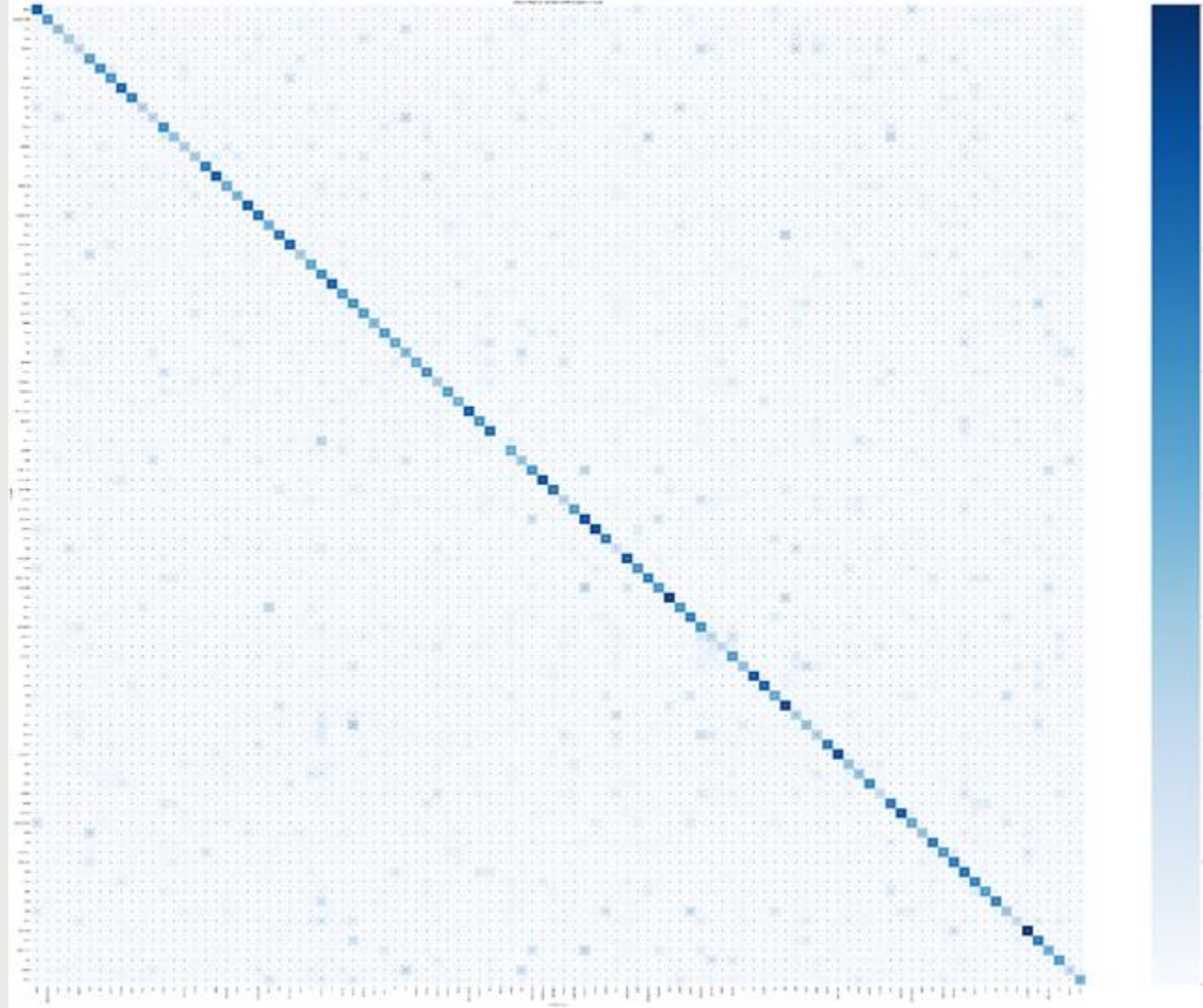




co_teaching1_cifar100_10_percent_nfl_rce.pt - Most Misclassified Classes (Noise Level: 10%)

Total Misclassifications: 4249

- oak_tree → maple_tree (22 times)
- bus → street (21 times)
- woman → girl (21 times)



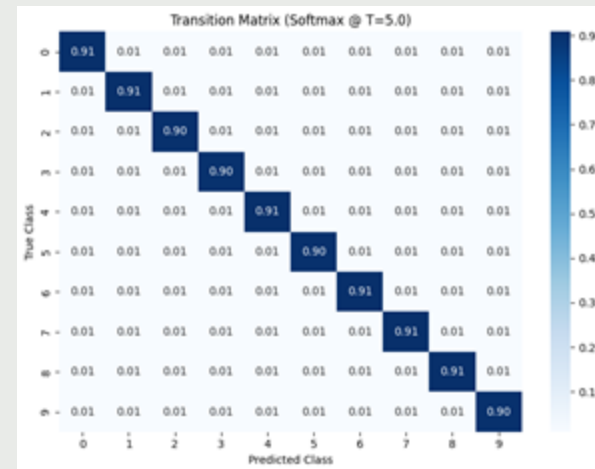
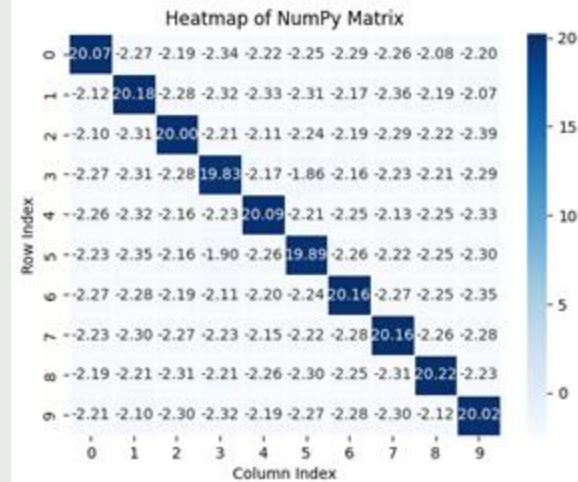
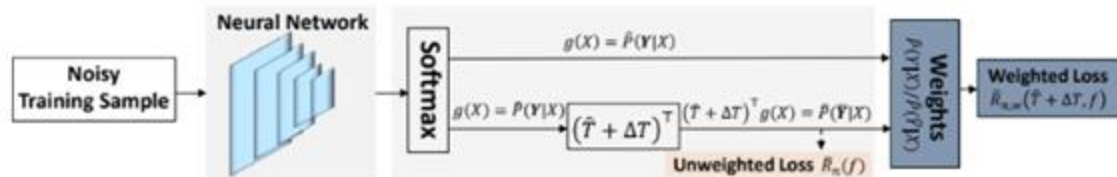

```
co_teaching1_cifar100_60_percent_nfl_rce.pt - Most Misclassified Classes (Noise Level: 60%):  
Total Misclassifications: 5433  
- shark → dolphin (22 times)  
- cloud → sea (21 times)  
- lizard → crocodile (21 times)
```

Comparing NR - CNN Performance with CIFAR-10 Variants Using Cross Entropy



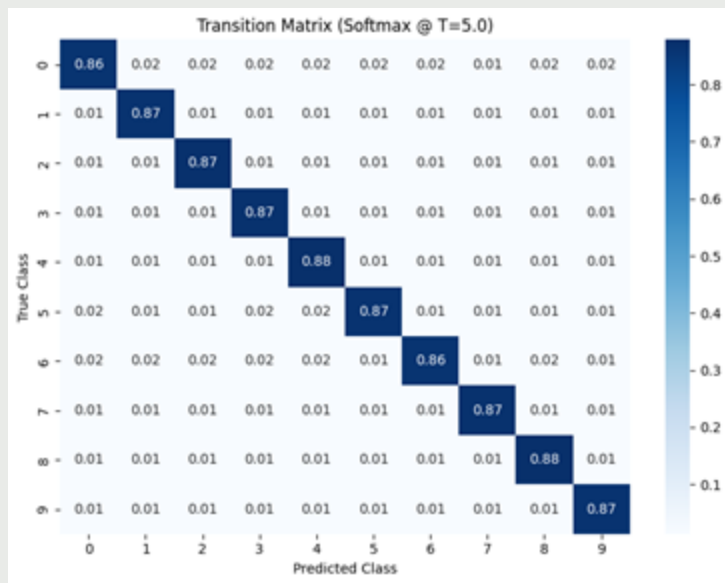
T - Revision

- Training begins with an identity matrix based on the assumption that all labels are correct, with no noise
- After a warm-up period the model learns enough so that it can detect mislabeled classes, these predictions help build the transition matrix by providing insight into how noisy the labels are
- T is applied to predicted probabilities to adjust the learning procedure by accounting for label noise
- T is updated during training (revision) so that it be more accurate at estimating label corruption



Insight: NR-CNN had to Crawl, so CT-NR CNNs Could Run

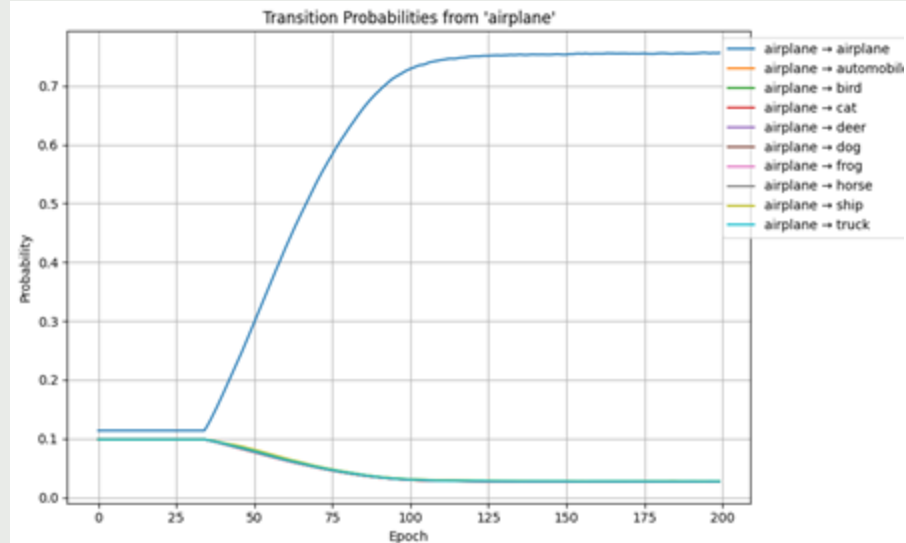
- Performance is better than base CNN at low - mid noise levels, but quickly decreases as more noise is introduced
- Not enough T regularization (Confidence based re-weighting) → Overfitting
 - Plans of combining with other techniques led to too much caution in terms of regularization



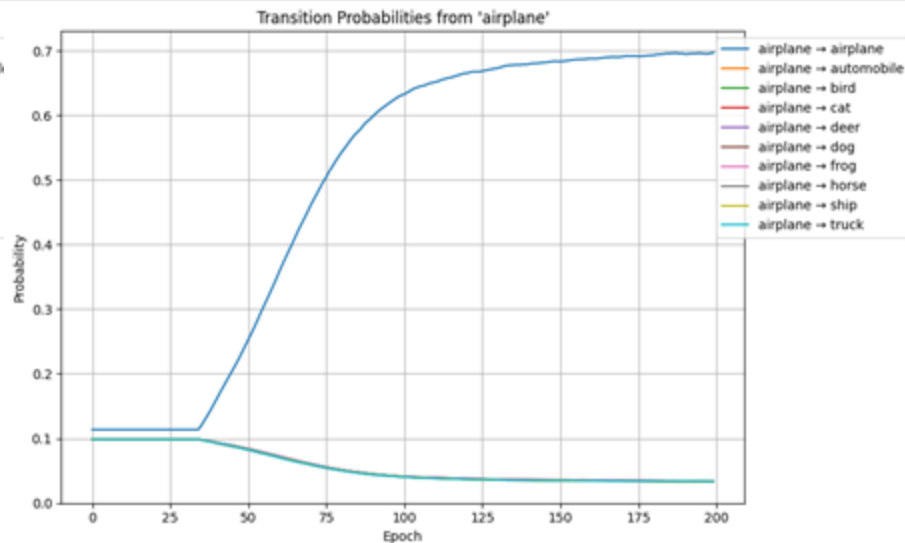
← Transition Matrix (T) for 60% Noise
(TOO GOOD TO BE TRUE)

A “correct” T should have ~40% across the diagonal, not high 80’s

More Confirmation of Overfitting :(



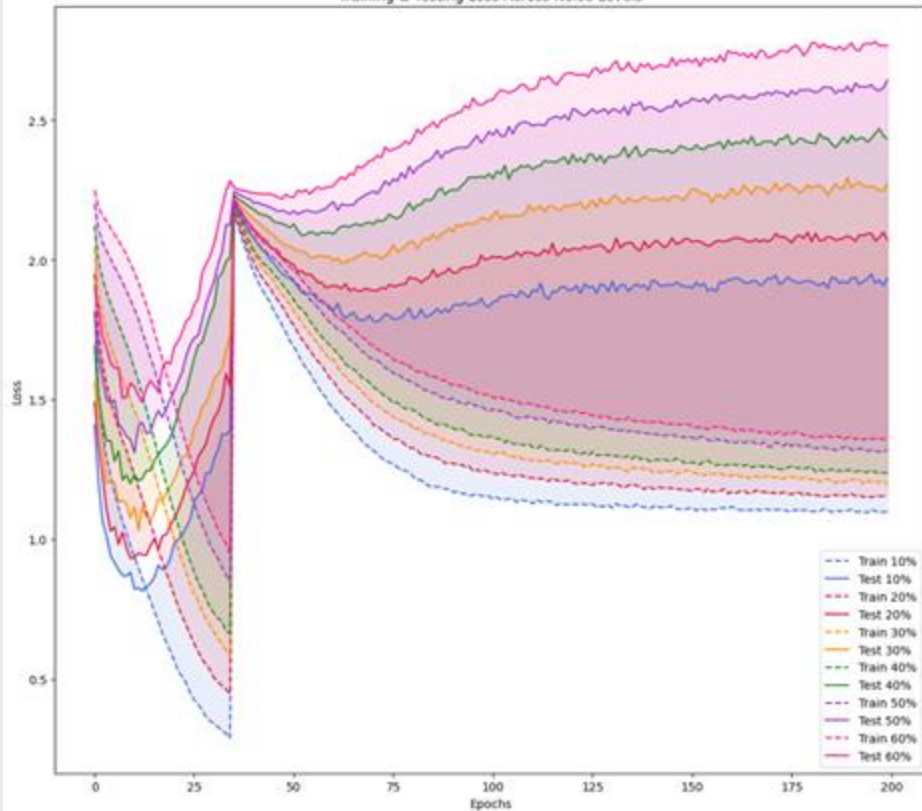
10% Noise



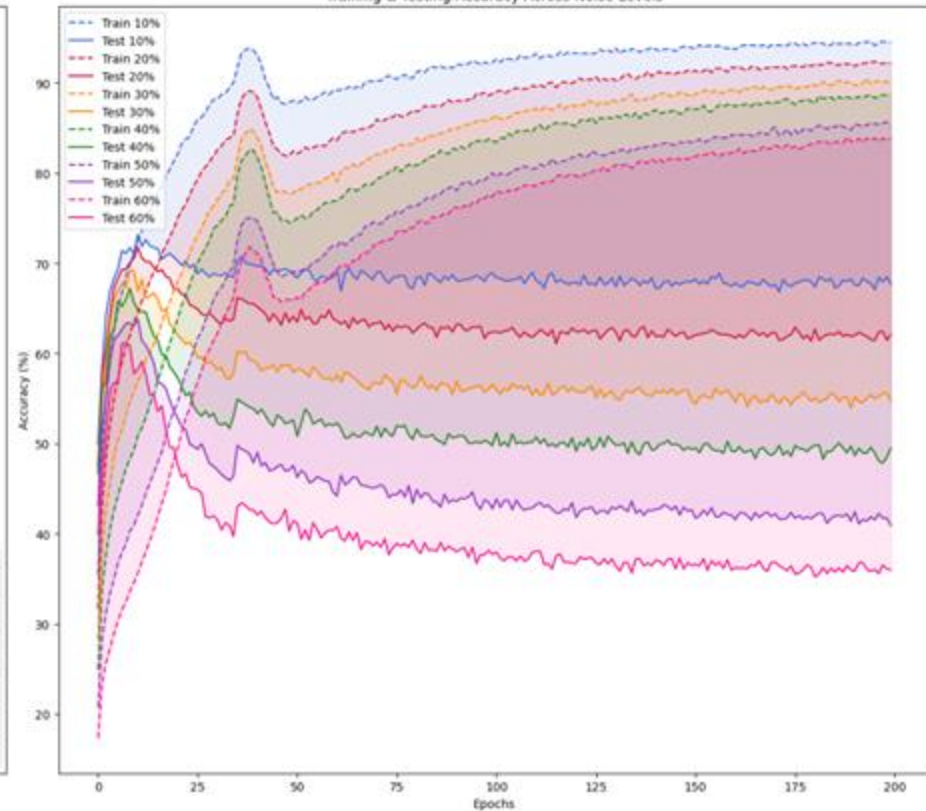
60% Noise

Loss and Accuracy Plots

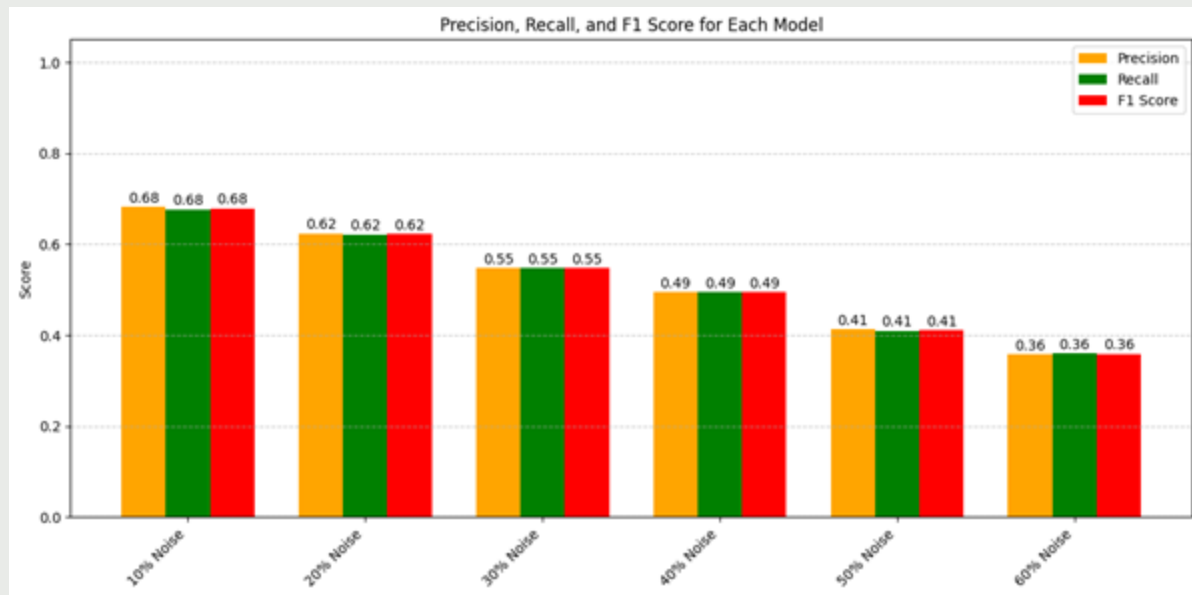
Training & Testing Loss Across Noise Levels



Training & Testing Accuracy Across Noise Levels



Classification Metrics:

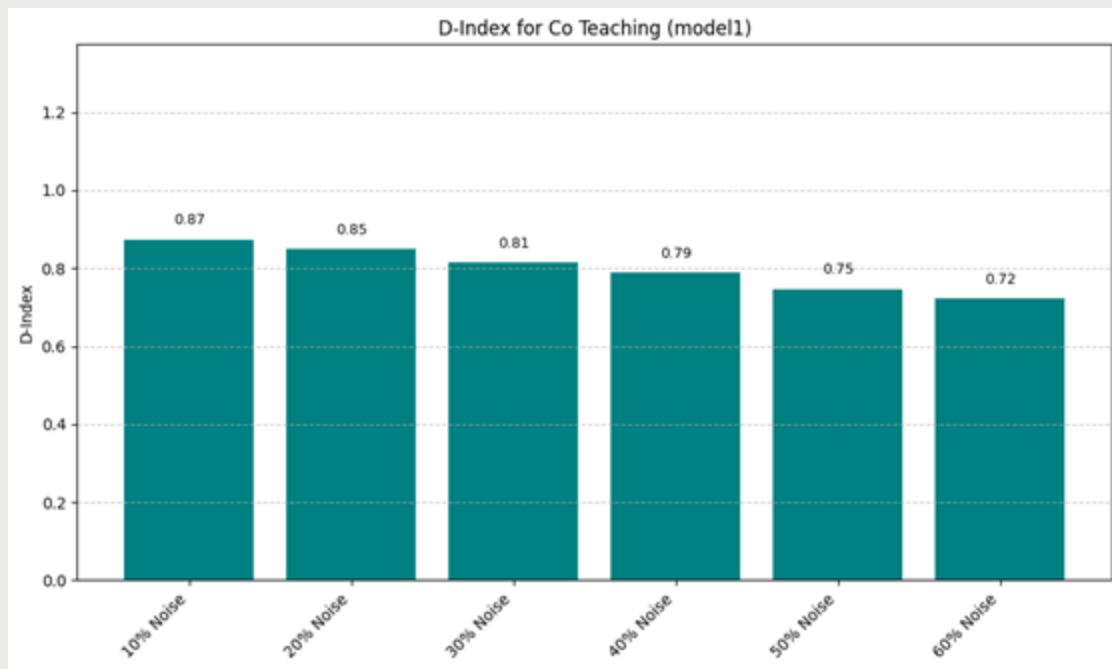


		Predicted	
		0	1
Actual	0	TN	FP
	1	FN	TP

$$\text{Precision} = \frac{TP}{TP + FP}$$
$$\text{Recall} = \frac{TP}{TP + FN}$$

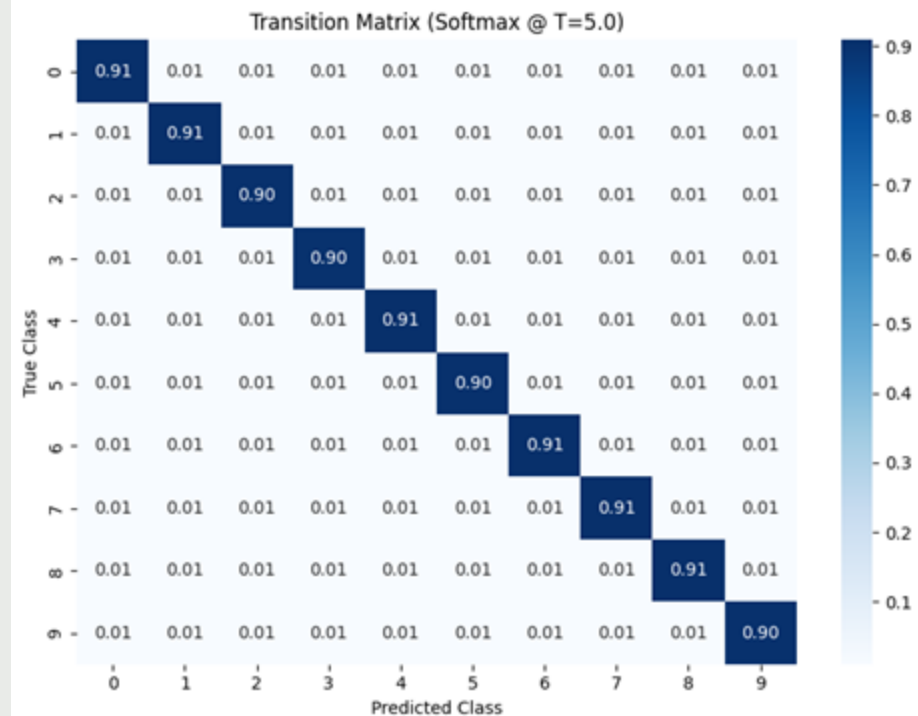
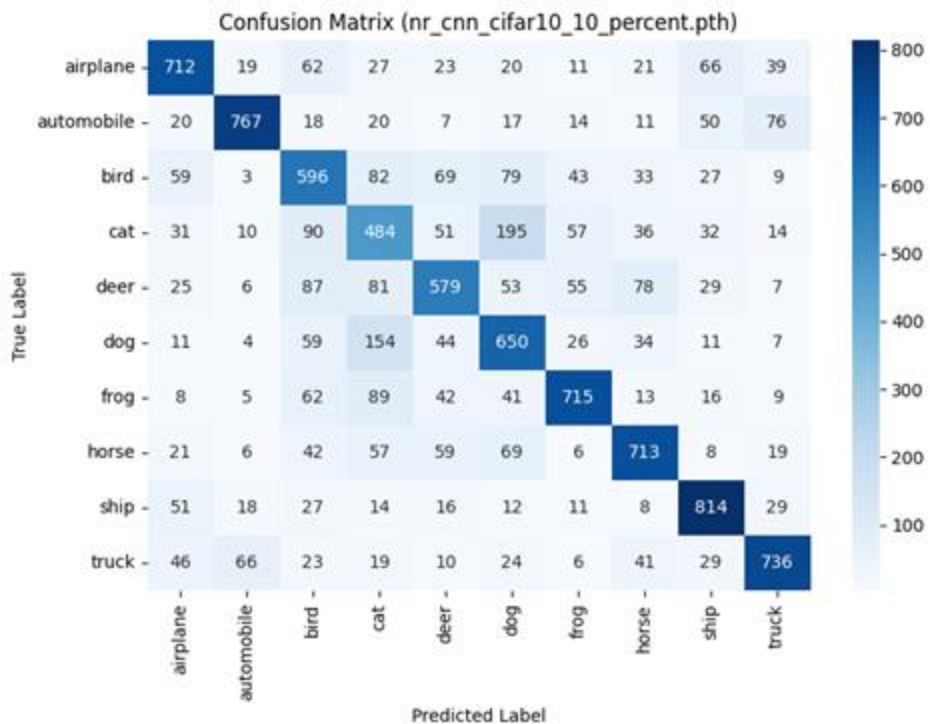
- Precision - accuracy of positive predictions
- Recall - ability to find correct instances
- F1 Score - harmonic mean of precision and recall

Diagnostic Index



- Increasing label noise lowers the model's ability to learn meaningful patterns declines
- Models struggle to extract consistent patterns when labels are unreliable
- A high d-index indicates the model is learning meaningful features; (0, 2]

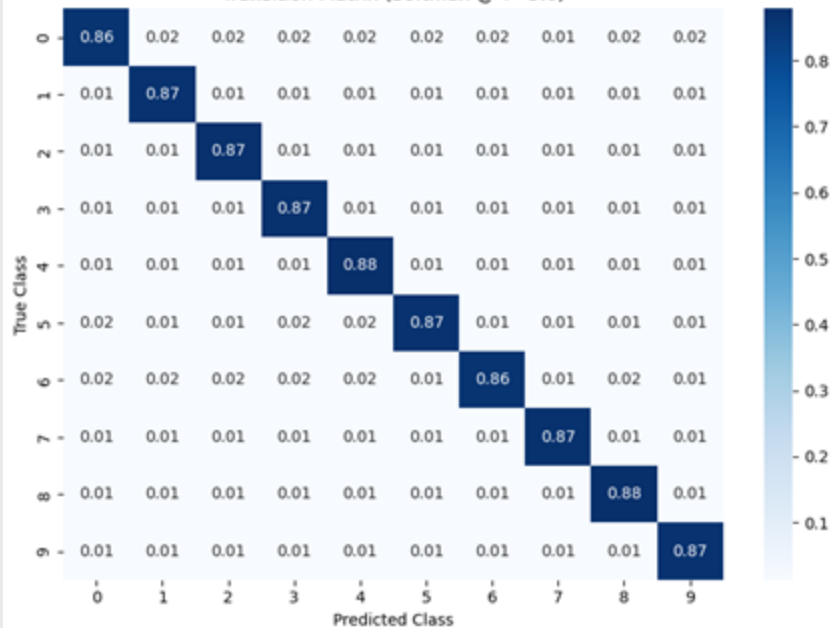
$$d = \log_2(1 + a) + \log_2(1 + \frac{s+p}{2})$$



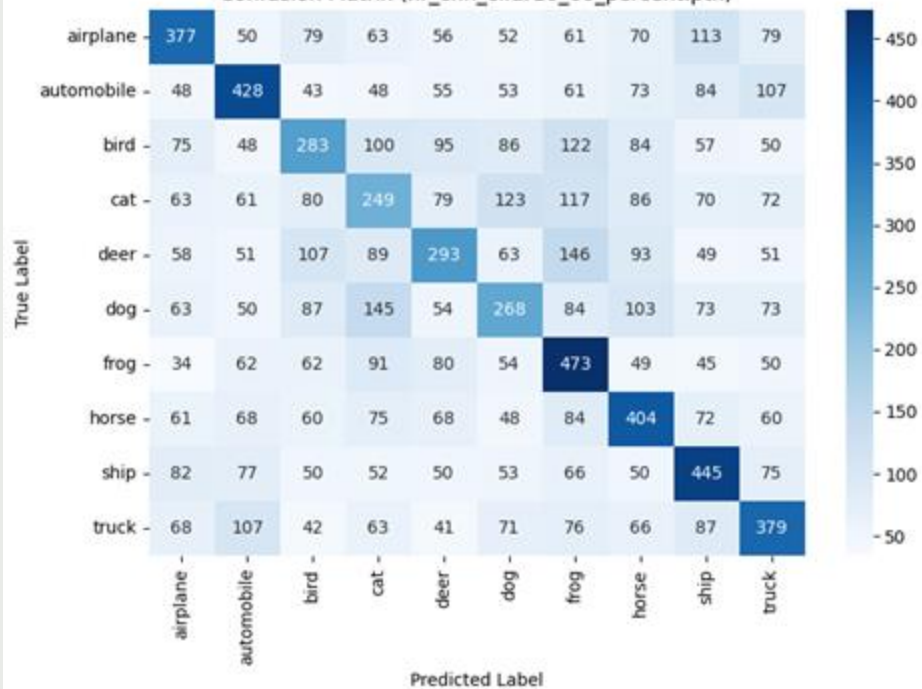
nr_cnn_cifar10_10_percent.pth -
 Total Misclassifications: 3234
 - cat → dog (195 times)
 - dog → cat (154 times)
 - cat → bird (90 times)



Transition Matrix (Softmax @ T=5.0)



Confusion Matrix (nr_cnn_cifar10_60_percent.pth)



Label: automobile



Label: truck



nr_cnn_cifar10_60_percent.pth -
 Total Misclassifications: 6401
 - deer → frog (146 times)
 - dog → cat (145 times)
 - cat → dog (123 times)

Comparing Co Teaching+ CNN Performance with CIFAR-100 Variants Using NFL+RCE



Co-Teaching+: Using Disagreement to Improve Noise Robustness

Disagreement Improving Generalization

- Modification to Co-Teaching that uses disagreement-based sample selection
- Two networks are trained simultaneously, each selecting small-loss examples where their predictions disagree $\rightarrow$ diversity and robustness.
- Improves over standard Co-Teaching by reducing confirmation bias and overfitting to noisy labels, specifically in later training stages

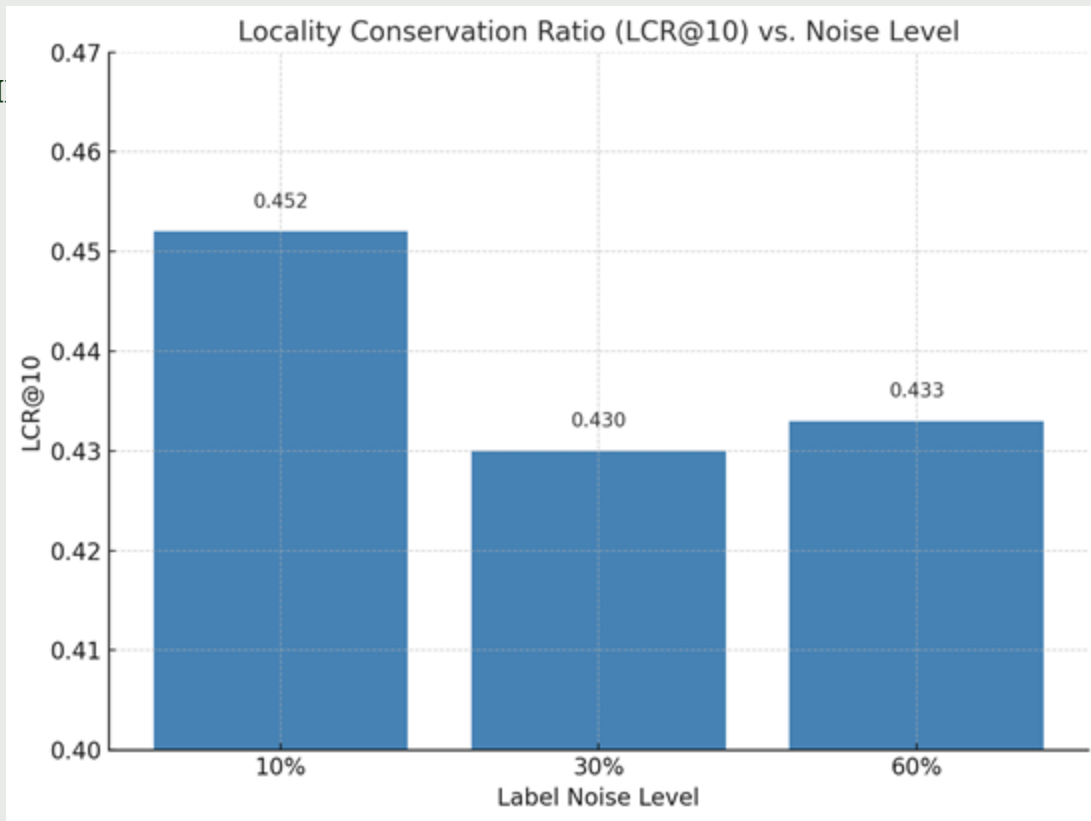
Algorithm 1 Co-teaching+. Step 4: disagreement-update; Step 5-8: cross-update.

```
1: Input  $w^{(1)}$  and  $w^{(2)}$ , training set  $\mathcal{D}$ , batch size  $B$ , learning rate  $\eta$ , estimated noise rate  $\tau$ , epoch  $E_k$  and  $E_{\max}$ ;  
for  $e = 1, 2, \dots, E_{\max}$  do  
  2: Shuffle  $\mathcal{D}$  into  $\frac{|\mathcal{D}|}{B}$  mini-batches; //noisy dataset  
  for  $n = 1, \dots, \frac{|\mathcal{D}|}{B}$  do  
    3: Fetch  $n$ -th mini-batch  $\bar{\mathcal{D}}$  from  $\mathcal{D}$ ;  
    4: Select prediction disagreement  $\bar{\mathcal{D}}'$  by Eq. (1);  
    5: Get  $\bar{\mathcal{D}}'^{(1)} = \arg \min_{\mathcal{D}': |\mathcal{D}'| \geq \lambda(e)|\bar{\mathcal{D}}'|} \ell(\mathcal{D}'; w^{(1)});$   
    //sample  $\lambda(e)\%$  small-loss instances  
    6: Get  $\bar{\mathcal{D}}'^{(2)} = \arg \min_{\mathcal{D}': |\mathcal{D}'| \geq \lambda(e)|\bar{\mathcal{D}}'|} \ell(\mathcal{D}'; w^{(2)});$   
    //sample  $\lambda(e)\%$  small-loss instances  
    7: Update  $w^{(1)} = w^{(1)} - \eta \nabla \ell(\bar{\mathcal{D}}'^{(2)}; w^{(1)});$  //update  $w^{(1)}$  by  $\bar{\mathcal{D}}'^{(2)}$ ;  
    8: Update  $w^{(2)} = w^{(2)} - \eta \nabla \ell(\bar{\mathcal{D}}'^{(1)}; w^{(2)});$  //update  $w^{(2)}$  by  $\bar{\mathcal{D}}'^{(1)}$ ;  
  end  
  9: Update  $\lambda(e) = 1 - \min\{\frac{e}{E_k}\tau, \tau\}$  or  $1 - \min\{\frac{e}{E_k}\tau, (1 + \frac{e - E_k}{E_{\max} - E_k})\tau\};$   
end  
10: Output  $w^{(1)}$  and  $w^{(2)}$ .
```

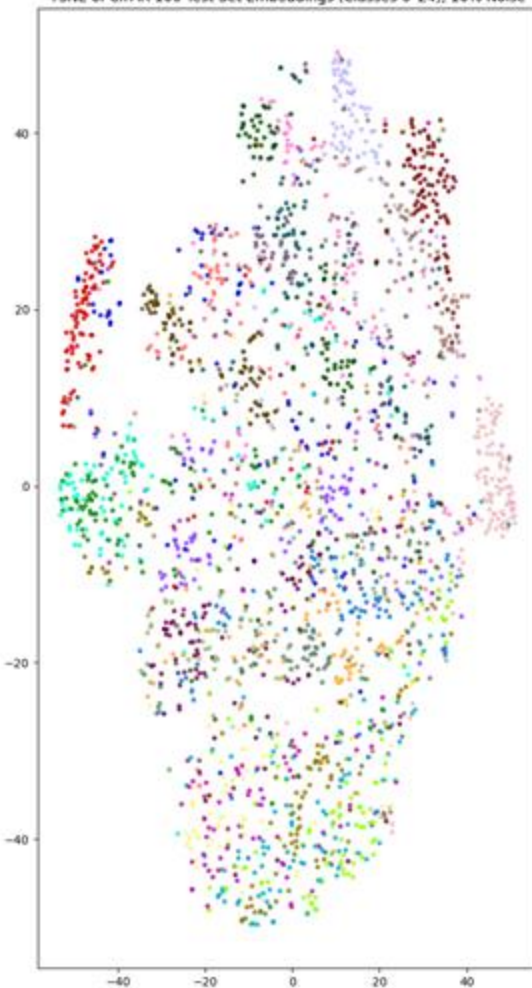
Co-Teaching Preserves Local Structure Even Under Heavy Noise

Evaluated using t-SNE & LCR@10 across CI
100 Variants

- Co-Teaching+/Co-Teaching become more selective at higher noise levels, discarding more uncertain samples & training on a cleaner subset of the data.
- Locality Conservation Ratio (LCR) remains stable as the model focuses on confident examples with more consistent local structure.
- t-SNE highlights confident clusters, preserving neighborhood integrity even as global label noise rises.

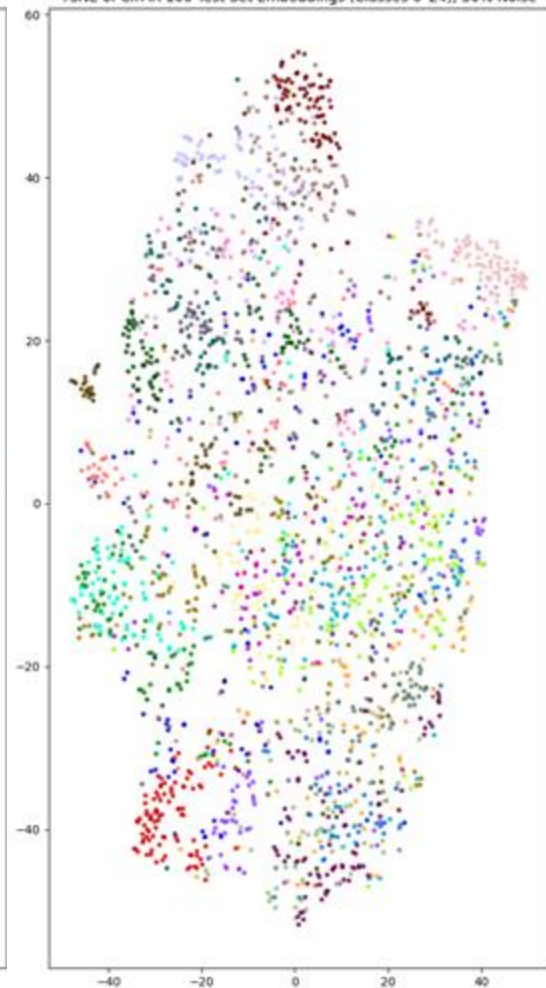


TSNE of CIFAR-100 Test Set Embeddings (Classes 0-24), 10% Noise



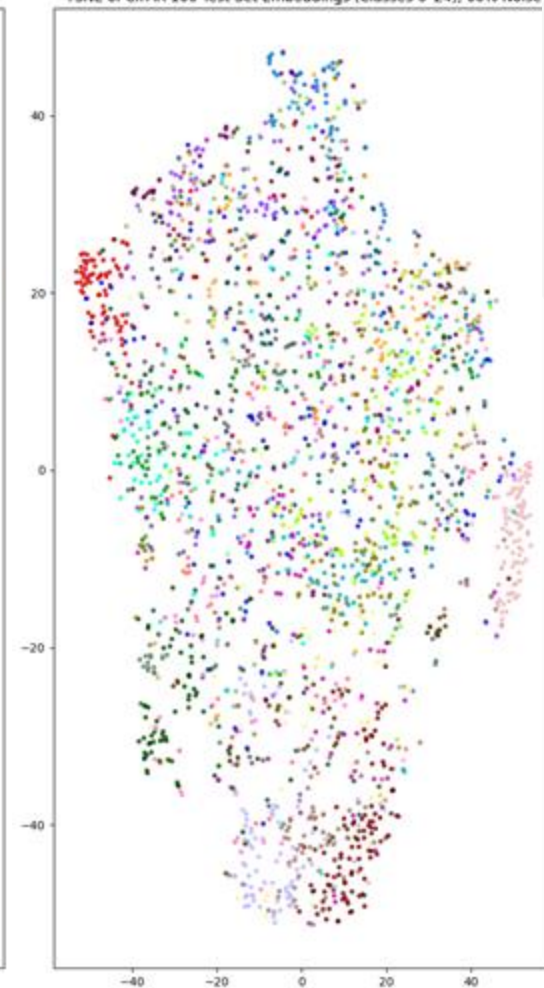
LCR = 0.452

TSNE of CIFAR-100 Test Set Embeddings (Classes 0-24), 30% Noise



LCR = 0.431

TSNE of CIFAR-100 Test Set Embeddings (Classes 0-24), 60% Noise

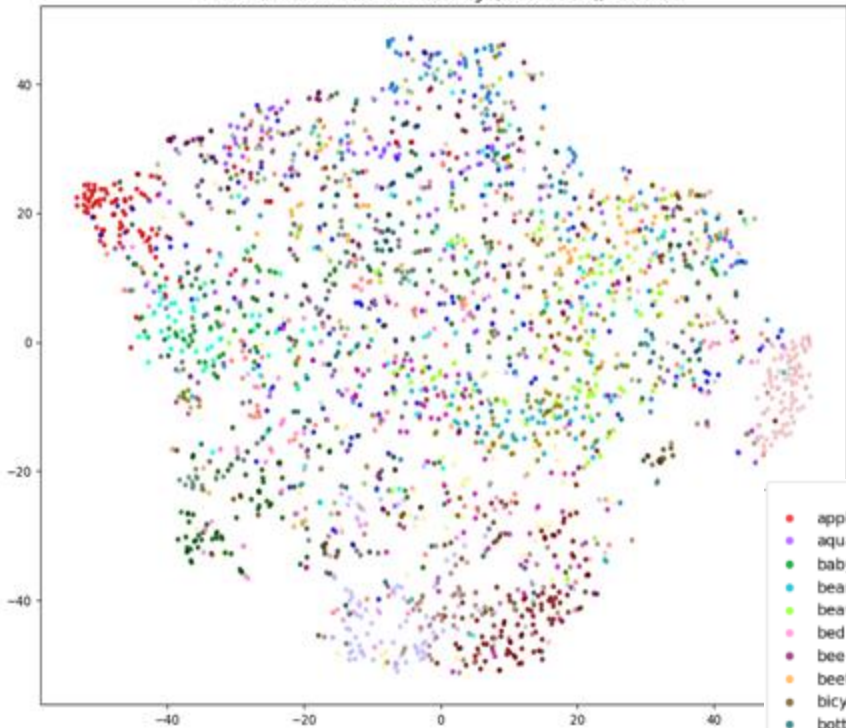


LCR = 0.433



t-SNE of Final Layer Embeddings & Confidence

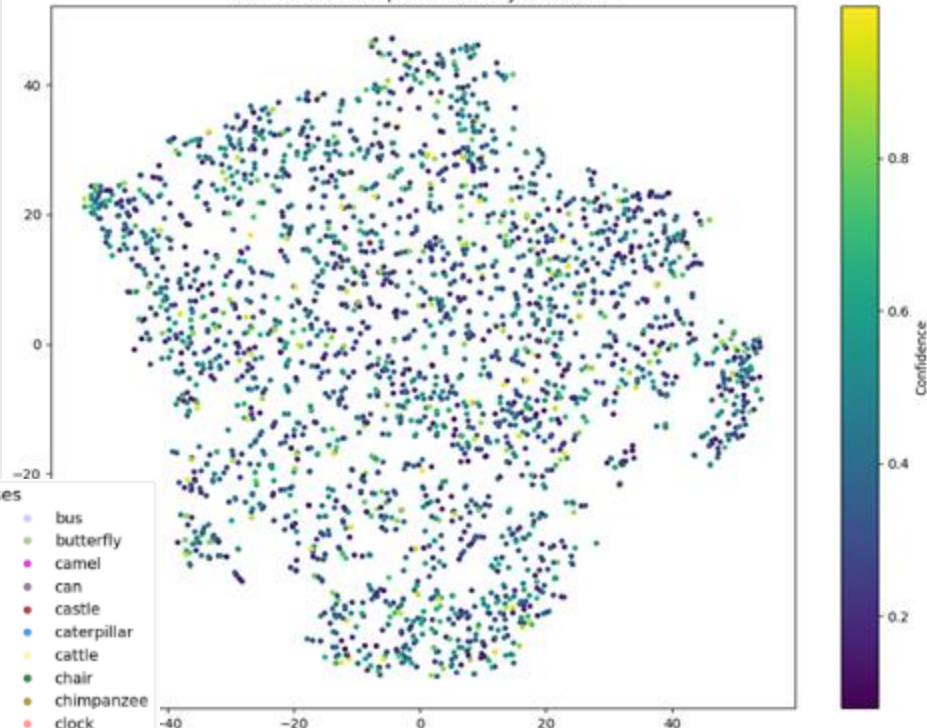
TSNE of CIFAR-100 Test Set Embeddings (Classes 0-24), 10% Noise



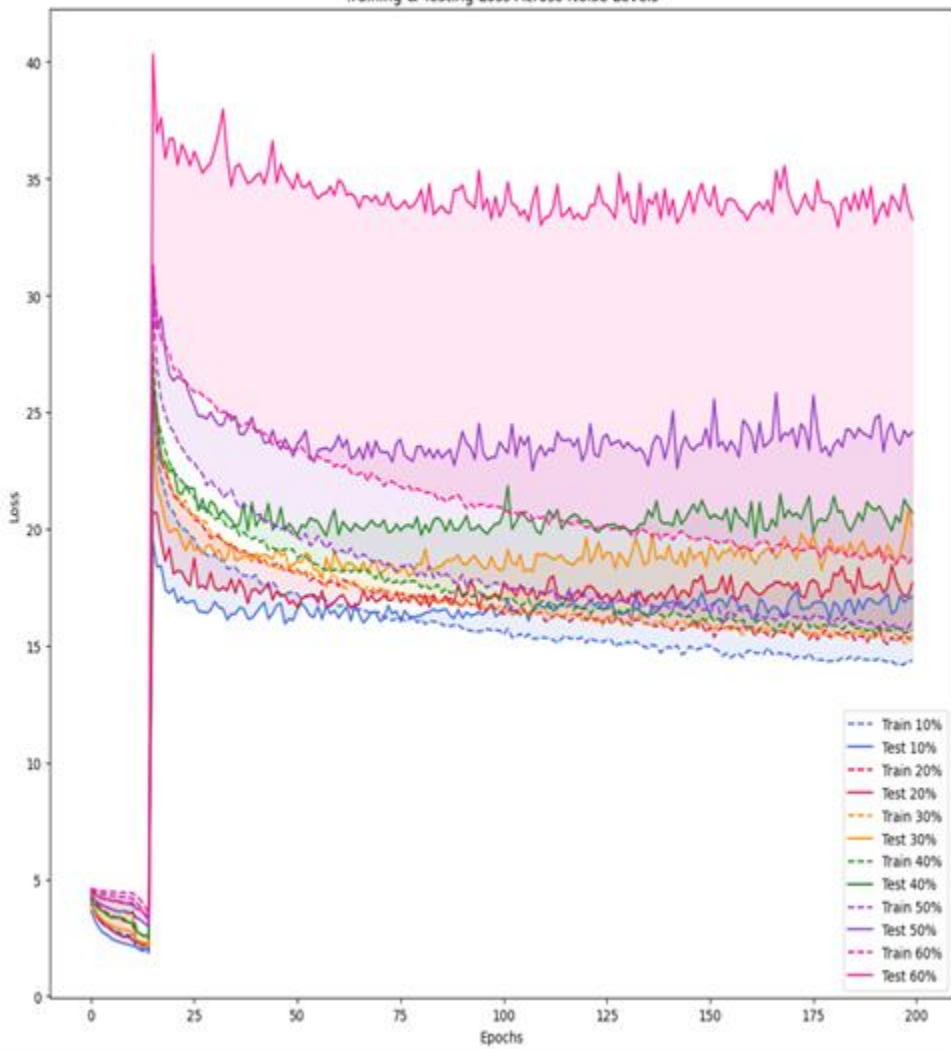
Classes

- | | |
|-----------------|---------------|
| • apple | • bus |
| • aquarium_fish | • butterfly |
| • baby | • camel |
| • bear | • can |
| • beaver | • castle |
| • bed | • caterpillar |
| • bee | • cattle |
| • beetle | • chair |
| • bicycle | • chimpanzee |
| • bottle | • clock |
| • bowl | • cloud |
| • boy | • cockroach |
| • bridge | |

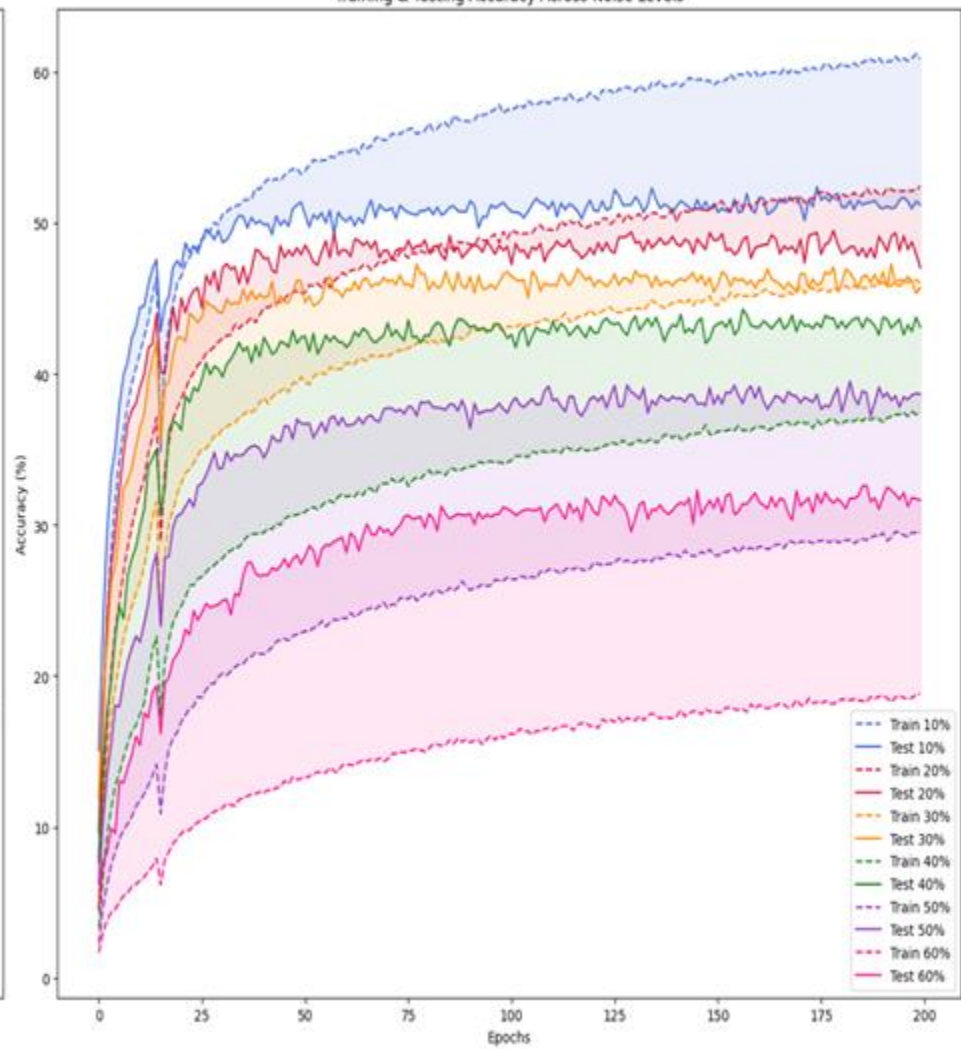
t-SNE of Model Outputs Colored by Confidence



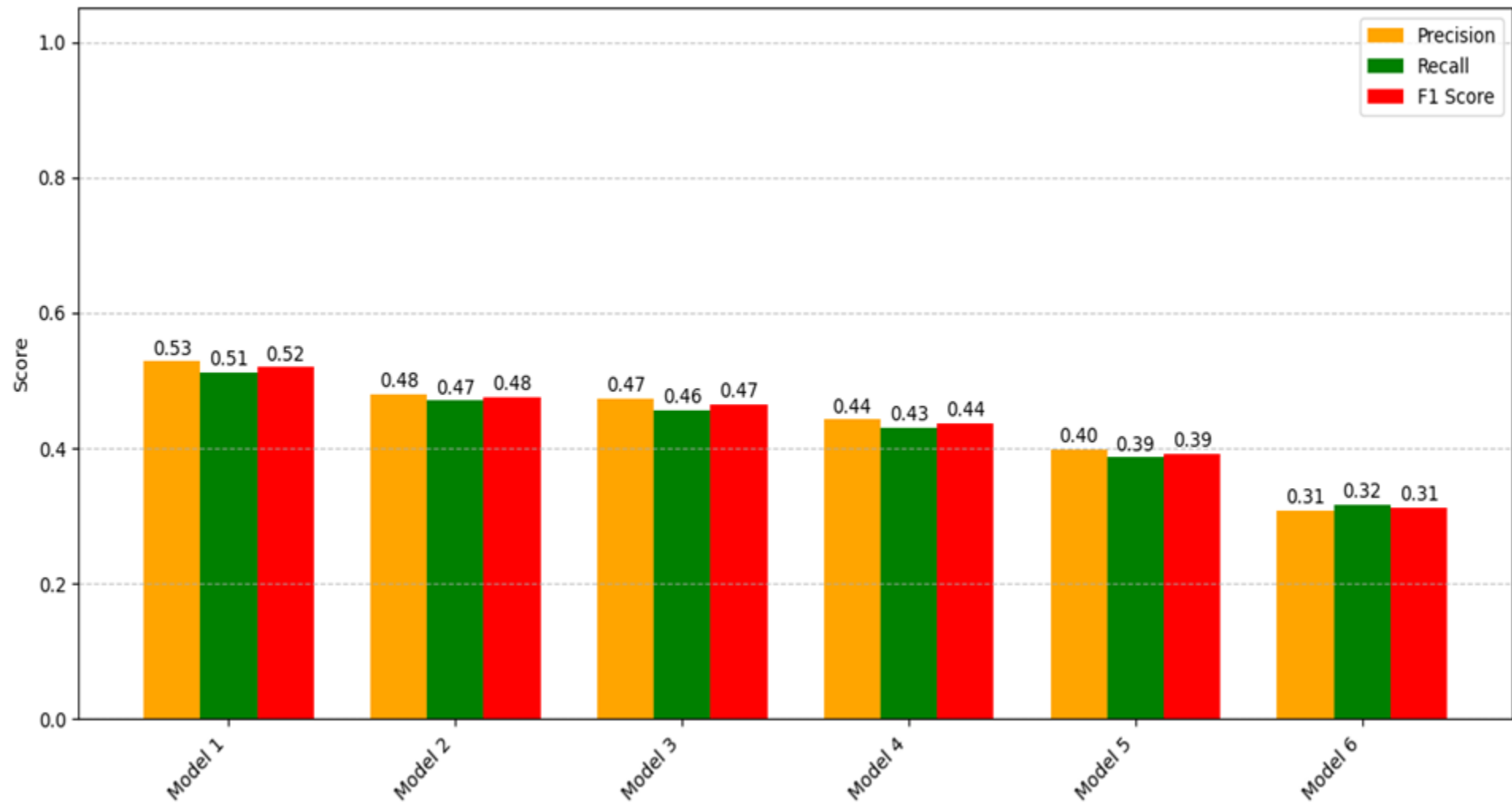
Training & Testing Loss Across Noise Levels



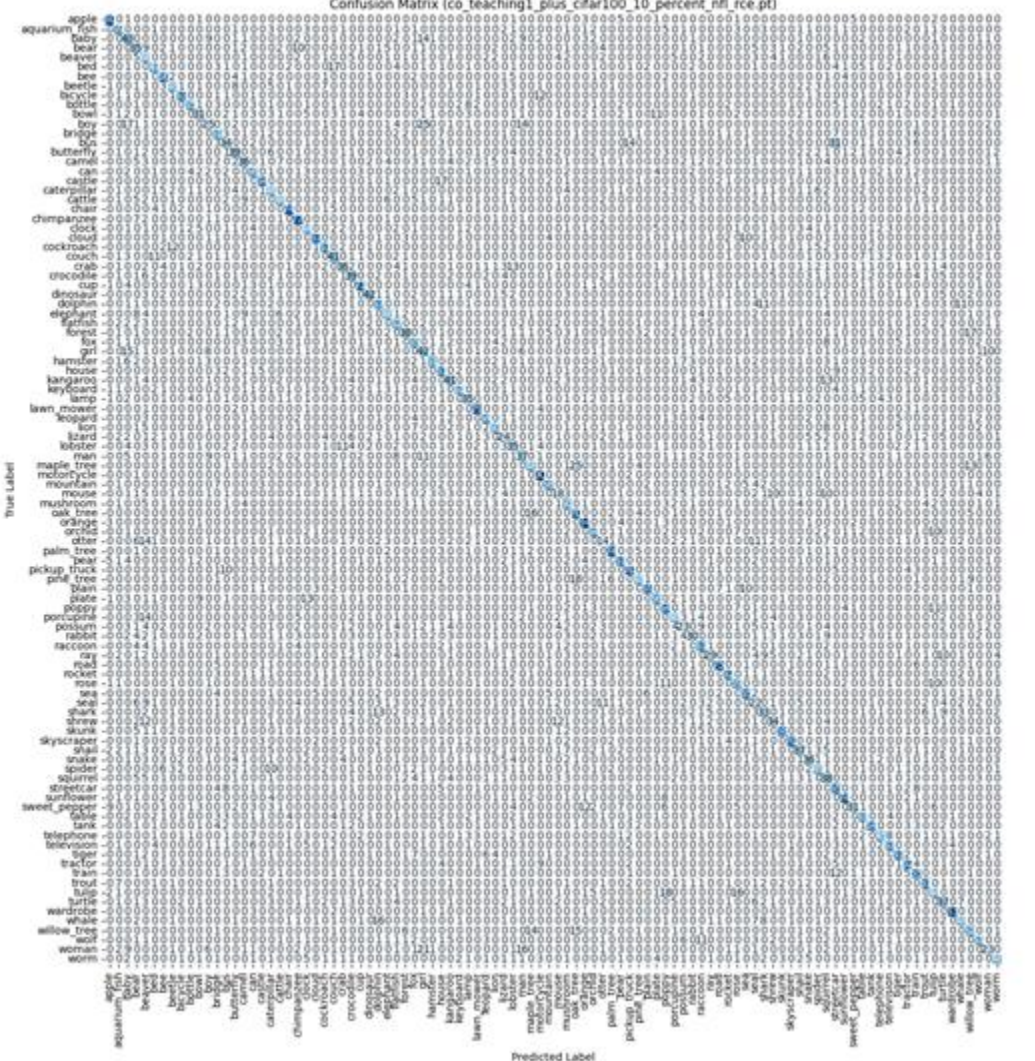
Training & Testing Accuracy Across Noise Levels



Precision, Recall, and F1 Score for Each Model



Confusion Matrix (co_teaching1_plus_cifar100_10_percent_nfl_rce.pt)



co_teaching1_plus_cifar100_10_percent_nfl_rce.pt

Total Misclassifications: 4881

- bus → streetcar (31 times)
- boy → girl (25 times)
- maple_tree → oak_tree (25 times)

co_teaching1_plus_cifar100_20_percent_nfl_rce.pt

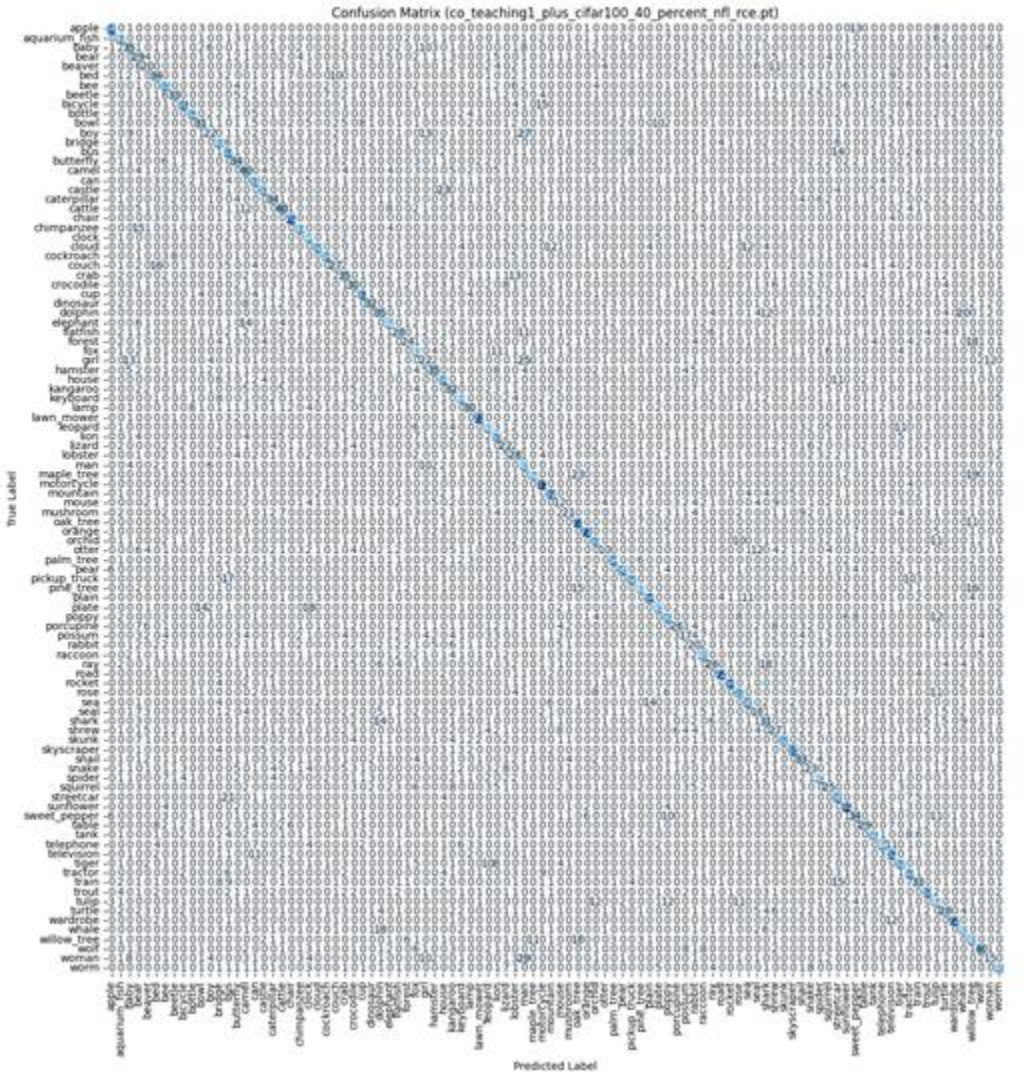
Total Misclassifications: 5296

- bus → streetcar (36 times)
- maple_tree → oak_tree (28 times)
- girl → woman (23 times)

co_teaching1_plus_cifar100_30_percent_nfl_rce.pt

Total Misclassifications: 5432

- poppy → tulip (30 times)
- woman → girl (26 times)
- maple_tree → oak_tree (25 times)



co_teaching1_plus_cifar100_40_percent_nfl_rce.pt

Total Misclassifications: 5690

- woman → man (29 times)
- boy → man (27 times)
- girl → man (25 times)

co_teaching1_plus_cifar100_50_percent_nfl_rce.pt

Total Misclassifications: 6133

- rose → poppy (28 times)
- bus → streetcar (27 times)
- maple_tree → oak_tree (25 times)

co_teaching1_plus_cifar100_60_percent_nfl_rce.pt

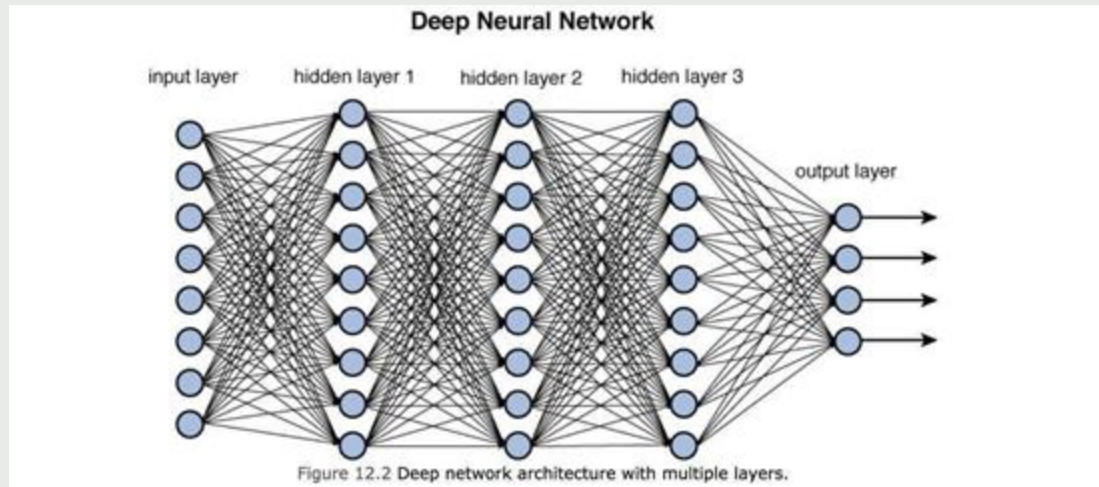
Total Misclassifications: 6835

- rose → tulip (34 times)
- dolphin → whale (33 times)
- sea → plain (25 times)

CNN (Overall)

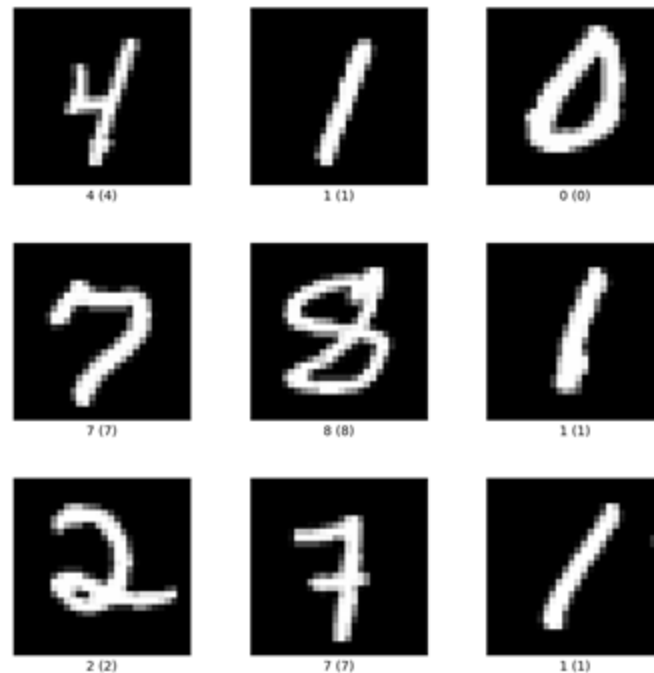
- Started off with CNN architectures to train on MNIST and CIFAR-10 datasets
- Wanted to implement co-teaching noise robust CNN and active passive loss functions to enhance the deep learning performance of CNN on MNIST and CIFAR-10
- Implemented T-Revision for the NR-CNN
- Compared Co-Taught CNN's performance with MNIST and CIFAR-10 variants using cross entropy
- Compared Co-Taught CNN's performance with CIFAR-10 Variants using NFL/RCE
- Trained Co-Taught CNN with CIFAR-100 using NFL/RCE
- Compared Co-Taught CNN performance with CIFAR-100 variants using NFL+RCE

Deep Neural Networks (DNN)

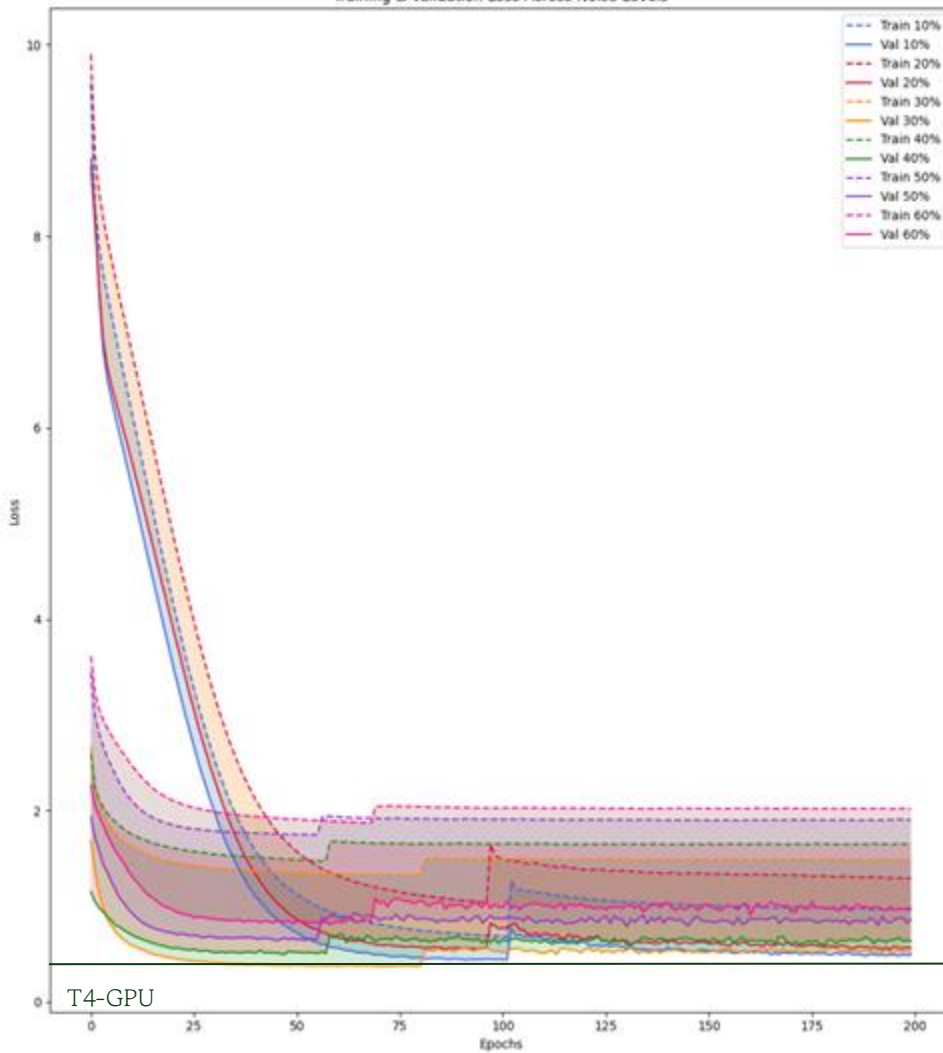


- Multiple layers of neurons detect patterns and deeper layers learn complex features for classification
- Full connections -> overfitting risk and high complexity
- Good for mapping, but not extracting features

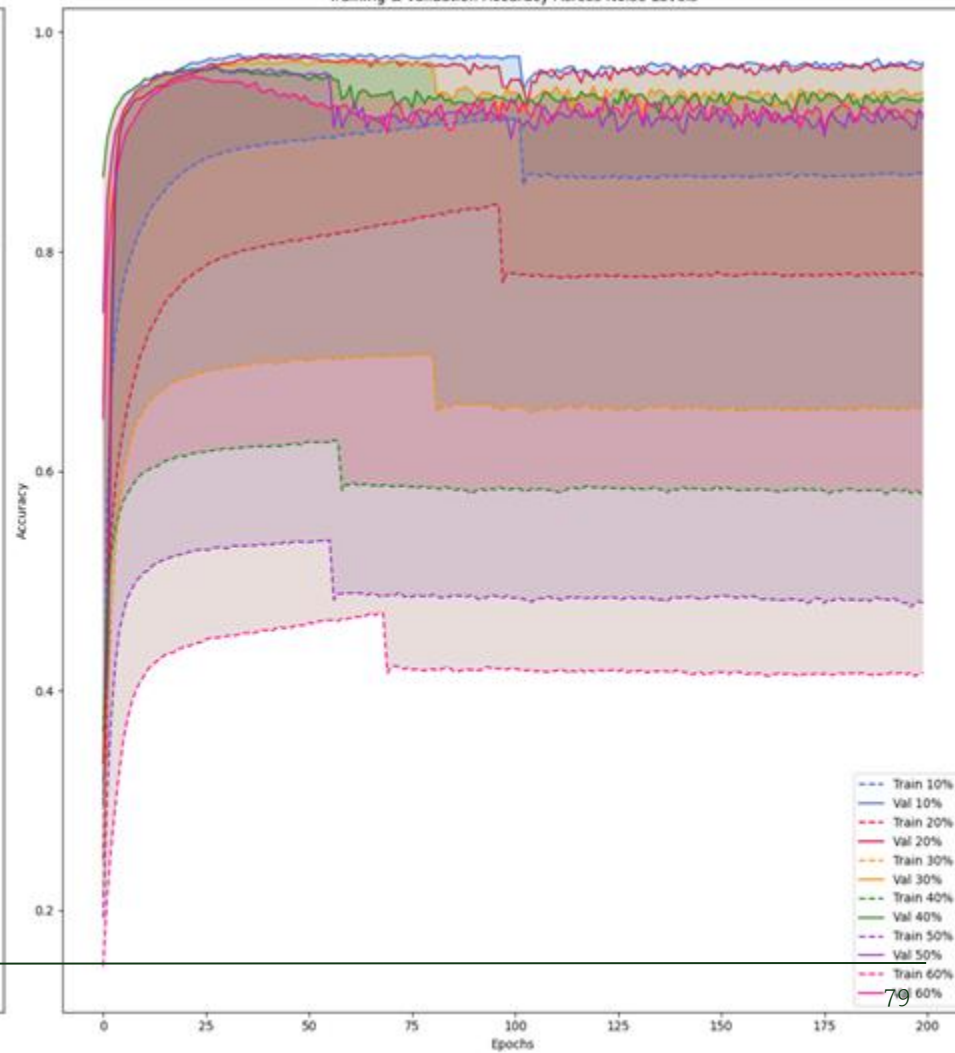
Comparing DNN Performance with MNIST Variants



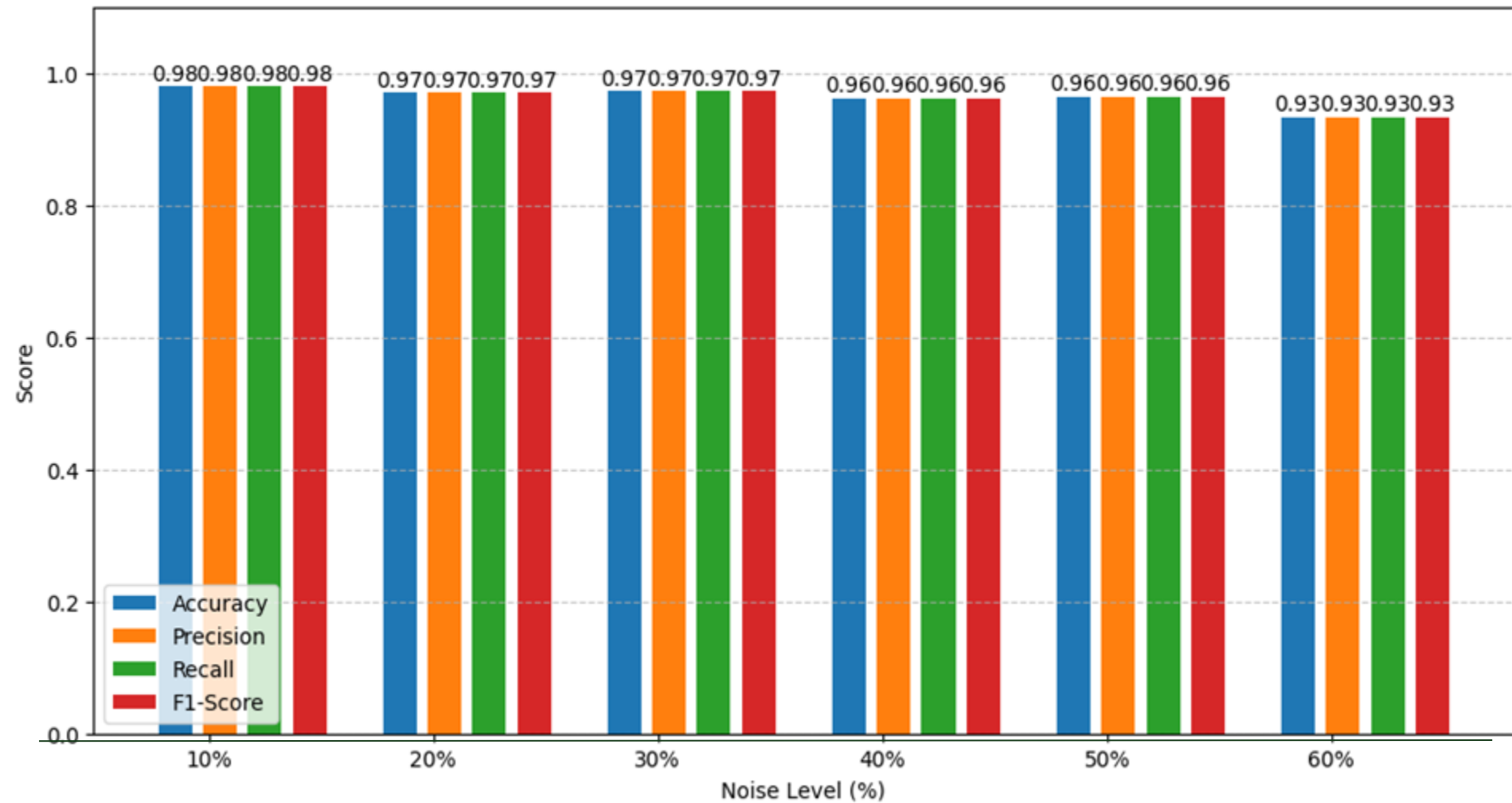
Training & Validation Loss Across Noise Levels



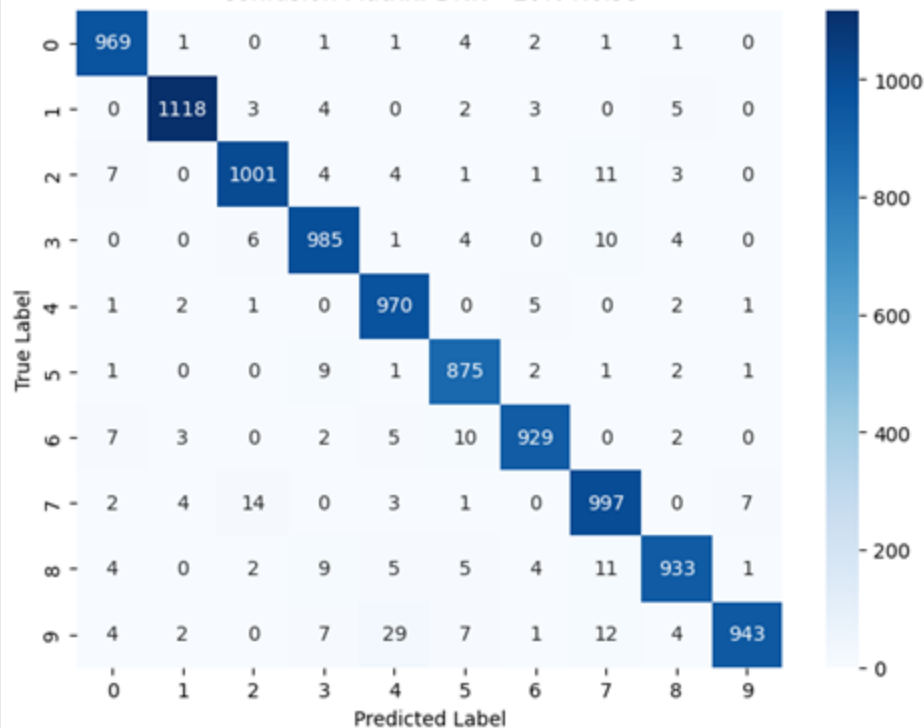
Training & Validation Accuracy Across Noise Levels



Model Performance Metrics Across Noise Levels



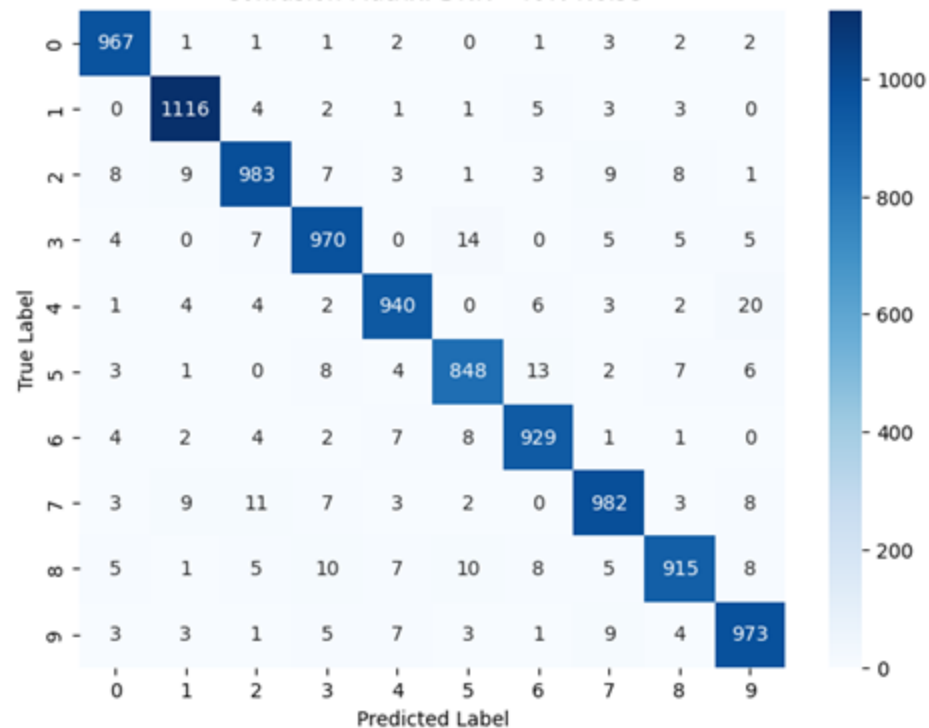
Confusion Matrix: DNN - 10% Noise



DNN - 10% Noise - Most Misclassified Classes:

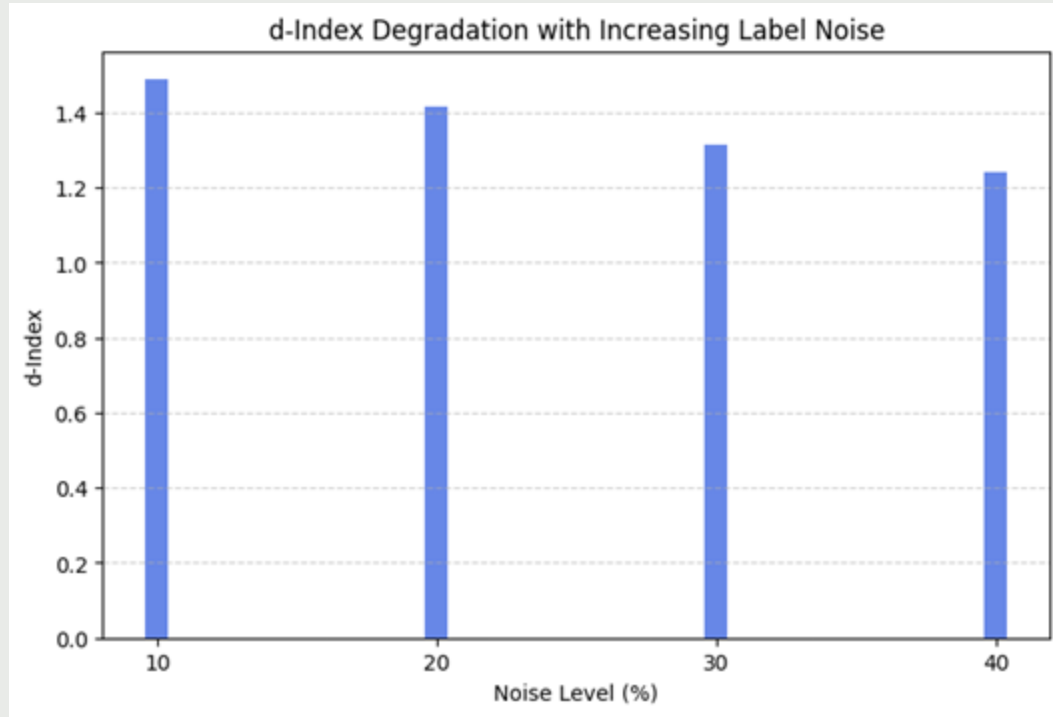
- 9 → 4 (29 times)
- 7 → 2 (14 times)
- 9 → 7 (12 times)

Confusion Matrix: DNN - 40% Noise



DNN - 40% Noise - Most Misclassified Classes:

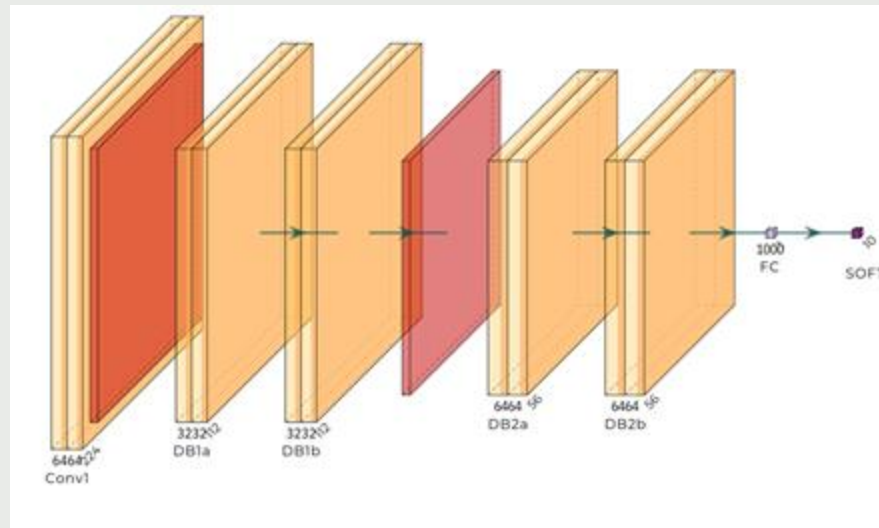
- 4 → 9 (20 times)
- 3 → 5 (14 times)
- 5 → 6 (13 times)



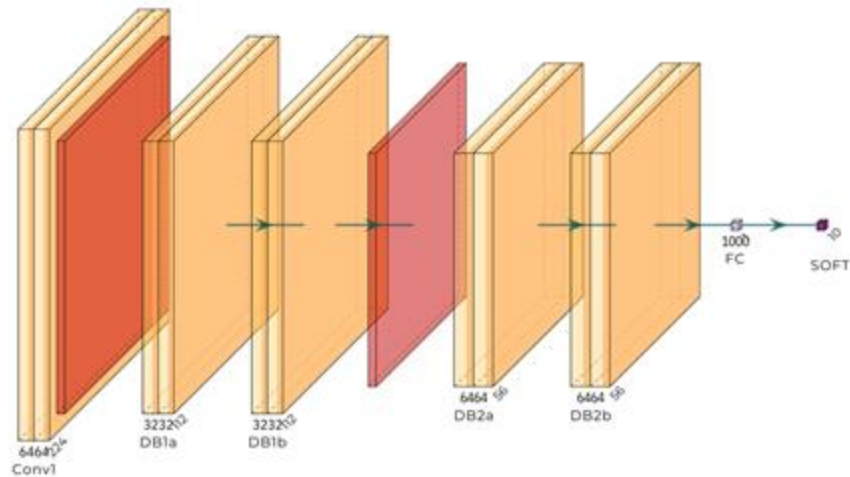
Diagnostic Index

DenseNet

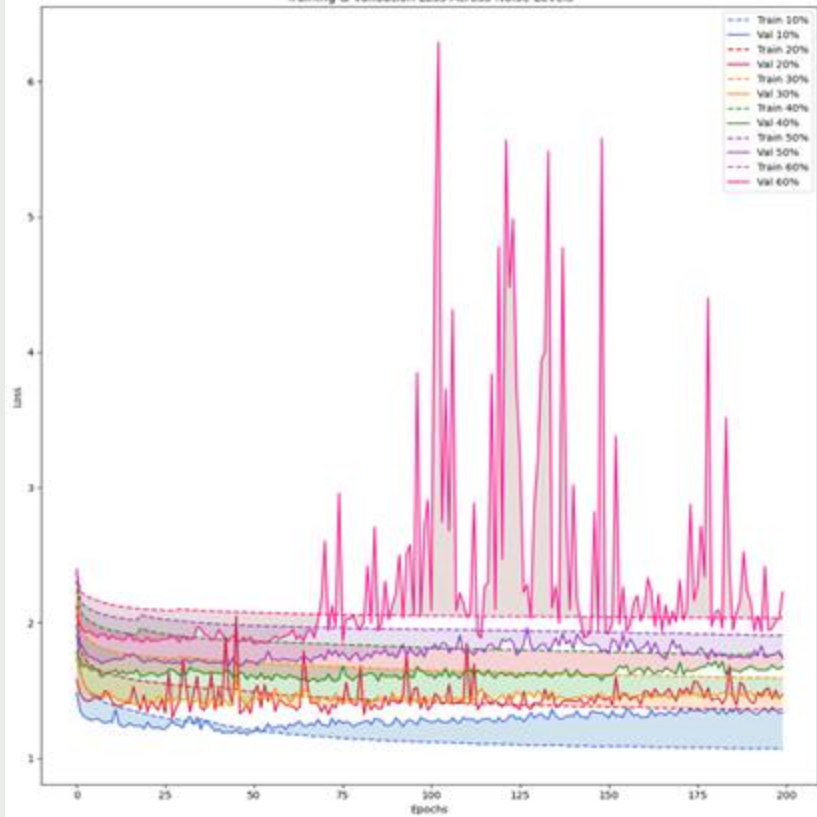
- DenseNet is a feed-forward convolutional neural network architecture
- DenseNet-169 is a DenseNet variant that is comprised of 169 layers with densely connected blocks, designed to maximize feature reuse while maintaining efficient parameter use
- The DenseNet architecture is composed of transition layers and dense blocks. In the dense blocks, each convolutional layer is linked to every other layer within the block, using the output of each layer as input to all subsequent layers



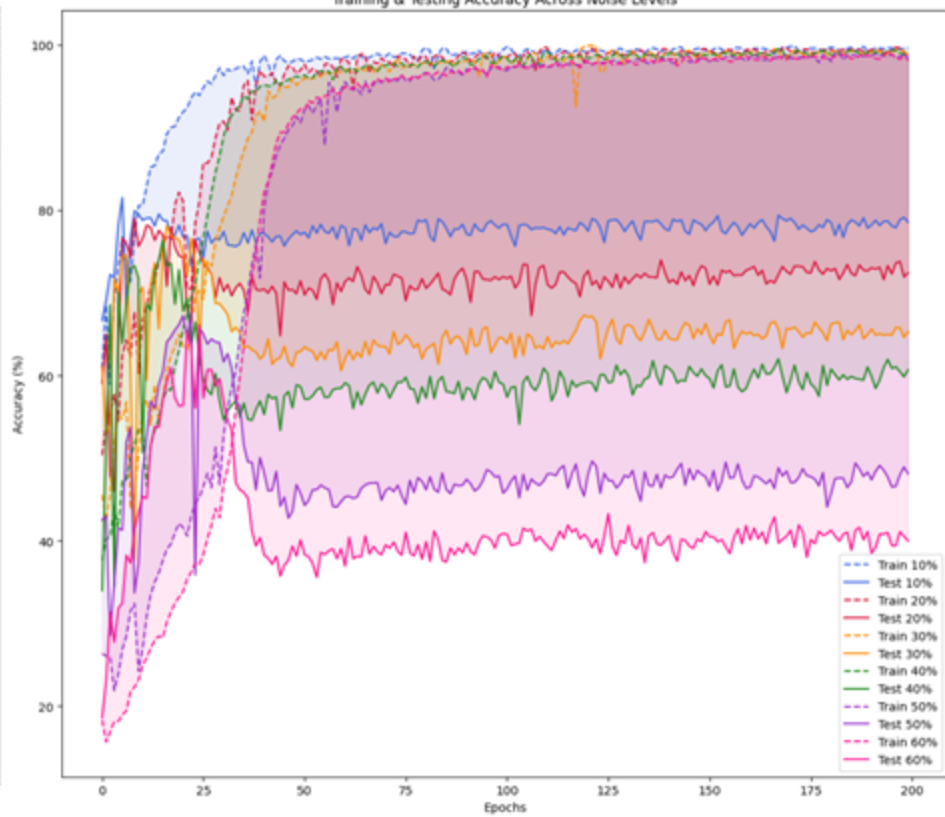
DenseNet Performance with CIFAR-10 Variants



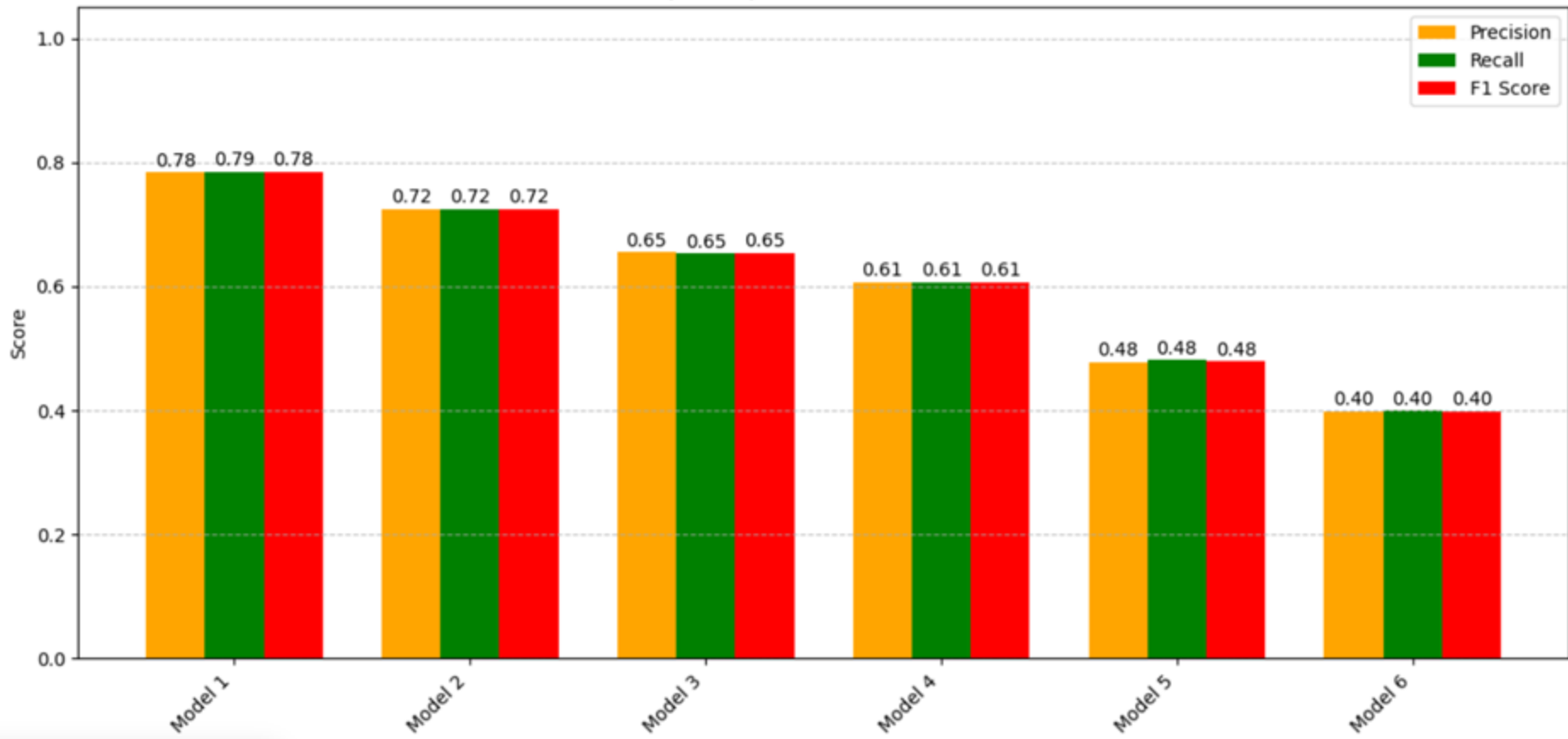
Training & Validation Loss Across Noise Levels

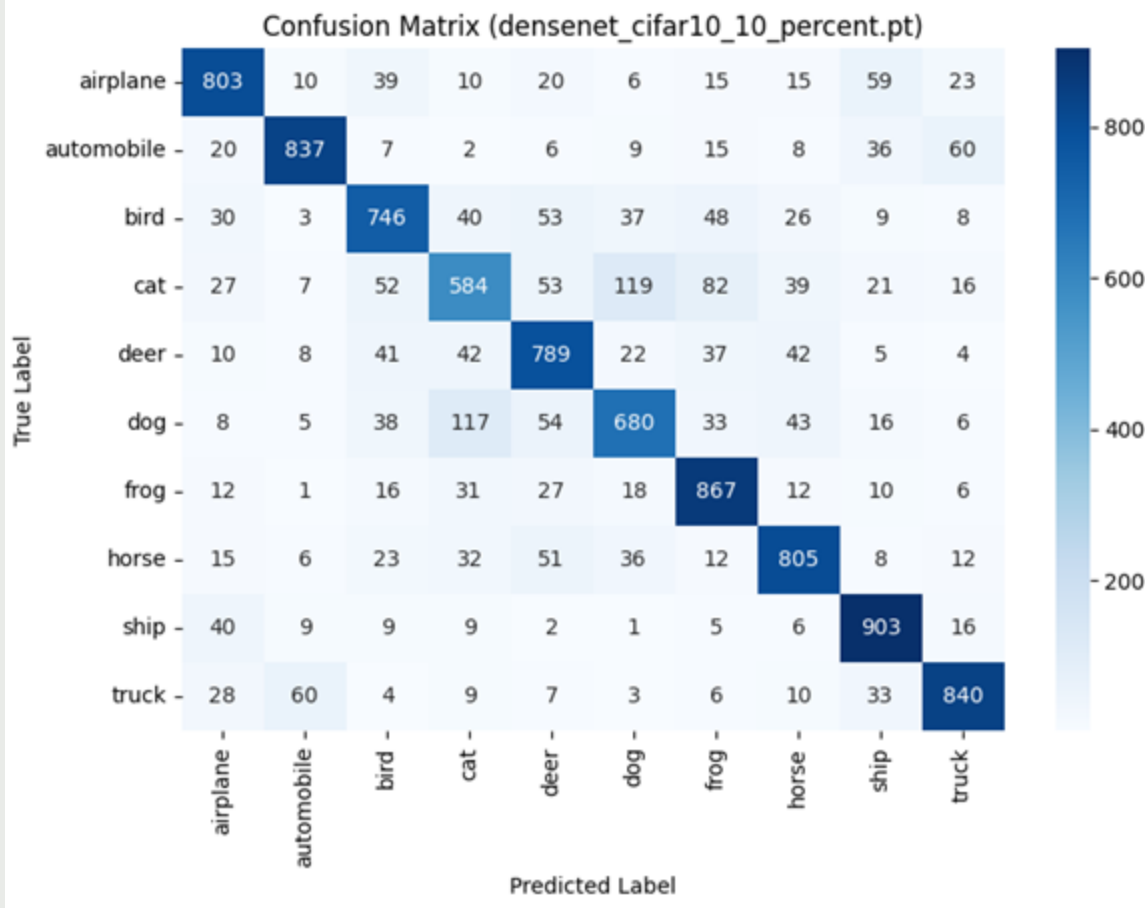


Training & Testing Accuracy Across Noise Levels

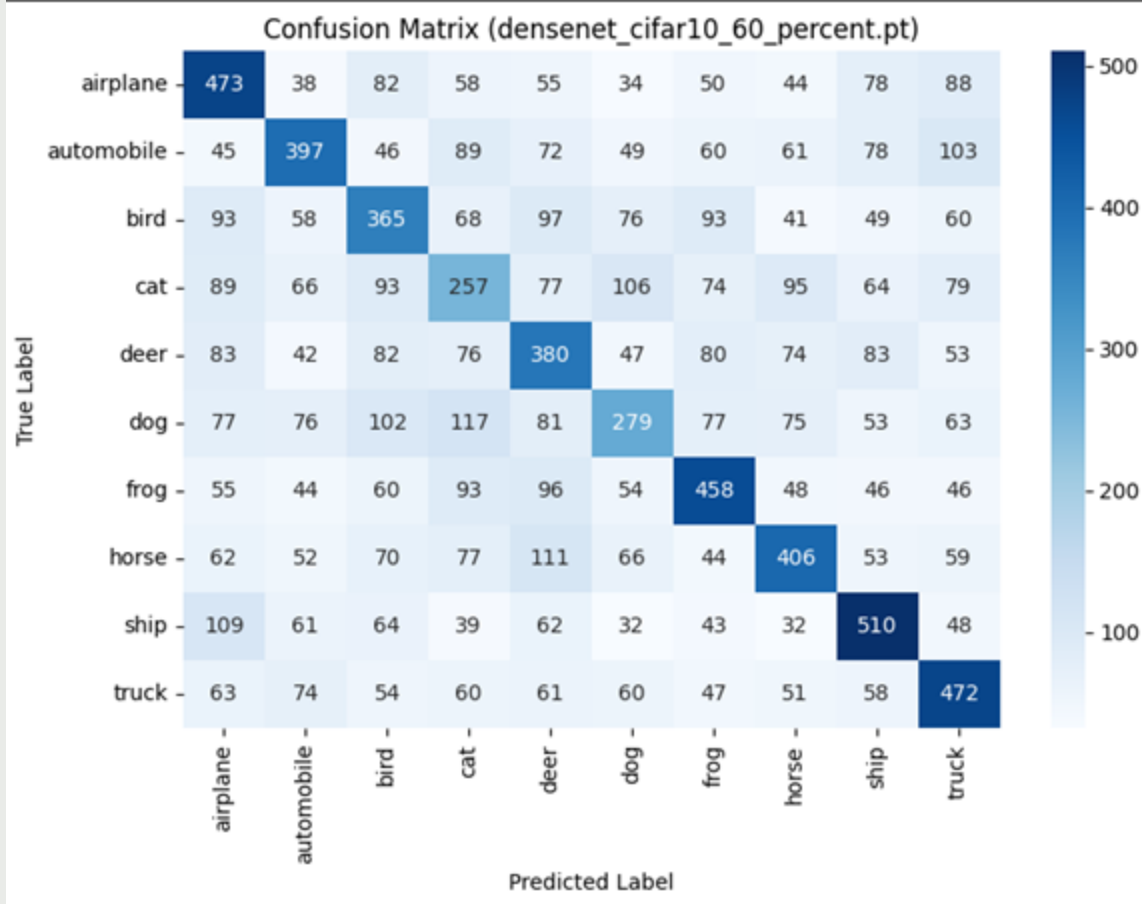


Precision, Recall, and F1 Score for Each Model



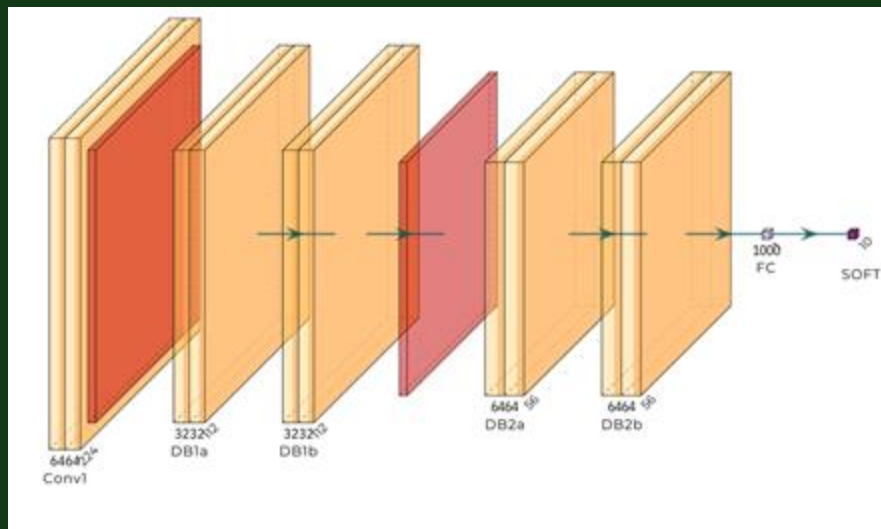


```
densenet_cifar10_10_percent.pt - Most Misclassified Classes (Noise Level: Unknown%):  
Total Misclassifications: {np.int64(2146)}  
- cat -> dog (119 times)  
- dog -> cat (117 times)  
- cat -> frog (82 times)
```

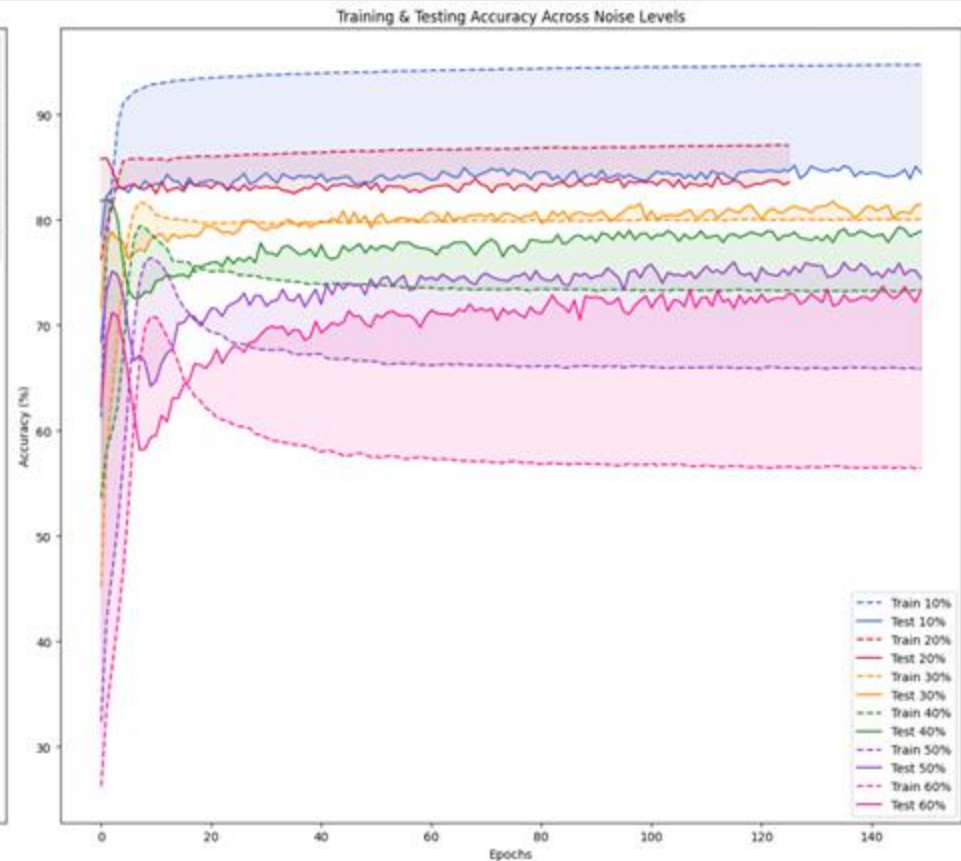
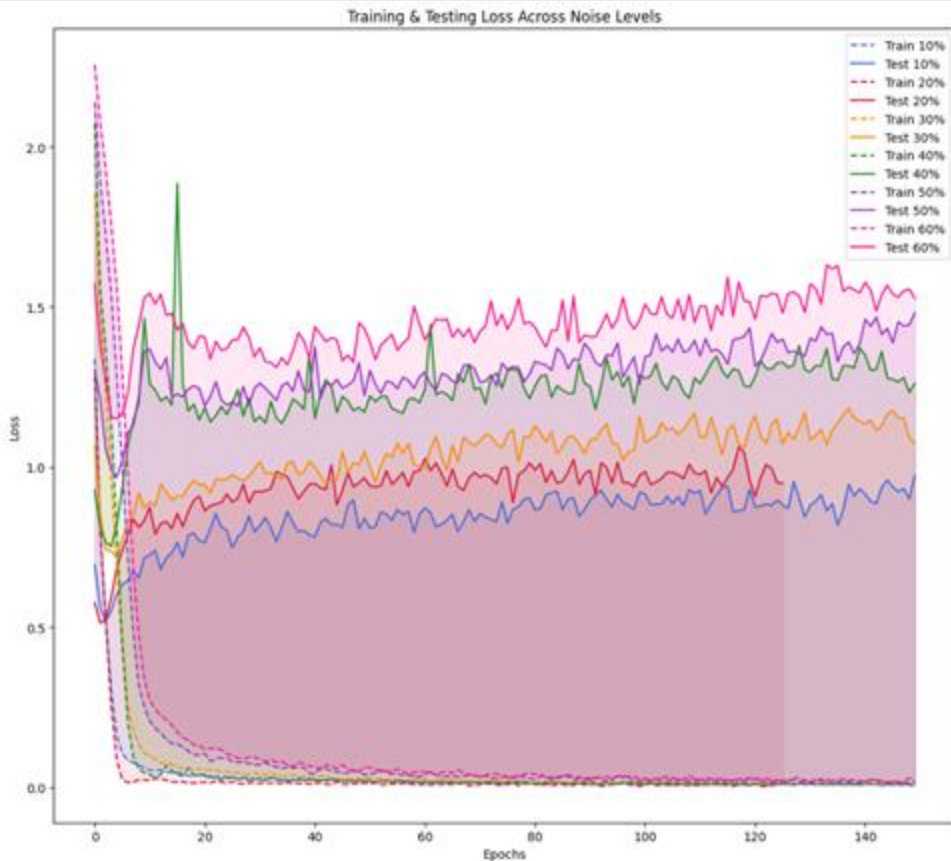



```
densenet_cifar10_60_percent.pt - Most Misclassified Classes (Noise Level: Unknown%):  
Total Misclassifications: {np.int64(6003)}  
- dog → cat (117 times)  
- horse → deer (111 times)  
- ship → airplane (109 times)
```

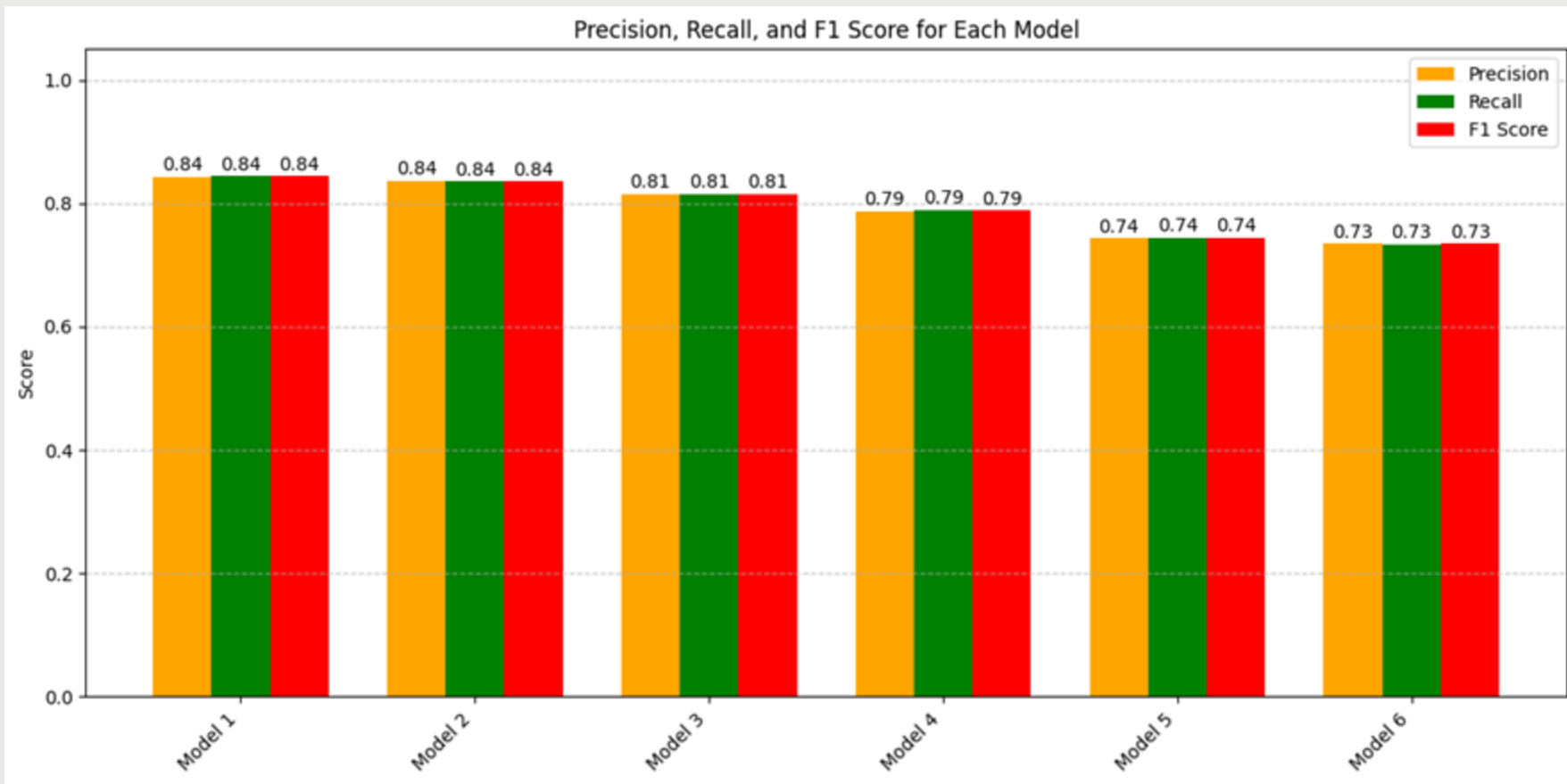
Co-Teaching DenseNet Performance with CIFAR-10 and CIFAR-100 Variants



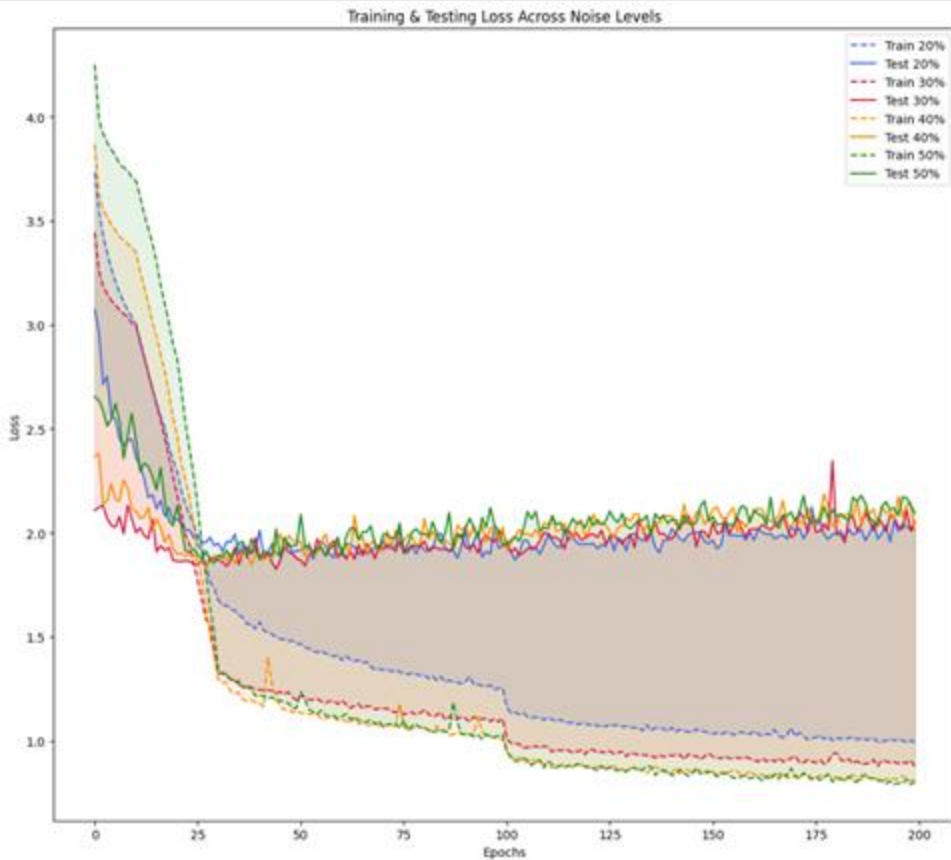
Co Teaching: DenseNet 169 on CIFAR-10



Densenet with SYMMETRICAL CIFAR-10

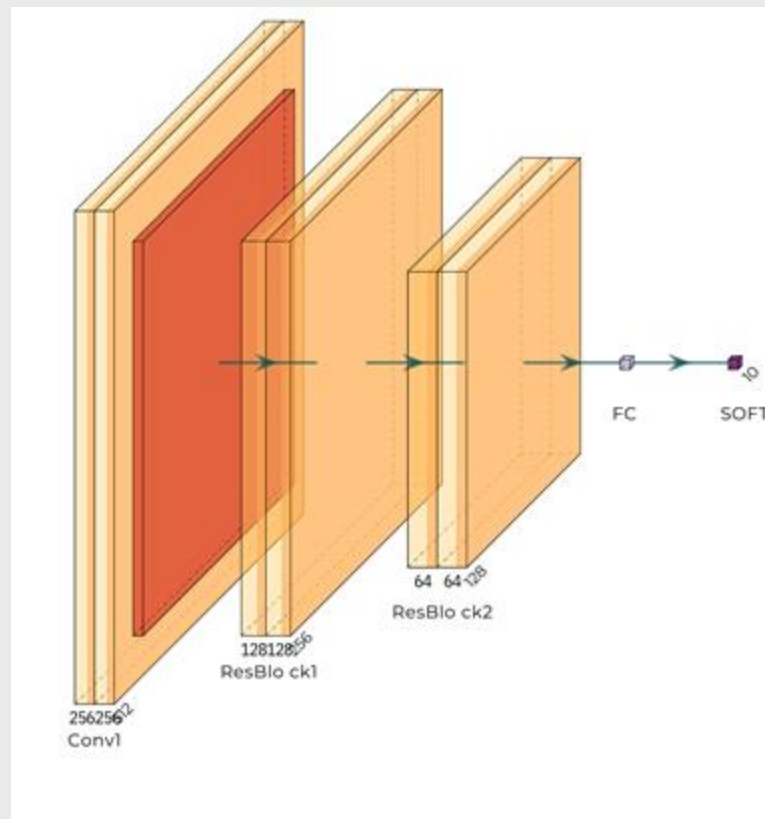


Co Teaching: DenseNet 169 on CIFAR-100

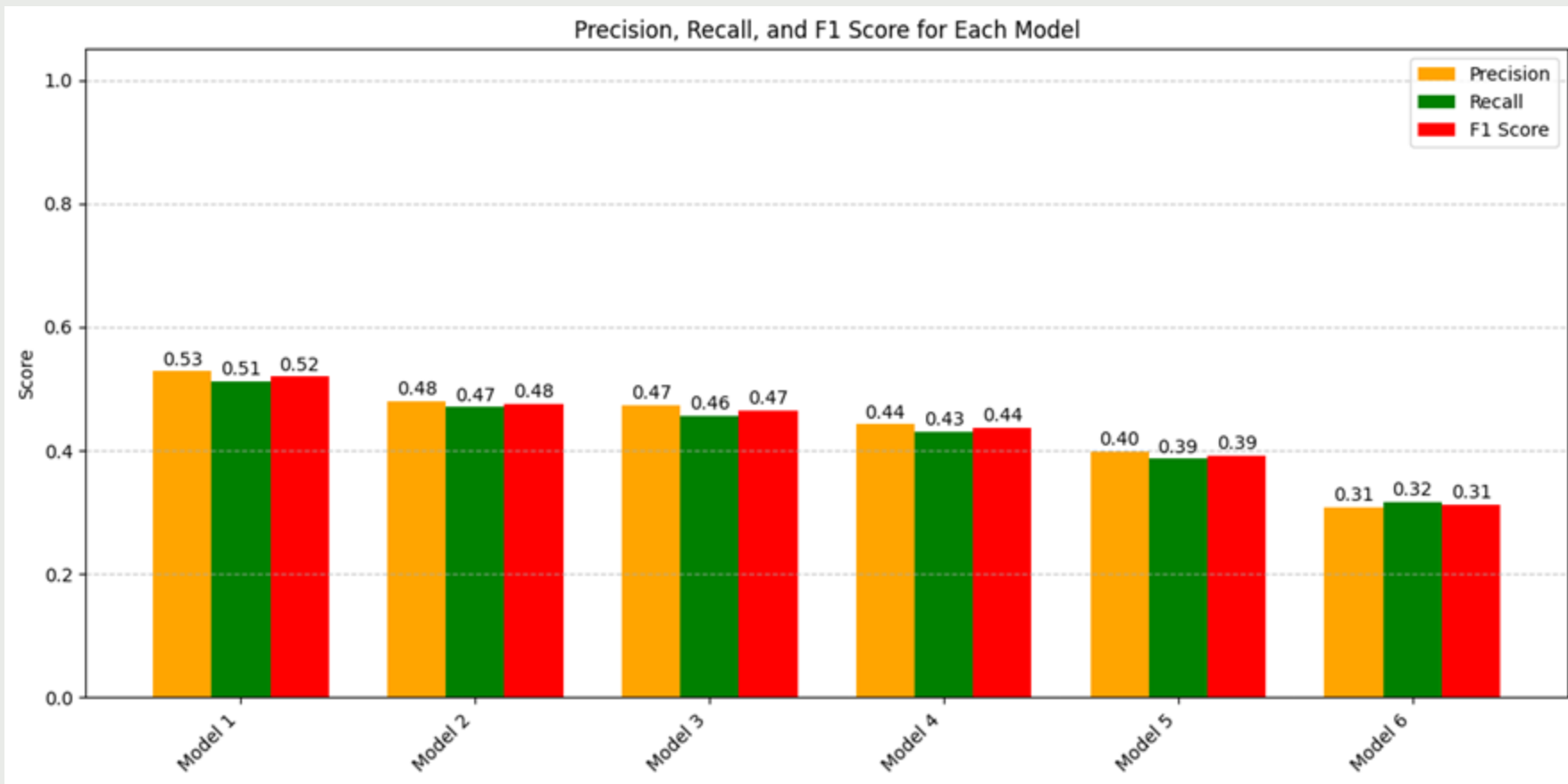


ResNet

- We implemented ResNet-18 and ResNet-50 variants in our research
 - The model we implement is a modified version of the original architecture with a design meant to balance computational efficiency and performance
- Derived from the design of the modern CNN and are frequently implemented in various types of tasks related to computer vision and image classification
- Special due to its objective of attempting to mitigate the issue of vanishing gradients, a common liability in the world of deep learning
- The residual block is the core piece of this model, which consists of convolutional layers, which then go through batch normalization and ReLU activation with several gradients bypassing straight to the output layer

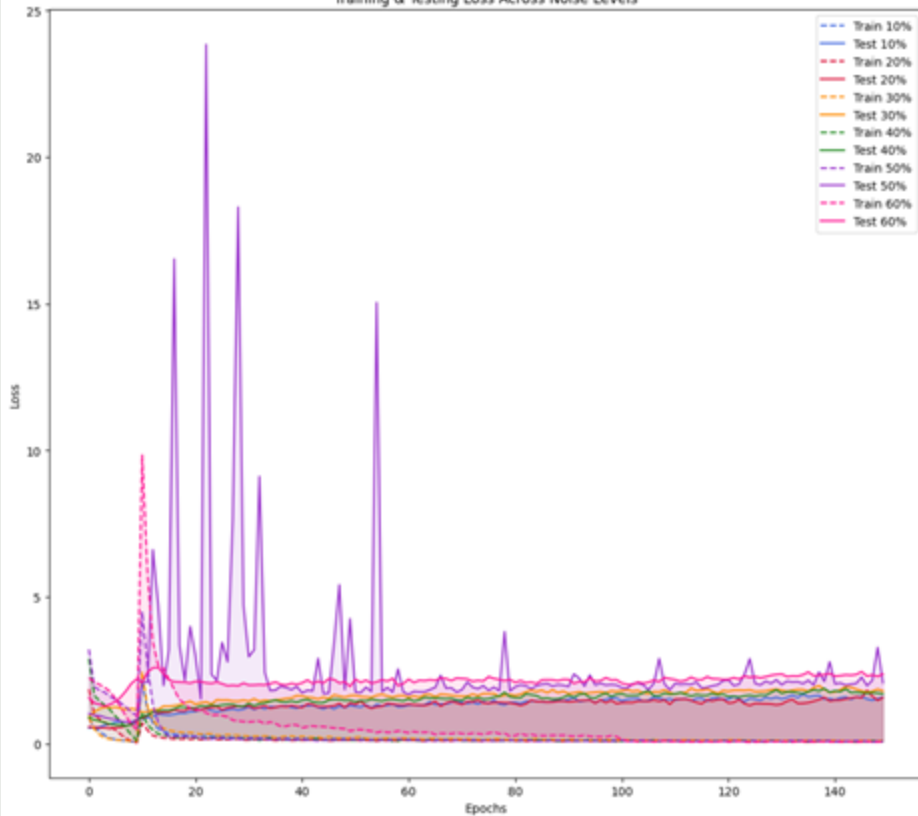


Densenet with SYMMETRICAL CIFAR-100

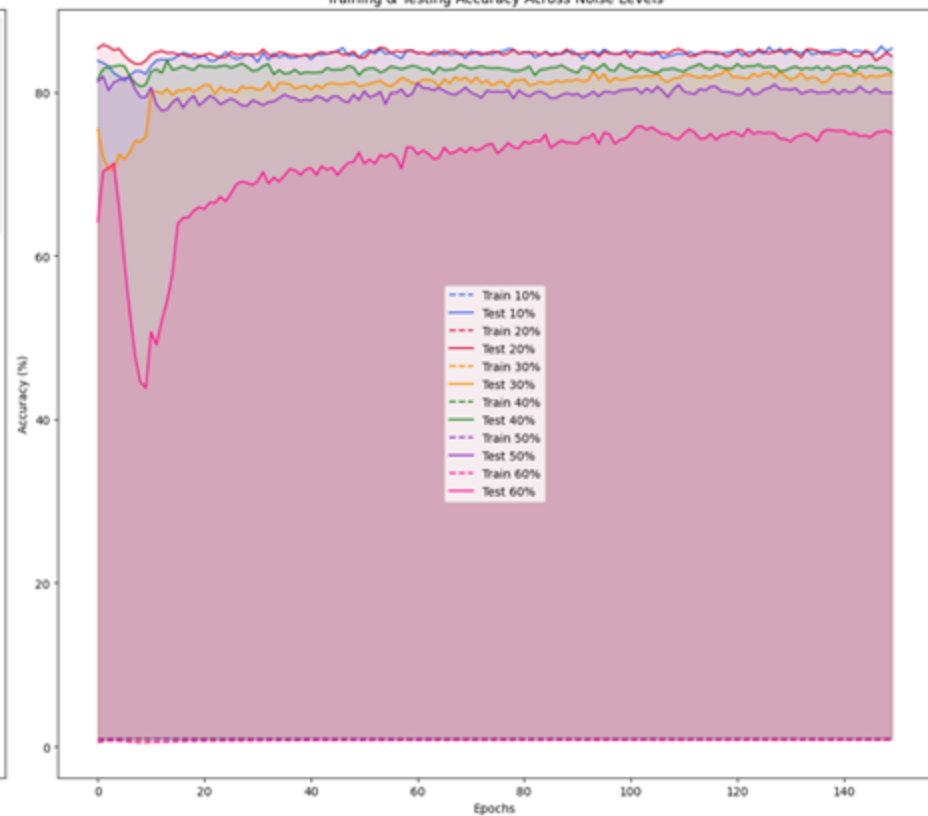


Densenet with NFL+RCE CIFAR-10

Training & Testing Loss Across Noise Levels

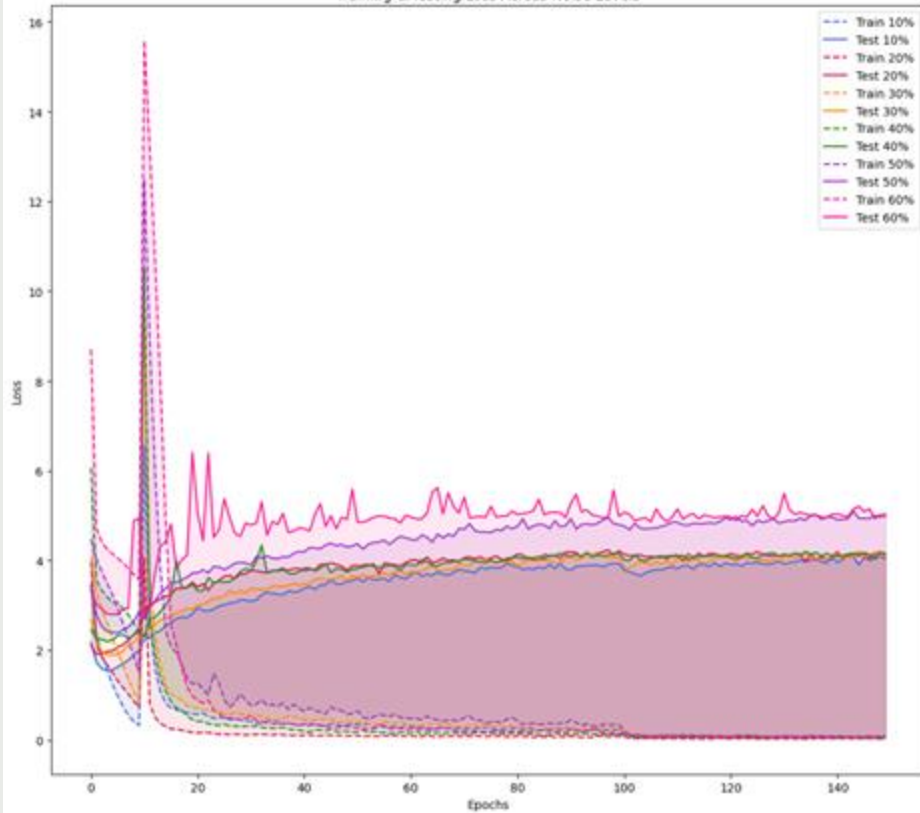


Training & Testing Accuracy Across Noise Levels

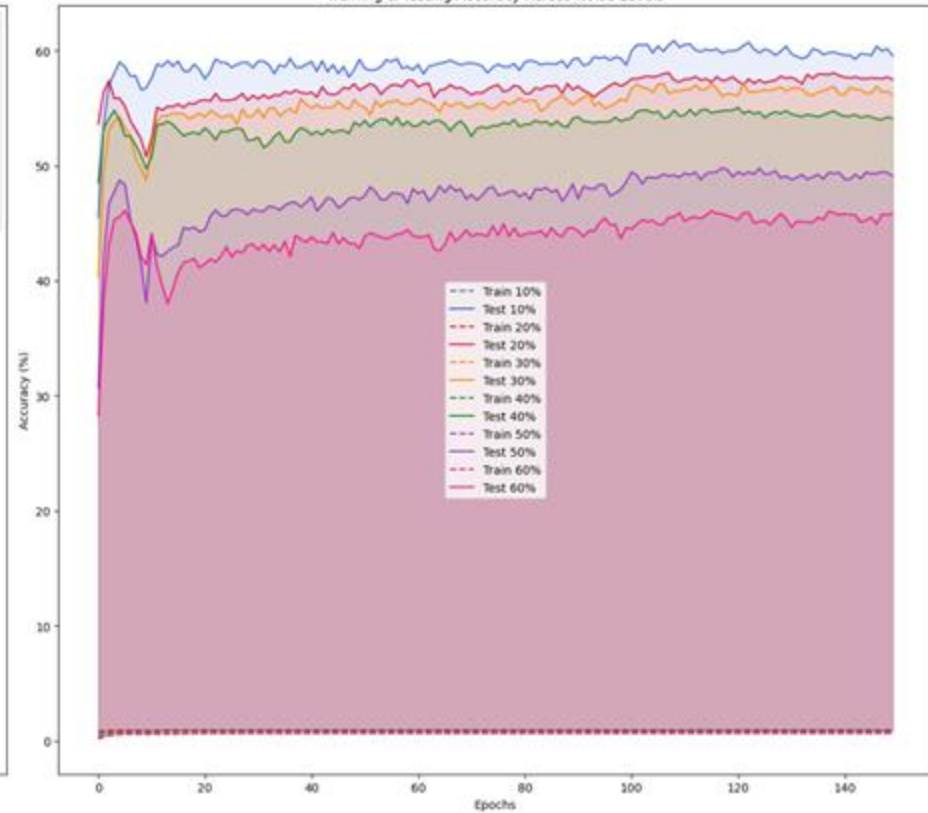


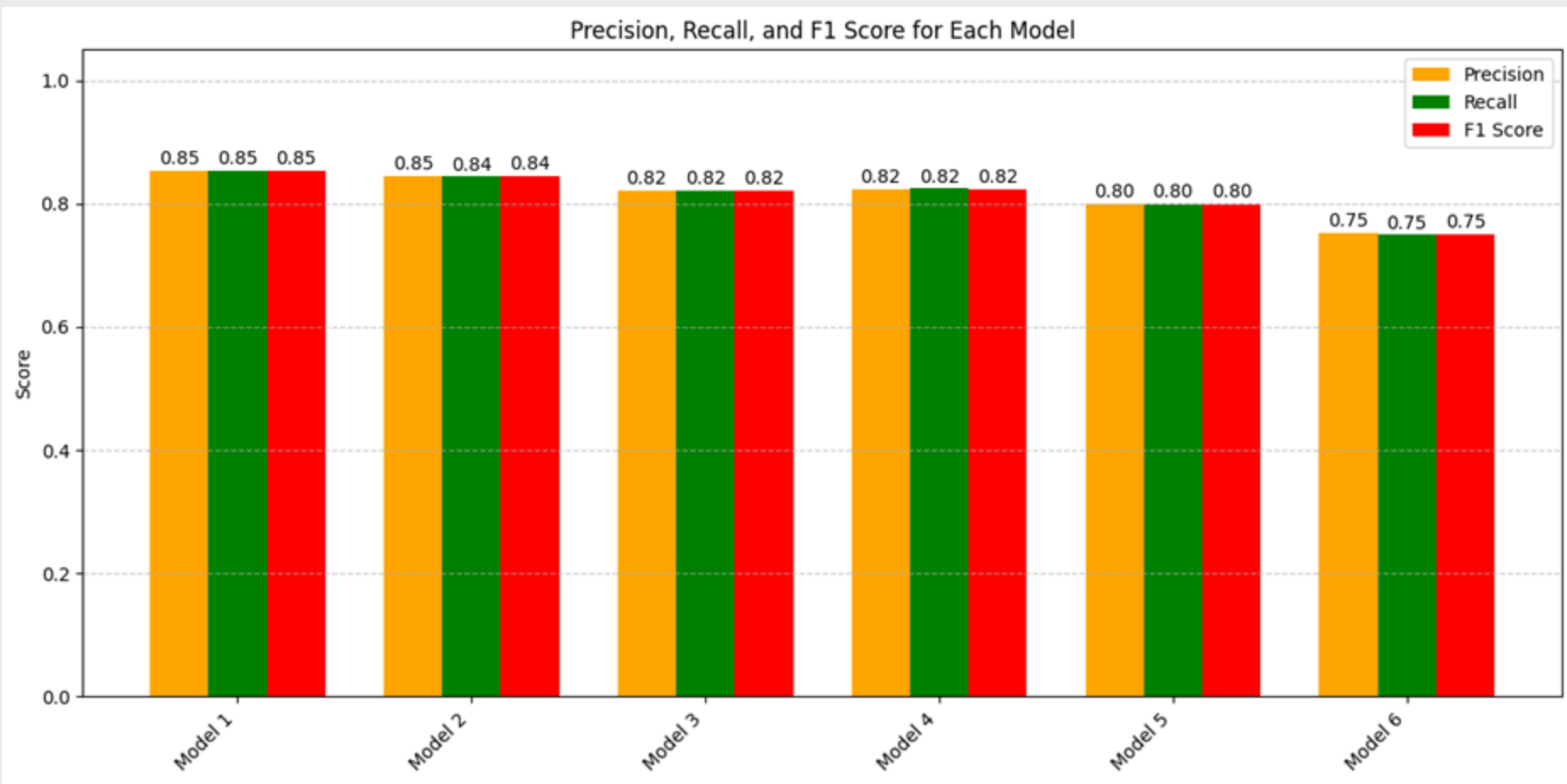
Densenet with NFL+RCE CIFAR-100

Training & Testing Loss Across Noise Levels

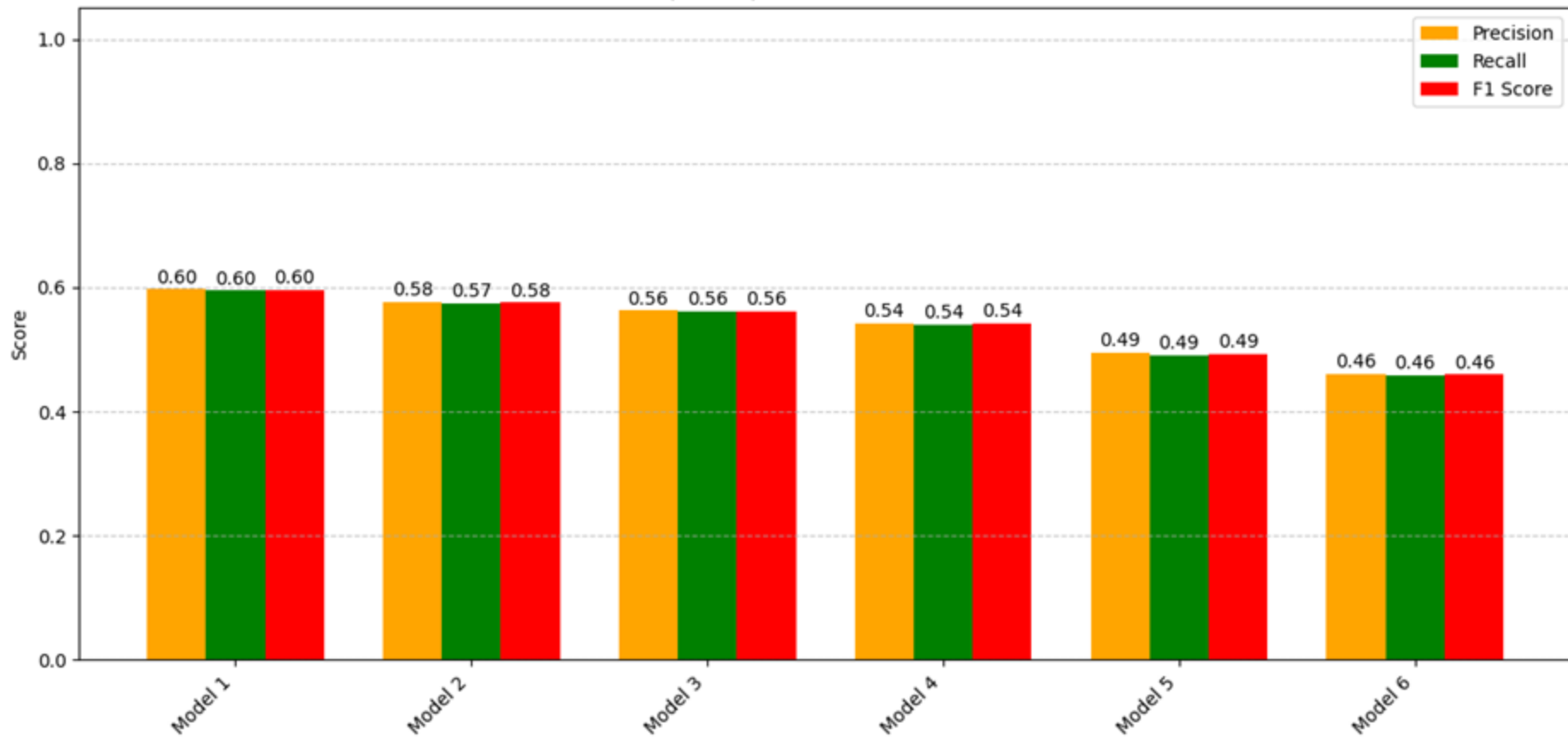


Training & Testing Accuracy Across Noise Levels





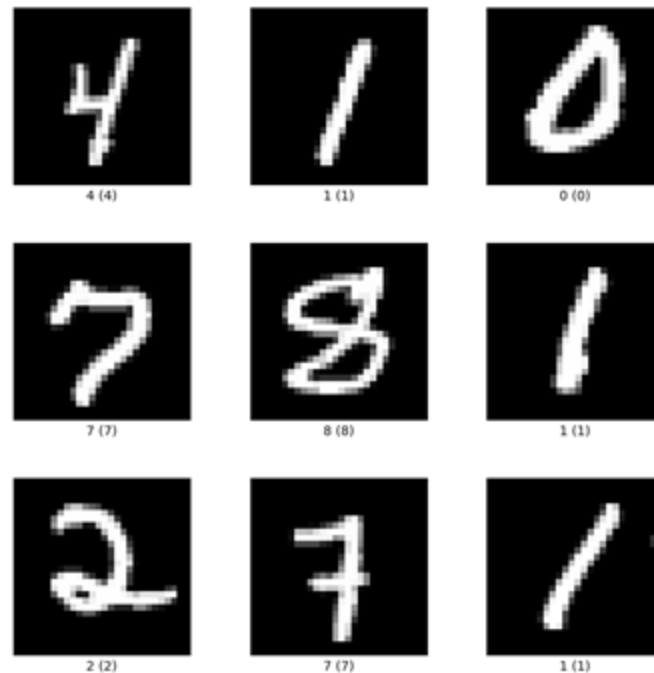
Precision, Recall, and F1 Score for Each Model



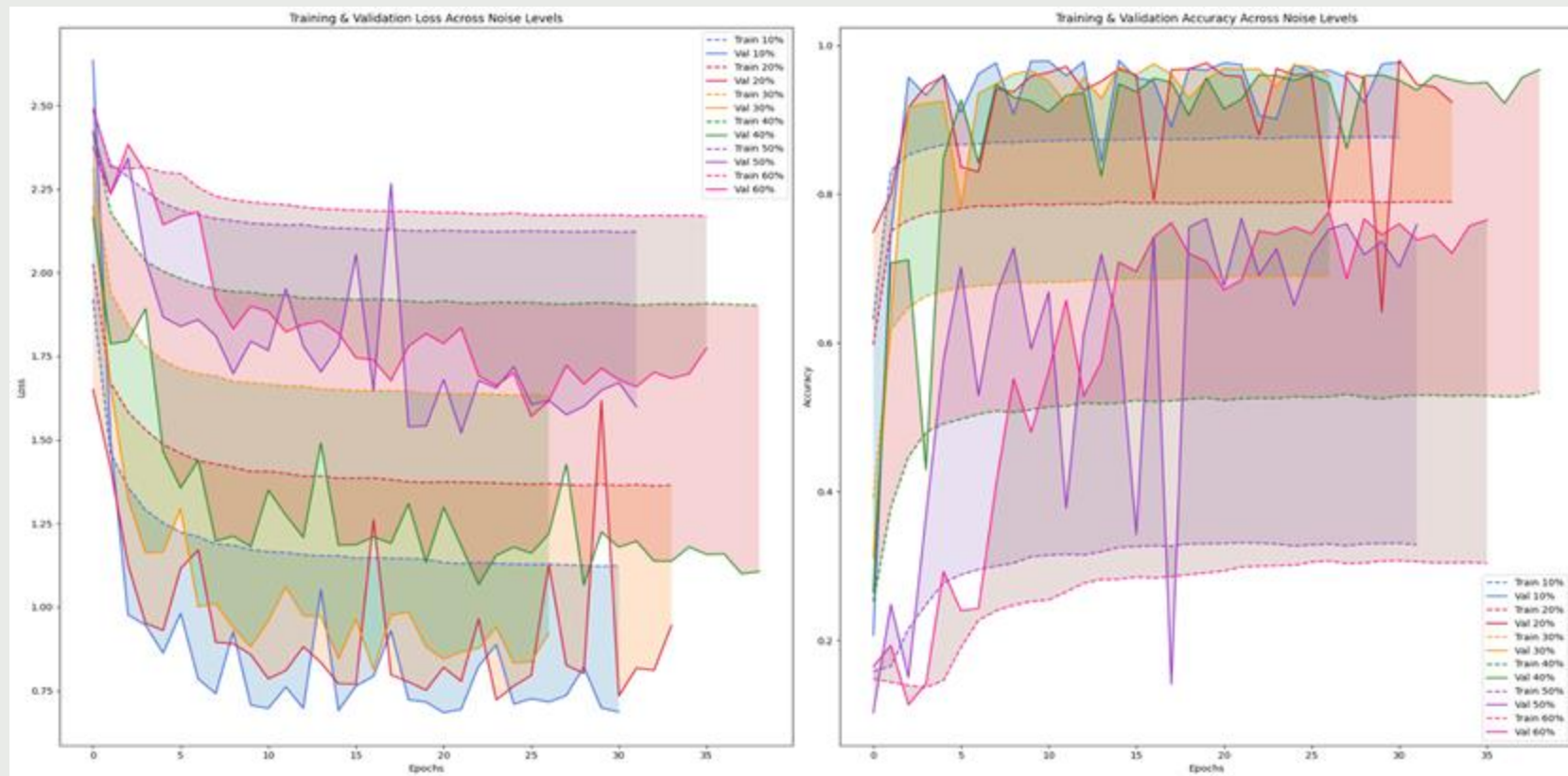
DenseNet (Overall)

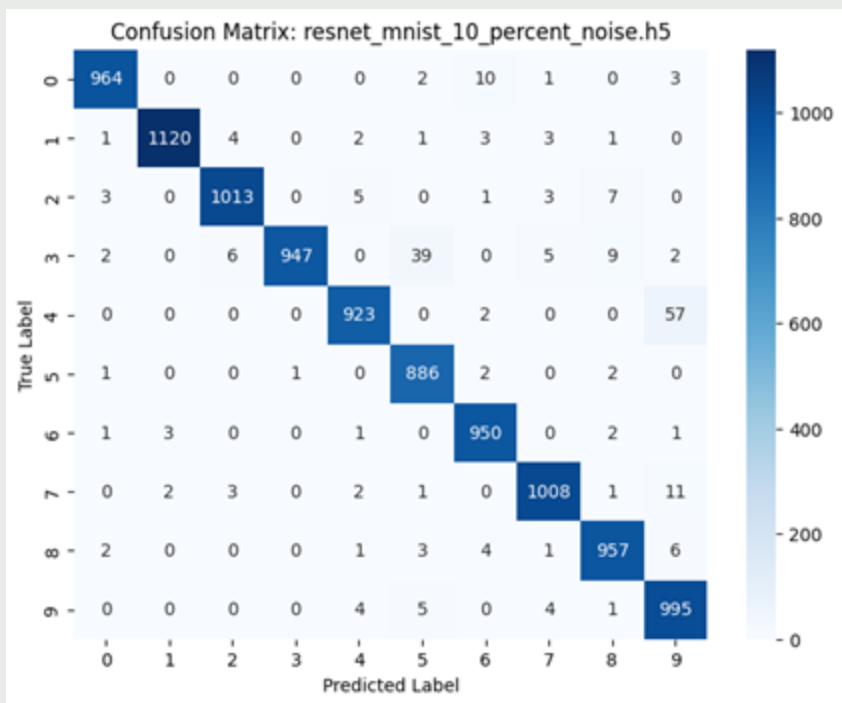
- Trained DenseNet with CIFAR-10 and CIFAR-100
- Trained Co-Taught DenseNet with CIFAR-10 and CIFAR-100
- Trained Co-Taught DenseNet using Cross Entropy with CIFAR-10 and CIFAR-100
- Trained Co-Taught DenseNet using Normalized Focal Loss + Reverse Cross Entropy (NFL + RCE) with CIFAR-10 and CIFAR-100

Comparing ResNet Performance with MNIST Variants

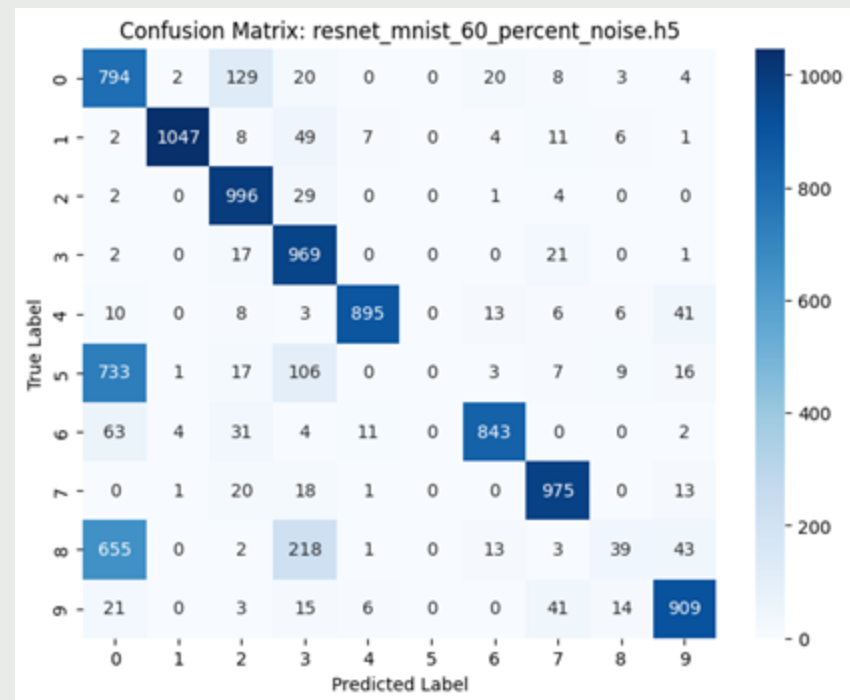


MNIST with Label Corruption 10%-60%

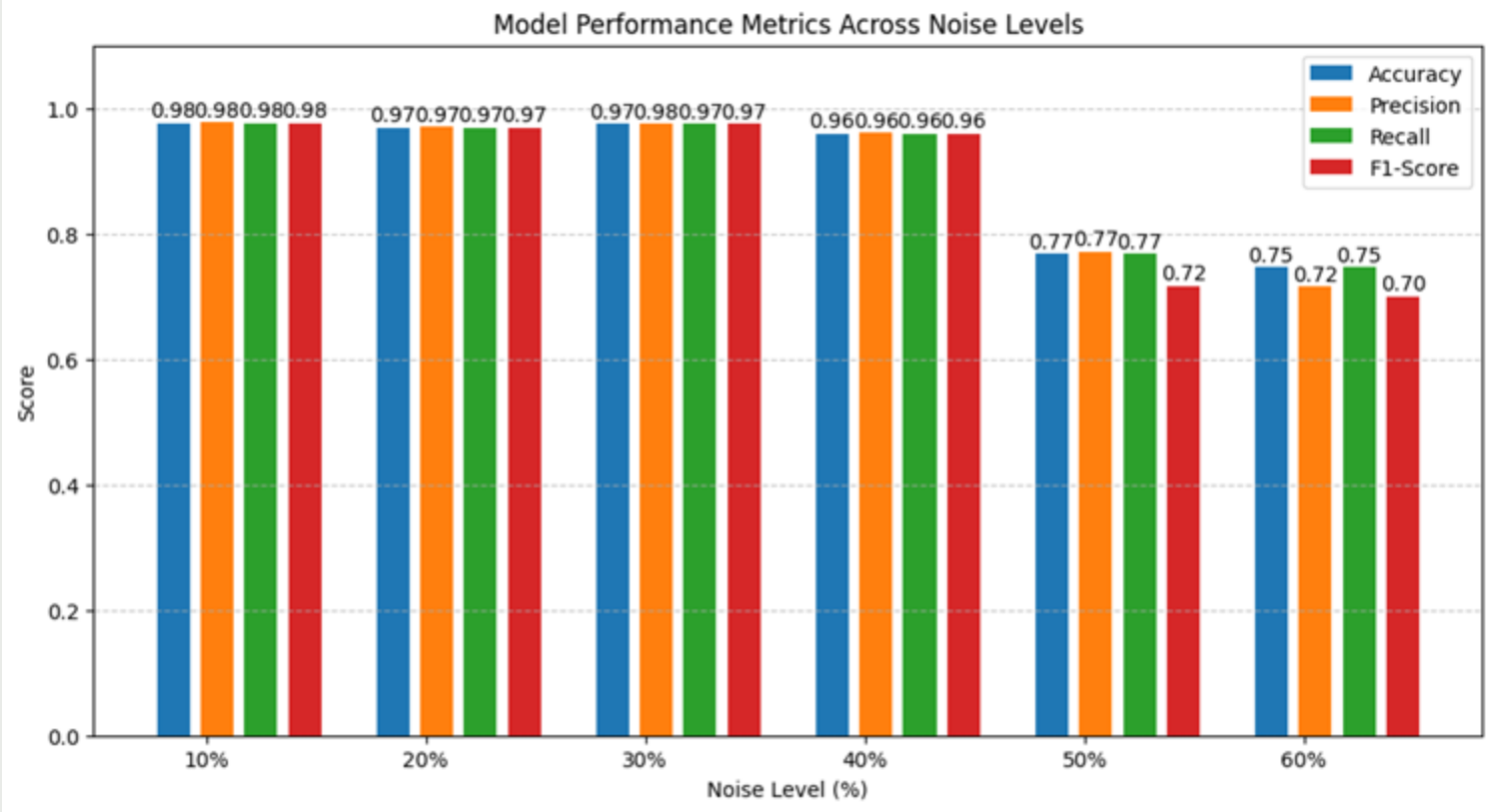




```
resnet_mnist_10_percent_noise.h5 - Most Misclassified Classes:**
- 4 → 9 (57 times)
- 3 → 5 (39 times)
- 7 → 9 (11 times)
```



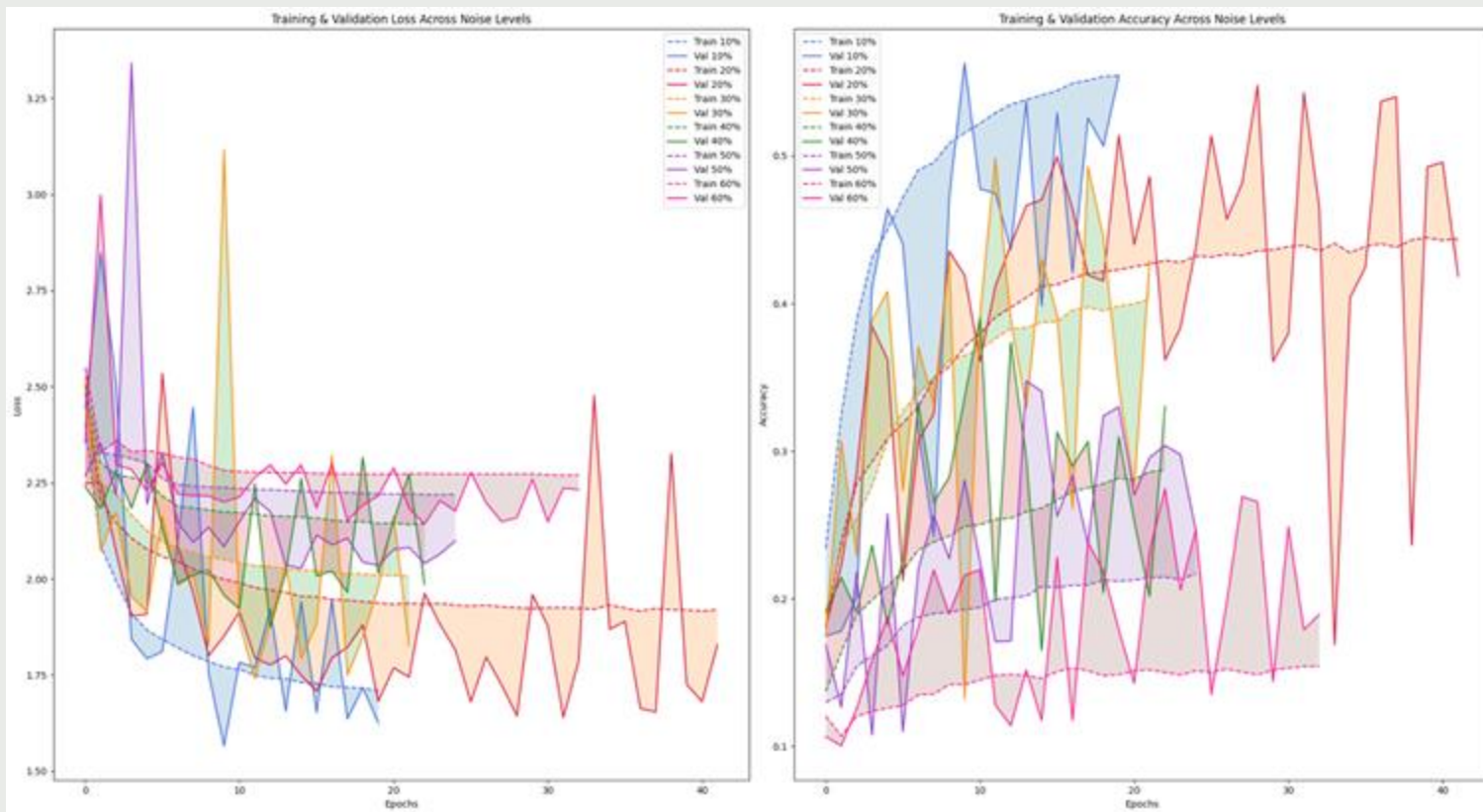
```
resnet_mnist_60_percent_noise.h5 - Most Misclassified Classes:**
- 5 → 0 (733 times)
- 8 → 0 (655 times)
- 8 → 3 (218 times)
```

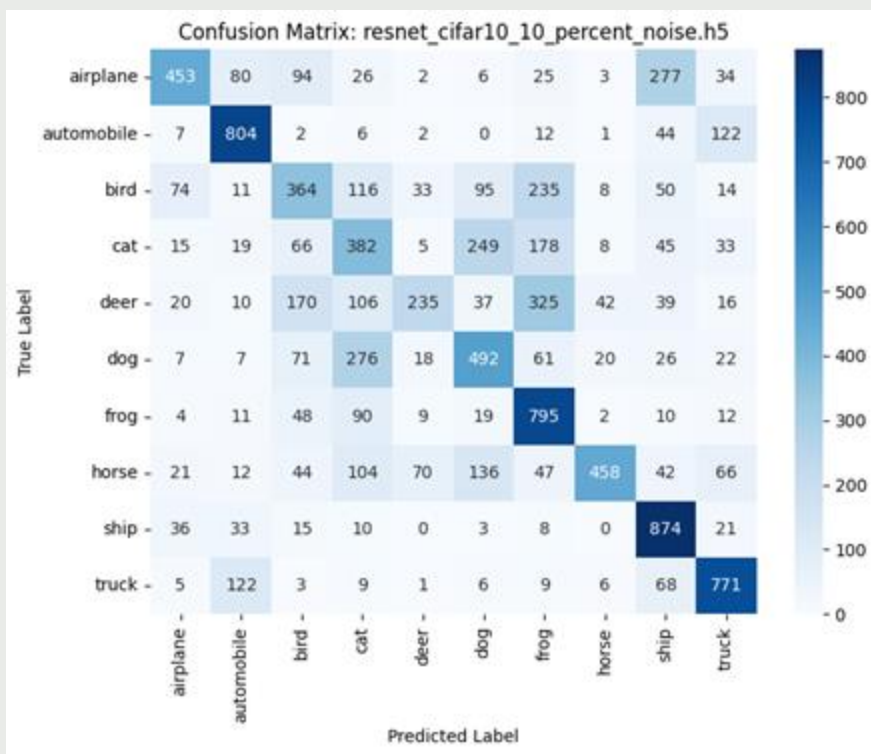


Comparing ResNet Performance with CIFAR-10 Variants



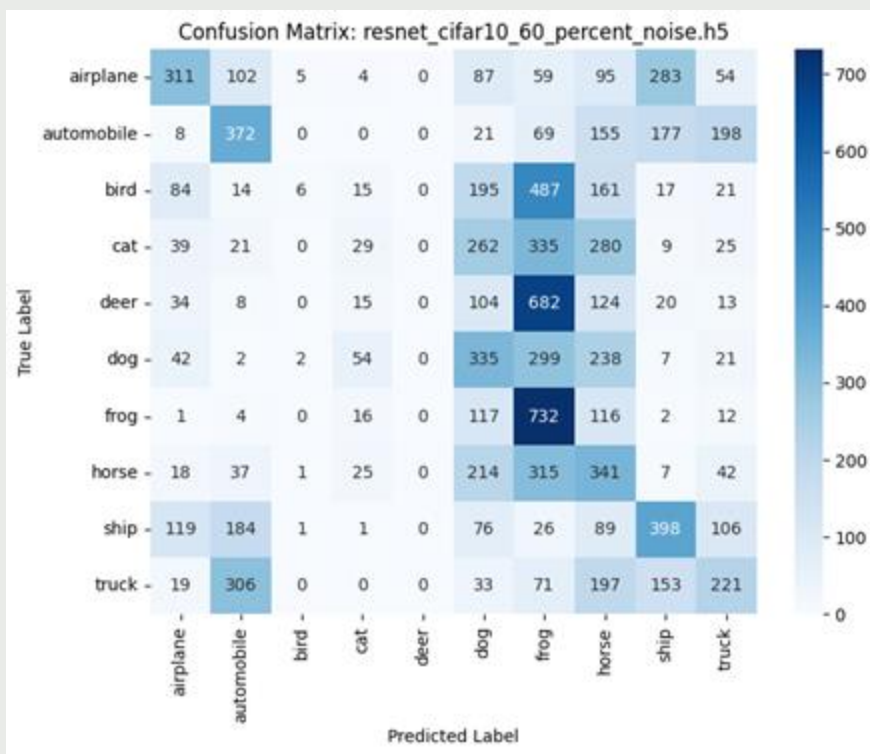
CIFAR-10 with Label Corruption 10%-60%





resnet_cifar10_10_percent_noise.h5 - Most Misclassified Classes:**

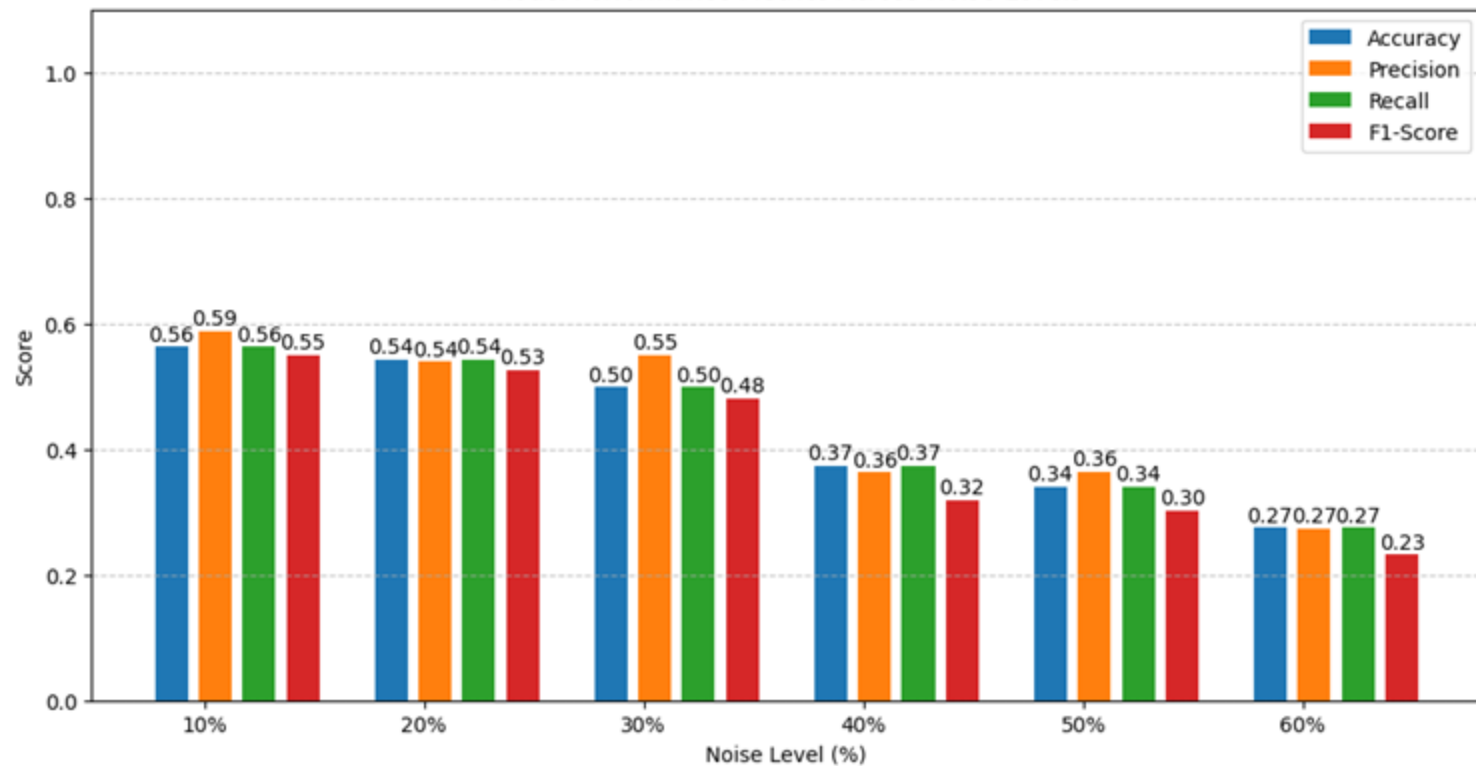
- deer → frog (325 times)
- airplane → ship (277 times)
- dog → cat (276 times)



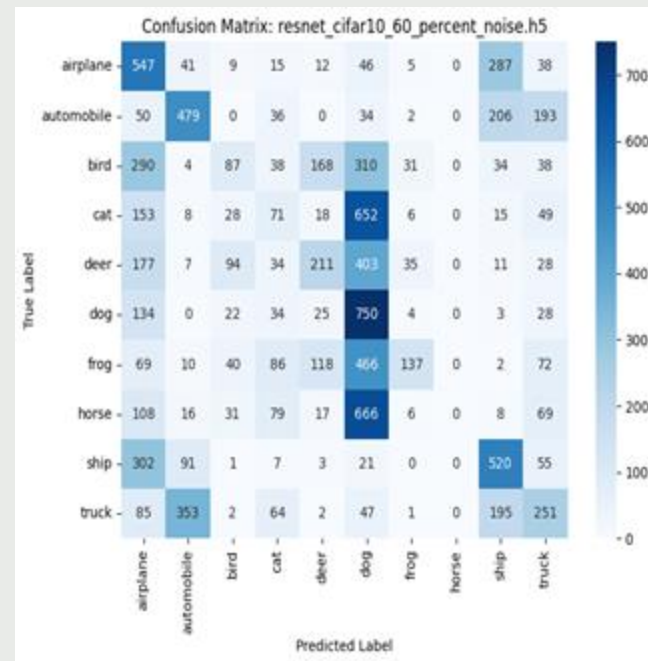
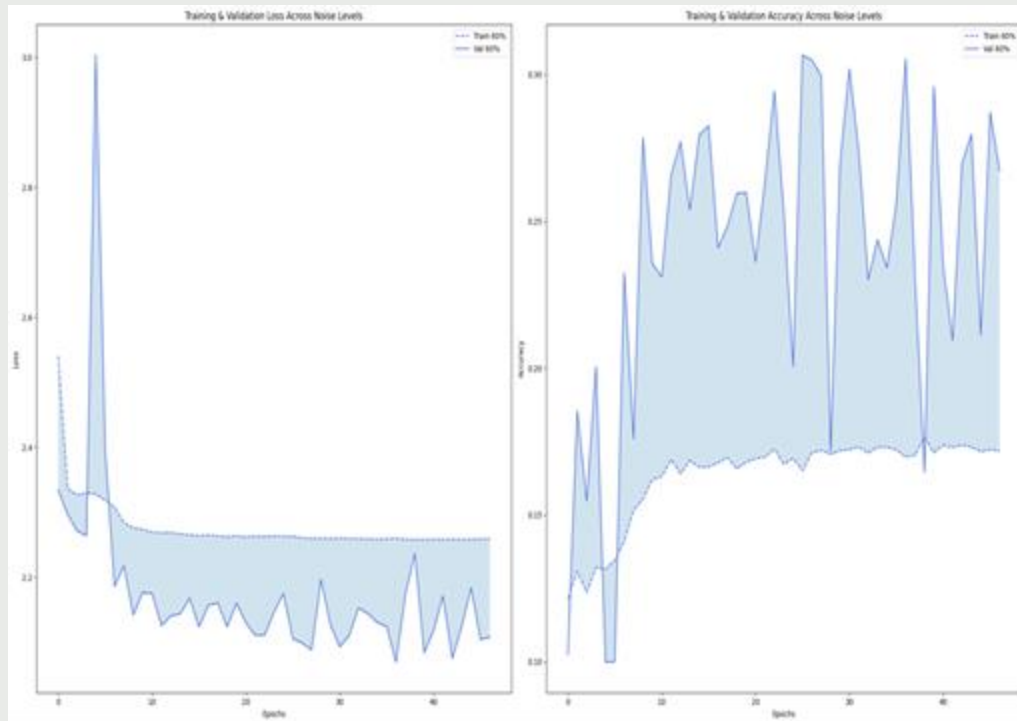
resnet_cifar10_60_percent_noise.h5 - Most Misclassified Classes:**

- deer → frog (682 times)
- bird → frog (487 times)
- cat → frog (335 times)

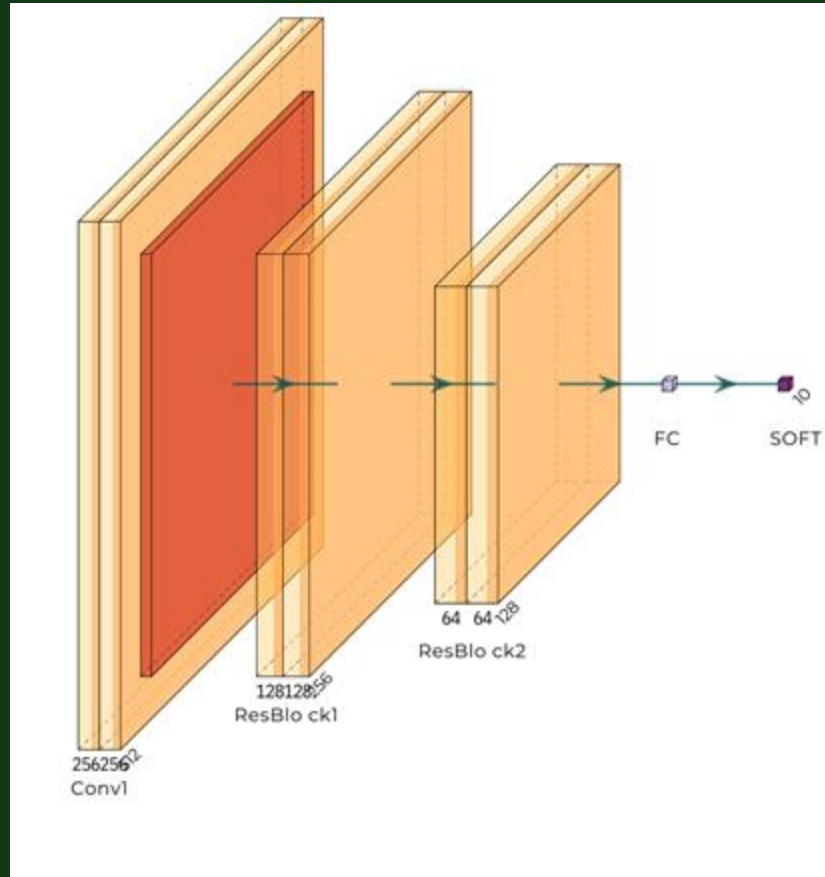
Model Performance Metrics Across Noise Levels



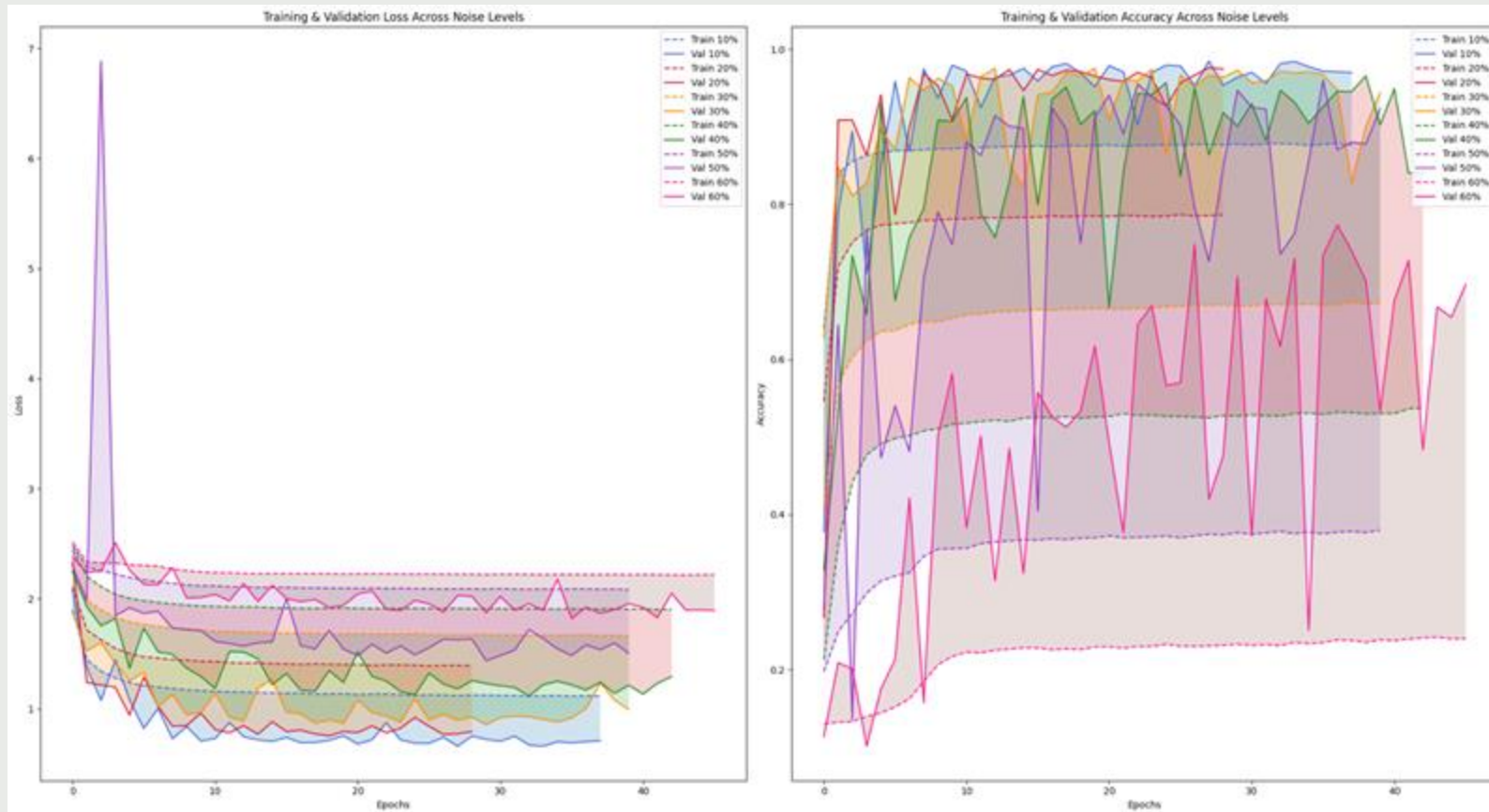
ResNet On 60% Noise Using Noise Aware Loss Function (Accuracy: 30% → 48% Improvement)

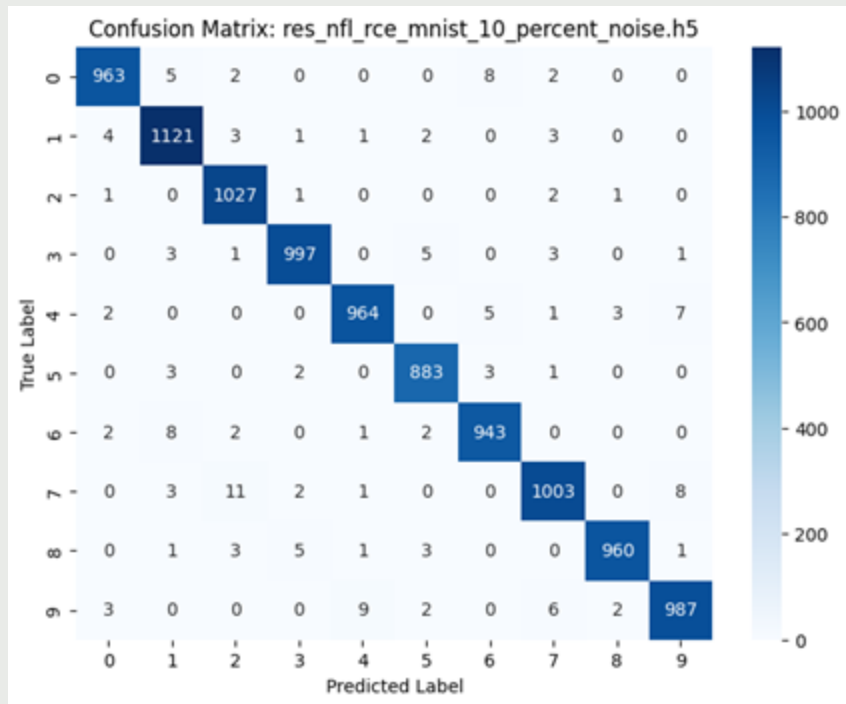


Comparing ResNet Performance with MNIST Variants Using NFL/RCE Loss Functions

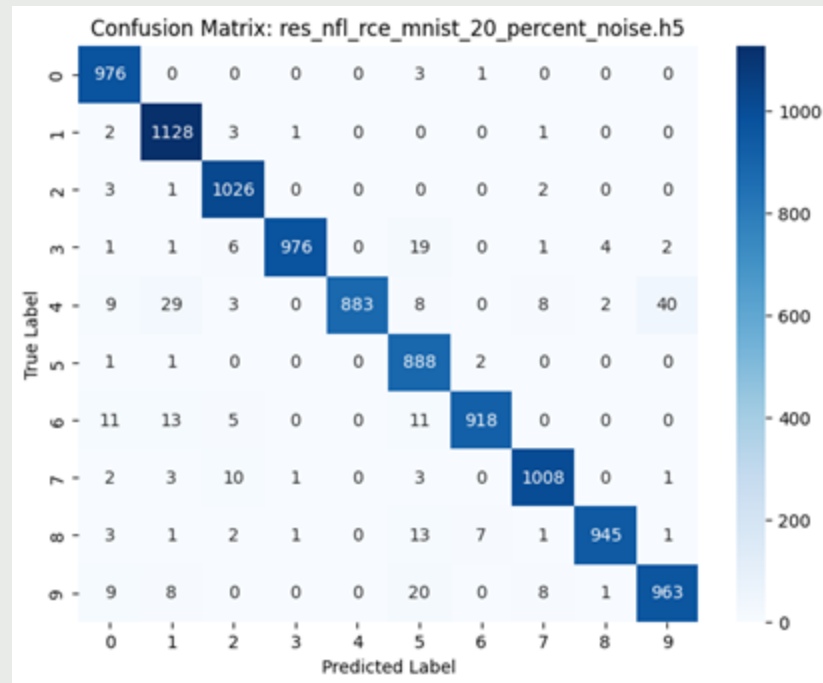


MNIST With Label Corruption 10-60%





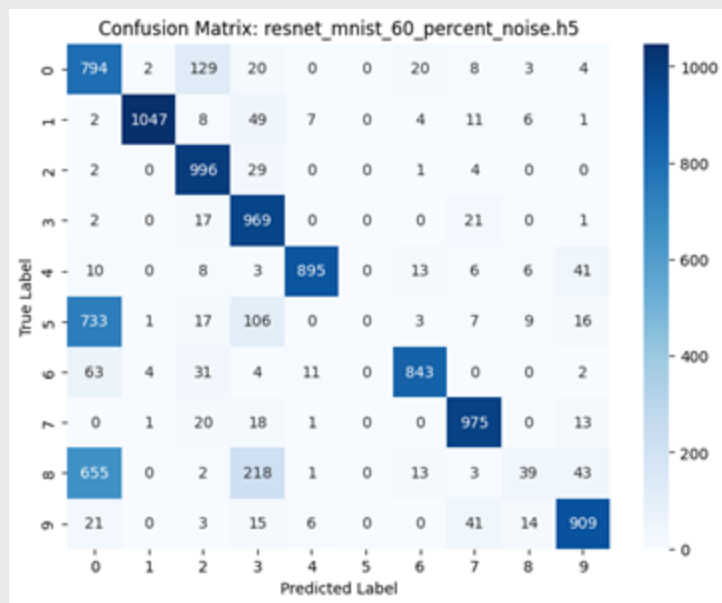
```
res_nfl_rce_mnist_10_percent_noise.h5 - Most Misclassified Classes:**
- 7 → 2 (11 times)
- 9 → 4 (9 times)
- 0 → 6 (8 times)
```



```
res_nfl_rce_mnist_20_percent_noise.h5 - Most Misclassified Classes:**
- 4 → 9 (40 times)
- 4 → 1 (29 times)
- 9 → 5 (20 times)
```

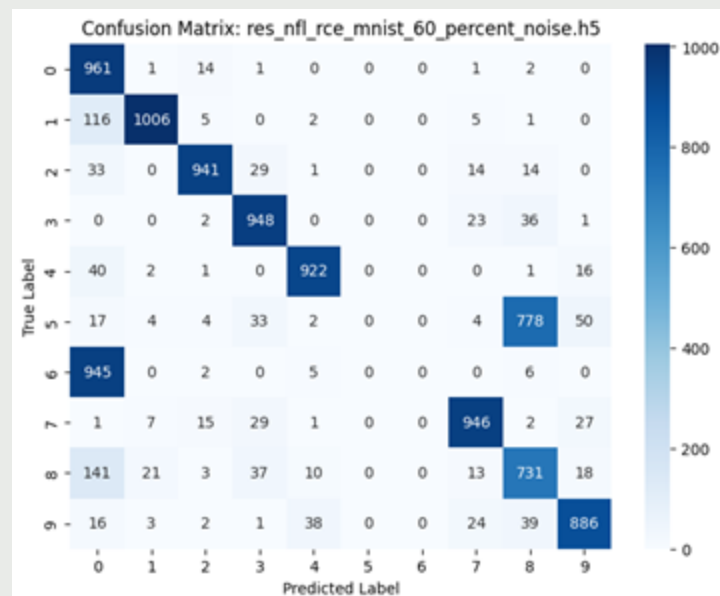
MNIST 60% Noise Before and After Implementing NFL/RCE

Categorical Cross Entropy



```
resnet_mnist_60_percent_noise.h5 - Most Misclassified Classes:**  
- 5 → 0 (733 times)  
- 8 → 0 (655 times)  
- 8 → 3 (218 times)
```

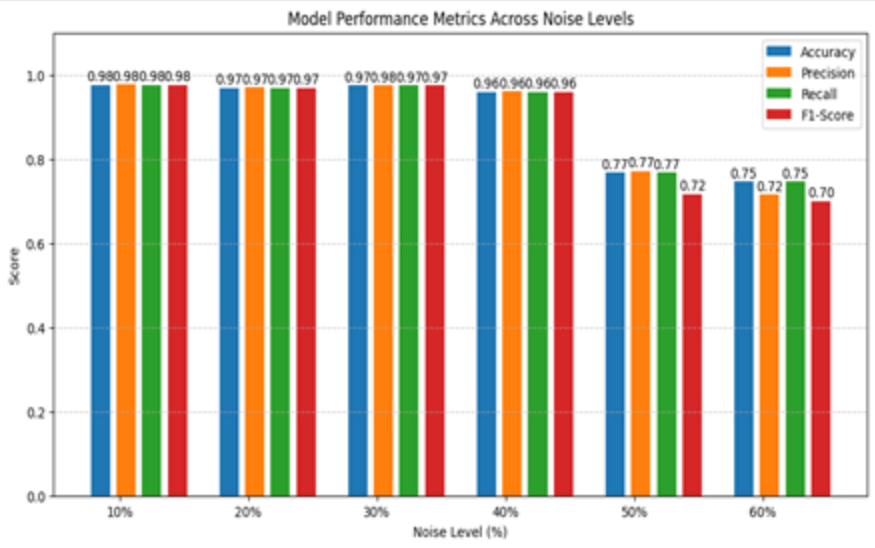
NFL/RCE



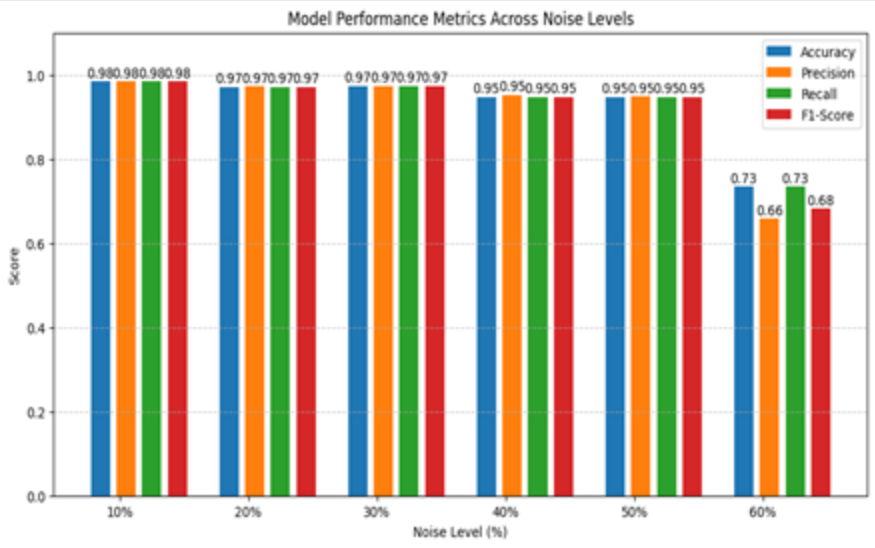
```
res_nfl_rce_mnist_60_percent_noise.h5 - Most Misclassified Classes:**  
- 6 → 0 (945 times)  
- 5 → 8 (778 times)  
- 8 → 0 (141 times)
```

Performance Metrics Before and After Implementing NFL/RCE

Categorical Cross Entropy

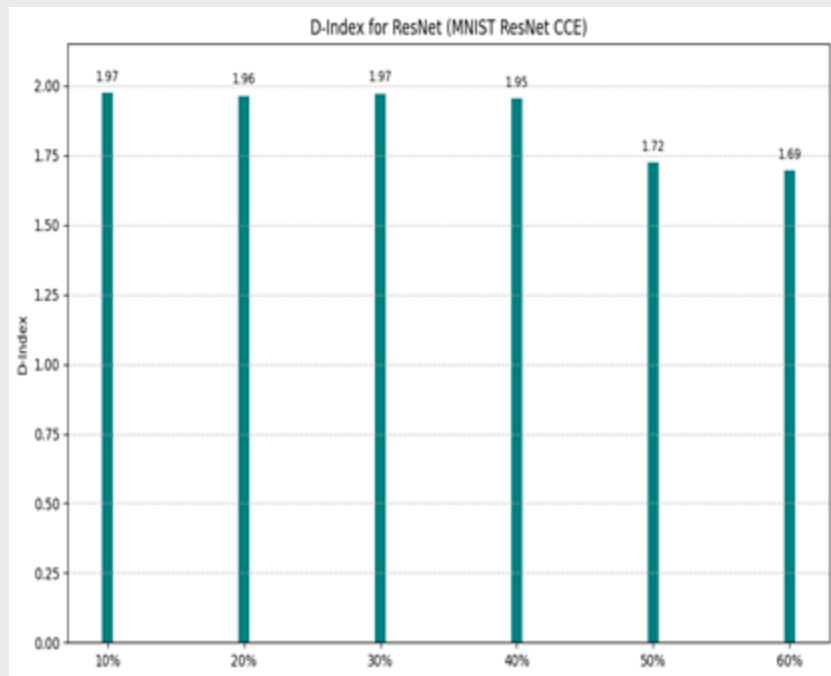


NFL/RCE

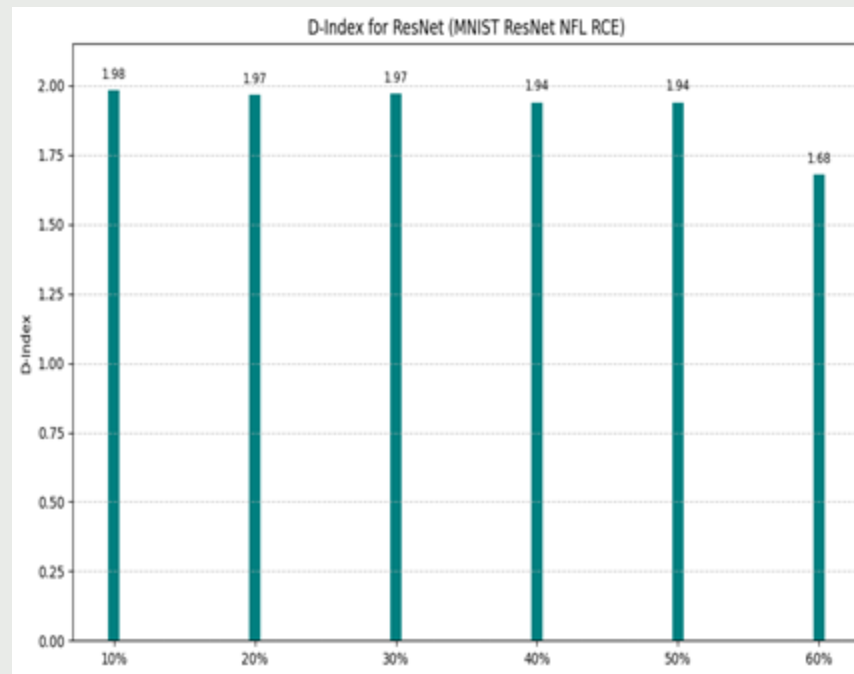


D Index Before and After Implementing NFL/RCE

Categorical Cross Entropy



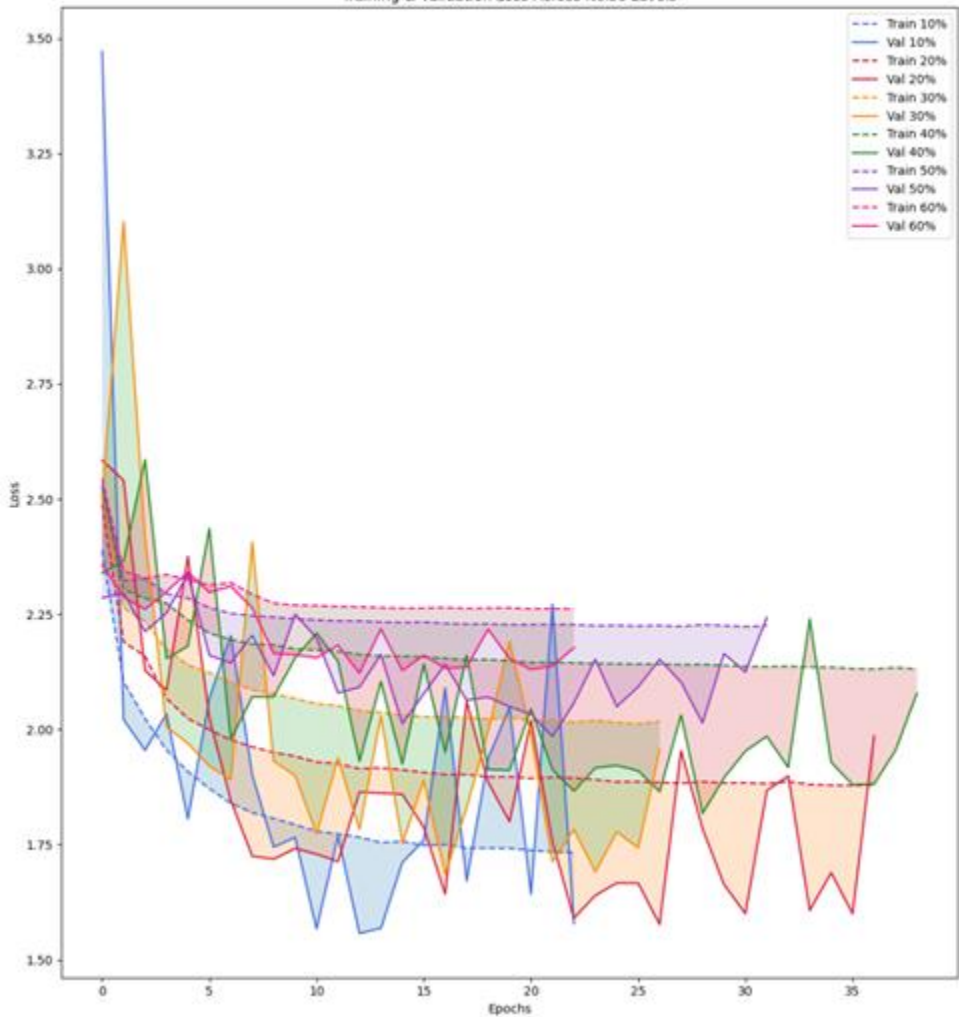
NFL/RCE



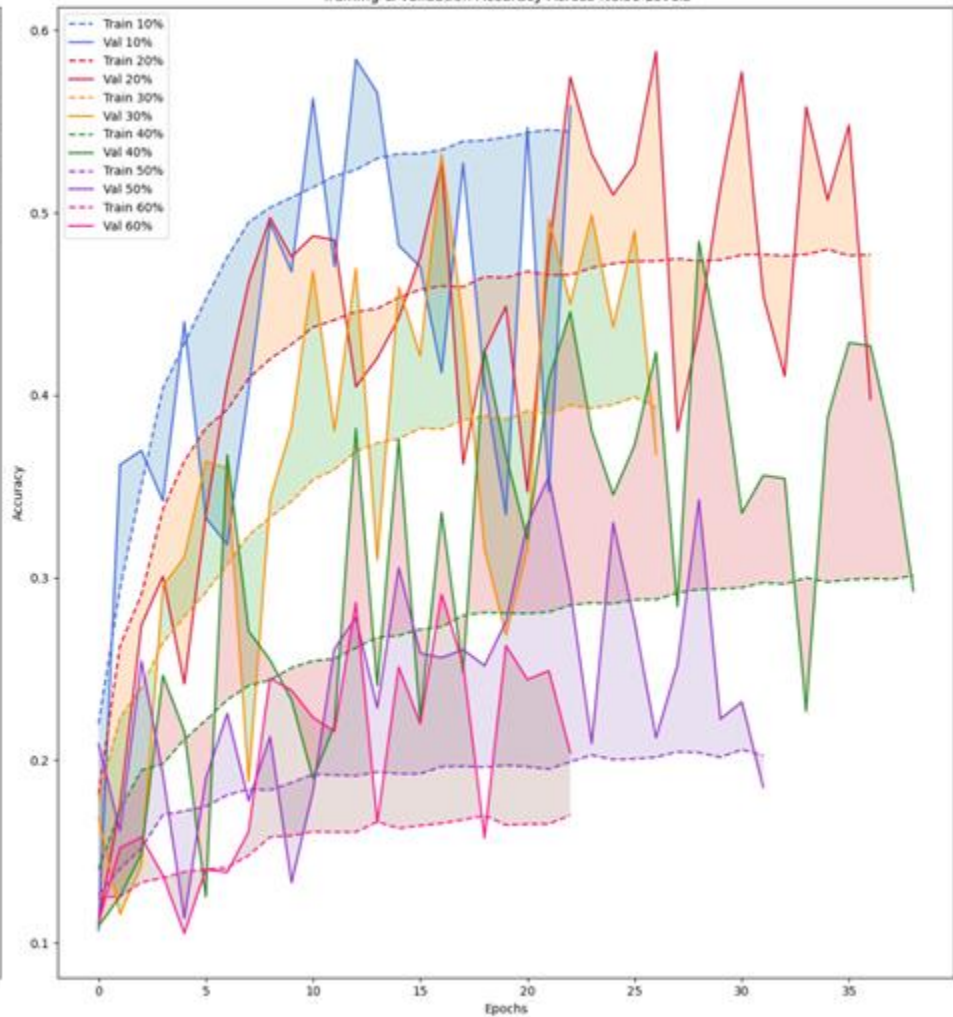
Comparing ResNet Performance with CIFAR-10 Variants Using NFL/RCE Loss Function



Training & Validation Loss Across Noise Levels

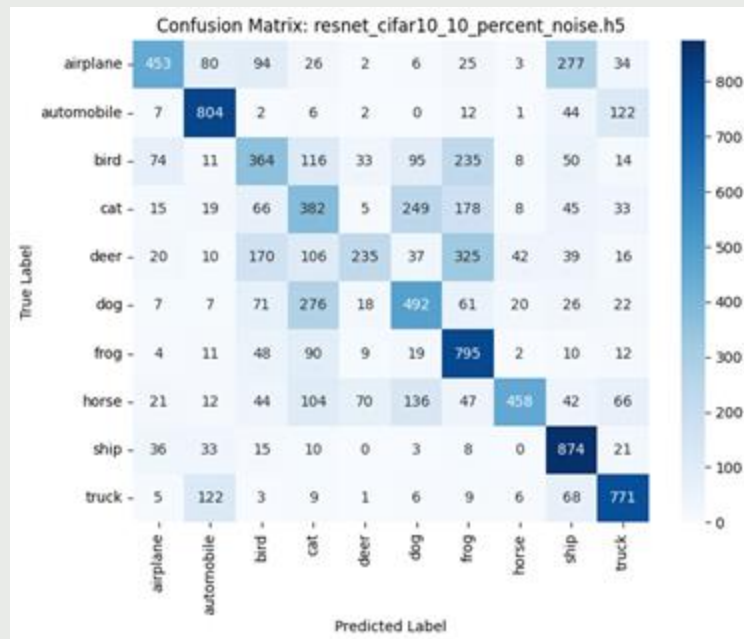


Training & Validation Accuracy Across Noise Levels



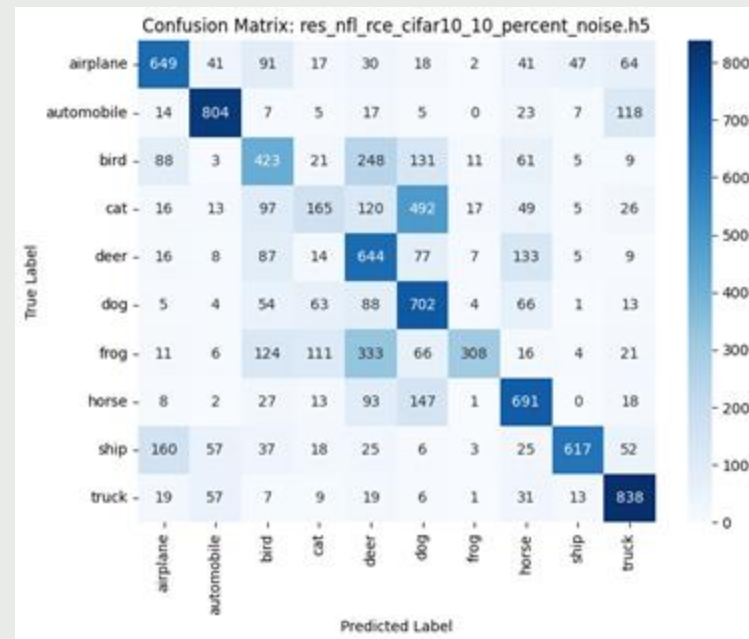
CIFAR-10 10% Noise Before and After Implementing NFL/RCE

Categorical Cross Entropy



```
resnet_cifar10_10_percent_noise.h5 - Most Misclassified Classes:**
- deer + frog (325 times)
- airplane + ship (277 times)
- dog + cat (276 times)
```

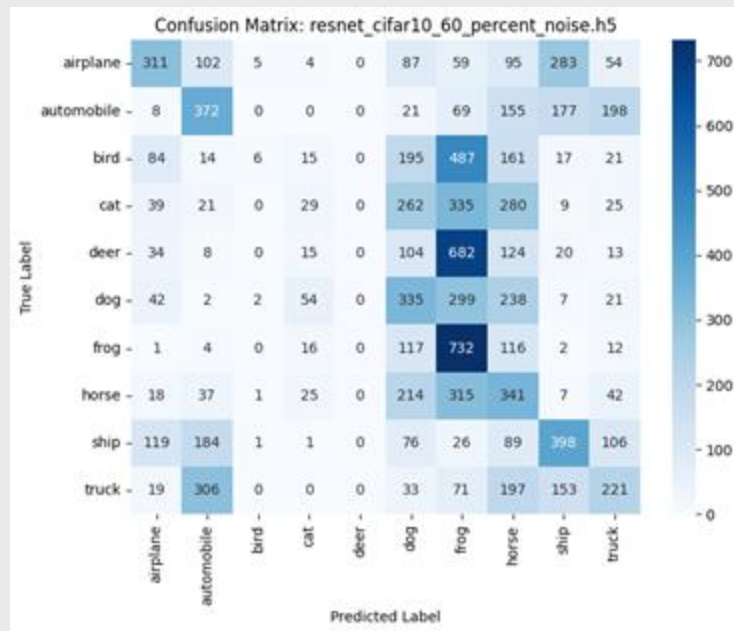
NFL/RCE



```
res_nfl_rce_cifar10_10_percent_noise.h5 - Most Misclassified Classes:**
- cat + dog (492 times)
- frog + deer (333 times)
- bird + deer (248 times)
```

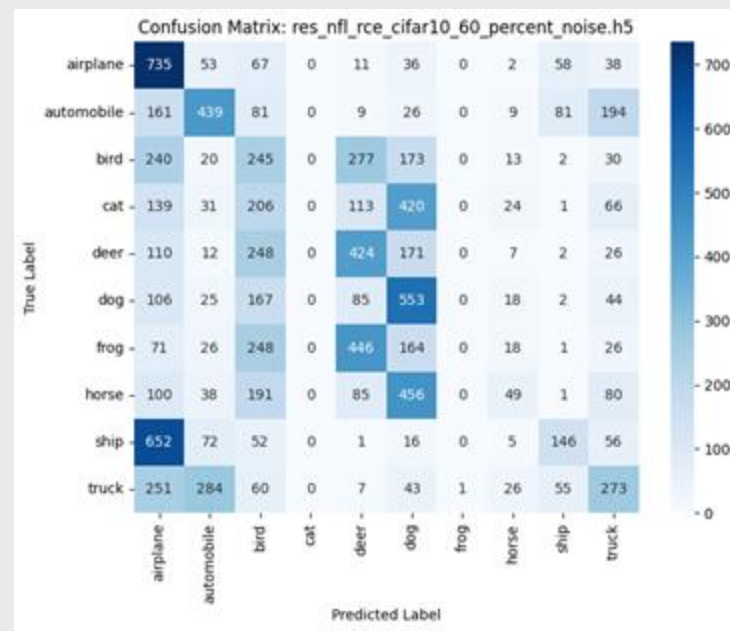

CIFAR-10 60% Noise Before and After Implementing NFL/RCE

Categorical Cross Entropy



```
resnet_cifar10_60_percent_noise.h5 - Most Misclassified Classes:**
- deer + frog (682 times)
- bird + frog (487 times)
- cat + frog (335 times)
```

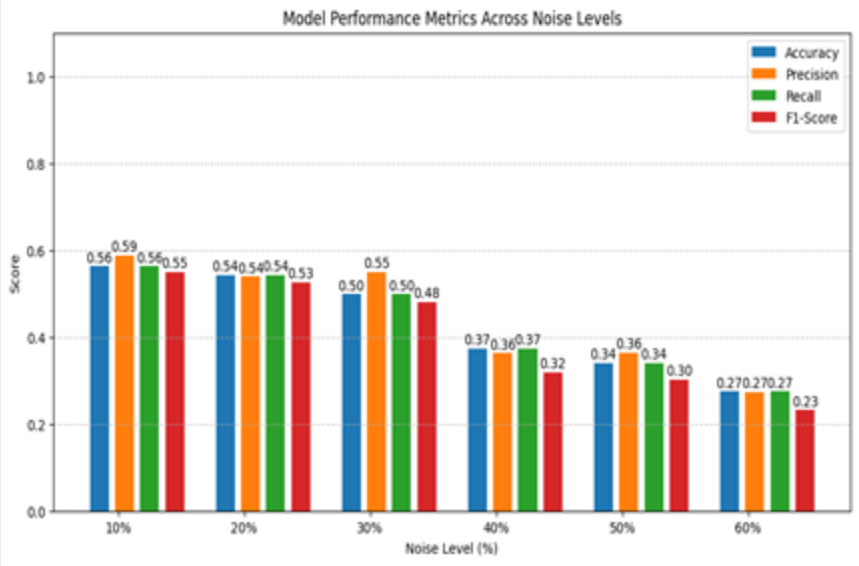
NFL/RCE



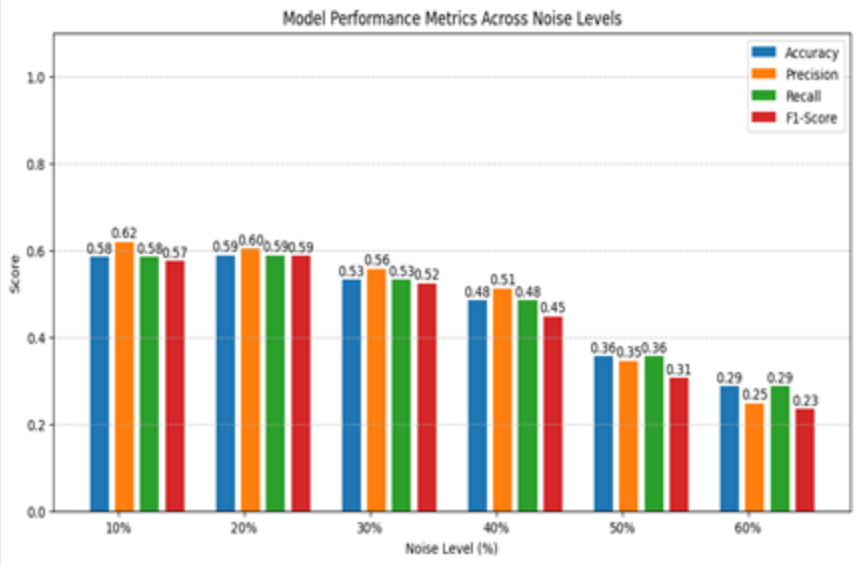
```
res_nfl_rce_cifar10_60_percent_noise.h5 - Most Misclassified Classes:**
- ship + airplane (652 times)
- horse + dog (456 times)
- frog + deer (446 times)
```

Performance Metrics Before and After Implementing NFL/RCE

Categorical Cross Entropy

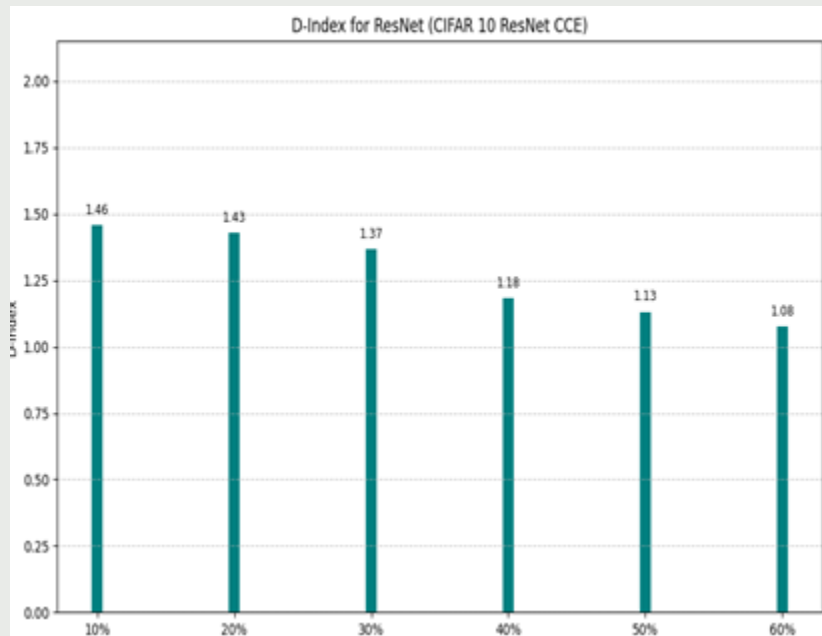


NFL/RCE

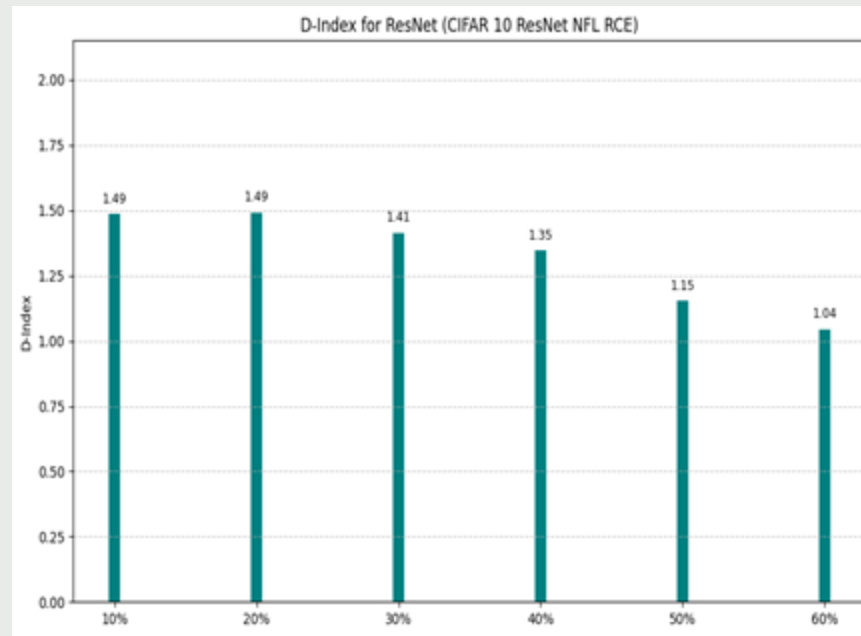


D Index Before and After Implementing NFL/RCE

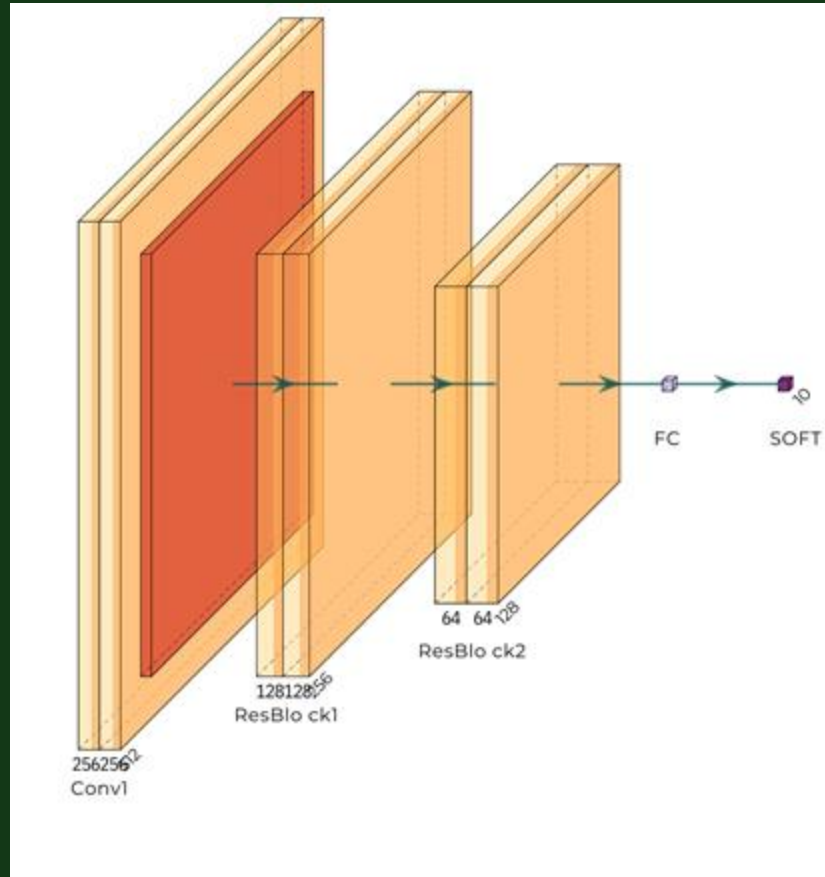
Categorical Cross Entropy



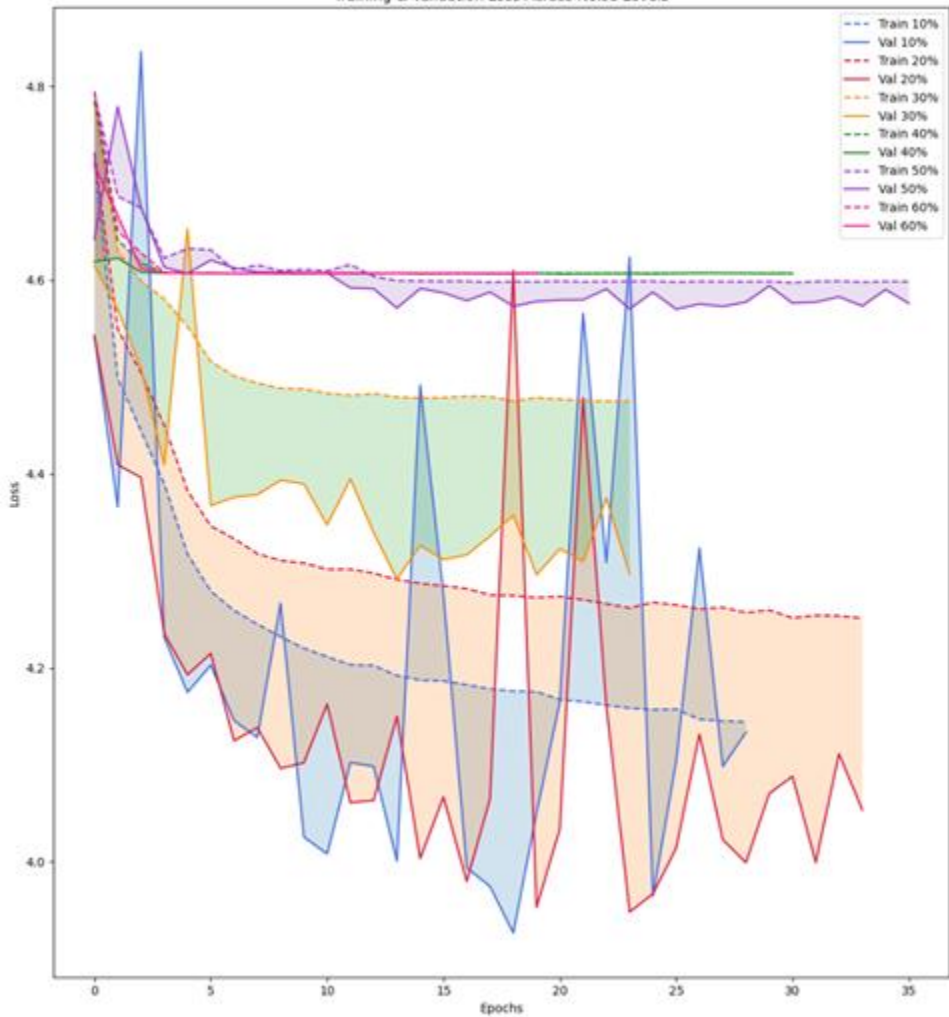
NFL/RCE



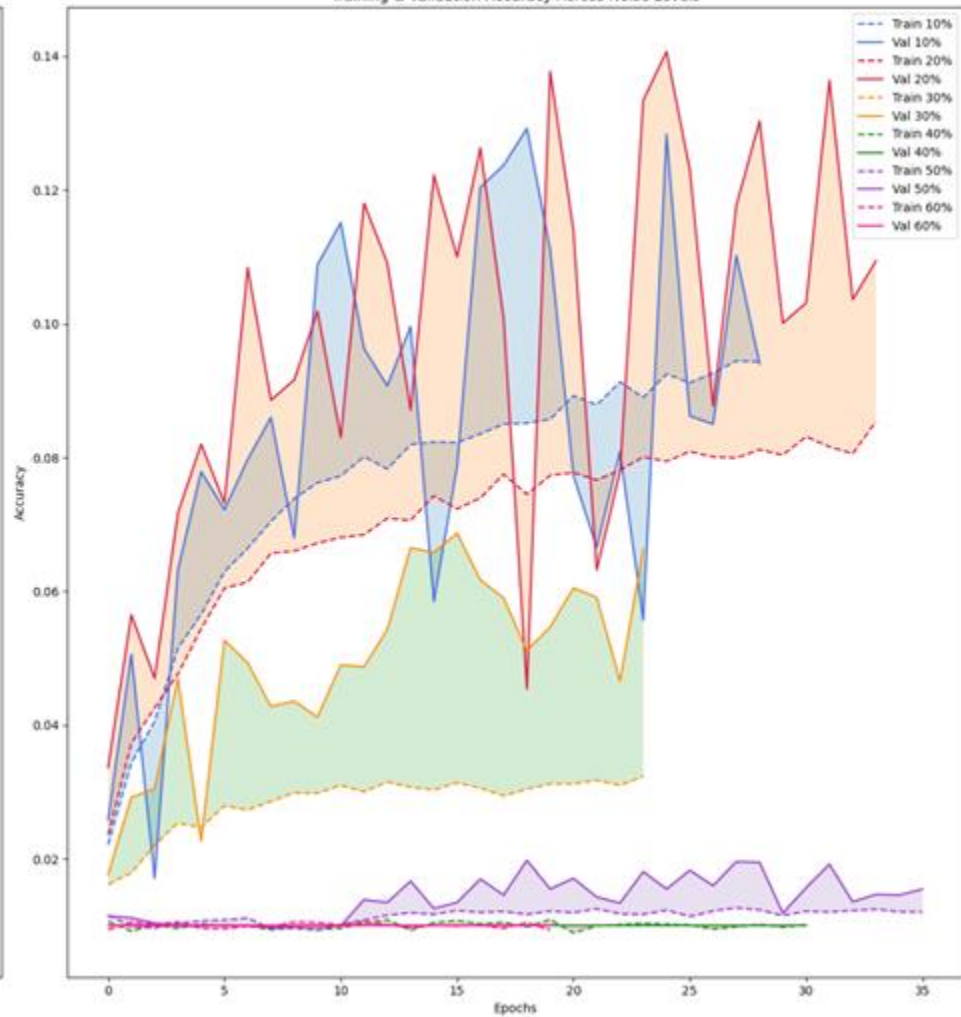
Comparing ResNet Performance with CIFAR - 100 Variants Using NFL+RCE Loss Function

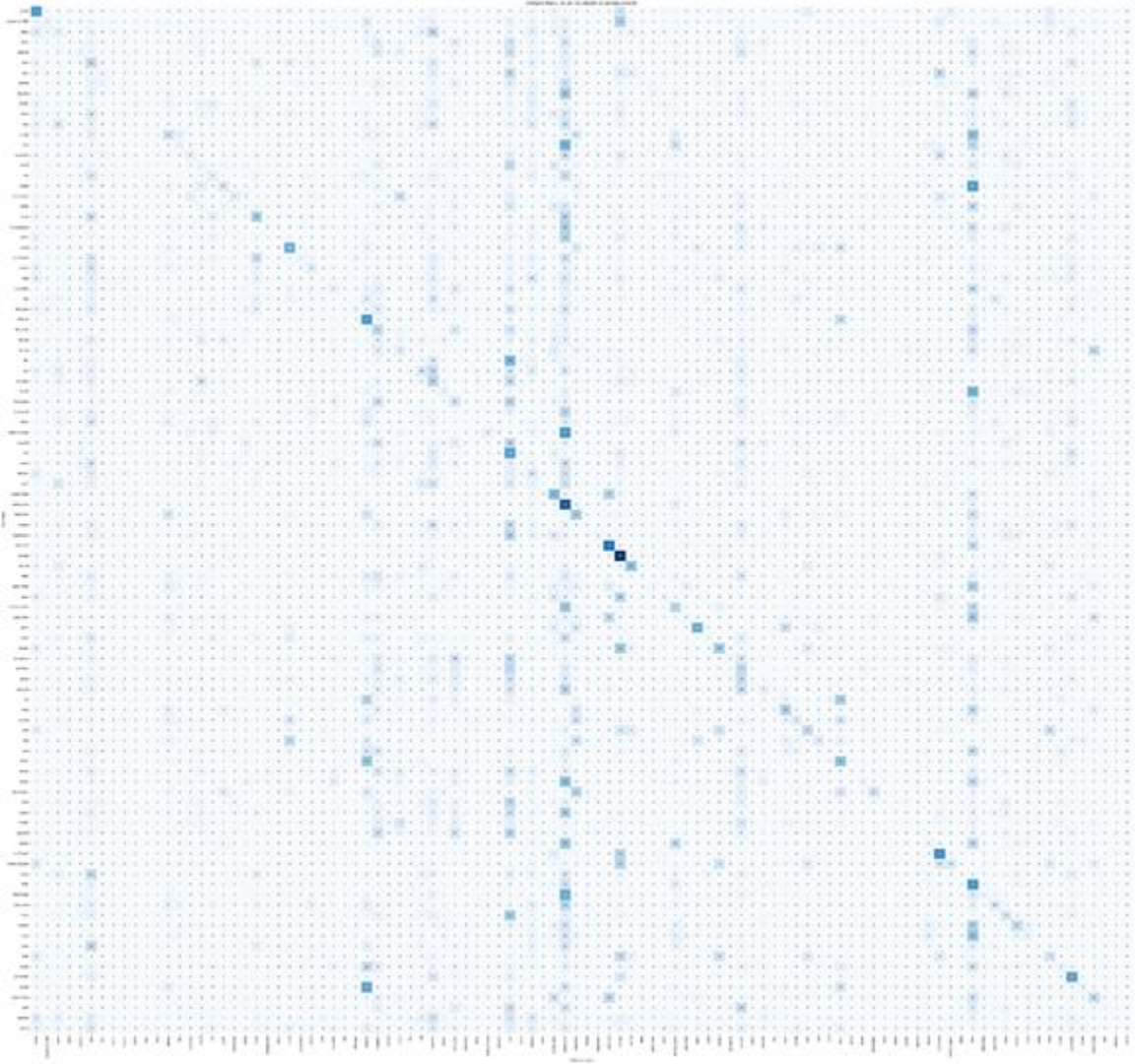


Training & Validation Loss Across Noise Levels



Training & Validation Accuracy Across Noise Levels





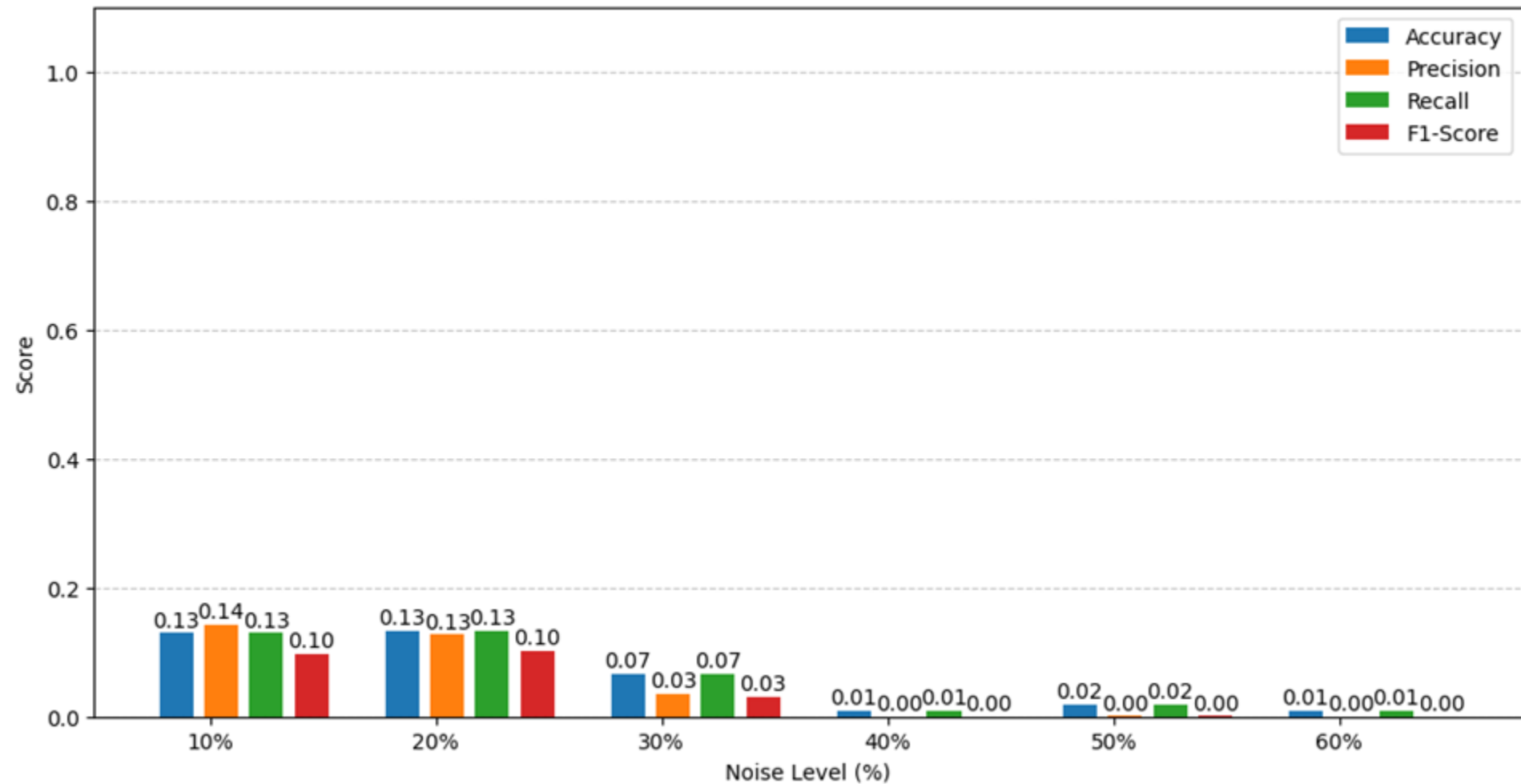
res_nfl_rce_cifar100_10_percent_noise.h5 - Most Misclassified Classes:**

- lawn_mower → motorcycle (52 times)
- castle → tank (50 times)
- whale → dolphin (47 times)
- telephone → motorcycle (44 times)
- bus → motorcycle (41 times)

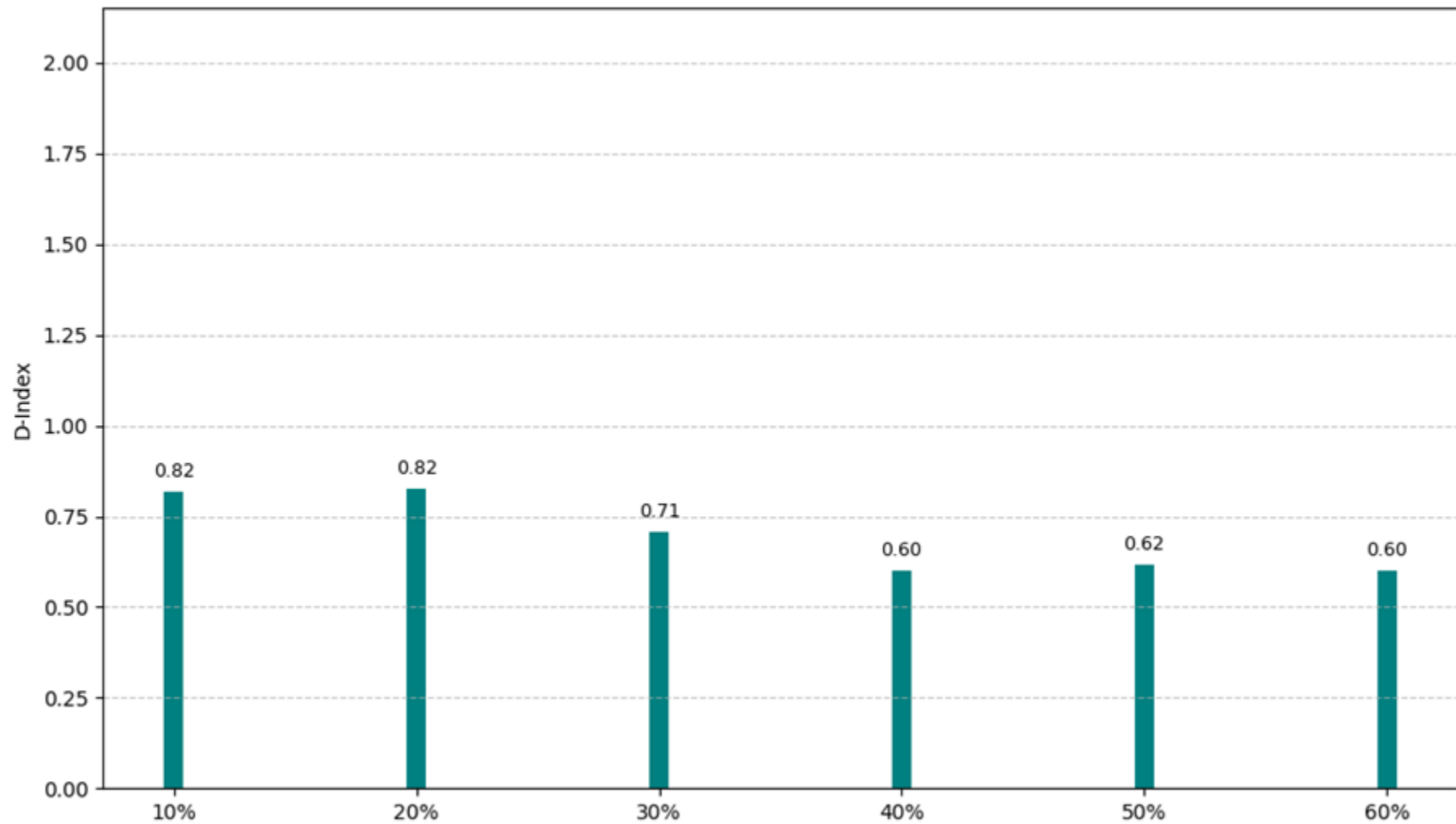
res_nfl_rce_cifar100_60_percent_noise.h5 - Most Misclassified Classes:**

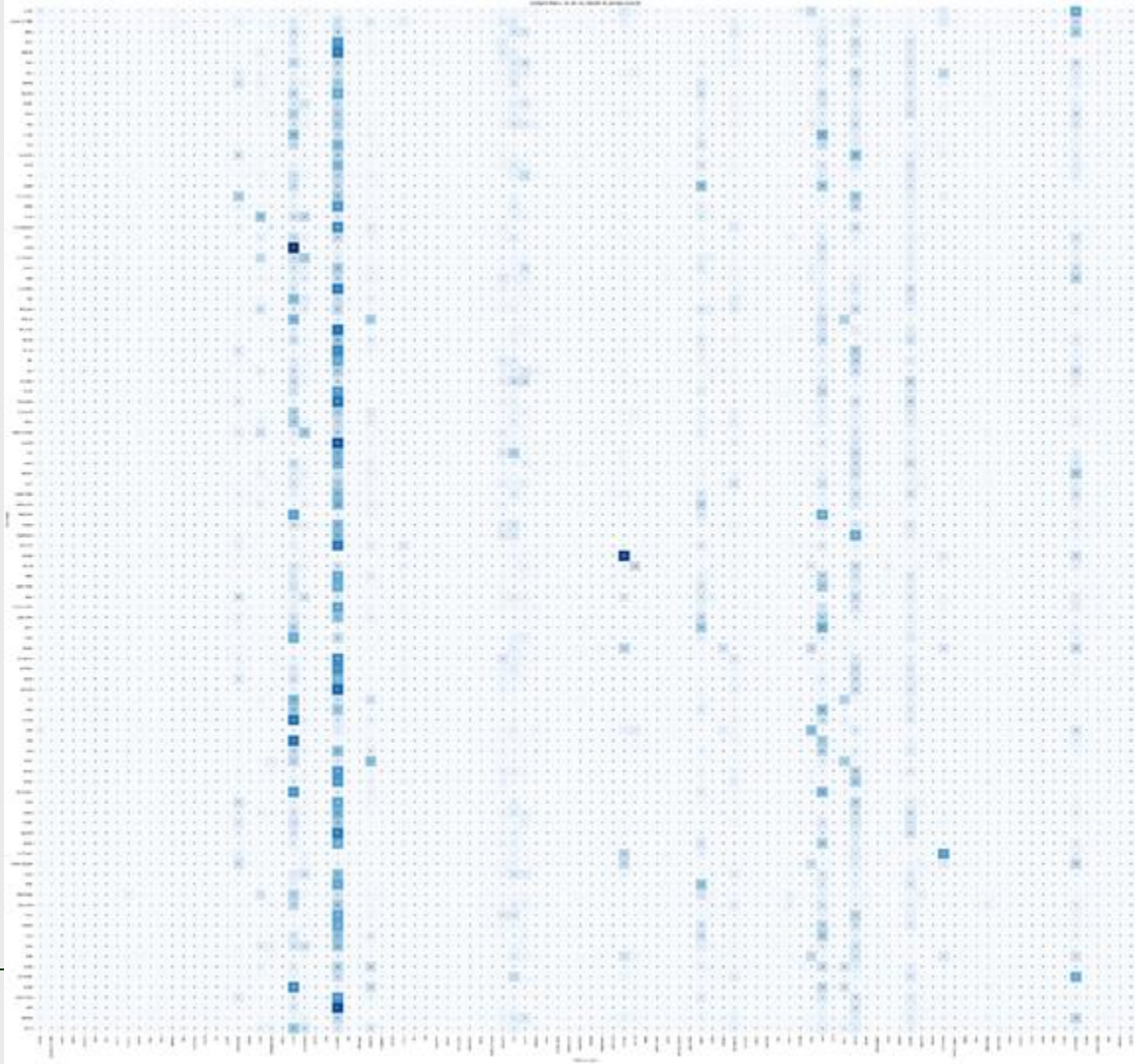
- apple → cockroach (100 times)
- aquarium_fish → cockroach (100 times)
- baby → cockroach (100 times)
- bear → cockroach (100 times)
- beaver → cockroach (100 times)

Model Performance Metrics Across Noise Levels



D-Index for ResNet (CIFAR 100 ResNet NFL RCE)

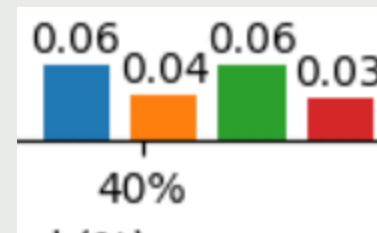




Striking Misclassifications

```
res_nfl_rce_cifar100_40_percent_noise.h5 - Most Misclassified Classes:**
```

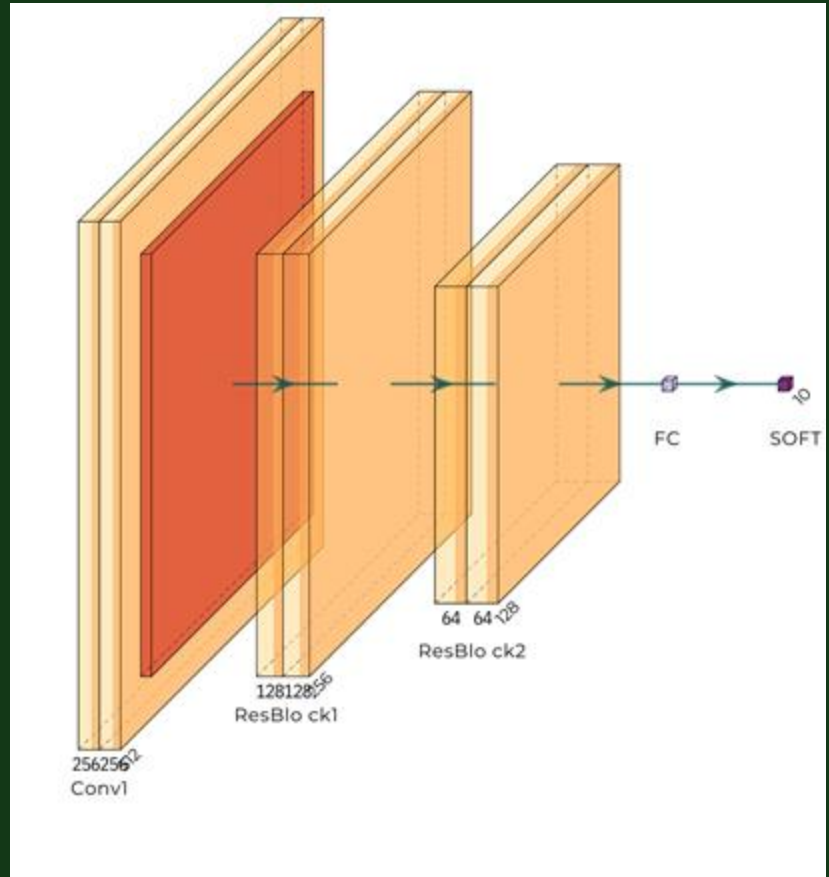
- apple → plate (100 times)
- aquarium_fish → plate (100 times)
- baby → plate (100 times)
- bear → plate (100 times)
- beaver → plate (100 times)



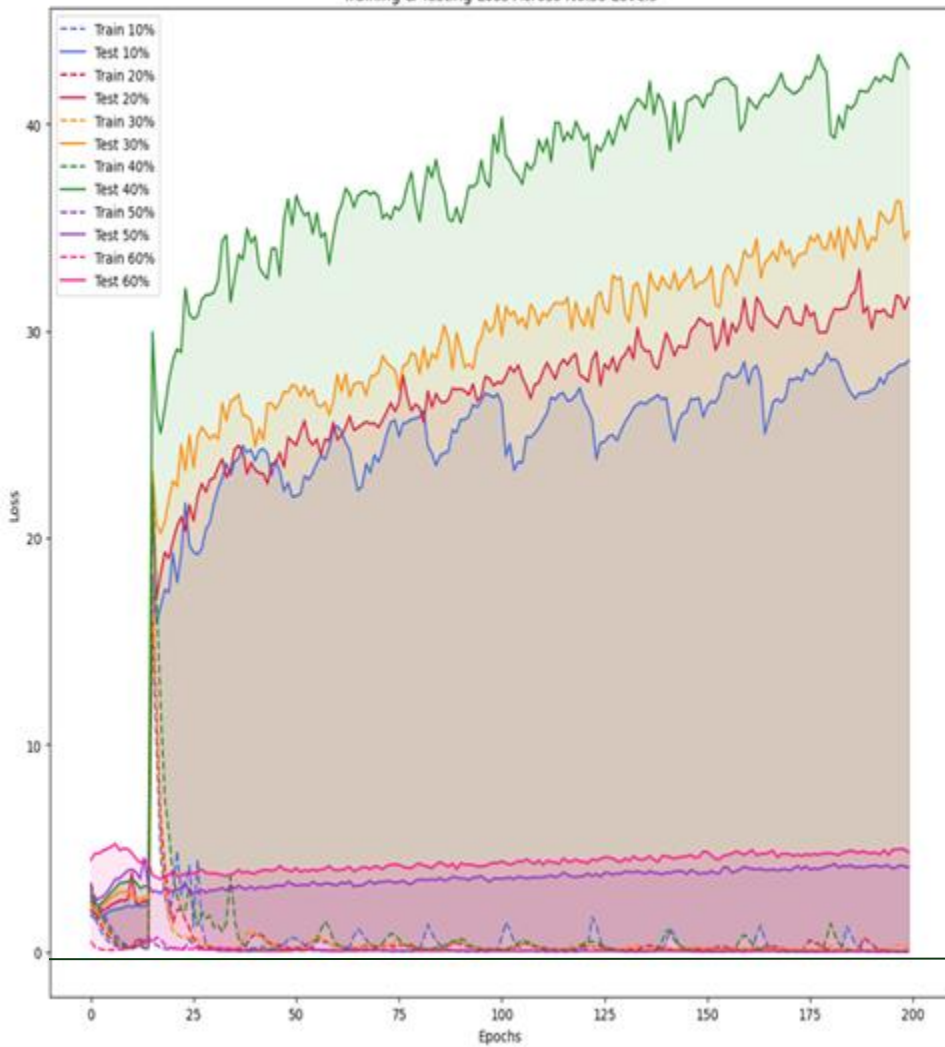
```
res_nfl_rce_cifar100_40_percent_noise.h5 - Most Misclassified Classes:**
```

- wolf → crocodile (67 times)
- leopard → crocodile (63 times)
- raccoon → crocodile (59 times)
- rocket → cloud (56 times)
- elephant → crocodile (55 times)

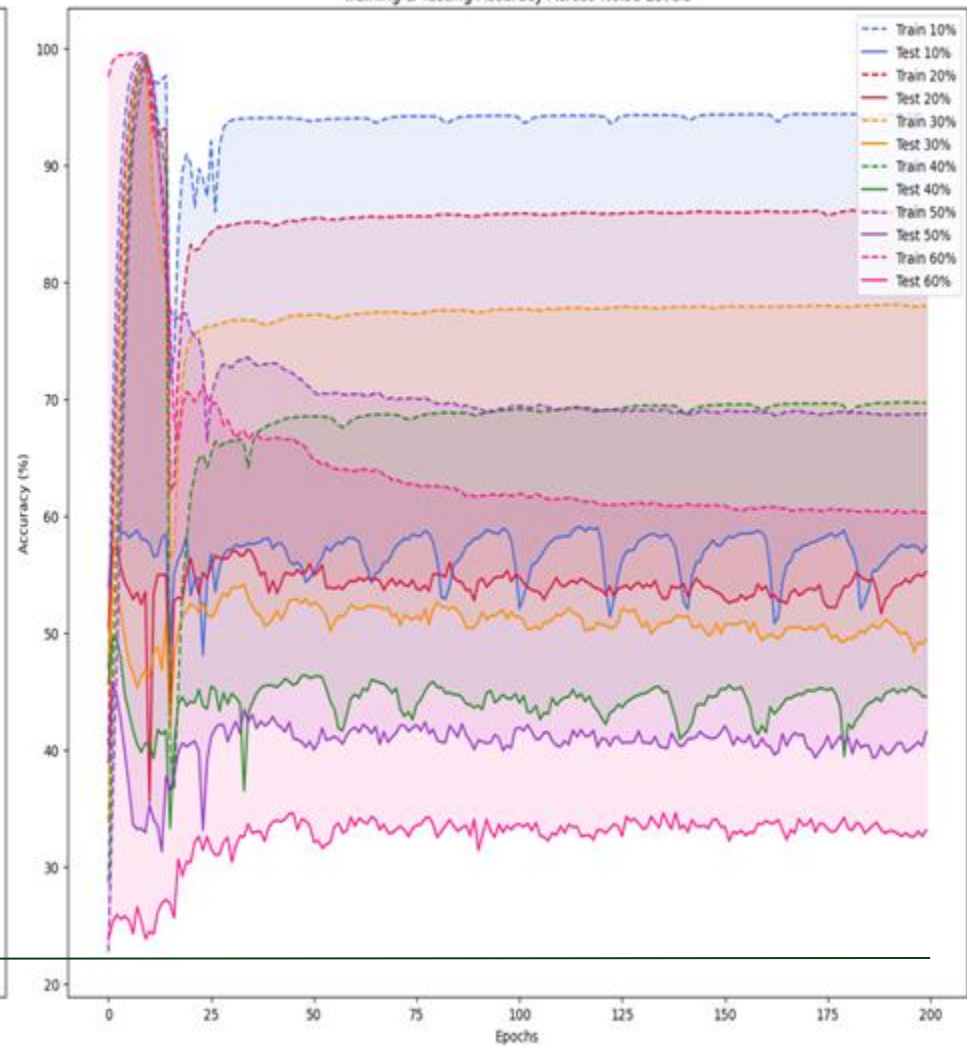
Comparing Co Teaching ResNet Performance with CIFAR - 100 Variants Using NFL+RCE Loss Function

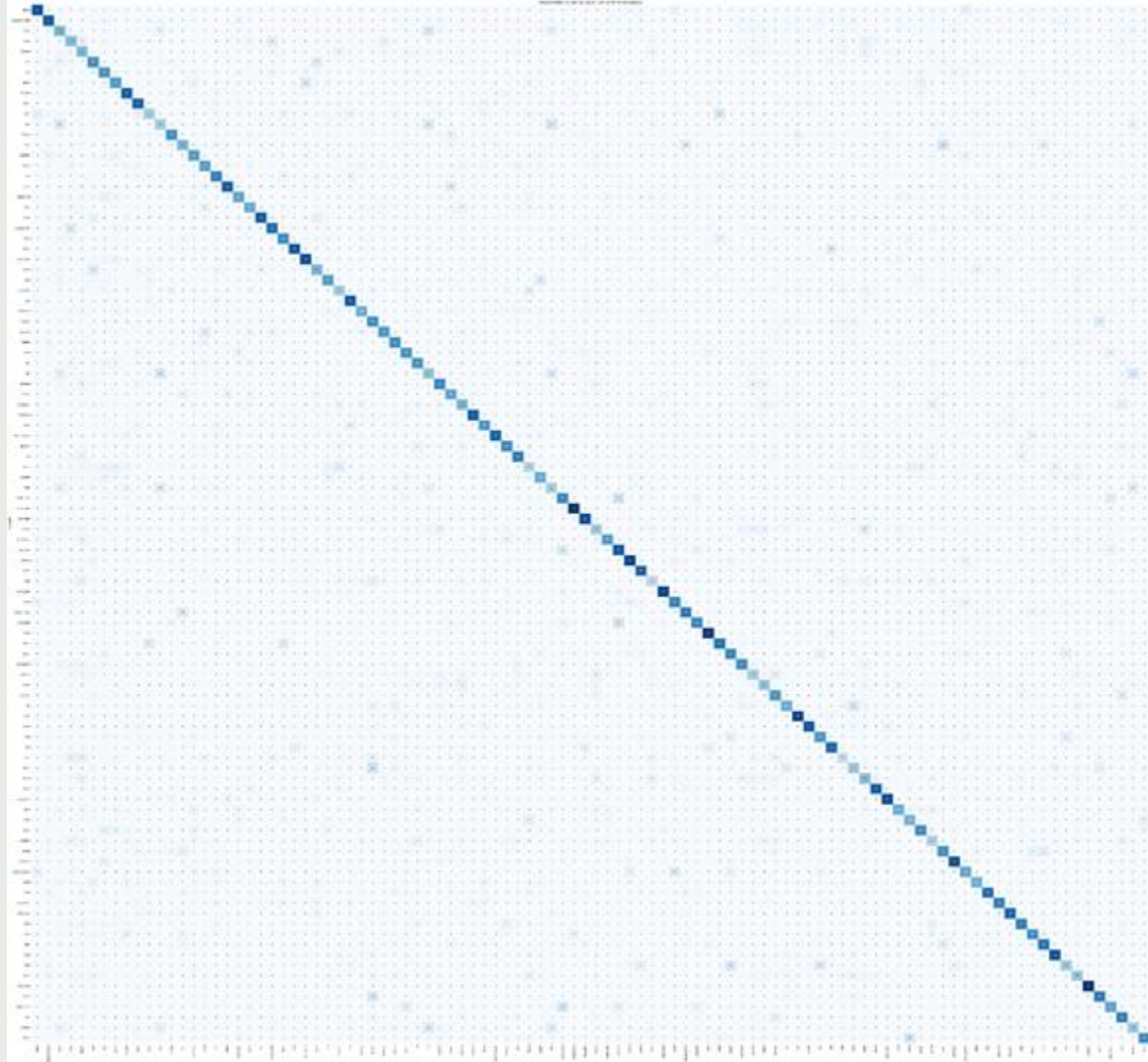


Training & Testing Loss Across Noise Levels



Training & Testing Accuracy Across Noise Levels

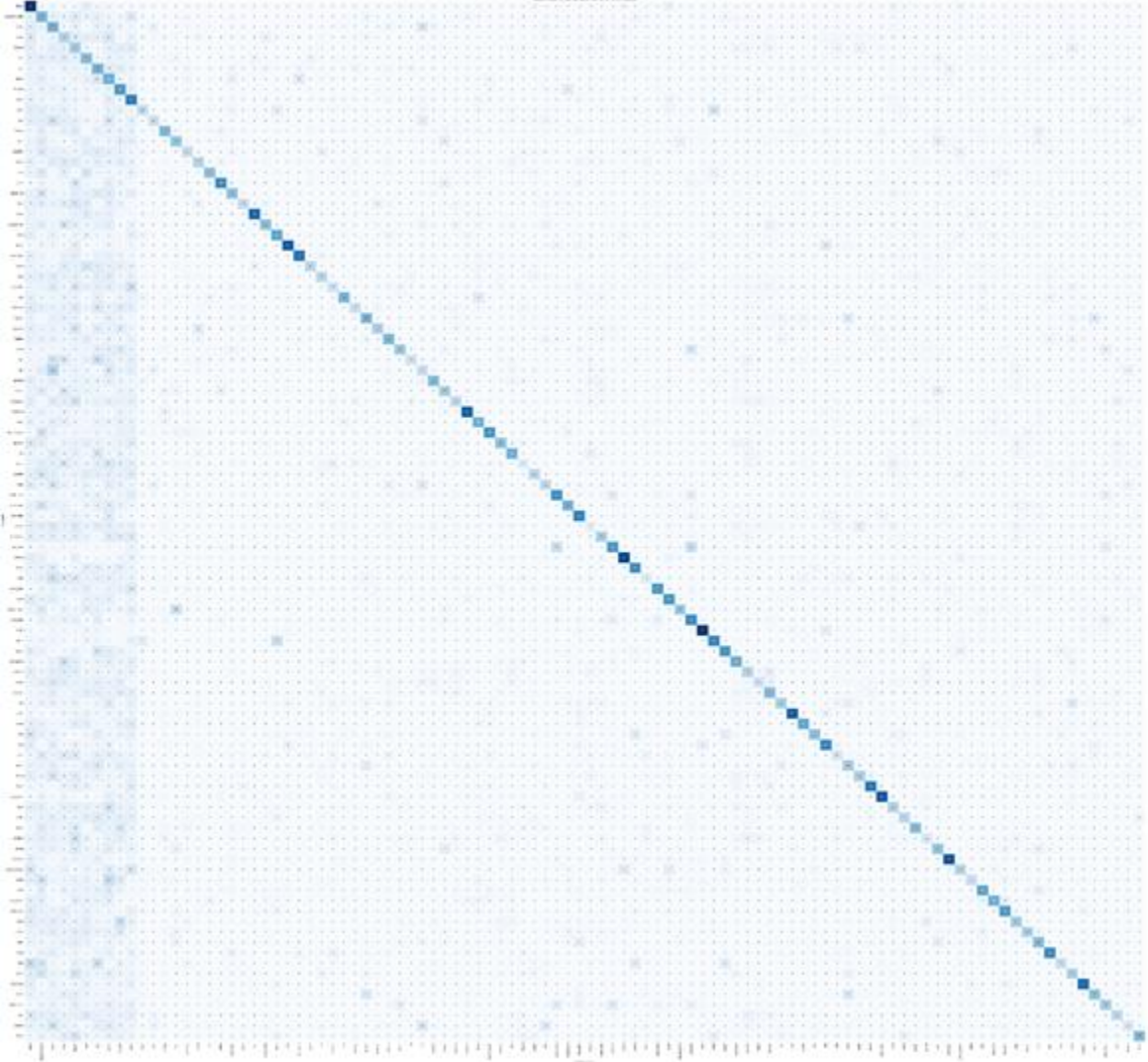




co_teaching_resnet1_cifar100_10_percent_noise.pt - Most Misclassified Classes (Noise Level: 10%):

Total Misclassifications: 4257

- maple_tree + oak_tree (18 times)
- shark + dolphin (18 times)
- bus + street (16 times)

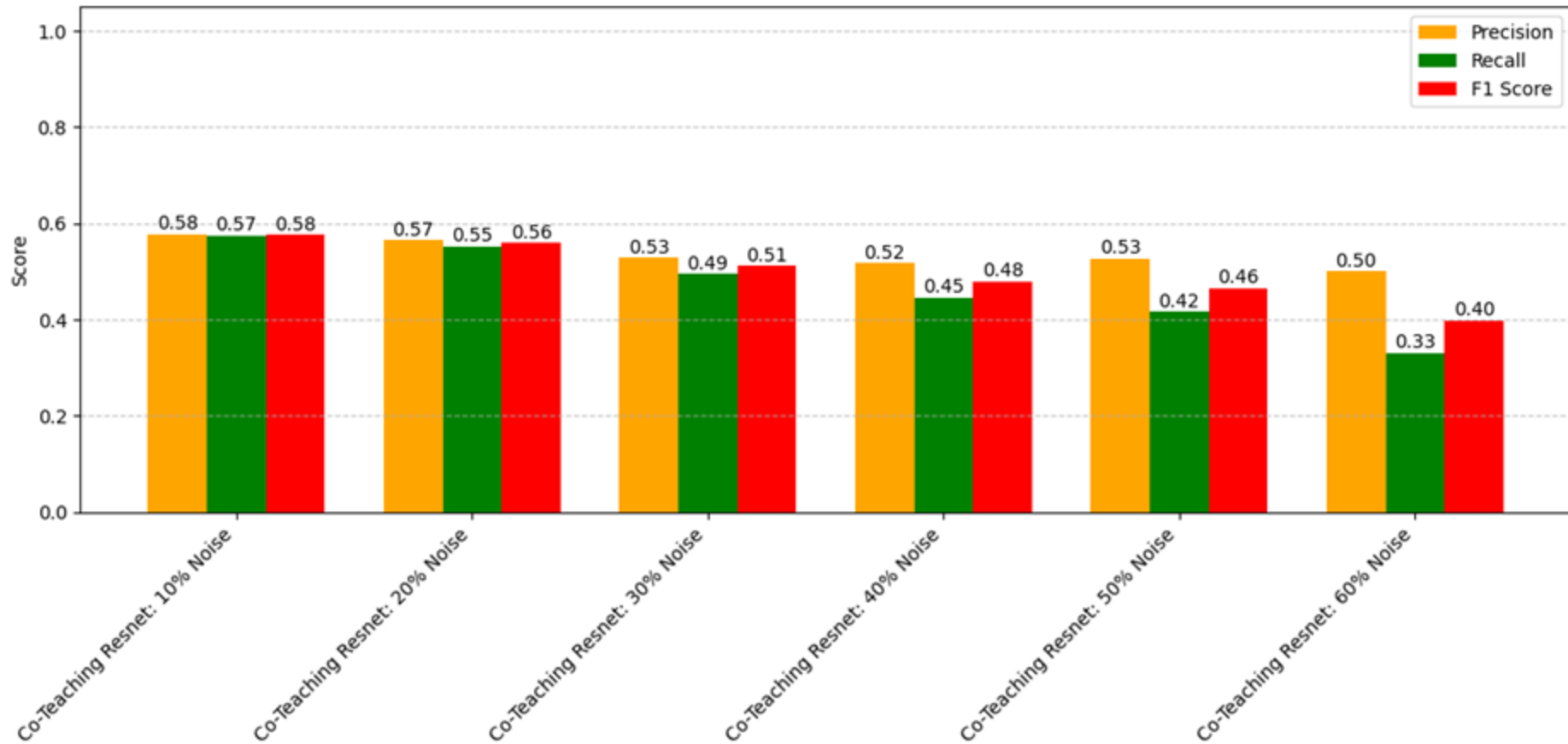


co_teaching_resnet1_cifar100_60_percent_noise.pt - Most Misclassified Classes (Noise Level: 60%):

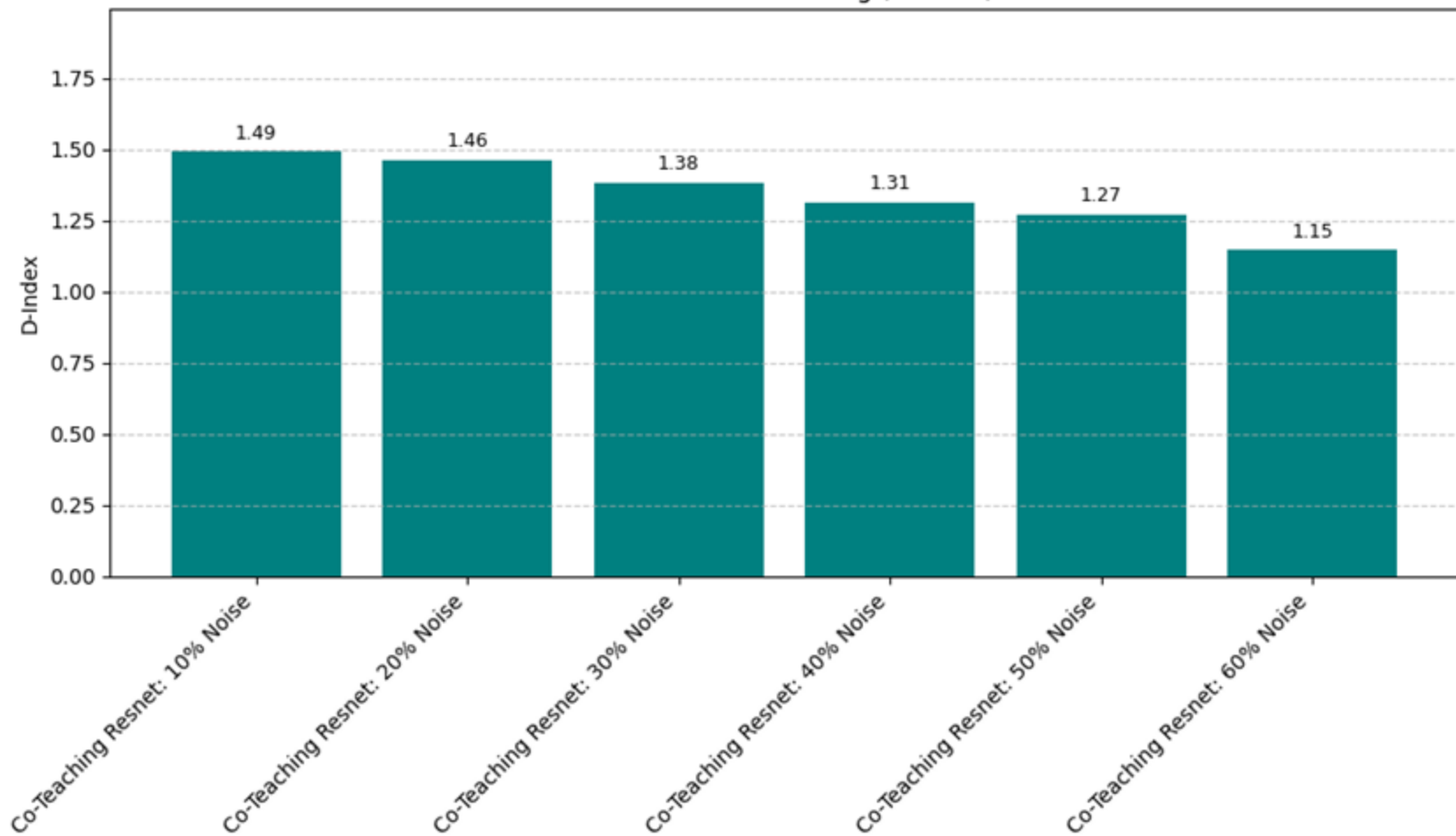
Total Misclassifications: 6689

- girl + baby (22 times)
- oak_tree + maple_tree (17 times)
- oak_tree + pine_tree (17 times)

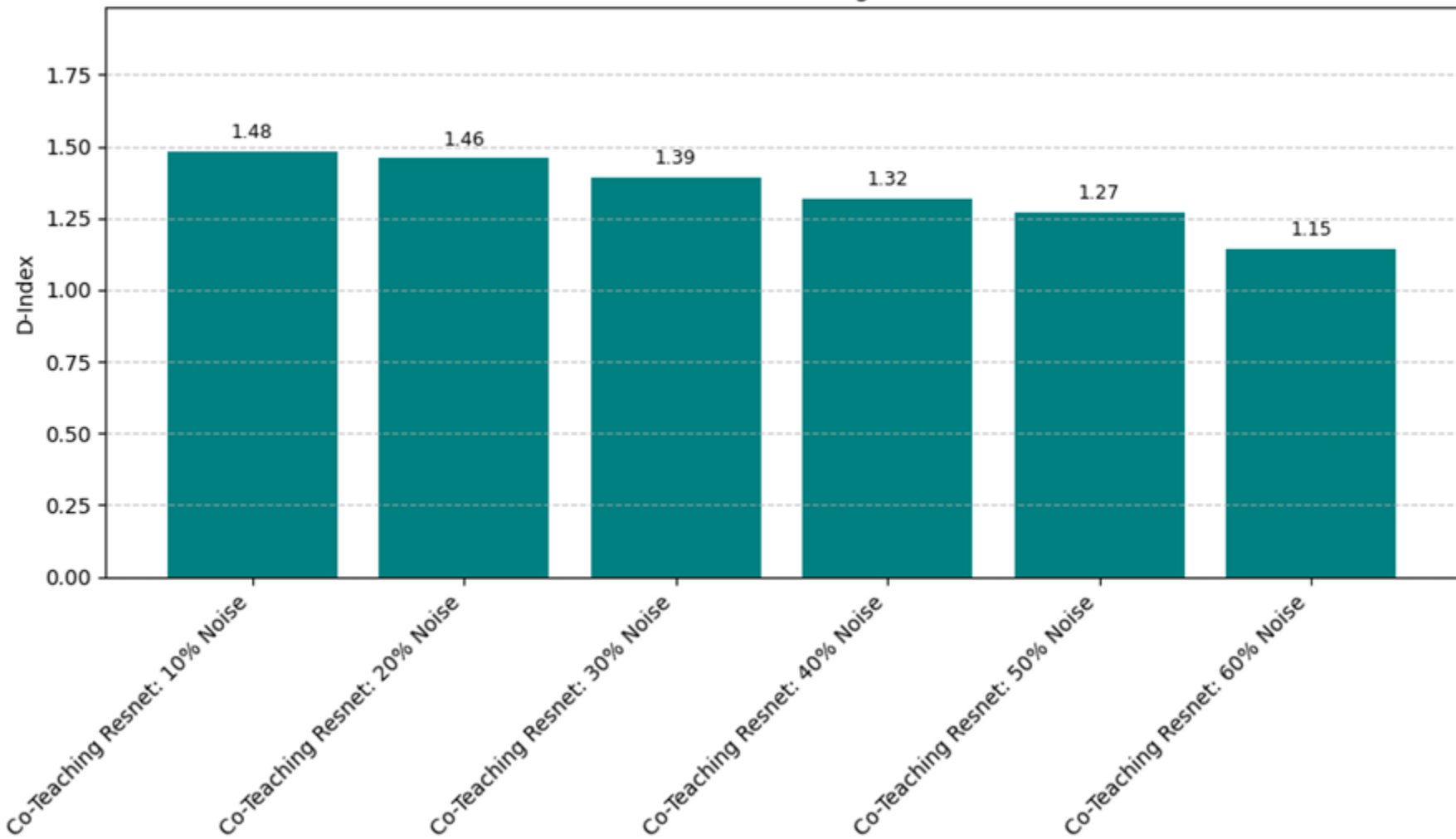
Precision, Recall, and F1 Score for Each Model



D-Index for Co Teaching (model1)



D-Index for Co Teaching (model2)



ResNet (Overall)

- Trained ResNet with MNIST and CIFAR-10, whose metrics began to fall off during higher levels of noise corruption on MNIST and even more on CIFAR-10 for any percentage of noise corruption
- Compared ResNet's performance with MNIST and CIFAR-10 variants using NFL/RCE loss functions
- Trained ResNet with CIFAR-100 using NFL/RCE loss functions; struggled to identify the images correctly when the images are corrupted
- Compared Co-Taught ResNet performance with CIFAR-100 variants using NFL/RCE loss function

Vision Transformers (ViT)

- Deep learning architecture designed for image classification tasks
- Images are divided into 16x16 patches and flattened into a sequence of patch vectors
- Patch vectors are linearly projected into a lower dimensional embedding space
- Positional encodings are added to each for spatial information
- CLS token is prepended to each
- Run through a transformer encoder and a final MLP head to make a prediction

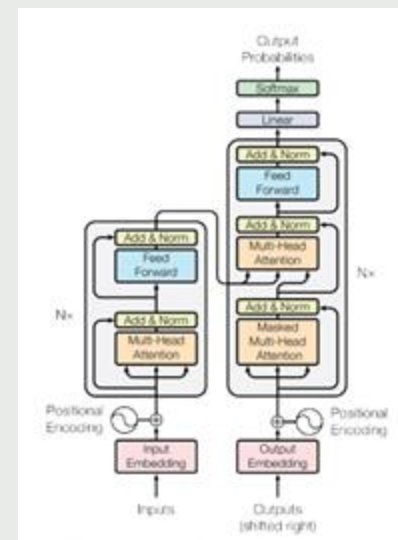


Figure 1: The Transformer - model architecture.

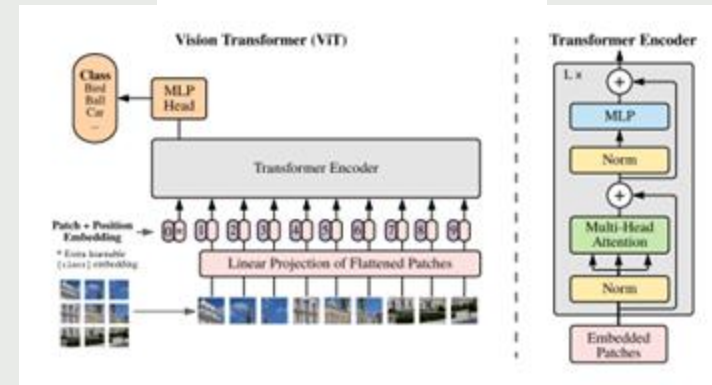


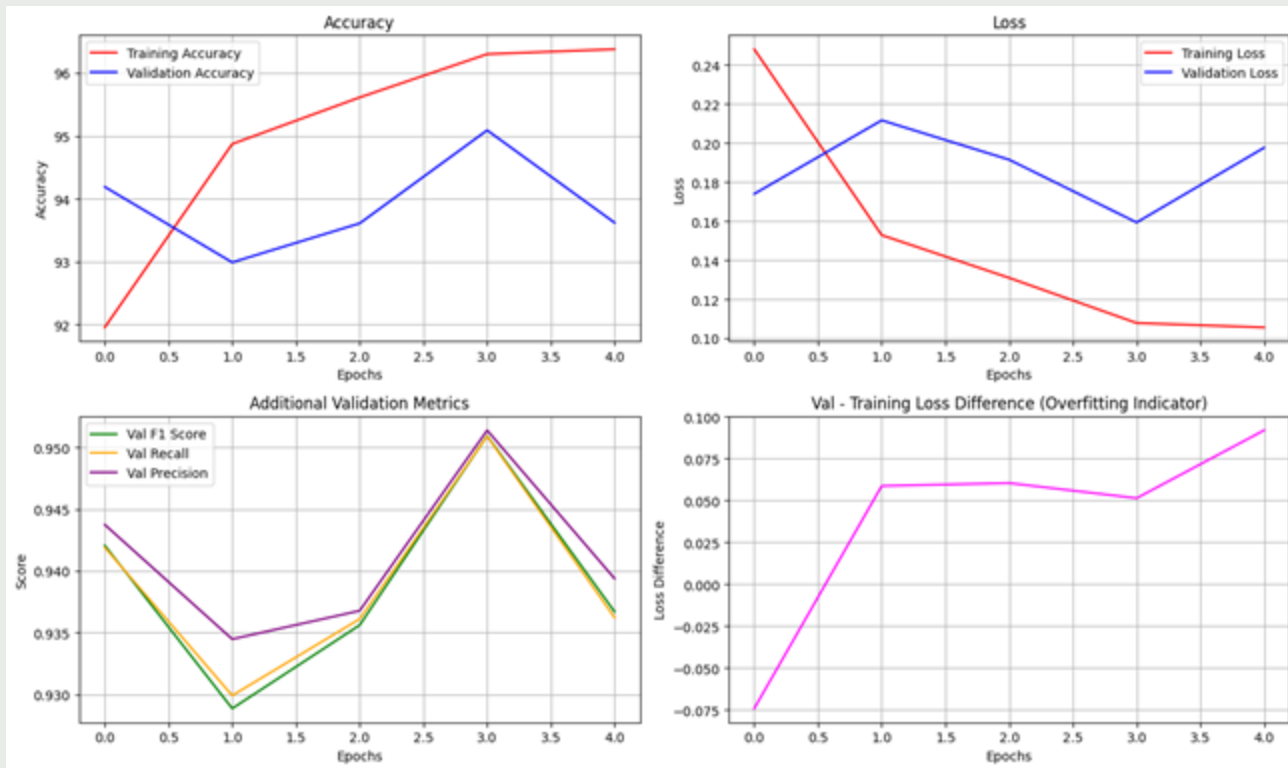
Image Source: Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). *Attention Is All You Need*. NeurIPS

Image Source: Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2020). *An image is worth 16x16 words: Transformers for image recognition at scale*. NeurIPS.

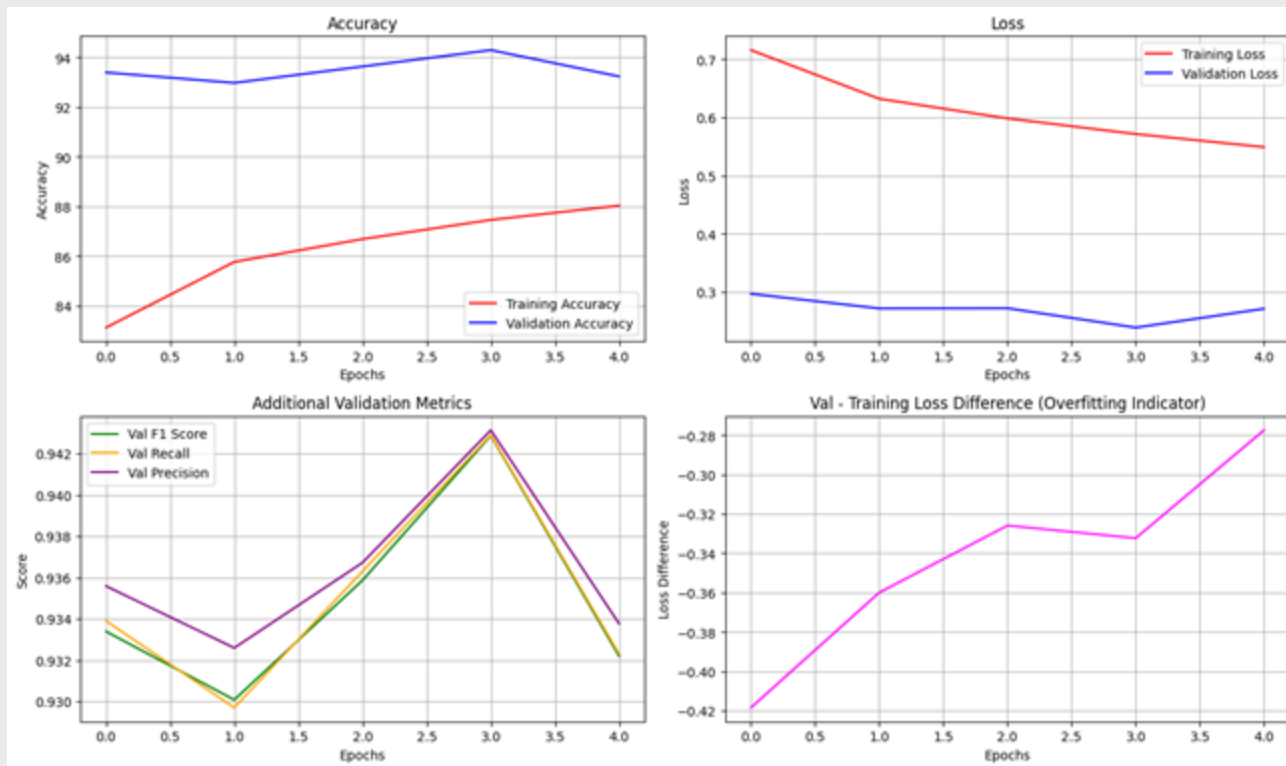
Comparing Vision Transformer Performance with CIFAR-10 Variants



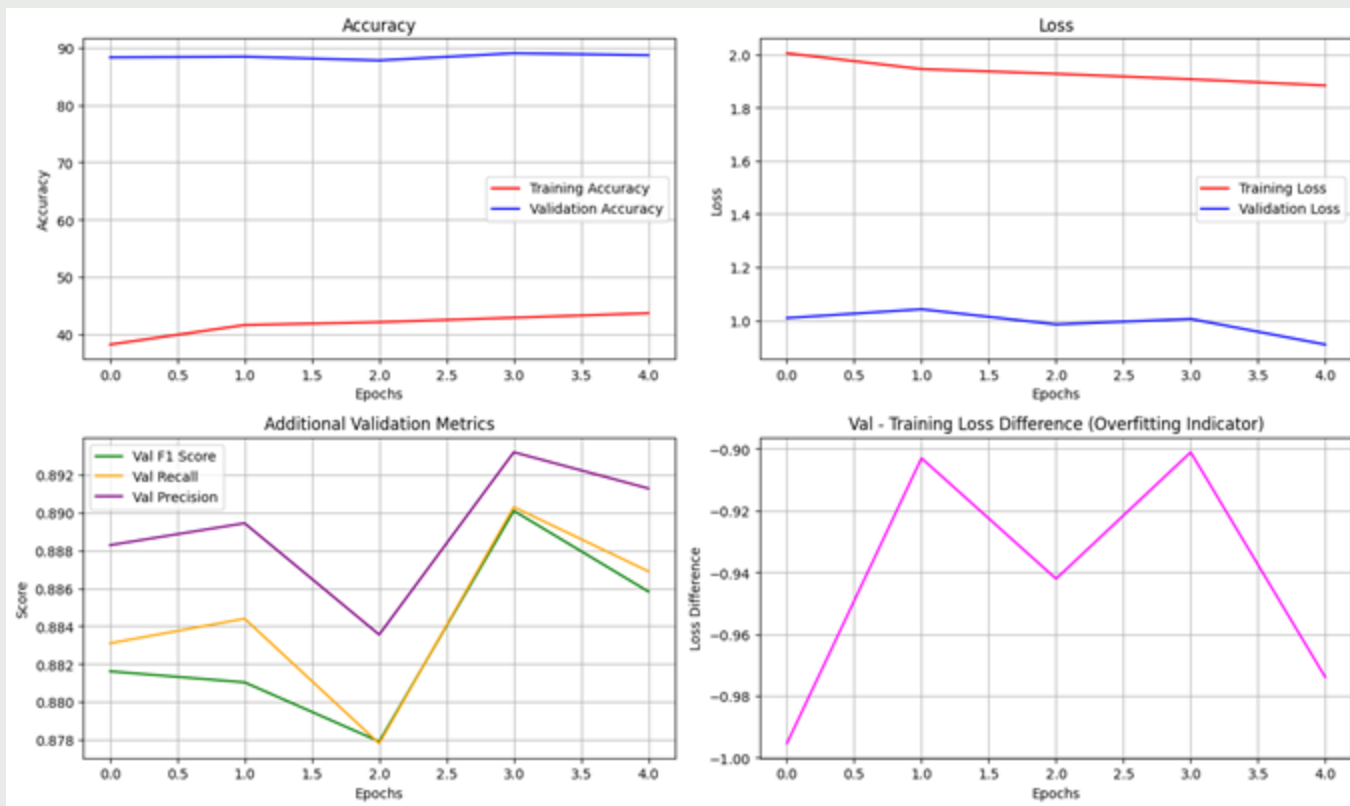
CIFAR10 with No Label Corruption



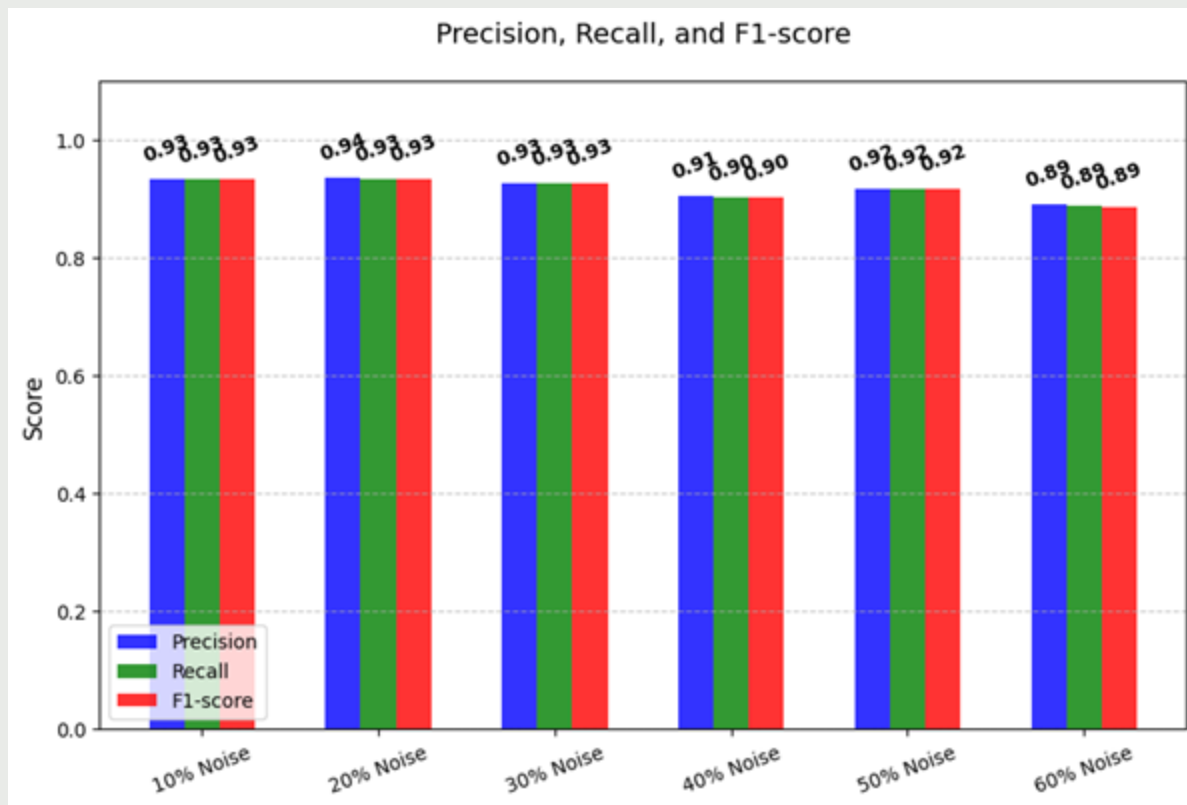
CIFAR10 with Label Corruption 10%



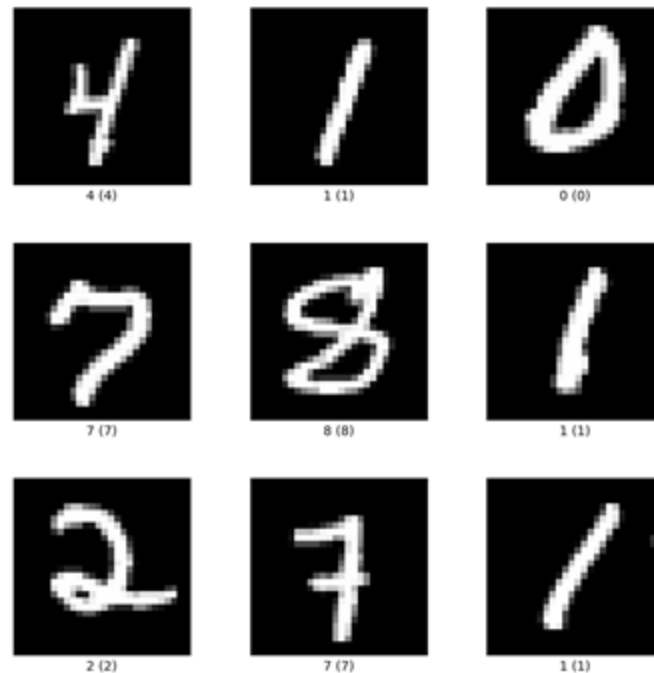
CIFAR10 with Label Corruption 60%



Classification Metrics



Comparing Vision Transformer Performance with MNIST Variants



MNIST with Label Corruption 10-30%

10%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [34:07:00:00, 2.18s/it, loss=0.39]

Epoch [1/5] - loss: 0.5798, Accuracy: 88.85% | Val loss: 0.1188, Val Accuracy: 99.40%

Epoch 2/5: 100% [██████████] 938/938 [34:12:00:00, 2.19s/it, loss=0.106]

Epoch [2/5] - loss: 0.5343, Accuracy: 90.12% | Val loss: 0.1246, Val Accuracy: 98.93%

Epoch 3/5: 100% [██████████] 938/938 [34:12:00:00, 2.19s/it, loss=0.667]

Epoch [3/5] - loss: 0.5272, Accuracy: 90.22% | Val loss: 0.1363, Val Accuracy: 98.23%

Epoch 4/5: 100% [██████████] 938/938 [34:14:00:00, 2.19s/it, loss=0.526]

Epoch [4/5] - loss: 0.5248, Accuracy: 90.31% | Val loss: 0.1353, Val Accuracy: 99.10%

Epoch 5/5: 100% [██████████] 938/938 [34:14:00:00, 2.19s/it, loss=0.313]

Epoch [5/5] - loss: 0.5394, Accuracy: 90.42% | Val loss: 0.1259, Val Accuracy: 99.30%

Model saved to /content/drive/MyDrive/vit_models/50percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/50percent/vit_mnist_history.pkl
{'loss': [0.5798126620637481,
0.5343033198926495,
0.527223778497944,
0.5247926487424655,
0.5194496129696027],
'val_loss': [0.1188228614218675,
0.12458365448511072,
0.13633675621763158,
0.135342316291882775,
0.12588541867749787],
'accuracy': [88.85333333333334,
90.11666666666666,
90.225,
90.31366666666667,
90.413],
'val_accuracy': [99.09, 98.93, 98.23, 99.1, 99.1]}
```

MNIST Final Accuracy
Train: 90.44%,
Test: 99.10%

20%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [33:50:00:00, 2.16s/it, loss=1.17]

Epoch [1/5] - loss: 0.9424, Accuracy: 80.04% | Val loss: 0.2483, Val Accuracy: 98.53%

Epoch 2/5: 100% [██████████] 938/938 [33:53:00:00, 2.17s/it, loss=0.84]

Epoch [2/5] - loss: 0.8998, Accuracy: 81.14% | Val loss: 0.2473, Val Accuracy: 98.35%

Epoch 3/5: 100% [██████████] 938/938 [33:53:00:00, 2.17s/it, loss=0.744]

Epoch [3/5] - loss: 0.8939, Accuracy: 81.32% | Val loss: 0.2980, Val Accuracy: 99.49%

Epoch 4/5: 100% [██████████] 938/938 [33:55:00:00, 2.17s/it, loss=0.906]

Epoch [4/5] - loss: 0.8858, Accuracy: 81.48% | Val loss: 0.2449, Val Accuracy: 98.34%

Epoch 5/5: 100% [██████████] 938/938 [33:53:00:00, 2.17s/it, loss=1.73]

Epoch [5/5] - loss: 0.8868, Accuracy: 81.43% | Val loss: 0.2205, Val Accuracy: 99.33%

Model saved to /content/drive/MyDrive/vit_models/20percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/20percent/vit_mnist_history.pkl
{'loss': [0.9423789918613332,
0.8997570901537246,
0.8929168311263452,
0.885763943244553,
0.886842878878127],
'val_loss': [0.2482578422233573,
0.2473339385754365,
0.24481928798529136,
0.24485118468382884,
0.22846951465545947],
'accuracy': [80.04333333333334,
81.14333333333333,
81.32333333333334,
81.47833333333334,
81.43166666666667],
'val_accuracy': [98.53, 98.35, 99.49, 98.34, 99.33]}
```

MNIST Final Accuracy
Train: 81.80%,
Test: 99.33%

30%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [35:48:00:00, 2.29s/it, loss=1.34]

Epoch [1/5] - loss: 1.2487, Accuracy: 71.13% | Val loss: 0.3934, Val Accuracy: 97.25%

Epoch 2/5: 100% [██████████] 938/938 [35:51:00:00, 2.29s/it, loss=0.986]

Epoch [2/5] - loss: 1.2078, Accuracy: 72.19% | Val loss: 0.3382, Val Accuracy: 99.35%

Epoch 3/5: 100% [██████████] 938/938 [35:52:00:00, 2.29s/it, loss=0.993]

Epoch [3/5] - loss: 1.2023, Accuracy: 72.25% | Val loss: 0.3488, Val Accuracy: 99.25%

Epoch 4/5: 100% [██████████] 938/938 [35:53:00:00, 2.30s/it, loss=1.42]

Epoch [4/5] - loss: 1.1983, Accuracy: 72.44% | Val loss: 0.3497, Val Accuracy: 99.34%

Epoch 5/5: 100% [██████████] 938/938 [35:51:00:00, 2.29s/it, loss=1.5]

Epoch [5/5] - loss: 1.1943, Accuracy: 72.44% | Val loss: 0.3464, Val Accuracy: 99.40%

Model saved to /content/drive/MyDrive/vit_models/30percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/30percent/vit_mnist_history.pkl
{'loss': [1.2487352775739455,
1.2077608323623666,
1.202285713826494,
1.2069497767177817,
1.1942835521977593],
'val_loss': [0.3934817858818968,
0.3282219597187091,
0.34880238721384897,
0.34879573988613535,
0.34832219634881834],
'accuracy': [71.13333333333333,
72.195,
72.25166666666667,
72.43833333333333,
72.44],
'val_accuracy': [97.25, 99.35, 99.25, 99.34, 99.4]}
```

MNIST Final Accuracy
Train: 72.79%,
Test: 99.40%

MNIST with Label Corruption 40-60%

40%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [36:25<00:00, 2.33s/it, loss=1.74]

Epoch [1/5] - loss: 1.5442, Accuracy: 62.07% | Val loss: 0.5032, Val Accuracy: 98.73%
Epoch 2/5: 100% [██████████] 938/938 [36:25<00:00, 2.33s/it, loss=1.70]

Epoch [2/5] - loss: 1.4751, Accuracy: 63.13% | Val loss: 0.4383, Val Accuracy: 99.27%
Epoch 3/5: 100% [██████████] 938/938 [36:25<00:00, 2.33s/it, loss=1.17]

Epoch [3/5] - loss: 1.4761, Accuracy: 63.21% | Val loss: 0.4672, Val Accuracy: 99.21%
Epoch 4/5: 100% [██████████] 938/938 [36:27<00:00, 2.33s/it, loss=0.888]

Epoch [4/5] - loss: 1.4043, Accuracy: 63.41% | Val loss: 0.4088, Val Accuracy: 99.31%
Epoch 5/5: 100% [██████████] 938/938 [36:16<00:00, 2.33s/it, loss=1.62]

Epoch [5/5] - loss: 1.4033, Accuracy: 63.33% | Val loss: 0.4542, Val Accuracy: 99.40%

Model saved to /content/drive/MyDrive/vit_models/40percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/40percent/vit_mnist_history.pkl
{"loss": [1.544208308668863,
1.475149319871458,
1.4761247356531467,
1.4042789531186417,
1.4056830493188917],
"val_loss": [0.503203689197483,
0.42534910888078237,
0.46723486073322946,
0.4667737378931434,
0.4546857227365377],
"accuracy": [62.07333333333333,
63.13,
63.206666666666666,
63.41166666666667,
63.325],
"val_accuracy": [98.73, 99.27, 99.33, 99.35, 99.40]}
```

50%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [36:45<00:00, 2.35s/it, loss=1.75]

Epoch [1/5] - loss: 1.7428, Accuracy: 53.81% | Val loss: 0.5743, Val Accuracy: 98.58%
Epoch 2/5: 100% [██████████] 938/938 [36:46<00:00, 2.35s/it, loss=2.03]

Epoch [2/5] - loss: 1.7953, Accuracy: 54.38% | Val loss: 0.6979, Val Accuracy: 97.87%
Epoch 3/5: 100% [██████████] 938/938 [36:44<00:00, 2.35s/it, loss=1.71]

Epoch [3/5] - loss: 1.6998, Accuracy: 54.23% | Val loss: 0.6165, Val Accuracy: 98.90%
Epoch 4/5: 100% [██████████] 938/938 [36:45<00:00, 2.35s/it, loss=1.63]

Epoch [4/5] - loss: 1.6867, Accuracy: 54.35% | Val loss: 0.6347, Val Accuracy: 98.97%
Epoch 5/5: 100% [██████████] 938/938 [36:45<00:00, 2.35s/it, loss=1.54]

Epoch [5/5] - loss: 1.6953, Accuracy: 54.37% | Val loss: 0.6483, Val Accuracy: 98.63%

Model saved to /content/drive/MyDrive/vit_models/50percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/50percent/vit_mnist_history.pkl
{"loss": [1.7428383273685585,
1.7953668228279732,
1.6997815299257543,
1.68678181348572922,
1.695324114398782],
"val_loss": [0.574278793573357,
0.697929934678515,
0.6165283785778183,
0.634789375879555,
0.6483666666666667],
"accuracy": [53.81, 54.381666666666665, 54.235, 54.355, 54.36833333333333],
"val_accuracy": [98.58, 97.87, 98.9, 98.97, 98.63]}
```

60%

```
Training VIT on MNIST:
No existing history found. Creating a new history dictionary.
Epoch 1/5: 100% [██████████] 938/938 [36:18<00:00, 2.31s/it, loss=2.22]

Epoch [1/5] - loss: 1.9688, Accuracy: 41.64% | Val loss: 0.9852, Val Accuracy: 98.18%
Epoch 2/5: 100% [██████████] 938/938 [35:59<00:00, 2.38s/it, loss=1.82]

Epoch [2/5] - loss: 1.9881, Accuracy: 44.88% | Val loss: 0.8866, Val Accuracy: 98.81%
Epoch 3/5: 100% [██████████] 938/938 [35:57<00:00, 2.38s/it, loss=2.38]

Epoch [3/5] - loss: 1.8973, Accuracy: 45.28% | Val loss: 0.8855, Val Accuracy: 98.78%
Epoch 4/5: 100% [██████████] 938/938 [35:57<00:00, 2.38s/it, loss=1.98]

Epoch [4/5] - loss: 1.8949, Accuracy: 45.38% | Val loss: 0.9038, Val Accuracy: 97.51%
Epoch 5/5: 100% [██████████] 938/938 [36:24<00:00, 2.33s/it, loss=1.58]

Epoch [5/5] - loss: 1.8932, Accuracy: 45.41% | Val loss: 0.7768, Val Accuracy: 98.71%

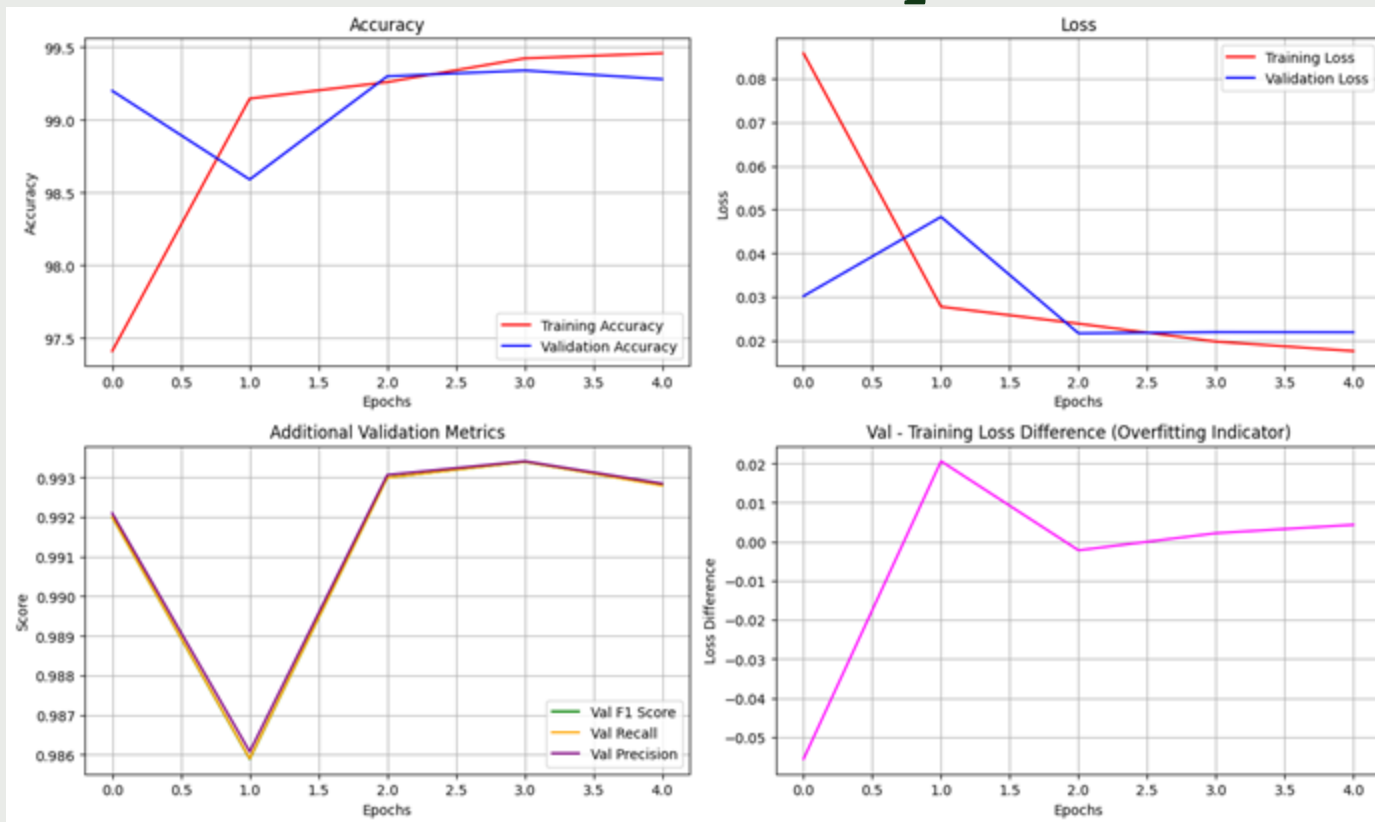
Model saved to /content/drive/MyDrive/vit_models/60percent/vit_mnist.pth
Training history saved to /content/drive/MyDrive/vit_models/60percent/vit_mnist_history.pkl
{"loss": [1.968802723825422,
1.9880512811268826,
1.89733786220888,
1.8948508598988836,
1.8921648667871802],
"val_loss": [0.985175337333753,
0.886599438373884,
0.885485123638887,
0.905764881454536,
0.776824678737367],
"accuracy": [41.64,
44.86333333333333,
45.285,
45.26333333333333,
45.408666666666666],
"val_accuracy": [98.18, 98.81, 98.78, 97.51, 98.71]}
```

MNIST Final Accuracy
Train: 63.69%,
Test: 99.05%

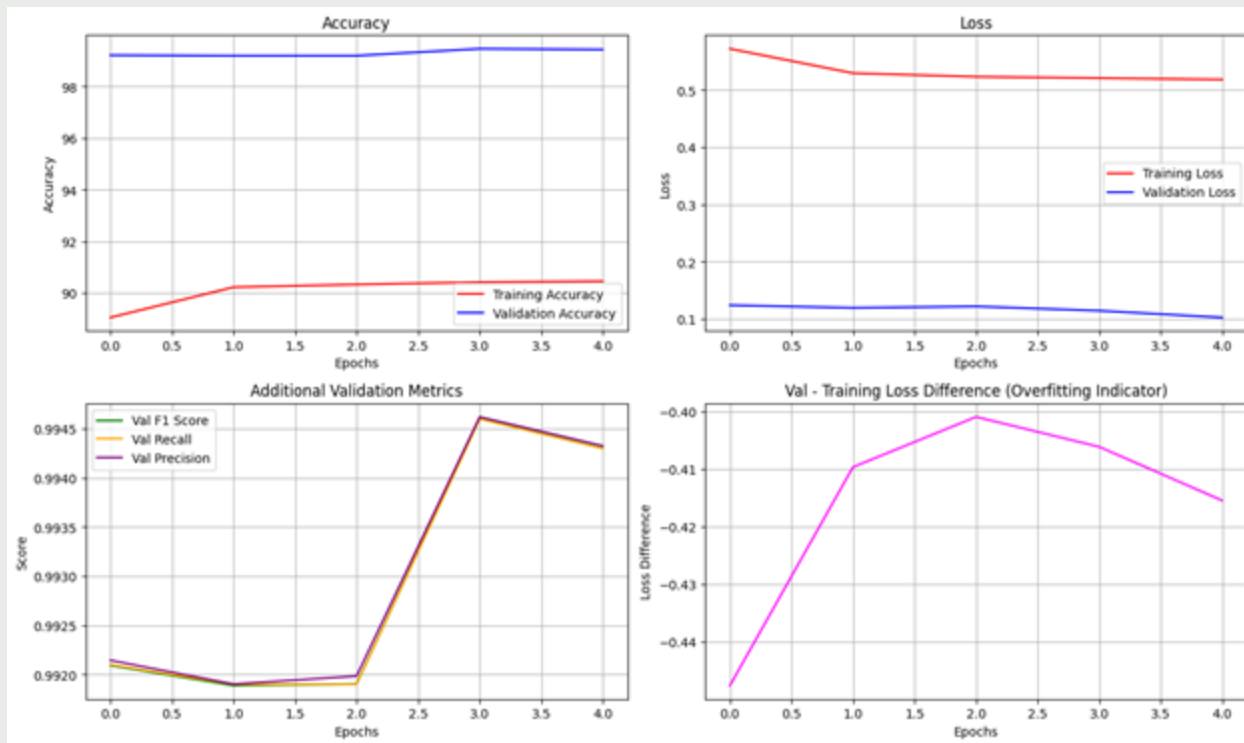
MNIST Final Accuracy
Train: 54.45%,
Test: 98.63%

MNIST Final Accuracy
Train: 45.48%,
Test: 98.25%

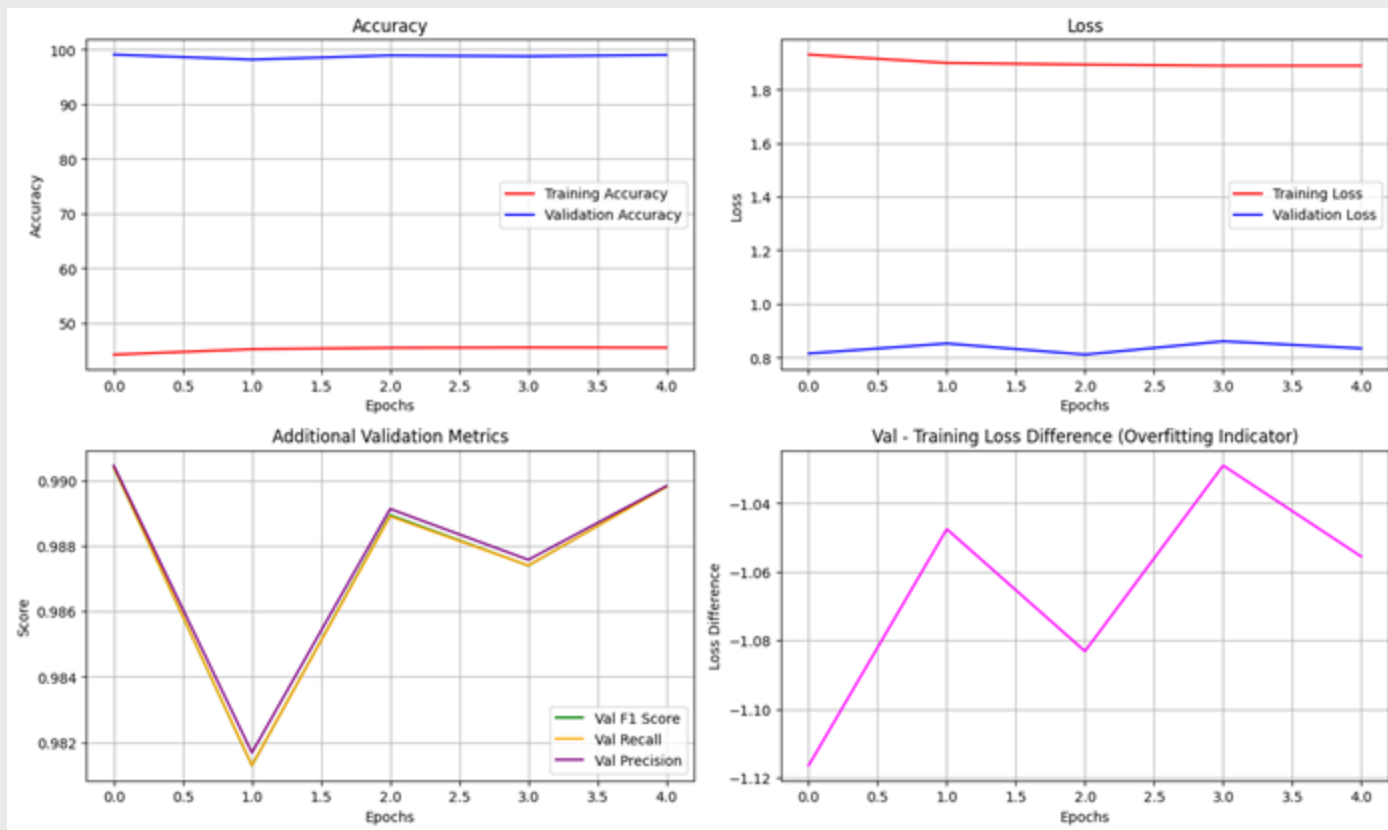
MNIST with No Label Corruption



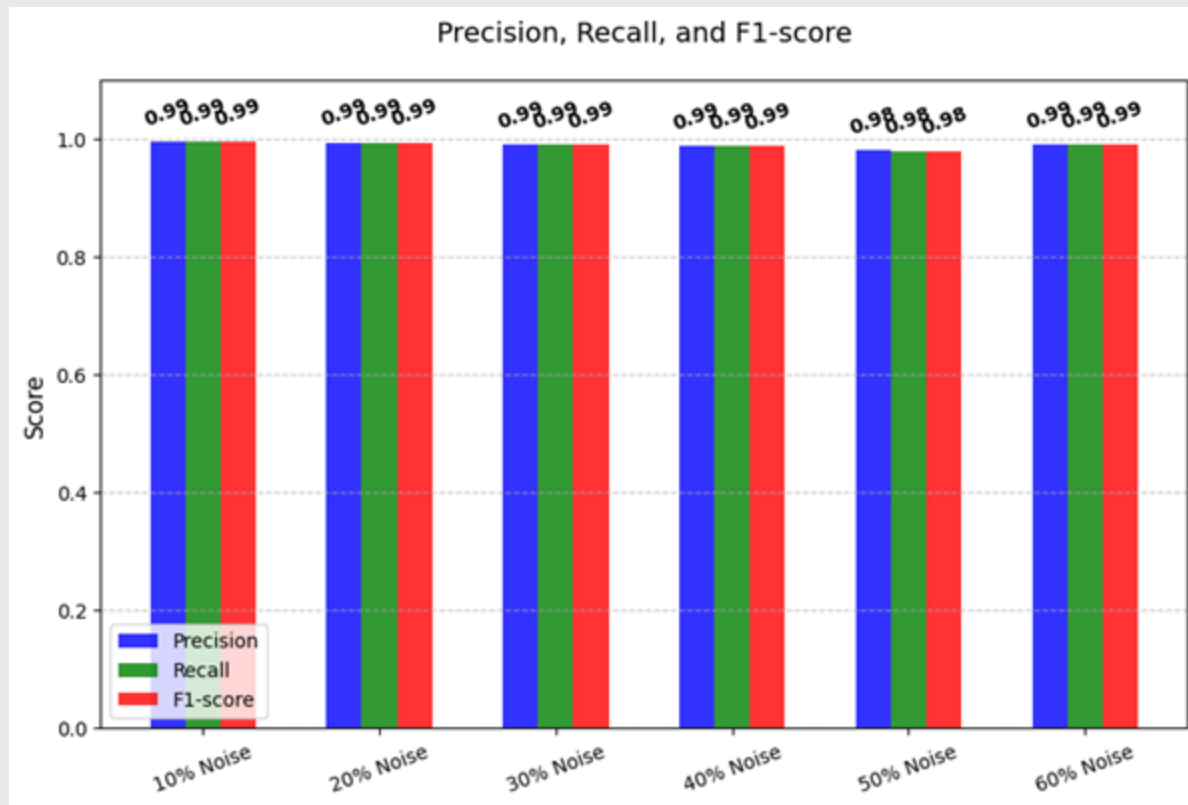
MNIST with Label Corruption 10%



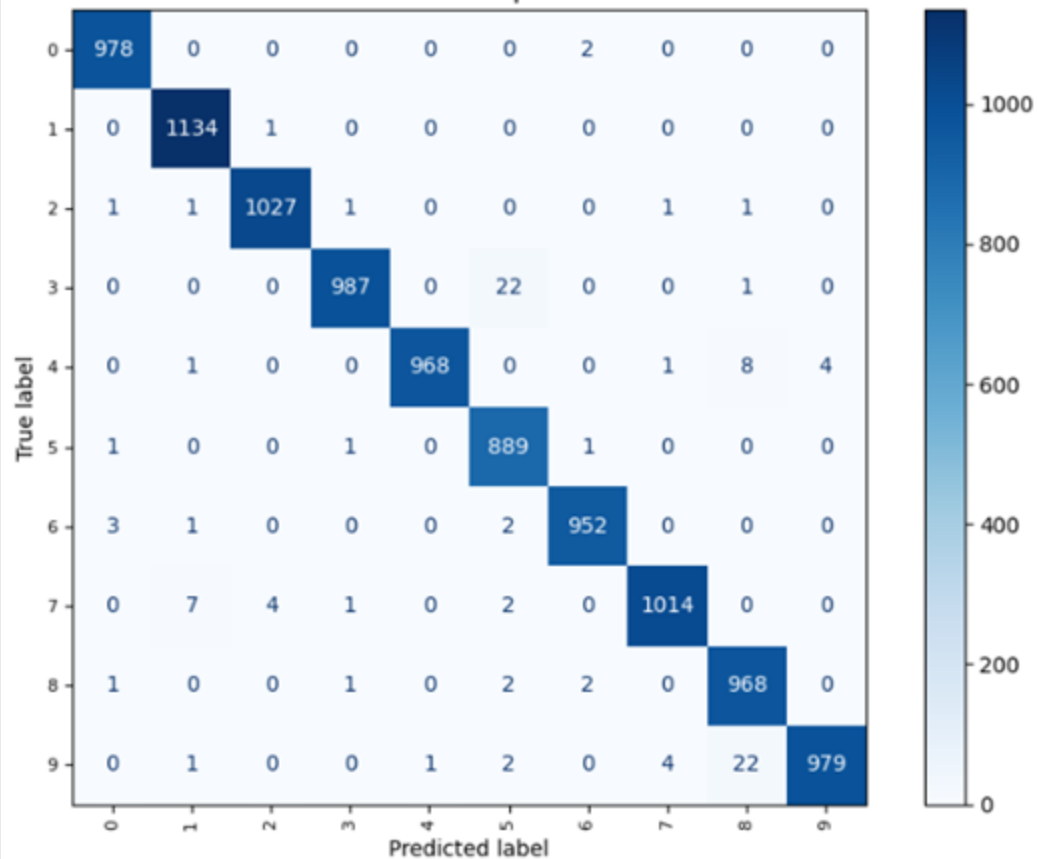
MNIST with Label Corruption 60%



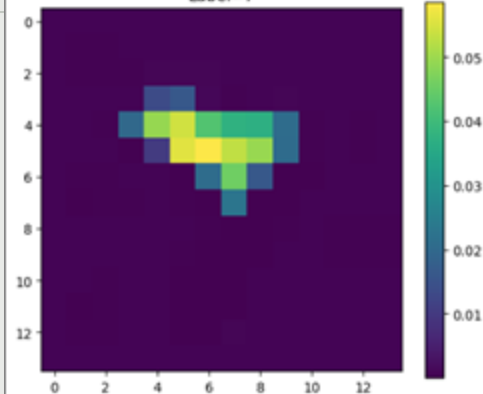
Classification Metrics



Confusion Matrix: 30percent Noise

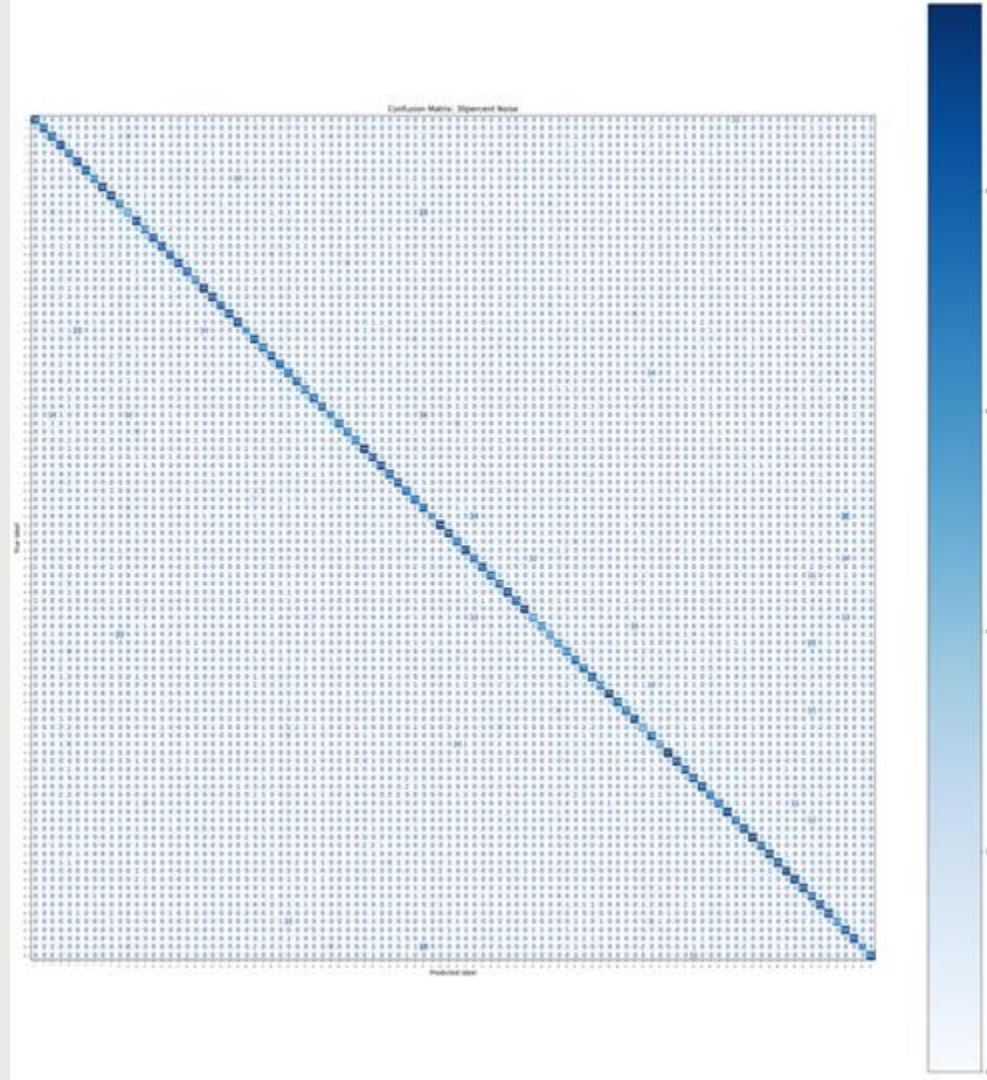


[CLS] → Patch Attention (Head 0)
Label=7

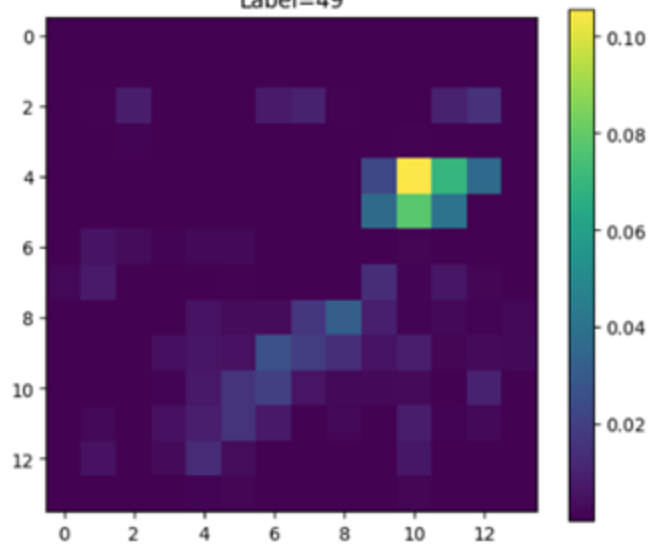


Comparing Vision Transformer Performance with CIFAR-100 Variants

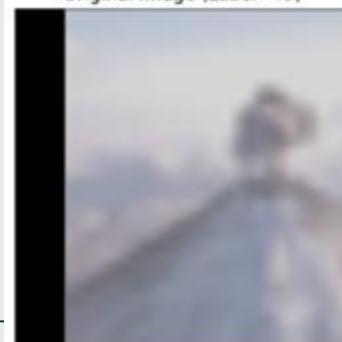




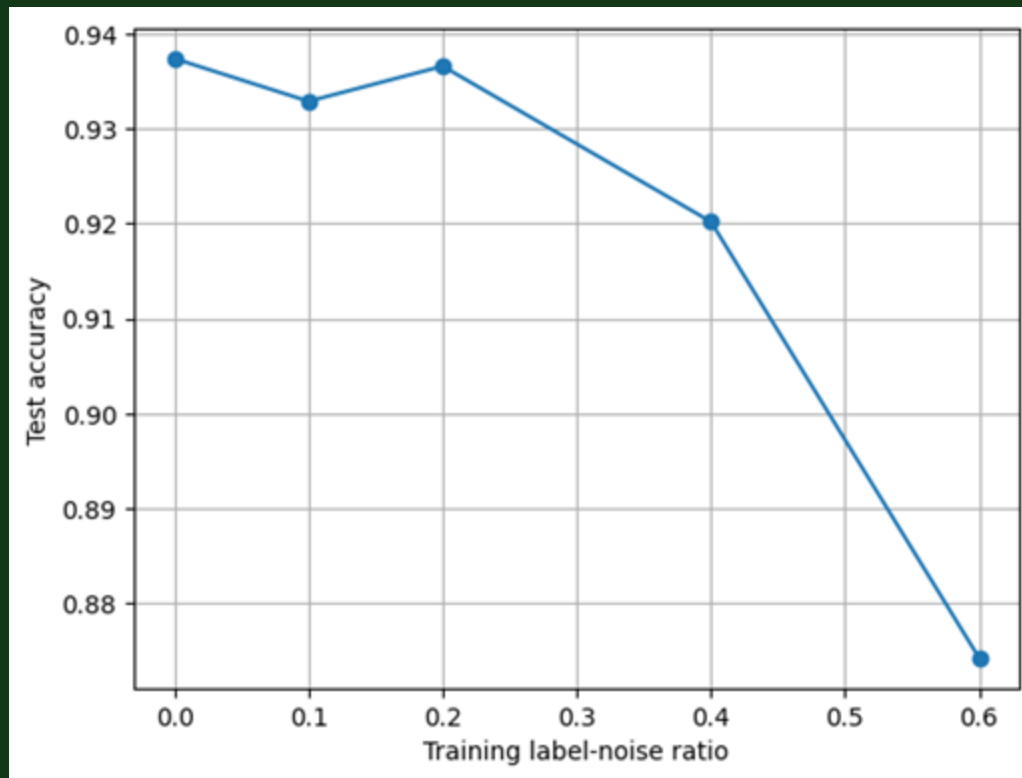
[CLS] → Patch Attention (Head 0)
Label=49



Original Image (Label=49)



Diagnosing & Mitigating Label-Noise through inference



Suspects

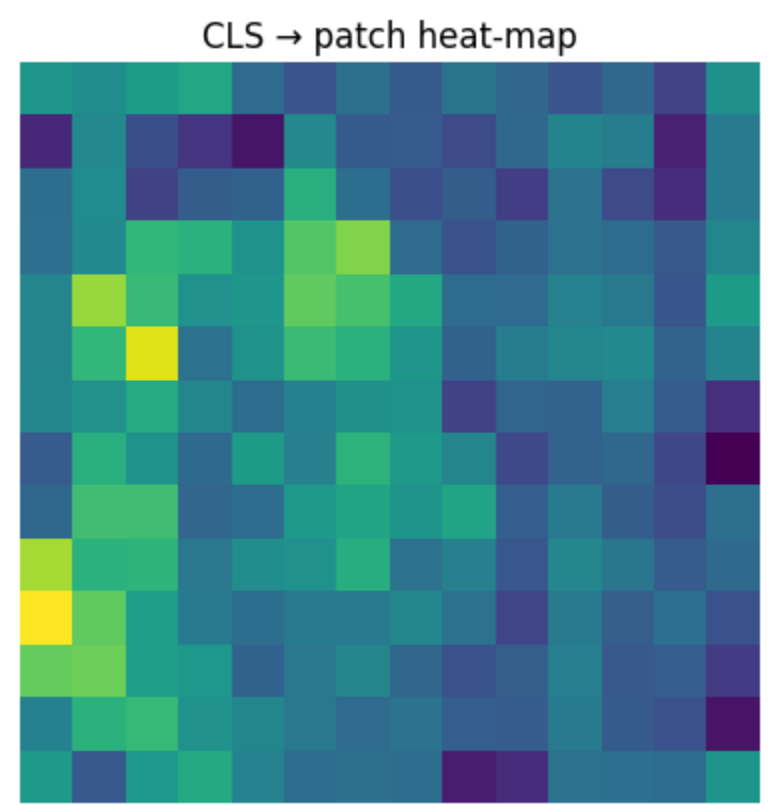
Confidence: 100%  782/782 [06:47<00:00, 2.49it/s]

Aug-consistency: 100%  782/782 [20:52<00:00, 1.24s/it]

Top-10 suspect indices: [49882, 10663, 21799, 32211, 30263, 19666, 36593, 32619, 21371, 35196]



CLS → patch heat-map



Vision Transformer (Overall)

- Tested Vision Transformer on MNIST, CIFAR-10, and CIFAR-100
- Compared Vision Transformer's performance with MNIST, CIFAR-10, and CIFAR-100 variants
- Takes quite some time to generate the results compared to the other architectures
- Had to retrain in order to provide more accurate results including adding in classification metrics